
DATA SCIENCE

Série 1 — Volume I

Fondations

Mathématiques · Probabilités · Statistique
Programmation · Bases de données · Outils

Un livre pour apprendre le métier pas à pas,
avec une boutique pour fil rouge.

Table des matières

Avant-propos	1
Bienvenue	1
À qui s'adresse ce livre ?	1
Comment ce livre est construit	2
Un livre, et son cahier	2
Les encadrés	2
Le principe « essentiel / pour aller plus loin »	3
Trois parcours de lecture	3
Le fil rouge : la boutique	4
Le code de ce livre	4
La carte du volume	5
Faites le point : un petit test de départ	5
Rappel express : les bases à avoir en tête	7
R.1 Fractions et pourcentages	7
R.2 Puissances et racines	7
R.3 Équations et fonctions affines	8
R.4 Le symbole $\sum$ (somme)	8
R.5 Exponentielle et logarithme	9
R.6 Fonctions, graphes et équations de droites	9
R.7 Un peu de vocabulaire de programmation	9
Faites le point	10
Chapitre 1 : Mathématiques pour la data science	11
Pourquoi des mathématiques ?	11
Le chemin de ce chapitre	11
1.1 Algèbre linéaire	12
1.1.1 Vecteurs : décrire un client par une liste de nombres	12
1.1.2 Matrices : le tableau de données devient un objet mathématique	15
1.1.3 Valeurs propres et vecteurs propres : les directions privilégiées	20
1.1.4 La décomposition en valeurs singulières (SVD)	24
1.2 Analyse : mesurer le changement	26
1.2.1 La dérivée : la vitesse à laquelle une fonction change	26
1.2.2 Dérivées partielles et gradient : plusieurs variables à la fois	30
1.2.3 Intégrales : accumuler les petits changements	32
1.3 Optimisation : trouver le meilleur choix	34
1.3.1 Le problème d'optimisation	35
1.3.2 La convexité : quand il n'y a qu'une vallée	35
1.3.3 La descente de gradient	37

1.3.4 Un exemple de décision : le prix qui maximise les recettes	40
1.3.5 Optimisation sous contraintes : les multiplicateurs de Lagrange	41
1.4 La fiche de notations	42
1.4.1 Conventions typographiques	42
1.4.2 Ensembles et logique	43
1.4.3 Sommes, produits, fonctions	43
1.4.4 Algèbre linéaire et analyse	44
1.4.5 L'alphabet grec en data science	44
1.4.6 Exemple : lire une formule du début à la fin	45
1.5 ⊕ Pour aller plus loin : analyse numérique	46
1.5.1 Pourquoi $0,1 + 0,2$ n'est pas égal à $0,3$	46
1.5.2 Les grands nombres avalent les petits	47
1.5.3 L'annulation catastrophique	47
1.5.4 Le conditionnement : quand un problème est « fragile »	48
1.5.5 Deux algorithmes classiques	48
1.6 ⊕ Pour aller plus loin : mathématiques discrètes et graphes	49
1.6.1 Ensembles : le langage de base	49
1.6.2 Compter : le principe multiplicatif	50
1.6.3 Les graphes : modéliser des relations	51
1.6.4 Parcourir un graphe : la recherche en largeur	52
Bilan du chapitre 1	52
Chapitre 2 : Probabilités	53
Le chemin de ce chapitre	53
2.1 Probabilités et formule de Bayes	54
2.1.1 Le vocabulaire : univers, événements, probabilité	54
2.1.2 Voir une probabilité : la simulation	55
2.1.3 Probabilité conditionnelle : changer d'information	55
2.1.4 Indépendance	56
2.1.5 La formule des probabilités totales	57
2.1.6 La formule de Bayes	57
2.1.7 Un classifieur à la main : l'idée du filtre naïf de Bayes	58
2.1.8 Trois paradoxes pour s'entraîner à se méfier de l'intuition	59
2.2 Variables aléatoires et lois usuelles	60
2.2.1 Qu'est-ce qu'une variable aléatoire ?	60
2.2.2 Loi de Bernoulli : le oui/non	61
2.2.3 Loi binomiale : compter les succès	62
2.2.4 Loi de Poisson : compter des événements rares	63
2.2.5 Les variables continues : la densité	63
2.2.6 Quelle loi choisir ?	66
2.3 Espérance, variance, covariance	66
2.3.1 L'espérance : la moyenne « à long terme »	66
2.3.2 La variance : mesurer l'étalement	68
2.3.3 Covariance et corrélation : bouger ensemble	69
2.4 Loi des grands nombres et théorème central limite	71
2.4.1 La moyenne d'échantillon est elle-même une variable aléatoire	72
2.4.2 La loi des grands nombres	72
2.4.3 Le théorème central limite	73
2.4.4 Trois usages du théorème central limite	75
2.4.5 Un mot de prudence	75
2.5 ⊕ Pour aller plus loin : la théorie de la mesure (ce que « probabilité » veut vraiment dire)	76

2.5.1 Le problème : on ne peut pas mesurer tous les ensembles	76
2.5.2 Le cadre : un espace probabilisé $(\Omega, \mathcal{F}, P)$	76
2.5.3 Une conséquence surprenante : événements de probabilité 0 qui arrivent . . .	77
2.5.4 L'espérance comme intégrale	77
2.5.5 Les modes de convergence	77
2.5.6 La densité comme dérivée d'une mesure	78
2.6 ⊕ Pour aller plus loin : les processus stochastiques	78
2.6.1 La marche aléatoire	79
2.6.2 Les chaînes de Markov : le futur ne dépend que du présent	79
2.6.3 Le processus de Poisson : des arrivées au hasard	81
Bilan du chapitre 2	82
Chapitre 3 : Statistique	83
Le chemin de ce chapitre	83
3.1 Statistique descriptive	84
3.1.1 Population, échantillon, variables	84
3.1.2 Le jeu de données du chapitre	84
3.1.3 Mesures de position : où est le centre ?	85
3.1.4 Mesures de dispersion : de combien ça varie ?	86
3.1.5 La forme : histogrammes, asymétrie, boîtes à moustaches	86
3.1.6 Variables qualitatives et comparaison de groupes	87
3.1.7 Relier deux variables : corrélation, et pourquoi il faut toujours dessiner	87
3.2 Estimation	89
3.2.1 Un estimateur est une variable aléatoire	89
3.2.2 Qu'est-ce qu'un bon estimateur ?	89
3.2.3 Pourquoi divise-t-on par $n - 1$ pour la variance ?	90
3.2.4 La méthode des moments	91
3.2.5 Le maximum de vraisemblance	91
3.2.6 Pourquoi le maximum de vraisemblance est la référence	93
3.3 Intervalles de confiance	94
3.3.1 Construire un intervalle pour une moyenne	94
3.3.2 Que veut dire « 95 % de confiance » ?	94
3.3.3 Quand l'écart-type est inconnu : la loi de Student	96
3.3.4 Intervalle pour une proportion	97
3.3.5 Quand il n'y a pas de formule : le bootstrap	97
3.3.6 Dimensionner un échantillon	98
3.4 Tests d'hypothèses	99
3.4.1 Le vocabulaire et la méthode	99
3.4.2 Test de Student sur une moyenne	99
3.4.3 Comparer deux groupes : le test de Welch	100
3.4.4 Test sur une proportion	101
3.4.5 Le test A/B : comparer deux proportions	102
3.4.6 Le test du khi-deux : deux variables qualitatives sont-elles liées ?	102
3.4.7 Comment choisir son test ?	103
3.5 p-valeurs, puissance et tests multiples	104
3.5.1 Ce que la p-valeur est vraiment	104
3.5.2 Ce que la p-valeur n'est pas	104
3.5.3 Signification statistique $\neq$ importance pratique	105
3.5.4 La puissance : savoir si l'on peut voir ce qu'on cherche	105
3.5.5 Les tests multiples : le piège du « fouillis de comparaisons »	107
3.5.6 Les bonnes pratiques, en dix lignes	108
3.6 ⊕ Pour aller plus loin : les sondages et l'échantillonnage	108

3.6.1 Le biais de sélection : le pire ennemi	109
3.6.2 L'échantillonnage aléatoire simple	109
3.6.3 L'échantillonnage stratifié	110
3.6.4 Autres plans de sondage	111
3.6.5 Redresser un échantillon biaisé : la pondération	111
3.7 ☉ Pour aller plus loin : les méthodes non paramétriques	112
3.7.1 Remplacer les valeurs par leurs rangs	112
3.7.2 Le test de Mann-Whitney (deux groupes indépendants)	112
3.7.3 Autres tests de rangs	113
3.7.4 Les tests de permutation : l'idée la plus simple de la statistique	113
3.7.5 Tester la normalité	114
Bilan du chapitre 3	114
Chapitre 4 : Programmation	116
Le chemin de ce chapitre	116
4.1 Les fondamentaux de Python	117
4.1.1 Pourquoi Python, et comment l'exécuter	117
4.1.2 Variables et types	118
4.1.3 Le texte et les f-strings	119
4.1.4 Les collections : listes, tuples, dictionnaires, ensembles	119
4.1.5 Décider : conditions	120
4.1.6 Répéter : boucles	121
4.1.7 Fonctions : écrire ses propres outils	122
4.1.8 Les erreurs : vos meilleures amies	123
4.1.9 Modules, fichiers et données réelles	124
4.1.10 Un petit programme complet : le ticket de caisse	125
4.2 Les fondamentaux de R	126
4.2.1 Lancer R, et premières différences	127
4.2.2 Vecteurs : la brique de base	127
4.2.3 Fonctions, conditions et boucles	128
4.2.4 Le data.frame : le tableau de données	129
4.2.5 Les statistiques « sortent de la boîte »	130
4.2.6 Le tidyverse : manipuler les tableaux avec des « phrases »	131
4.2.7 Python ou R ? Un tableau de correspondance	132
4.3 Algorithmes et structures de données	133
4.3.1 Qu'est-ce qu'un algorithme ?	133
4.3.2 Choisir sa structure de données	134
4.3.3 Piles et files	135
4.3.4 La récursion : une fonction qui s'appelle elle-même	136
4.3.5 Rechercher : de la liste à la dichotomie	137
4.3.6 Trier	138
4.3.7 Une dernière application : les « top k »	140
4.4 NumPy et pandas : manipuler des données	141
4.4.1 NumPy : pourquoi un tableau n'est pas une liste	141
4.4.2 Créer, indexer, trancher	142
4.4.3 Le broadcasting : calculer entre tableaux de formes différentes	143
4.4.4 Agréger, trier, tirer au hasard	145
4.4.5 pandas : Series et DataFrame	146
4.4.6 Sélectionner : colonnes, lignes, conditions	147
4.4.7 Créer et transformer des colonnes	148
4.4.8 Copie, vue et pièges de pandas 3.0	149
4.4.9 Regrouper : le « split-apply-combine » (groupby)	150

4.4.10 Combiner plusieurs tables : merge et concat	152
4.4.11 Valeurs manquantes	153
4.4.12 Travailler avec des dates	154
4.4.13 Du tableau au rapport	155
4.5 Premières visualisations	156
4.5.1 Quel graphique pour quelle question ?	156
4.5.2 Anatomie d'un graphique matplotlib	157
4.5.3 Les quatre graphiques essentiels, avec pandas	158
4.5.4 seaborn : des graphiques statistiques en une ligne	160
4.5.5 ggplot2 : la grammaire des graphiques en R	162
4.5.6 Bien faire, mal faire : les pièges du graphique trompeur	163
4.5.7 Enregistrer, réutiliser : de la figure au rapport	164
4.6 ⊕ Pour aller plus loin : programmation orientée objet, code propre et tests	165
4.6.1 Pourquoi des objets ? Le problème des dictionnaires	166
4.6.2 Votre première classe	166
4.6.3 Le cahier des charges du module	167
4.6.4 Code propre : lisible avant tout	167
4.6.5 Tester son code : le filet de sécurité	168
4.6.6 Tester une fonction d'analyse	171
4.7 ⊕ Pour aller plus loin : d'autres langages (SAS, MATLAB, Julia)	171
4.7.1 La tâche : un résumé par canal	172
4.7.2 SAS : le langage des grandes organisations	172
4.7.3 MATLAB : le calcul numérique des ingénieurs	173
4.7.4 Julia : la promesse « rapide comme C, simple comme Python »	173
4.7.5 Le même calcul, vu de près : un dictionnaire de traduction	174
4.7.6 Alors, lequel choisir ?	175
4.8 ⊕ Pour aller plus loin : complexité algorithmique et optimisation du code	175
4.8.1 L'idée : compter les opérations, pas les secondes	176
4.8.2 La notation $O(\cdot)$: une définition rigoureuse	176
4.8.3 Mesurer : le temps, avec méthode	178
4.8.4 Boucles Python contre calcul vectorisé	179
4.8.5 La mémoïsation : ne jamais calculer deux fois la même chose	179
4.8.6 Profiler : mesurer avant d'optimiser	180
4.8.7 Un exemple : les paires de produits achetés ensemble	180
Bilan du chapitre 4	181
Chapitre 5 : Bases de données et SQL	183
Le chemin de ce chapitre	183
5.1 Modèle relationnel et conception de bases de données	184
5.1.1 Pourquoi pas simplement un fichier CSV ?	184
5.1.2 Le modèle relationnel	185
5.1.3 Créer des tables : un petit exemple, puis la base fournie	187
5.1.4 Types, contraintes : la base se défend	189
5.1.5 SQL sur un fichier CSV : l'import en cinq lignes	190
5.2 Requêtes SQL : sélection, jointures, agrégation	191
5.2.1 Anatomie d'une requête : SELECT ... FROM	191
5.2.2 Filtrer avec WHERE	192
5.2.3 Trier, limiter, dédoublonner	193
5.2.4 Agréger : COUNT, SUM, AVG, GROUP BY	193
5.2.5 Les jointures : relier les tables	195
5.2.6 Les sous-requêtes : une requête dans une requête	198
5.2.7 Les pièges : NULL et dates	199

5.2.8 CASE WHEN : classer et croiser	201
5.3 Fonctions fenêtres et CTE	202
5.3.1 Le principe : fonction(...) OVER (...)	202
5.3.2 Les classements : ROW_NUMBER, RANK, DENSE_RANK	203
5.3.3 Cumuls et moyennes mobiles : le cadre de la fenêtre	204
5.3.4 Comparer avec la ligne précédente : LAG et LEAD	206
5.3.5 Les CTE : donner un nom à une étape	207
5.3.6 Les CTE récursives : des requêtes qui se rappellent elles-mêmes	207
5.4 Normalisation et conception de schémas	209
5.4.1 Le problème : la grande feuille unique	209
5.4.2 Dépendances fonctionnelles : la théorie derrière la normalisation	210
5.4.3 Les trois premières formes normales	212
5.4.4 Aller plus loin, ou s'arrêter plus tôt ?	213
5.4.5 Les index : retrouver une ligne sans tout lire	214
5.4.6 Les transactions : tout ou rien	215
5.5 ⊕ Pour aller plus loin : les bases NoSQL (MongoDB, Redis)	217
5.5.1 Pourquoi sortir du relationnel ?	217
5.5.2 Les bases de documents : l'exemple de MongoDB	218
5.5.3 Les bases clé-valeur : l'exemple de Redis	219
5.5.4 Alors, que choisir ?	220
Bilan du chapitre 5	221
Chapitre 6 : Outils de travail	222
Le chemin de ce chapitre	222
L'atelier de la gérante	223
6.1 Git et gestion de versions	224
6.1.1 Pourquoi pas simplement « analyse_FINAL_vrai.py » ?	224
6.1.2 Premiers réglages	224
6.1.3 Le premier dépôt : init, add, commit	225
6.1.4 Ce qu'il y a vraiment dans un commit	226
6.1.5 Modifier, comparer, enregistrer : le cycle de travail	227
6.1.6 Défaire une erreur	227
6.1.7 Ce qu'il ne faut pas suivre : .gitignore	228
6.1.8 Les branches : tester une idée sans risque	228
6.1.9 Étiqueter une version : les tags	230
6.1.10 Dépôts distants : partager et sauvegarder	230
6.1.11 Bonnes pratiques pour la data science	231
6.2 Notebooks Jupyter	232
6.2.1 Les ingrédients : cellules et noyau	232
6.2.2 Un notebook est un fichier texte (JSON)	233
6.2.3 Le piège : l'état caché	234
6.2.4 Repérer un notebook douteux	235
6.2.5 Bonnes pratiques	235
6.2.6 Notebooks et Git : le problème des sorties	235
6.2.7 Exporter : du notebook au rapport, au script	236
6.2.8 Pour finir : choisir son outil	236
6.3 Ligne de commande et environnements	236
6.3.1 S'orienter dans les dossiers	237
6.3.2 Regarder le contenu d'un fichier	238
6.3.3 Interroger des données avec des outils de texte	238
6.3.4 Pipes et redirections : assembler de petits outils	239
6.3.5 Lancer des programmes Python depuis le terminal	240

6.3.6 Le problème des bibliothèques : les environnements virtuels	240
6.3.7 Variables d'environnement et PATH	241
6.3.8 Et sous Windows ?	241
6.4 ⊕ Pour aller plus loin : scripts shell, environnements avancés et Docker	242
6.4.1 Écrire un script shell	242
6.4.2 Enchaîner des étapes : échouer bruyamment	243
6.4.3 Faire tourner un script tout seul : cron	243
6.4.4 Isoler plus finement : plusieurs versions d'une même bibliothèque	244
6.4.5 Docker : emballer le projet avec son système	244
6.5 ⊕ Pour aller plus loin : recherche reproductible	246
6.5.1 Reproductible, répliquable : de quoi parle-t-on ?	246
6.5.2 L'aléa maîtrisé : les graines	247
6.5.3 Relever l'environnement	247
6.5.4 Un rapport qui se fabrique tout seul	248
6.5.5 make : ne refaire que ce qui a changé	249
6.5.6 Les outils « tout-en-un » : R Markdown et Quarto	249
6.5.7 LaTeX : composer proprement les formules et les PDF	250
6.5.8 Garder une empreinte des données	250
Bilan du chapitre 6	251
Points clés	253
Les points clés, chapitre par chapitre	253
Trois idées qui traversent tout le volume	254
Et maintenant ?	254

Avant-propos

Bienvenue

Ce livre est le premier d'une série de six volumes qui racontent, pas à pas, comment on devient data scientist : on part de ce qu'il y a de plus simple (additionner des nombres, ranger des données dans un tableau) et on arrive à ce qu'il y a de plus récent (modèles de langage, mise en production, modélisation du risque en banque et en assurance).

Ce premier volume est le **socle**. Il ne contient aucun modèle « à la mode » : ni réseau de neurones, ni intelligence artificielle spectaculaire. Il contient ce qui rend tout le reste possible :

- les **mathématiques** dont on a vraiment besoin (et seulement celles-là) ;
- les **probabilités**, qui donnent un langage précis à l'incertitude ;
- la **statistique**, qui apprend à tirer des conclusions honnêtes à partir de données imparfaites ;
- la **programmation**, pour que la machine fasse les calculs à notre place ;
- les **bases de données et le SQL**, pour aller chercher l'information là où elle vit ;
- les **outils de travail** (Git, notebooks, ligne de commande), pour travailler proprement et reproduire ses résultats.

Si vous ne deviez retenir qu'une idée de ce livre, ce serait celle-ci :

La data science n'est pas de la magie. C'est de l'artisanat : des mathématiques simples, du code propre et beaucoup de bon sens, appliqués avec méthode.

Un artisan, ça se forme avec des gestes répétés. C'est pourquoi ce livre est rempli d'exemples chiffrés que vous pouvez refaire à la main, puis refaire avec l'ordinateur.

À qui s'adresse ce livre ?

À toute personne qui veut apprendre le métier depuis le début :

- l'étudiant ou l'étudiante qui entre dans une filière de statistique, d'informatique ou d'ingénierie ;
- la personne en reconversion qui veut comprendre ce qu'il y a sous le capot des outils ;
- l'analyste qui sait faire des tableaux croisés et veut passer à la modélisation ;
- l'ingénieur ou l'ingénieure déjà formé(e) qui veut un ouvrage de référence clair pour réviser ses bases.

Prérequis : le niveau de mathématiques du lycée, de la curiosité, et un ordinateur. Aucune expérience de programmation n'est supposée. Si certaines notions de base sont rouillées, le « Rappel

express » qui suit cet avant-propos les remet en route en quelques pages.

Comment ce livre est construit

Les livres de mathématiques ont souvent deux défauts opposés. Certains sont **rigoureux mais froids** : des définitions, des théorèmes, des démonstrations, et on se demande à quoi tout cela sert. D'autres sont **sympathiques mais flous** : on a l'impression d'avoir compris, puis on ne sait plus rien refaire seul.

Ce volume essaie d'éviter les deux, en **alternant systématiquement** :

1. une **intuition** simple, avec une image ou une histoire ;
2. un **exemple chiffré** assez petit pour être refait à la main ;
3. quand c'est utile, une **démonstration rigoureuse**, pour que la certitude remplace la confiance aveugle ;
4. un **point à retenir** et les pièges à éviter.

Chaque idée importante reçoit au moins un exemple « sans issue de secours » : un exemple si concret qu'on ne puisse plus se dire « je n'ai pas compris ».

Un livre, et son cahier

Pour que la lecture reste fluide, **le livre ne contient ni exercices, ni corrigés, ni longues applications** : il explique, il montre des exemples, il démontre. Tout ce qui sert à **s'entraîner** vit dans un second ouvrage, le **Cahier d'exercices et d'applications** de ce volume : des exercices corrigés, classés par difficulté, des applications guidées sur des données réalistes, et le projet de clôture du volume.

Les deux ouvrages se répondent. À la fin des sections concernées, une ligne de ce type vous indique quoi faire ensuite :

Pour s'entraîner. Cahier, chapitre 3 : application 3.1, exercices 3.1 à 3.4.

Le cahier est organisé comme le livre : le chapitre 3 du cahier accompagne le chapitre 3 du livre. Une manière efficace de travailler : **lire une section du livre, puis faire sans attendre les exercices indiqués**, avec un crayon, avant de regarder les corrigés.

Les encadrés

Pour que la lecture reste fluide, chaque type de contenu a son pictogramme :

Pictogramme	Rôle
■ Intuition	L'idée en langage courant, sans formule. À lire en premier.
■ Exemple	Un calcul concret, souvent refait à la main.
■ Démonstration	Le raisonnement rigoureux. On peut la sauter à la première lecture, mais elle est ce qui rend la compréhension durable.
△ Piège	Une erreur classique, que presque tout le monde fait une fois.

Pictogramme	Rôle
✓ À retenir	Le résumé à garder en mémoire.
■ Repère	Un guide de lecture, ou l'annonce d'une section optionnelle.
■ Pour s'entraîner	Un renvoi vers les exercices et applications du cahier.
⊕ Pour aller plus loin	Section facultative : approfondissement, outil ou sujet connexe.

Le principe « essentiel / pour aller plus loin »

Dans chaque chapitre, tout ce qui n'est **pas** marqué ⊕ constitue le **parcours essentiel**. Il est complet : un lecteur qui suit uniquement ce parcours acquiert la vue d'ensemble et les compétences utiles pour la suite de la série.

Les sections marquées ⊕ **Pour aller plus loin** sont des cadeaux, pas des obligations : elles approfondissent un point, présentent un outil de plus ou ouvrent une porte. Certains lecteurs préfèrent se concentrer sur la grande idée ; d'autres aiment creuser. Les deux manières de lire ce livre sont bonnes.

Trois parcours de lecture

Parcours	Pour qui	Que lire
Essentiel	Vous voulez la vue d'ensemble, vite	Le livre sans les démonstrations ■ ni les sections ⊕ ; les ■, ■, ▲ et ✓ suffisent
Complet	Vous voulez comprendre en profondeur	Tout le parcours essentiel, démonstrations comprises, et les exercices du cahier après chaque section
Praticien	Vous aimez coder avant de théoriser	Le chapitre 6 (outils), puis le 4 (programmation), puis le 5 (SQL), avec les applications du cahier ; les chapitres 1 à 3 ensuite

Vous n'avez jamais programmé ? Pas de panique : dans les chapitres de programmation, chaque bloc de code est suivi de **sa sortie réelle**, et chaque bloc est court. On peut donc lire le livre sans rien exécuter et comprendre ce qui se passe. Quand vous serez prêt(e), le chapitre 4 (section 4.1) vous apprend Python depuis zéro, et le cahier vous propose de quoi pratiquer.

Le fil rouge : la boutique

Apprendre sur des données abstraites est ennuyeux. Tout au long de ce volume, nous travaillerons donc sur une même histoire :

La gérante dirige **une boutique** qui vend des produits artisanaux (céramiques, textiles, bijoux, cosmétiques) dans son magasin, sur son site web et sur les réseaux sociaux. Elle vend, elle livre, elle reçoit des retours. Elle a des questions ; vous allez l'aider à y répondre avec des données.

Voici quelques-unes de ses questions, et le chapitre où nous y répondrons :

Question de la gérante	Outil	Chapitre
« Quels clients se ressemblent ? »	vecteurs, similarité	1
« Comment ajuster mes prix pour maximiser le chiffre d'affaires ? »	optimisation	1
« Cette commande est-elle une fraude ? »	théorème de Bayes	2
« Combien de commandes dois-je prévoir à 14 h ? »	loi de Poisson	2
« Mon nouveau transporteur est-il vraiment plus rapide ? »	tests d'hypothèses	3
« Combien vaut mon panier moyen, à ± 2 euros près ? »	intervalle de confiance	3
« Comment automatiser mes rapports de ventes ? »	Python, pandas	4
« Quels sont mes dix meilleurs clients ? »	SQL	5
« Comment retrouver la version de mon analyse d'il y a un mois ? »	Git	6

Une précision importante : cette boutique, son personnel, ses clients et toutes ses données sont **fictifs**. Les chiffres sont générés par ordinateur, avec des règles simples que nous connaissons, ce qui permet de vérifier que nos méthodes retrouvent bien la réalité. Toute ressemblance avec une entreprise réelle serait une coïncidence.

Le code de ce livre

Ce livre n'est pas un manuel de programmation, sauf dans les chapitres 4, 5 et 6, qui enseignent précisément Python, SQL, Git et la ligne de commande. La règle est donc simple :

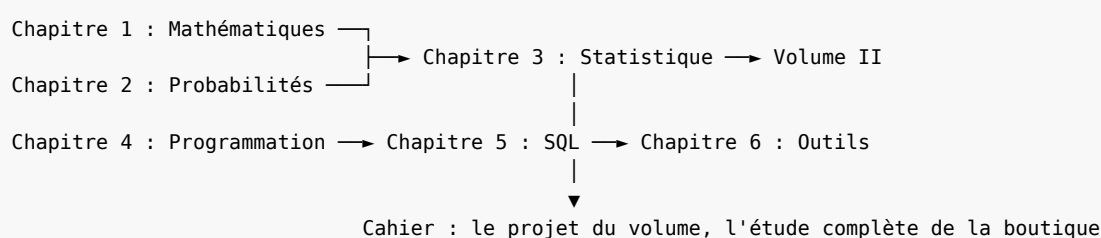
- **le code n'apparaît que lorsqu'il aide à comprendre** : quand on apprend à programmer, quand l'exemple *est* un morceau de code (une requête SQL, une commande Git), ou quand un phénomène numérique se montre mieux qu'il ne se décrit ;
- **les blocs sont courts** (une quinzaine de lignes au plus) et chacun est suivi de sa sortie réelle ;
- **les calculs, les simulations et les graphiques qui illustrent un résultat** sont bien produits par du code, mais ce code n'encombre pas le texte : les chiffres cités sont reproductibles, et les versions complètes se trouvent dans le cahier et dans le dépôt du livre.

Les exemples sont écrits en **Python** (langage principal), avec quelques exemples en **R** et en **SQL**. Les données aléatoires sont générées avec une **graine** (*seed*) fixée : en rejouant le code, vous obtiendrez les mêmes nombres que dans le livre, à de très légères variations près selon les versions des bibliothèques. Les jeux de données utilisés (un tableau de commandes, puis une petite base SQL) sont fournis dans le dossier `donnees/`.

Pour installer ce qu'il faut, un seul jeu de commandes suffit (détaillé au chapitre 6) :

```
python -m venv .venv
source .venv/bin/activate          # sous Windows : .venv\Scripts\activate
pip install numpy pandas scipy matplotlib seaborn
```

La carte du volume



Chapitre	Il répond à la question...	À la fin, vous saurez...
1. Mathématiques	De quels outils mathématiques a-t-on besoin ?	manipuler vecteurs et matrices, dériver, minimiser une fonction
2. Probabilités	Comment parler d'incertitude avec précision ?	calculer des probabilités, reconnaître les lois usuelles, comprendre la loi des grands nombres
3. Statistique	Que peut-on conclure à partir d'un échantillon ?	estimer, construire un intervalle de confiance, tester une hypothèse
4. Programmation	Comment faire calculer l'ordinateur ?	écrire du Python, manipuler des données avec pandas, tracer des graphiques
5. SQL	Comment interroger une base de données ?	écrire des requêtes avec jointures, agrégations et fonctions fenêtres
6. Outils	Comment travailler proprement ?	utiliser Git, des notebooks, la ligne de commande

Faites le point : un petit test de départ

Avant de commencer, le **cahier** propose un petit **test de départ** de dix questions, à faire en dix minutes, au crayon, **sans calculatrice** : un peu de calcul, de probabilités, de dérivation et une question sur la programmation. Les réponses et la manière d'interpréter votre score y sont données.

Pour s'entraîner. Cahier, mode d'emploi : test de départ.

La règle de lecture est simple. Si vous êtes à l'aise, commencez par le chapitre 1. Si le test vous a fait hésiter, lisez d'abord le « Rappel express » qui suit : ce n'est pas une affaire de talent, seulement de pratique.

Maintenant, au travail. La gérante nous attend, et sa boutique reçoit une commande à l'instant.

Rappel express : les bases à avoir en tête

Quelques pages pour remettre en route ce qu'on a vu au collège et au lycée. Rien ici n'est nouveau : tout sert dans la suite du livre. Si tout vous semble évident, passez directement au chapitre 1.

R.1 Fractions et pourcentages

Une **fraction** est une division : $\frac{3}{4}$ veut dire « 3 divisé par 4 », soit 0,75. Pour additionner deux fractions, on les met sur le même dénominateur :

$$\frac{3}{4} + \frac{5}{6} = \frac{3 \times 3}{4 \times 3} + \frac{5 \times 2}{6 \times 2} = \frac{9}{12} + \frac{10}{12} = \frac{19}{12}.$$

Un **pourcentage** est une fraction de dénominateur 100. « Prendre 25 % de 120 € » revient à calculer $120 \times \frac{25}{100} = 30$ €.

Exemple (la boutique). Un article coûte 240 €. La gérante propose une remise de 15 %.

- Montant de la remise : $240 \times 0,15 = 36$ €.
- Nouveau prix : $240 - 36 = 204$ €, ou directement $240 \times 0,85 = 204$ €.

Astuce : « baisser de 15 % » revient à **multiplier par 0,85**. « Augmenter de 19 % » (la TVA) revient à multiplier par 1,19.

Piège. Une hausse de 20 % suivie d'une baisse de 20 % **ne ramène pas** au prix de départ : $100 \rightarrow 120 \rightarrow 96$. Les pourcentages se multiplient, ils ne s'additionnent pas.

R.2 Puissances et racines

a^n signifie « a multiplié n fois par lui-même » : $2^3 = 2 \times 2 \times 2 = 8$. Les règles à connaître :

$$a^m \times a^n = a^{m+n}, \quad \frac{a^m}{a^n} = a^{m-n}, \quad (a^m)^n = a^{mn}, \quad a^0 = 1, \quad a^{-n} = \frac{1}{a^n}.$$

La **racine carrée** $\sqrt{a}$ est le nombre positif dont le carré vaut a : $\sqrt{49} = 7$. On a aussi $\sqrt{a} = a^{1/2}$.

Exemple. $2^{10} = 1024$; $10^{-2} = 0,01$; $\sqrt{2} \approx 1,414$. Et $2^3 \times 2^4 = 2^7 = 128$, qu'on vérifie : $8 \times 16 = 128$.

R.3 Équations et fonctions affines

Résoudre une équation du premier degré, c'est isoler l'inconnue en faisant la **même opération des deux côtés** :

$$2x - 7 = 11 \implies 2x = 18 \implies x = 9.$$

Une **fonction affine** s'écrit $y = ax + b$. Le nombre a est la **pente** (de combien y augmente quand x augmente de 1), et b est l'**ordonnée à l'origine** (la valeur de y quand $x = 0$).

Exemple. Les frais de livraison de la boutique valent 5 € de forfait plus 0,80 € par kilo. Si x est le poids en kilos :

$$\text{frais}(x) = 0,8x + 5.$$

Pour un colis de 6 kg : $0,8 \times 6 + 5 = 9,8$ €. La pente (0,8) est le coût d'un kilo de plus ; l'ordonnée à l'origine (5) est le forfait.

Cette idée, « une pente et une ordonnée à l'origine », reviendra souvent : c'est la base de la régression linéaire.

R.4 Le symbole $\sum$ (somme)

En statistique, on additionne sans arrêt. Pour ne pas écrire $x_1 + x_2 + \dots + x_n$ à chaque fois, on utilise le grand sigma :

$$\sum_{i=1}^n x_i = x_1 + x_2 + \dots + x_n.$$

Cela se lit : « somme, pour i allant de 1 à n , des x_i ». C'est exactement une boucle for qui additionne.

Exemple. Les ventes de la boutique sur cinq jours (en euros) : $x_1 = 120$, $x_2 = 80$, $x_3 = 200$, $x_4 = 100$, $x_5 = 150$.

$$\sum_{i=1}^5 x_i = 120 + 80 + 200 + 100 + 150 = 650 \text{ €}.$$

La **moyenne** est $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \frac{650}{5} = 130$ €.

Deux propriétés utiles (elles se vérifient en écrivant les sommes) :

$$\sum (x_i + y_i) = \sum x_i + \sum y_i, \quad \sum c x_i = c \sum x_i \quad (c \text{ constante}).$$

R.5 Exponentielle et logarithme

Le **logarithme** répond à la question : « à quelle puissance faut-il élever la base pour obtenir ce nombre ? ». Ainsi $\log_{10}(1000) = 3$, car $10^3 = 1000$.

En data science, on utilise surtout le **logarithme népérien** $\ln$ (base $e \approx 2,718$), et son opposé, la **fonction exponentielle** e^x . Ce qu'il faut savoir :

$$\ln(ab) = \ln a + \ln b, \quad \ln(a^n) = n \ln a, \quad \ln(e^x) = x, \quad e^{a+b} = e^a e^b.$$

Intuition. Le logarithme transforme les **produits en sommes**. C'est précieux : une somme est bien plus facile à manipuler (et à dériver) qu'un produit. C'est la raison pour laquelle, plus tard, nous prendrons le log de produits de probabilités pour estimer des paramètres.

Exemple. Un placement double tous les 10 ans. Après 30 ans, il a été multiplié par $2^3 = 8$. Combien de « doubléments » faut-il pour multiplier par 1000 ? $\log_2(1000) \approx 9,97$, soit environ 10 doubléments.

R.6 Fonctions, graphes et équations de droites

Une **fonction** associe à chaque entrée x une sortie $f(x)$. Son **graphe** est la courbe des points $(x, f(x))$. Quelques fonctions que nous croiserons :

Fonction	Forme	Allure
affine	$ax + b$	droite
carré	x^2	parabole (« bol », avec un minimum en 0)
exponentielle	e^x	croît de plus en plus vite
logarithme	$\ln x$	croît de plus en plus lentement
racine	$\sqrt{x}$	croît lentement

R.7 Un peu de vocabulaire de programmation

Avant de lire du code, reprenez trois mots :

- une **variable** : un nom pour une valeur (`prix = 45.0`) ;
- une **fonction** : une recette réutilisable (`sum([1, 2, 3])` renvoie 6) ;
- une **liste** : une suite ordonnée de valeurs (`[120, 80, 200]`).

Le reste sera expliqué quand il apparaîtra, et le chapitre 4 reprend tout depuis le début.

Faites le point

Cinq petites questions de vérification, avec leurs réponses, vous attendent dans le cahier : si vous savez y répondre, vous avez tout ce qu'il faut pour commencer.

Pour s'entraîner. Cahier, mode d'emploi : exercices du rappel express (0.1 à 0.5).

Chapitre 1 : Mathématiques pour la data science

« Un tableau de données, c'est une **matrice**. Un modèle, c'est une **fonction**. Apprendre, c'est **minimiser une erreur**. »

Ces trois phrases résument ce chapitre. Si elles vous paraissent mystérieuses, c'est normal : à la fin du chapitre, elles vous sembleront évidentes.

Pourquoi des mathématiques ?

On entend parfois que « la data science, ce sont des bibliothèques Python : on appelle `fit()` et c'est fini ». C'est vrai... jusqu'au jour où le modèle donne un résultat absurde et où l'on doit comprendre pourquoi. Les mathématiques sont ce qui permet alors de **raisonner au lieu de deviner**.

Rassurez-vous : il ne s'agit pas de refaire un cursus de mathématiques. Il suffit de **quatre familles d'idées**, que nous construirons de zéro :

Idée	Ce que c'est	À quoi ça sert en data science
Vecteurs et matrices	des listes et des tableaux de nombres, avec des règles de calcul	représenter les données, comparer des individus, réduire la dimension
Dérivées et gradients	la mesure de « à quelle vitesse ça change »	savoir dans quel sens améliorer un modèle
Intégrales	la mesure d'une « quantité accumulée »	calculer des probabilités (chapitre 2)
Optimisation	trouver le meilleur choix selon un critère	entraîner (presque) tous les modèles

Le chemin de ce chapitre

Nous suivrons une petite boutique et ses questions concrètes :

- **1.1 Algèbre linéaire** : la gérante veut savoir quels clients se ressemblent, calculer son chiffre d'affaires par mois sans boucle interminable, et résumer un tableau de ventes en quelques tendances.
- **1.2 Analyse** : comment son bénéfice change-t-il quand elle modifie un prix un tout petit peu ? Et quelle est la probabilité qu'une commande arrive dans les trois prochaines minutes ?

- **1.3 Optimisation** : quel prix maximise ses recettes ? Comment répartir son budget publicitaire ?
- **1.4 Fiche de notations** : toutes les notations du livre, au même endroit.
- **⊕ Pour aller plus loin** : pourquoi l'ordinateur se trompe parfois (analyse numérique), et comment compter et relier des objets (mathématiques discrètes et graphes).

Comment lire ce chapitre. Chaque notion suit le même rythme : une intuition, un exemple chiffré à la main, puis, quand c'est utile, une démonstration. Si une démonstration vous décourage, sautez-la : l'exemple suffit pour avancer. Revenez-y plus tard.

Le cahier. Les applications guidées et les exercices corrigés de ce chapitre sont dans le **Cahier d'exercices et d'applications** du volume (chapitre 1). Ce livre se concentre sur les idées, les démonstrations et les exemples faits à la main : le code n'y apparaît que lorsqu'il aide vraiment à comprendre.

1.1 Algèbre linéaire

L'algèbre linéaire est le langage des tableaux de nombres. Tout ce que fait un modèle de data science, de la régression la plus simple au réseau de neurones le plus profond, s'écrit avec elle. Nous allons en construire les quatre briques : **vecteurs**, **matrices**, **valeurs propres** et **décomposition en valeurs singulières**.

1.1.1 Vecteurs : décrire un client par une liste de nombres

Intuition. Un **vecteur** est une liste ordonnée de nombres. On peut le voir de deux façons, qui sont les deux faces d'une même pièce :

- comme une **flèche** partant de l'origine et pointant vers un point ;
- comme la **fiche d'identité** d'un individu : une ligne de tableau où chaque case décrit une caractéristique.

En data science, la seconde vision est la plus utile : *un individu = un vecteur*.

Un exemple concret

La boutique compte, pour chaque client, le nombre d'achats dans trois catégories : **poteries**, **textiles**, **bijoux**. Chaque client devient un vecteur à trois composantes :

Client	Poteries	Textiles	Bijoux	Vecteur
Alix	3	1	0	$\mathbf{a} = (3, 1, 0)$
Basile	6	2	0	$\mathbf{b} = (6, 2, 0)$
Camille	0	1	4	$\mathbf{c} = (0, 1, 4)$

On note $\mathbf{a} = (a_1, a_2, \dots, a_p)$ un vecteur à p composantes, et $\mathbb{R}^p$ l'ensemble de tous les vecteurs à p composantes réelles. Ici $p = 3$: nos clients vivent dans $\mathbb{R}^3$.

Convention du livre. Un nombre seul (un *scalaire*) s'écrit en italique : a, λ . Un vecteur s'écrit en gras minuscule : $\mathbf{a}$. Une matrice s'écrit en gras majuscule : $\mathbf{A}$.

Les opérations de base

Addition. On additionne composante par composante. Si Alix et Camille regroupent leurs achats :

$$\mathbf{a} + \mathbf{c} = (3 + 0, 1 + 1, 0 + 4) = (3, 2, 4).$$

Multiplication par un scalaire. On multiplie chaque composante :

$$2\mathbf{a} = (2 \times 3, 2 \times 1, 2 \times 0) = (6, 2, 0) = \mathbf{b}.$$

Regardez bien : **Basile est exactement « deux fois Alix »**. Il achète les mêmes choses dans les mêmes proportions, simplement en plus grande quantité. Nous allons voir comment la mathématique capture cette idée.

Combinaison linéaire. Combiner les deux opérations donne une *combinaison linéaire* : $\lambda_1 \mathbf{u}_1 + \lambda_2 \mathbf{u}_2$. Par exemple, « la moitié d'Alix plus un tiers de Camille » est $\frac{1}{2}\mathbf{a} + \frac{1}{3}\mathbf{c}$. Presque tout ce que nous ferons en modélisation est une combinaison linéaire de quelque chose.

Pour manipuler ces vecteurs sur un ordinateur, la bibliothèque Python **NumPy** les traite comme des objets à part entière (un tout petit aperçu, le chapitre 4 y reviendra en détail) :

```
import numpy as np

alix = np.array([3, 1, 0])
basile = np.array([6, 2, 0])
camille = np.array([0, 1, 4])

print("Alix + Camille :", alix + camille)
print("2 x Alix      :", 2 * alix)
print("Basile == 2 x Alix ?", np.array_equal(basile, 2 * alix))
```

```
Alix + Camille : [3 2 4]
2 x Alix      : [6 2 0]
Basile == 2 x Alix ? True
```

Le produit scalaire : mesurer l'accord entre deux vecteurs

Le **produit scalaire** de deux vecteurs de même taille est la somme des produits des composantes :

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^p a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_p b_p.$$

Exemple à la main.

$$\mathbf{a} \cdot \mathbf{b} = 3 \times 6 + 1 \times 2 + 0 \times 0 = 18 + 2 + 0 = 20.$$

$$\mathbf{a} \cdot \mathbf{c} = 3 \times 0 + 1 \times 1 + 0 \times 4 = 0 + 1 + 0 = 1.$$

Intuition. Le produit scalaire est grand quand les deux vecteurs « vont dans le même sens » (ils ont des composantes fortes aux mêmes endroits), proche de 0 quand ils n'ont rien en commun, et négatif quand ils s'opposent. Alix et Basile s'entendent bien (20) ; Alix et Camille presque pas (1) : le seul point commun est un textile.

La **norme** d'un vecteur est sa longueur, donnée par le théorème de Pythagore généralisé :

$$\|\mathbf{a}\| = \sqrt{\mathbf{a} \cdot \mathbf{a}} = \sqrt{a_1^2 + a_2^2 + \dots + a_p^2}.$$

Pour nos clients : $\|\mathbf{a}\| = \sqrt{9 + 1 + 0} = \sqrt{10} \approx 3,162$, $\|\mathbf{b}\| = \sqrt{36 + 4} = \sqrt{40} \approx 6,325$ et $\|\mathbf{c}\| = \sqrt{0 + 1 + 16} = \sqrt{17} \approx 4,123$. On remarque que $\|\mathbf{b}\| = 2\|\mathbf{a}\|$: doubler un vecteur double sa longueur.

Le produit scalaire a aussi une **interprétation géométrique** qui est fondamentale :

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta,$$

où θ est l'angle entre les deux flèches. En isolant le cosinus, on obtient la **similarité cosinus** :

$$\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}.$$

Exemple à la main.

- Alix et Basile : $\cos \theta = \frac{20}{\sqrt{10} \sqrt{40}} = \frac{20}{\sqrt{400}} = \frac{20}{20} = 1$. Même direction : **mêmes goûts**.
- Alix et Camille : $\cos \theta = \frac{1}{\sqrt{10} \sqrt{17}} = \frac{1}{\sqrt{170}} \approx 0,077$. Presque perpendiculaires : **goûts très différents**.

La similarité cosinus ignore la *taille* des vecteurs et ne regarde que leur *direction*. C'est exactement ce que l'on veut pour comparer des profils d'achat : un gros acheteur et un petit acheteur qui ont les mêmes goûts doivent apparaître comme semblables.

Démonstration : pourquoi le cosinus reste toujours entre -1 et 1.

Il faut montrer que $|\mathbf{a} \cdot \mathbf{b}| \leq \|\mathbf{a}\| \|\mathbf{b}\|$. C'est l'**inégalité de Cauchy-Schwarz**. Supposons $\mathbf{b} \neq \mathbf{0}$ et considérons, pour un réel t quelconque, la quantité

$$q(t) = \|\mathbf{a} - t\mathbf{b}\|^2 = \|\mathbf{a}\|^2 - 2t(\mathbf{a} \cdot \mathbf{b}) + t^2\|\mathbf{b}\|^2.$$

Une norme au carré est toujours positive ou nulle, donc $q(t) \geq 0$ **pour tout** t . Or q est un trinôme du second degré en t ; pour qu'il ne prenne jamais de valeur négative, son discriminant doit être négatif ou nul :

$$\Delta = 4(\mathbf{a} \cdot \mathbf{b})^2 - 4\|\mathbf{a}\|^2\|\mathbf{b}\|^2 \leq 0 \iff (\mathbf{a} \cdot \mathbf{b})^2 \leq \|\mathbf{a}\|^2\|\mathbf{b}\|^2.$$

En prenant la racine carrée, $|\mathbf{a} \cdot \mathbf{b}| \leq \|\mathbf{a}\| \|\mathbf{b}\|$, donc $-1 \leq \cos \theta \leq 1$. ■

L'égalité (cosinus égal à ± 1) a lieu exactement quand le discriminant est nul, c'est-à-dire quand il existe un t tel que $\mathbf{a} = t\mathbf{b}$: les deux vecteurs sont alignés. C'est le cas d'Alix et Basile.

La distance entre deux vecteurs

La **distance euclidienne** entre deux individus est la norme de leur différence :

$$d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\| = \sqrt{\sum_{i=1}^p (a_i - b_i)^2}.$$

Exemple.

- $d(\mathbf{a}, \mathbf{b}) = \|(3 - 6, 1 - 2, 0 - 0)\| = \|(-3, -1, 0)\| = \sqrt{10} \approx 3,162.$
- $d(\mathbf{a}, \mathbf{c}) = \|(3 - 0, 1 - 1, 0 - 4)\| = \|(3, 0, -4)\| = \sqrt{9 + 0 + 16} = 5.$

Remarquez la nuance. Selon le **cosinus**, Alix et Basile sont *identiques* (1) ; selon la **distance**, ils sont à 3,16 unités l'un de l'autre, parce que Basile achète plus. Ni l'une ni l'autre mesure n'est « la bonne » : elles répondent à des questions différentes.

Mesure	Elle répond à...	À utiliser quand...
Cosinus	« Ont-ils les mêmes goûts ? »	seule la <i>proportion</i> compte (profils, textes, recommandations)
Distance	« Sont-ils proches en valeur absolue ? »	la <i>quantité</i> compte (regroupement par niveau de dépense, par exemple)

Piège : les unités. La distance additionne des carrés de différences, donc une variable exprimée en grands nombres écrase les autres. Soit trois clients décrits par (âge, panier moyen en €) : $P = (30, 100)$, $Q = (60, 102)$ et $R = (31, 160)$.

- $d(P, Q) = \sqrt{30^2 + 2^2} = \sqrt{904} \approx 30,1$
- $d(P, R) = \sqrt{1^2 + 60^2} = \sqrt{3601} \approx 60,0$

Selon la distance, P (30 ans) ressemble davantage à Q (60 ans) qu'à R (31 ans), simplement parce que 60 euros « pèsent » plus que 30 ans dans le calcul ! La cause : les deux variables n'ont pas la même échelle. Le remède est de **standardiser** les variables avant de les comparer, ce que nous ferons au chapitre 3.

À retenir (vecteurs).

- Un individu est un vecteur ; ses composantes sont ses caractéristiques.
- Le produit scalaire $\sum a_i b_i$ mesure l'accord entre deux vecteurs ; la norme mesure leur longueur.
- Le cosinus compare les **directions**, la distance compare les **positions**. Toujours se demander laquelle des deux correspond à la question posée.
- Avant de comparer des variables d'échelles différentes, on les standardise.

Pour s'entraîner. Cahier, chapitre 1 : exercice 1.1.

1.1.2 Matrices : le tableau de données devient un objet mathématique

Intuition. Une **matrice** est un tableau rectangulaire de nombres, rangés en lignes et en colonnes. Si un vecteur est « un individu », une matrice est « **tous les individus à la fois** » : chaque ligne est un individu, chaque colonne est une variable. Un fichier de données, c'est une matrice.

On note $\mathbf{A} \in \mathbb{R}^{n \times p}$ une matrice à n lignes et p colonnes (on dit « n par p »), et a_{ij} le nombre situé à la ligne i et à la colonne j . Une matrice carrée a autant de lignes que de colonnes ($n = p$).

Un exemple concret

La gérante a noté ses ventes des trois premiers mois pour trois produits (poteries, textiles, bijoux). Chaque **ligne** est un mois, chaque **colonne** un produit :

$$\mathbf{V} = \begin{pmatrix} 10 & 40 & 20 \\ 12 & 35 & 25 \\ 15 & 50 & 18 \end{pmatrix} \begin{array}{l} \leftarrow \text{janvier} \\ \leftarrow \text{février} \\ \leftarrow \text{mars} \end{array}$$

Les prix unitaires sont $\mathbf{p} = (45, 12, 30)$ euros. Question : quel est le chiffre d'affaires de chaque mois ?

Le produit matrice-vecteur

Pour janvier, c'est un produit scalaire : $10 \times 45 + 40 \times 12 + 20 \times 30 = 450 + 480 + 600 = 1\,530$ €. On fait de même pour février et mars. Calculer ces trois produits scalaires d'un coup, c'est ce qu'on appelle le **produit matrice-vecteur** $\mathbf{Vp}$:

$$\mathbf{Vp} = \begin{pmatrix} 10 \cdot 45 + 40 \cdot 12 + 20 \cdot 30 \\ 12 \cdot 45 + 35 \cdot 12 + 25 \cdot 30 \\ 15 \cdot 45 + 50 \cdot 12 + 18 \cdot 30 \end{pmatrix} = \begin{pmatrix} 1530 \\ 1710 \\ 1815 \end{pmatrix}.$$

Formellement, $(\mathbf{Vp})_i = \sum_j v_{ij} p_j$: la composante i du résultat est le produit scalaire de la **ligne** i de $\mathbf{V}$ avec $\mathbf{p}$.

Une seconde lecture, tout aussi utile. On peut aussi voir $\mathbf{Vp}$ comme une **combinaison linéaire des colonnes** de $\mathbf{V}$:

$$\mathbf{Vp} = 45 \begin{pmatrix} 10 \\ 12 \\ 15 \end{pmatrix} + 12 \begin{pmatrix} 40 \\ 35 \\ 50 \end{pmatrix} + 30 \begin{pmatrix} 20 \\ 25 \\ 18 \end{pmatrix}.$$

Le chiffre d'affaires est « 45 fois les ventes de poteries, plus 12 fois les ventes de textiles, plus 30 fois les ventes de bijoux ». C'est exactement ce que fait une régression linéaire : un mélange pondéré de colonnes.

```
import numpy as np

V = np.array([[10, 40, 20],
              [12, 35, 25],
              [15, 50, 18]]) # janvier : poteries, textiles, bijoux
                                # février
                                # mars
prix = np.array([45, 12, 30])

print("Forme de V :", V.shape)
print("CA par mois :", V @ prix)
```

```
Forme de V : (3, 3)
CA par mois : [1530 1710 1815]
```

L'opérateur `@` est le produit matriciel de Python. Il évite d'écrire une boucle.

Le produit matrice-matrice

Et si la gérante veut comparer deux grilles de prix : les prix actuels et les prix augmentés de 10 % ? On met les deux grilles côte à côte dans une matrice $\mathbf{P}$ (une colonne par scénario) et on calcule $\mathbf{VP}$.

Règle de calcul. Le produit de $\mathbf{A} \in \mathbb{R}^{n \times p}$ par $\mathbf{B} \in \mathbb{R}^{p \times q}$ est la matrice $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{n \times q}$ dont l'élément (i, j) est le produit scalaire de la **ligne** i de $\mathbf{A}$ avec la **colonne** j de $\mathbf{B}$:

$$c_{ij} = \sum_{k=1}^p a_{ik} b_{kj}.$$

Règle d'or des dimensions. Pour multiplier, le **nombre de colonnes de A doit égaier le nombre de lignes de B** : $(n \times p)(p \times q) \rightarrow (n \times q)$. Les deux p « du milieu » doivent être identiques et disparaissent ; il reste $n \times q$. Quand un code plante avec une erreur de « forme » (*shape*), c'est presque toujours cette règle qui est violée.

Chaque ligne est un mois, chaque colonne un scénario. La deuxième colonne vaut 1,1 fois la première, comme attendu.

Piège : le produit matriciel n'est pas commutatif. En général, $\mathbf{AB} \neq \mathbf{BA}$. Un petit exemple suffit à s'en convaincre :

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Calcul de $\mathbf{AB}$: première ligne $(1 \cdot 0 + 2 \cdot 1, 1 \cdot 1 + 2 \cdot 0) = (2, 1)$; seconde ligne $(3 \cdot 0 + 4 \cdot 1, 3 \cdot 1 + 4 \cdot 0) = (4, 3)$. Donc $\mathbf{AB} = \begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix}$: multiplier à droite par $\mathbf{B}$ **échange les colonnes** de $\mathbf{A}$.

Calcul de $\mathbf{BA}$: on trouve $\begin{pmatrix} 3 & 4 \\ 1 & 2 \end{pmatrix}$: multiplier à gauche par $\mathbf{B}$ **échange les lignes** de $\mathbf{A}$. Les deux résultats sont différents.

Ce que l'on garde, en revanche : le produit est **associatif** ($(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$) et **distributif** ($\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$).

La transposée

La **transposée** $\mathbf{A}^\top$ d'une matrice échange ses lignes et ses colonnes : $(\mathbf{A}^\top)_{ij} = a_{ji}$. Une matrice 3×2 devient une matrice 2×3 . On a la règle $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$ (l'ordre s'inverse).

Pour un vecteur colonne $\mathbf{x}$, le nombre $\mathbf{x}^\top \mathbf{x} = \sum x_i^2 = \|\mathbf{x}\|^2$ est son carré de norme : le produit scalaire s'écrit donc $\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^\top \mathbf{b}$. Vous croiserez cette écriture partout.

La matrice identité et l'inverse

La **matrice identité** $\mathbf{I}$ (des 1 sur la diagonale, des 0 ailleurs) est le « 1 » des matrices : $\mathbf{IA} = \mathbf{AI} = \mathbf{A}$.

L'**inverse** d'une matrice carrée $\mathbf{A}$, quand il existe, est la matrice $\mathbf{A}^{-1}$ telle que

$$\mathbf{AA}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}.$$

C'est l'analogue de $a \times \frac{1}{a} = 1$. Pour une matrice 2×2 , il existe une formule simple. Si $\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, son **déterminant** est $\det \mathbf{A} = ad - bc$, et lorsque $\det \mathbf{A} \neq 0$:

$$\mathbf{A}^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

Exemple. Soit $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 5 & 3 \end{pmatrix}$. Son déterminant vaut $2 \times 3 - 1 \times 5 = 1$, donc

$$\mathbf{A}^{-1} = \begin{pmatrix} 3 & -1 \\ -5 & 2 \end{pmatrix}.$$

Vérification : $\mathbf{A}\mathbf{A}^{-1} = \begin{pmatrix} 2 \cdot 3 + 1 \cdot (-5) & 2 \cdot (-1) + 1 \cdot 2 \\ 5 \cdot 3 + 3 \cdot (-5) & 5 \cdot (-1) + 3 \cdot 2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$. ✓

Résoudre un système d'équations linéaires

Voici l'utilité majeure de l'inverse. La gérante a perdu sa liste de prix, mais retrouve deux factures :

- commande 1 : 2 poteries + 1 écharpe = 102 € ;
- commande 2 : 1 poterie + 3 écharpes = 81 €.

Notons x_1 le prix d'une poterie et x_2 celui d'une écharpe. Le système s'écrit $\mathbf{M}\mathbf{x} = \mathbf{b}$ avec

$$\mathbf{M} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 102 \\ 81 \end{pmatrix}.$$

Résolution à la main. $\det \mathbf{M} = 2 \times 3 - 1 \times 1 = 5$ et $\mathbf{M}^{-1} = \frac{1}{5} \begin{pmatrix} 3 & -1 \\ -1 & 2 \end{pmatrix}$. Donc

$$\mathbf{x} = \mathbf{M}^{-1}\mathbf{b} = \frac{1}{5} \begin{pmatrix} 3 \cdot 102 - 81 \\ -102 + 2 \cdot 81 \end{pmatrix} = \frac{1}{5} \begin{pmatrix} 225 \\ 60 \end{pmatrix} = \begin{pmatrix} 45 \\ 12 \end{pmatrix}.$$

Une poterie coûte **45 €**, une écharpe **12 €**. Vérifions : $2 \times 45 + 12 = 102$ ✓ et $45 + 3 \times 12 = 81$ ✓.

```
M = np.array([[2, 1], [1, 3]])
b = np.array([102, 81])
print("Prix retrouvés :", np.linalg.solve(M, b))
```

Prix retrouvés : [45. 12.]

Piège : en pratique, on évite d'inverser. Pour résoudre $\mathbf{M}\mathbf{x} = \mathbf{b}$, on utilise `np.linalg.solve`, jamais `np.linalg.inv(M) @ b` : c'est plus rapide et plus précis (voir la section ④ sur l'analyse numérique). L'inverse est un outil de **raisonnement** ; la machine, elle, résout directement.

Quand le système n'a pas de solution unique : déterminant et rang

Imaginons que la commande 2 soit en réalité « 4 poteries + 2 écharpes = 204 € » : c'est exactement le **double** de la commande 1. Elle n'apporte **aucune information nouvelle**. Il y a alors une infinité de couples de prix qui conviennent. Le déterminant le détecte : $\det \begin{pmatrix} 2 & 1 \\ 4 & 2 \end{pmatrix} = 2 \times 2 - 1 \times 4 = 0$. On dit que la matrice est **singulière** (non inversible).

Une notion plus générale est le **rang** : le nombre de colonnes (ou de lignes) *linéairement indépendantes*, c'est-à-dire qui ne s'obtiennent pas comme combinaison des autres. Ici le rang vaut 1 au lieu de 2.

Des colonnes redondantes dans de vraies données

Cette situation est fréquente. Si un fichier contient le prix hors taxes (HT) *et* le prix toutes taxes comprises (TTC = 1,19 × HT), la seconde colonne est un multiple de la première : elle ne dit rien de plus, et le rang du tableau vaut 1 au lieu de 2. Un modèle de régression qui utiliserait ces deux colonnes serait incapable de départager leurs rôles : c'est le problème de la **multicolinéarité**, que nous retrouverons au volume II.

Toutes les similarités d'un coup

Revenons à la similarité cosinus. L'astuce : si l'on **normalise** chaque ligne d'une matrice de profils **X** pour que sa norme vaille 1, on obtient une matrice **U** dont chaque ligne est un vecteur de longueur 1. Alors le produit scalaire de deux lignes de **U** est directement leur cosinus, et **tous** les produits scalaires entre lignes sont rassemblés dans un seul produit matriciel :

$$(\mathbf{UU}^T)_{ij} = \mathbf{u}_i \cdot \mathbf{u}_j = \cos \theta_{ij}.$$

Pour nos trois clients du début, cela donne

$$\mathbf{UU}^T \approx \begin{pmatrix} 1 & 1 & 0,08 \\ 1 & 1 & 0,08 \\ 0,08 & 0,08 & 1 \end{pmatrix}.$$

On y lit que la matrice est **symétrique** (la ressemblance d'Alix à Basile est celle de Basile à Alix), que sa **diagonale vaut 1** (chacun ressemble parfaitement à lui-même), et qu'Alix et Basile sont identiques tandis que Camille est très différente d'eux. Avec cinq mille clients, ce serait la même opération : une seule multiplication de matrices.

À retenir (matrices).

- Une matrice est un tableau $n \times p$: lignes = individus, colonnes = variables.
- $(\mathbf{AB})_{ij}$ = ligne i de **A** · colonne j de **B** ; les dimensions doivent s'emboîter : $(n \times p)(p \times q)$.
- Le produit matriciel n'est **pas commutatif**.
- Résoudre $\mathbf{Mx} = \mathbf{b}$: on utilise `np.linalg.solve`. Si $\det \mathbf{M} = 0$ (rang insuffisant), il n'y a pas de solution unique.
- Des colonnes redondantes font chuter le rang : c'est la source de la multicolinéarité.

Pour s'entraîner. Cahier, chapitre 1 : applications 1.1 et 1.2, exercice 1.2.

1.1.3 Valeurs propres et vecteurs propres : les directions privilégiées

Intuition. Une matrice carrée agit sur le plan comme une machine à déformer : elle étire, elle fait pivoter, elle cisaille. Si vous lui donnez un vecteur, elle en renvoie un autre, en général dans une **autre direction**.

Mais il existe parfois des vecteurs « têtus » : la machine les **étire (ou les écrase) sans les faire tourner**. Ce sont les **vecteurs propres**. Le facteur d'étirement est la **valeur propre**.

Formellement, un vecteur $\mathbf{v} \neq \mathbf{0}$ est un **vecteur propre** de la matrice carrée $\mathbf{A}$ s'il existe un nombre λ (la **valeur propre**) tel que

$$\mathbf{A}\mathbf{v} = \lambda \mathbf{v}.$$

Un exemple que l'on vérifie à la main

Prenons $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ et essayons trois vecteurs.

Test 1. $\mathbf{v} = (1, 1)$: $\mathbf{A}\mathbf{v} = (2 \cdot 1 + 1 \cdot 1, 1 \cdot 1 + 2 \cdot 1) = (3, 3) = 3\mathbf{v}$. ✓ La direction est conservée, le vecteur est triplé : **vecteur propre, valeur propre 3**.

Test 2. $\mathbf{w} = (1, -1)$: $\mathbf{A}\mathbf{w} = (2 - 1, 1 - 2) = (1, -1) = 1 \cdot \mathbf{w}$. ✓ **Vecteur propre, valeur propre 1**.

Test 3. $\mathbf{u} = (1, 0)$: $\mathbf{A}\mathbf{u} = (2, 1)$. Ce n'est pas un multiple de $(1, 0)$: la direction a changé. ✗ **Pas un vecteur propre**.

La figure ci-dessous montre ce que fait $\mathbf{A}$ sur tous les vecteurs de longueur 1 (le cercle unité) : elle le transforme en une **ellipse**, allongée d'un facteur 3 dans la direction $(1, 1)$ et conservée (facteur 1) dans la direction $(1, -1)$. Les axes de l'ellipse sont exactement les vecteurs propres.

Comment trouver les valeurs propres ?

On cherche les λ pour lesquels l'équation $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ admet une solution non nulle. On la réécrit $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$. Un système homogène a une solution non nulle exactement lorsque sa matrice est **singulière**, c'est-à-dire lorsque son déterminant est nul :

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0.$$

C'est l'**équation caractéristique**. Pour notre matrice :

$$\det \begin{pmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{pmatrix} = (2 - \lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = (\lambda - 1)(\lambda - 3).$$

Les valeurs propres sont donc $\lambda_1 = 3$ et $\lambda_2 = 1$. On trouve ensuite les vecteurs propres en résolvant $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$:

- pour $\lambda = 3$: $\begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \mathbf{v} = \mathbf{0}$ donne $v_1 = v_2$, soit $\mathbf{v} = (1, 1)$ (à un multiple près) ;
- pour $\lambda = 1$: $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \mathbf{v} = \mathbf{0}$ donne $v_2 = -v_1$, soit $\mathbf{w} = (1, -1)$.

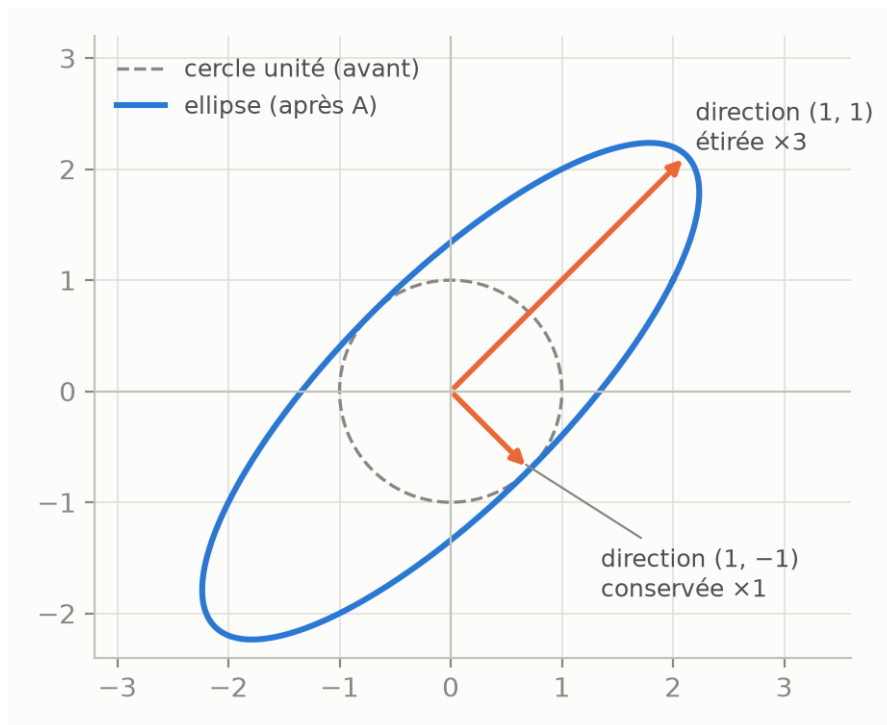


Figure 1: Action de la matrice A sur le cercle unité : l'ellipse est étirée d'un facteur 3 le long de $(1, 1)$ et inchangée le long de $(1, -1)$.

Deux contrôles rapides. La **somme** des valeurs propres vaut la **trace** de la matrice (somme de la diagonale) : $3 + 1 = 4 = 2 + 2$. Le **produit** des valeurs propres vaut le **déterminant** : $3 \times 1 = 3 = 2 \cdot 2 - 1 \cdot 1$. Utile pour détecter une erreur de calcul.

```
import numpy as np

A = np.array([[2, 1], [1, 2]])
valeurs, vecteurs = np.linalg.eigh(A)    # eigh : pour les matrices symétriques
print("valeurs propres :", valeurs)
print("vecteurs propres (en colonnes) :\n", vecteurs.round(4))

valeurs propres : [1. 3.]
vecteurs propres (en colonnes) :
[[-0.7071  0.7071]
 [ 0.7071  0.7071]]
```

NumPy range les valeurs propres par ordre croissant et renvoie des vecteurs **de longueur 1** (le signe est arbitraire : si $\mathbf{v}$ est propre, $-\mathbf{v}$ l'est aussi).

Le cas des matrices symétriques

Une matrice est **symétrique** si $\mathbf{A}^\top = \mathbf{A}$ (la matrice de similarité de la section précédente en est une, et la matrice de covariance, que nous rencontrerons dans un instant, aussi). Ces matrices ont une propriété exceptionnelle :

Théorème spectral. Une matrice symétrique réelle de taille $p \times p$ a p valeurs propres **réelles** et on peut choisir pour elle p vecteurs propres **orthogonaux** deux à deux (et de longueur 1). Si $\mathbf{V}$ est la matrice qui les contient en colonnes et Λ la matrice diagonale des valeurs propres, alors

$$\mathbf{A} = \mathbf{V}\Lambda\mathbf{V}^\top.$$

Nous admettons l'existence (la preuve complète se trouve dans tout cours d'algèbre linéaire), mais voici pourquoi les vecteurs propres associés à des valeurs propres différentes sont orthogonaux :

Démonstration : orthogonalité. Soient $\mathbf{A}\mathbf{v}_1 = \lambda_1\mathbf{v}_1$ et $\mathbf{A}\mathbf{v}_2 = \lambda_2\mathbf{v}_2$ avec $\lambda_1 \neq \lambda_2$ et $\mathbf{A}$ symétrique. Calculons de deux façons le nombre $(\mathbf{A}\mathbf{v}_1) \cdot \mathbf{v}_2$:

- directement : $(\mathbf{A}\mathbf{v}_1) \cdot \mathbf{v}_2 = \lambda_1 (\mathbf{v}_1 \cdot \mathbf{v}_2)$;
- en déplaçant $\mathbf{A}$: comme $\mathbf{A}$ est symétrique, $(\mathbf{A}\mathbf{v}_1) \cdot \mathbf{v}_2 = \mathbf{v}_1^\top \mathbf{A}^\top \mathbf{v}_2 = \mathbf{v}_1^\top \mathbf{A} \mathbf{v}_2 = \mathbf{v}_1 \cdot (\mathbf{A}\mathbf{v}_2) = \lambda_2 (\mathbf{v}_1 \cdot \mathbf{v}_2)$.

En soustrayant : $(\lambda_1 - \lambda_2) (\mathbf{v}_1 \cdot \mathbf{v}_2) = 0$. Comme $\lambda_1 \neq \lambda_2$, on a $\mathbf{v}_1 \cdot \mathbf{v}_2 = 0$: les vecteurs sont orthogonaux. ■

Dans notre exemple : $(1, 1) \cdot (1, -1) = 1 - 1 = 0$. ✓

Une conséquence pratique de $\mathbf{A} = \mathbf{V}\Lambda\mathbf{V}^\top$ est que les **puissances** deviennent triviales : $\mathbf{A}^k = \mathbf{V}\Lambda^k\mathbf{V}^\top$, et élever une matrice diagonale à la puissance k , c'est simplement élever ses éléments. (Nous nous en servons pour les chaînes de Markov dans la section $\oplus$ du chapitre 2.)

Un nuage de clients : la direction principale

La gérante mesure, pour 200 clients, le nombre de visites mensuelles sur son site et leur dépense mensuelle. Elle soupçonne que les deux sont liées. On **standardise** chaque variable (on retranche la moyenne et on divise par l'écart-type, pour qu'elles aient la même échelle, comme promis dans la section sur les vecteurs), puis on calcule la **matrice de covariance** des deux variables standardisées :

$$\mathbf{C} = \begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix},$$

où r est la corrélation entre les deux variables. C'est une matrice symétrique : le théorème spectral s'applique, et le calcul se fait **à la main** pour n'importe quel $r > 0$.

À la main. L'équation caractéristique est $\det(\mathbf{C} - \lambda\mathbf{I}) = (1 - \lambda)^2 - r^2 = 0$, donc $\lambda = 1 \pm r$.

- Pour $\lambda_1 = 1 + r$: $\begin{pmatrix} -r & r \\ r & -r \end{pmatrix} \mathbf{v} = \mathbf{0}$ donne $v_1 = v_2$, soit la direction $\frac{1}{\sqrt{2}}(1, 1)$: la **diagonale**.
- Pour $\lambda_2 = 1 - r$: on trouve $\frac{1}{\sqrt{2}}(1, -1)$: l'autre diagonale, perpendiculaire à la première.

Quelle que soit la corrélation, les axes principaux de deux variables standardisées sont donc les deux diagonales, et la part de variance portée par le premier axe vaut $\frac{1+r}{2}$. Pour $r = 0,9$, cela fait 95 %.

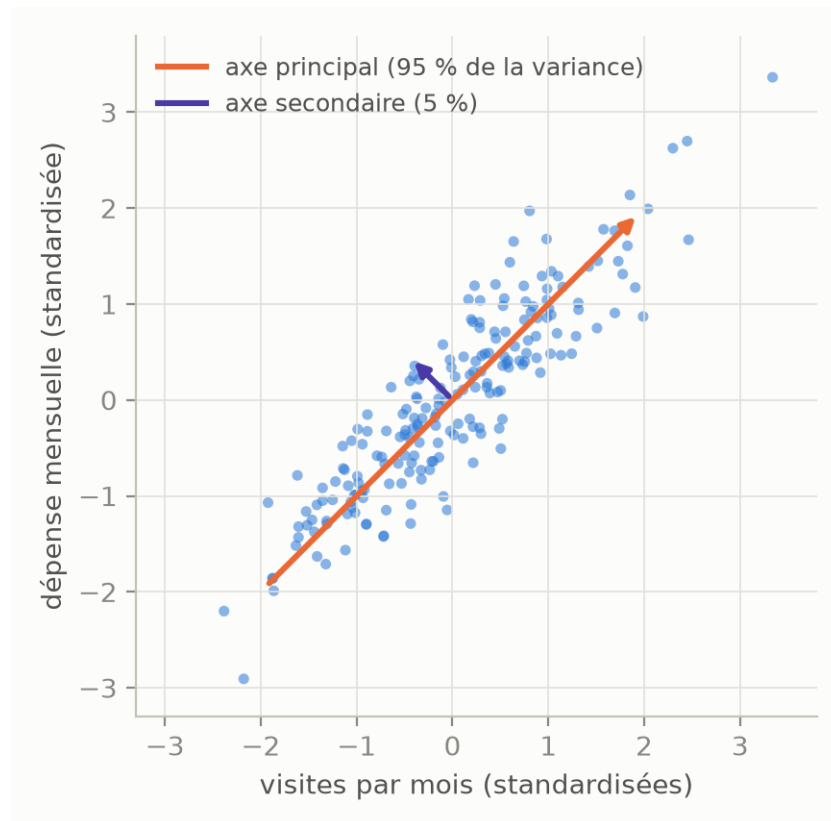


Figure 2: Nuage des 200 clients (variables standardisées) et son axe principal : le long de la diagonale, le nuage est étiré ; perpendiculairement, il est mince.

Sur les données simulées. La corrélation mesurée vaut environ 0,90 : la plus grande valeur propre (environ 1,90) correspond à la direction (0,71; 0,71), la diagonale : c'est l'axe le long duquel le nuage de clients est le plus étiré, l'axe « client actif et dépensier ». Elle capte environ 95 % de toute la variabilité. La seconde (0,10) correspond à la direction perpendiculaire, qui ne représente que les écarts « dépense inhabituelle pour ce nombre de visites ».

Autrement dit, on peut résumer ces deux variables par **une seule** (la position le long de l'axe principal), en ne perdant que 5 % de l'information. C'est le principe de l'**analyse en composantes principales** (ACP), que nous étudierons en détail au volume II : *trouver les directions de plus grande variance, ce sont les vecteurs propres de la matrice de covariance.*

À retenir (valeurs propres).

- $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$: un vecteur propre garde sa direction, la valeur propre dit de combien il est étiré.
- On les trouve avec $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$. Somme des valeurs propres = trace ; produit = déterminant.
- Une matrice symétrique a des valeurs propres réelles et des vecteurs propres orthogonaux ; $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$.
- Les vecteurs propres de la matrice de covariance sont les axes principaux d'un nuage de données.

Pour s'entraîner. Cahier, chapitre 1 : application 1.3, exercice 1.3.

1.1.4 La décomposition en valeurs singulières (SVD)

Les valeurs propres ne sont définies que pour des matrices **carrées**. Or un jeu de données est rarement carré : n individus, p variables, avec $n \neq p$. La **décomposition en valeurs singulières**, ou **SVD** (*Singular Value Decomposition*), étend la même idée à **n'importe quelle matrice**. C'est l'un des outils les plus puissants de toute la data science.

Intuition. Toute matrice, même rectangulaire, agit en trois temps : **une rotation, un étirement le long des axes, puis une autre rotation**. On écrit :

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top.$$

Les matrices $\mathbf{U}$ et $\mathbf{V}$ sont des rotations (leurs colonnes sont des vecteurs de longueur 1, orthogonaux entre eux) ; $\mathbf{\Sigma}$ est « diagonale » et contient les facteurs d'étirement $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$, appelés **valeurs singulières**.

On peut aussi lire la SVD comme une **somme de couches simples** (« de rang 1 »), classées de la plus importante à la moins importante :

$$\mathbf{A} = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^\top + \dots + \sigma_r \mathbf{u}_r \mathbf{v}_r^\top.$$

Chaque couche $\mathbf{u}_i \mathbf{v}_i^\top$ est un tableau « produit » : (profil des lignes) $\times$ (profil des colonnes). La première couche, celle de plus grande valeur singulière, capture la structure dominante des données ; les suivantes en sont les raffinements.

Un exemple minuscule

Exemple à la main. Prenons $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ (rang 1 : les deux lignes sont identiques). Calculons

$\mathbf{A}^\top \mathbf{A} = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$. Ses valeurs propres sont 4 et 0 (trace 4, déterminant 0). La plus grande valeur singulière est donc $\sigma_1 = \sqrt{4} = 2$ et la seconde est $\sigma_2 = 0$. Le vecteur propre associé à 4 est $\mathbf{v}_1 = \frac{1}{\sqrt{2}}(1, 1)$, et $\mathbf{u}_1 = \mathbf{A} \mathbf{v}_1 / \sigma_1 = \frac{1}{\sqrt{2}}(1, 1)$. On vérifie :

$$\sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top = 2 \cdot \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \mathbf{A}. \quad \checkmark$$

Une seule valeur singulière non nulle : la matrice n'a qu'une couche. Plus généralement, le **nombre de valeurs singulières non nulles est le rang** de la matrice.

Démonstration : d'où viennent les valeurs singulières ? (l'idée de la construction)

Soit $\mathbf{A} \in \mathbb{R}^{n \times p}$. La matrice $\mathbf{A}^\top \mathbf{A}$ est carrée ($p \times p$), **symétrique**, et **positive** : pour tout vecteur $\mathbf{x}$, $\mathbf{x}^\top \mathbf{A}^\top \mathbf{A} \mathbf{x} = \|\mathbf{A} \mathbf{x}\|^2 \geq 0$. Le théorème spectral nous donne donc des vecteurs propres orthonormés $\mathbf{v}_1, \dots, \mathbf{v}_p$ associés à des valeurs propres $\lambda_i \geq 0$ (elles sont positives car $\lambda_i = \mathbf{v}_i^\top \mathbf{A}^\top \mathbf{A} \mathbf{v}_i = \|\mathbf{A} \mathbf{v}_i\|^2$).

On pose $\sigma_i = \sqrt{\lambda_i}$ et, pour $\sigma_i > 0$, $\mathbf{u}_i = \mathbf{A} \mathbf{v}_i / \sigma_i$. Ces vecteurs $\mathbf{u}_i$ sont **orthonormés** :

$$\mathbf{u}_i \cdot \mathbf{u}_j = \frac{(\mathbf{A} \mathbf{v}_i)^\top (\mathbf{A} \mathbf{v}_j)}{\sigma_i \sigma_j} = \frac{\mathbf{v}_i^\top \mathbf{A}^\top \mathbf{A} \mathbf{v}_j}{\sigma_i \sigma_j} = \frac{\lambda_j \mathbf{v}_i \cdot \mathbf{v}_j}{\sigma_i \sigma_j} = \begin{cases} 1 & i = j \\ 0 & i \neq j. \end{cases}$$

Enfin, par construction $\mathbf{A} \mathbf{v}_i = \sigma_i \mathbf{u}_i$: c'est exactement l'égalité $\mathbf{A} \mathbf{V} = \mathbf{U} \mathbf{\Sigma}$, qui donne $\mathbf{A} =$

$U\Sigma V^T$. ■

Autrement dit : les valeurs singulières de A sont les racines carrées des valeurs propres de $A^T A$. Le lien avec la section précédente est direct.

Approximer une matrice par un nombre réduit de couches

Voici la propriété qui rend la SVD si utile en pratique.

Théorème (Eckart-Young). Parmi toutes les matrices de rang k , la meilleure approximation de A (au sens de la somme des carrés des erreurs) est obtenue en **ne gardant que les k premières couches** de la SVD :

$$A_k = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \dots + \sigma_k \mathbf{u}_k \mathbf{v}_k^T.$$

L'erreur vaut alors $\|A - A_k\|_F = \sqrt{\sigma_{k+1}^2 + \dots + \sigma_r^2}$, où $\|\cdot\|_F$ est la racine de la somme des carrés de tous les éléments.

Autrement dit : si les premières valeurs singulières sont grandes et les suivantes minuscules, on peut **jeter** les dernières couches sans presque rien perdre. C'est de la **compression**.

Un tableau de ventes : combien de couches faut-il ?

La gérante a les ventes hebdomadaires de 6 produits sur 8 semaines (un tableau de 48 nombres). Les données sont simulées selon une règle simple : *ventes = popularité du produit × effet de la semaine + un peu de bruit*. Une telle structure « produit × saison » doit se retrouver dans la première couche de la SVD : c'est la matrice la plus simple possible, un produit extérieur $\mathbf{u}\mathbf{v}^T$ (comme dans l'exemple à la main ci-dessus, mais bruitée).

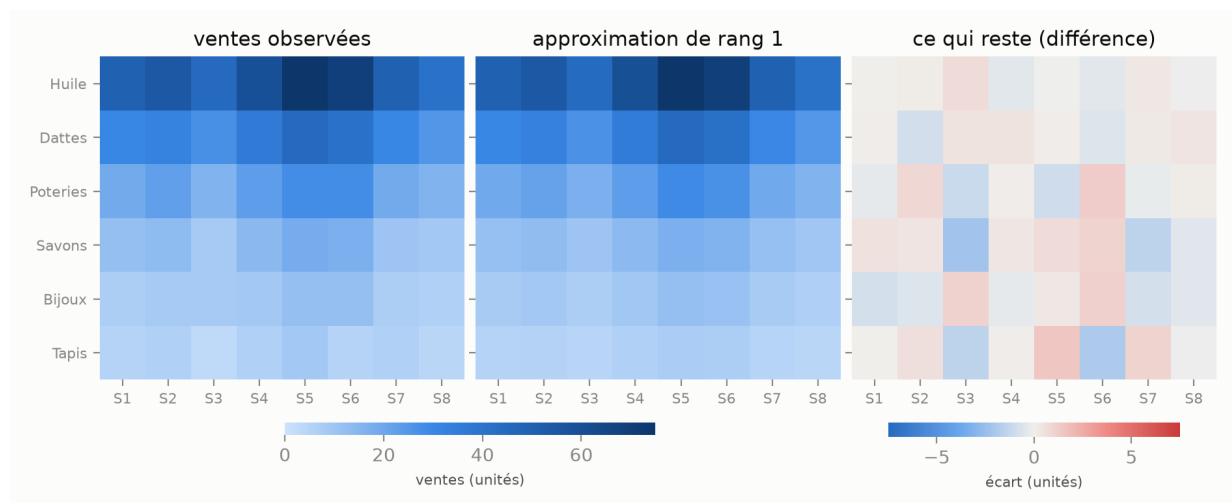


Figure 3: Ventes observées (à gauche), approximation par une seule couche (au centre) et ce qui reste (à droite). L'échelle de couleur est la même pour les deux premiers panneaux.

Résultat. La SVD du tableau donne une première valeur singulière (environ 202) près de soixante fois plus grande que la deuxième (environ 3,5) : **99,9 %** de l'« énergie » du tableau est dans une seule couche. Une approximation de rang 1, qui ne stocke que 15 nombres au lieu de 48, reproduit le tableau avec une erreur relative d'environ 2,7 %. Le reste (les couches 2 à 6) est du bruit.

La SVD a donc **retrouvé toute seule** la structure « popularité × saison » que nous avons mise dans les données, sans qu'on lui dise de la chercher. Vous venez de voir le principe de la **réduction de dimension** et celui des **systèmes de recommandation** par factorisation de matrices, deux sujets que nous développerons aux volumes suivants.

À retenir (SVD).

- Toute matrice se décompose en $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^T$: rotation, étirement, rotation.
- Les valeurs singulières mesurent l'importance de chaque couche ; leur nombre non nul est le rang.
- Garder les k premières couches donne la meilleure approximation de rang k (Eckart-Young) : c'est la base de la compression, du débruitage et de l'ACP.
- L'ACP n'est rien d'autre que la SVD des données centrées.

Pour s'entraîner. Cahier, chapitre 1 : application 1.4.

1.2 Analyse : mesurer le changement

L'algèbre linéaire nous a appris à décrire des données. L'**analyse** nous apprend à décrire comment les choses **changent**. C'est ce qui permet de répondre à des questions comme : « si j'augmente mon prix d'un euro, mes recettes montent-elles ou descendent-elles, et de combien ? ». Elle repose sur deux notions : la **dérivée** (le changement instantané) et l'**intégrale** (le changement accumulé).

1.2.1 La dérivée : la vitesse à laquelle une fonction change

Intuition. Si une fonction décrit **où vous en êtes**, sa dérivée décrit à **quelle vitesse vous avancez**. Dans une voiture, la distance parcourue est la fonction ; le compteur de vitesse affiche sa dérivée. Graphiquement, la dérivée en un point est la **pente de la tangente** à la courbe en ce point.

Une fonction pour commencer

Une **fonction** associe à chaque valeur d'entrée x une valeur de sortie $f(x)$. Voici celle que nous utiliserons dans cette section : le bénéfice hebdomadaire de la gérante pour une poterie, en fonction du nombre q de pièces vendues. Plus elle en vend, plus elle doit baisser le prix, et il y a 300 € de frais fixes :

$$P(q) = -2q^2 + 80q - 300.$$

Par exemple, $P(20) = -2 \cdot 400 + 1600 - 300 = 500$ €.

D'abord, une idée de limite

Pour définir la dérivée, il faut la notion de **limite** : la valeur *vers laquelle tend* une quantité quand on s'approche d'un point, même si l'on ne peut pas l'atteindre.

Exemple. Considérons $f(x) = \frac{x^2 - 1}{x - 1}$. Elle n'est pas définie en $x = 1$ (division par zéro). Mais que se passe-t-il quand x s'en approche ? Calculons quelques valeurs (comme $x^2 - 1 = (x - 1)(x + 1)$, on a $f(x) = x + 1$ dès que $x \neq 1$) :

x	0,9	0,99	0,999	1,001	1,01	1,1
$f(x)$	1,9	1,99	1,999	2,001	2,01	2,1

On voit que $f(x)$ se rapproche de 2. On écrit $\lim_{x \rightarrow 1} f(x) = 2$. Une limite décrit un comportement **au voisinage** d'un point, pas au point lui-même.

Définition de la dérivée

Comment mesurer la pente d'une courbe en un seul point ? Une pente se mesure entre deux points. L'idée est donc de prendre deux points très proches, x et $x + h$, et de regarder la pente de la droite qui les relie : le **taux d'accroissement**

$$\frac{f(x+h) - f(x)}{h}.$$

Puis de faire tendre h vers zéro. La **dérivée** est la limite, quand elle existe :

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

Exemple chiffré. Prenons $f(x) = x^2$ au point $x = 3$ et rapprochons h de zéro. Le taux d'accroissement $\frac{(3+h)^2 - 3^2}{h}$ vaut 7 pour $h = 1$, puis 6,1 ; 6,01 ; 6,001 ; 6,0001 pour $h = 0,1$; 0,01 ; 0,001 ; 0,0001.

Les valeurs se rapprochent de 6. Et $2 \times 3 = 6$: la dérivée de x^2 semble être $2x$. Prouvons-le.

Démonstration : la dérivée de x^2 est $2x$.

$$\frac{(x+h)^2 - x^2}{h} = \frac{x^2 + 2xh + h^2 - x^2}{h} = \frac{2xh + h^2}{h} = 2x + h.$$

Quand $h \rightarrow 0$, cette quantité tend vers $2x$. Donc $(x^2)' = 2x$. ■

Notez que les calculs numériques ci-dessus suivent exactement ce résultat : le taux vaut $2 \cdot 3 + h = 6 + h$, soit 7, 6,1, 6,01, 6,001, 6,0001.

Les règles de calcul

On ne recalcule pas une limite à chaque fois. On utilise des règles, qu'il faut connaître par cœur :

Fonction	Dérivée	Exemple
constante c	0	$(7)' = 0$
x^n	$n x^{n-1}$	$(x^3)' = 3x^2$; $(\sqrt{x})' = \frac{1}{2\sqrt{x}}$
e^x	e^x	
$\ln x$	$\frac{1}{x}$	
somme $f + g$	$f' + g'$	$(x^2 + 5x)' = 2x + 5$
multiple cf	cf'	$(4x^3)' = 12x^2$
produit fg	$f'g + fg'$	$(x e^x)' = e^x + x e^x$
composée $f(g(x))$	$f'(g(x)) \cdot g'(x)$	voir ci-dessous

La dernière règle, celle de la **dérivée d'une composée** (ou « règle de la chaîne »), est la plus importante pour la suite : c'est elle qui fait fonctionner l'entraînement de tous les réseaux de neurones. Elle dit : *dérivez l'extérieur sans toucher à l'intérieur, puis multipliez par la dérivée de l'intérieur*.

Exemple. $h(x) = (3x + 1)^2$. L'extérieur est « carré », l'intérieur est $3x + 1$.

$$h'(x) = \underbrace{2(3x + 1)}_{\text{dérivée de l'extérieur}} \times \underbrace{3}_{\text{dérivée de l'intérieur}} = 6(3x + 1) = 18x + 6.$$

Contrôle : en développant, $h(x) = 9x^2 + 6x + 1$, dont la dérivée est $18x + 6$. ✓

Autre exemple, qui servira bientôt : $\frac{d}{dx} e^{-0,5x} = -0,5 e^{-0,5x}$ (l'intérieur est $-0,5x$, de dérivée $-0,5$).

Pour **vérifier** une dérivée calculée à la main (ou dénicher une erreur), on peut la comparer à une dérivée numérique, la pente entre $x - h$ et $x + h$ pour un très petit h (la « différence centrée ») :

$$f'(x) \approx \frac{f(x + h) - f(x - h)}{2h}, \quad h \approx 10^{-6}.$$

Pour x^3 en $x = 2$, $(3x + 1)^2$ en $x = 1$, $e^{-0,5x}$ en $x = 2$ et $\ln(2x)$ en $x = 4$, cette formule redonne les valeurs de la dérivée exacte (12 ; 24 ; $-0,1839$; 0,25) à six décimales près.

Bonne habitude. Chaque fois que vous dérivez une expression compliquée, comparez-la à une dérivée numérique. Les erreurs de signe et les oublis de « fois la dérivée de l'intérieur » sont les fautes les plus courantes.

Un exemple de décision : à quel niveau de ventes le bénéfice est-il maximal ?

Reprenons $P(q) = -2q^2 + 80q - 300$. Observons-la d'abord, pour quelques valeurs de q :

q	10	15	20	25	30
$P(q)$ (€)	300	450	500	450	300

Le bénéfice monte, atteint un sommet, puis redescend : c'est une parabole « en cloche ». **Au sommet, la tangente est horizontale, donc la dérivée est nulle.** C'est la clé de l'optimisation. On dérive :

$$P'(q) = -4q + 80.$$

On résout $P'(q) = 0$: $-4q + 80 = 0$, donc $q = 20$ pièces, pour un bénéfice $P(20) = 500$ €.

La dérivée a aussi une lecture économique directe : c'est le **bénéfice marginal**, le gain approximatif que rapporte la pièce supplémentaire. En $q = 10$, $P'(10) = -40 + 80 = 40$ €. Comparons avec la vraie différence : $P(11) - P(10) = 338 - 300 = 38$ €.

La pente (40) est une bonne approximation du gain réel (38) : elle est exacte pour une variation infiniment petite, et approchée pour une variation d'une pièce. Une recherche numérique du maximum, sans utiliser la dérivée, retrouve bien $q = 20$ et un bénéfice de 500 €.

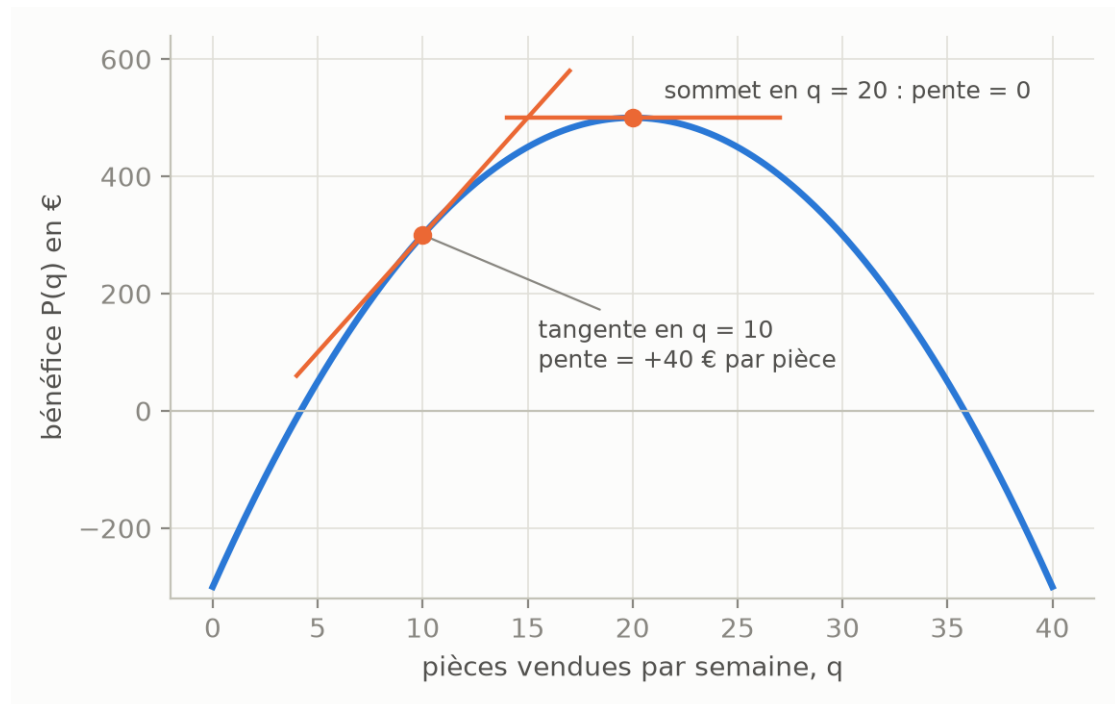


Figure 4: Bénéfice hebdomadaire en fonction du nombre de pièces vendues. En $q = 10$ la tangente monte avec une pente de 40 ; en $q = 20$ elle est horizontale : c'est le sommet.

Comment savoir si c'est un maximum ou un minimum ?

Une dérivée nulle signale un point **candidat**, pas forcément un maximum. On regarde alors la **dérivée seconde** f'' (la dérivée de la dérivée, qui mesure comment la pente elle-même change) :

- $f''(x) < 0$: la pente décroît, la courbe est une « bosse », c'est un **maximum local** ;
- $f''(x) > 0$: la pente croît, la courbe est un « creux », c'est un **minimum local** ;
- $f''(x) = 0$: on ne peut pas conclure.

Ici $P''(q) = -4 < 0$: c'est bien un maximum.

Piège. « Dérivée nulle » ne veut pas dire « extremum ». Pour $f(x) = x^3$, on a $f'(0) = 3 \cdot 0^2 = 0$, et pourtant la fonction ne fait que croître : en 0, la tangente est horizontale mais la courbe continue de monter (c'est un *point d'inflexion*). Vérifiez toujours avec la dérivée seconde ou en comparant les valeurs de part et d'autre.

À retenir (dérivée).

- $f'(x)$ est la pente de la tangente en x : c'est le taux de variation instantané.
- Les cinq règles à maîtriser : puissances, somme, multiple, produit, composée.
- Pour optimiser : on cherche $f'(x) = 0$, puis on contrôle avec f'' .
- Comparer une dérivée à la main à une dérivée numérique permet de détecter les erreurs.

Pour s'entraîner. Cahier, chapitre 1 : application 1.5, exercice 1.4.

1.2.2 Dérivées partielles et gradient : plusieurs variables à la fois

Un modèle de data science a rarement un seul paramètre : il en a deux, dix, ou un milliard. Il nous faut donc dériver des fonctions de **plusieurs variables**.

Intuition. Imaginez que vous êtes debout sur le flanc d'une colline brumeuse et que vous voulez monter le plus vite possible. À vos pieds, le sol penche plus dans une direction que dans les autres. Le **gradient** est la flèche qui indique **la direction de la plus forte montée**, et sa longueur dit à quel point c'est raide. Pour descendre, on marche dans la direction opposée.

Dérivée partielle

Pour une fonction $f(x, y)$, la **dérivée partielle par rapport à x** , notée $\frac{\partial f}{\partial x}$, s'obtient en dérivant par rapport à x **en traitant y comme une constante**. De même pour y .

Exemple. Soit $f(x, y) = x^2 + 3y^2$ (un « bol » plus raide dans la direction y).

$$\frac{\partial f}{\partial x} = 2x, \quad \frac{\partial f}{\partial y} = 6y.$$

(Pour $\partial f / \partial x$, le terme $3y^2$ est une constante : sa dérivée est 0.)

Le **gradient** est le vecteur qui rassemble toutes les dérivées partielles :

$$\nabla f(x, y) = \begin{pmatrix} \partial f / \partial x \\ \partial f / \partial y \end{pmatrix} = \begin{pmatrix} 2x \\ 6y \end{pmatrix}.$$

En $(1, 1)$, on obtient $\nabla f(1, 1) = (2, 6)$: la pente est 3 fois plus forte dans la direction y que dans la direction x .

Pourquoi le gradient indique-t-il la plus forte montée ?

Dans une direction donnée par un vecteur $\mathbf{u}$ de longueur 1, la pente de f (la **dérivée directionnelle**) vaut

$$D_{\mathbf{u}}f = \nabla f \cdot \mathbf{u}.$$

Et ce produit scalaire, nous savons le majorer.

Démonstration. Par la formule du produit scalaire vue en 1.1.1,

$$D_{\mathbf{u}}f = \nabla f \cdot \mathbf{u} = \|\nabla f\| \|\mathbf{u}\| \cos \theta = \|\nabla f\| \cos \theta,$$

où θ est l'angle entre $\mathbf{u}$ et le gradient. Cette quantité est maximale quand $\cos \theta = 1$, c'est-à-dire quand $\mathbf{u}$ pointe **dans la même direction que le gradient**, et alors la pente vaut $\|\nabla f\|$. Elle est minimale (la descente la plus forte, de pente $-\|\nabla f\|$) quand $\theta = 180^\circ$, c'est-à-dire dans la direction $-\nabla f$. ■

(Au passage : on retrouve la même inégalité de Cauchy-Schwarz que dans la section 1.1.1.)

Vérifions en $(1, 1)$, où $\nabla f = (2, 6)$ et $\|\nabla f\| = \sqrt{40} \approx 6,325$, en calculant la pente de f dans quatre directions $\mathbf{u}$ (de longueur 1), par $\nabla f \cdot \mathbf{u}$:

Direction	(1,0)	(0,1)	celle du gradient	opposée au gradient
Pente	2	6	+6,325	-6,325

(Un calcul numérique de la pente par de petits déplacements, mené en parallèle, donne les mêmes valeurs.)

La pente est maximale ($\|\nabla f\| = \sqrt{40}$) dans la direction du gradient, et minimale ($-\sqrt{40}$) dans la direction opposée. Aucune autre direction ne fait mieux.

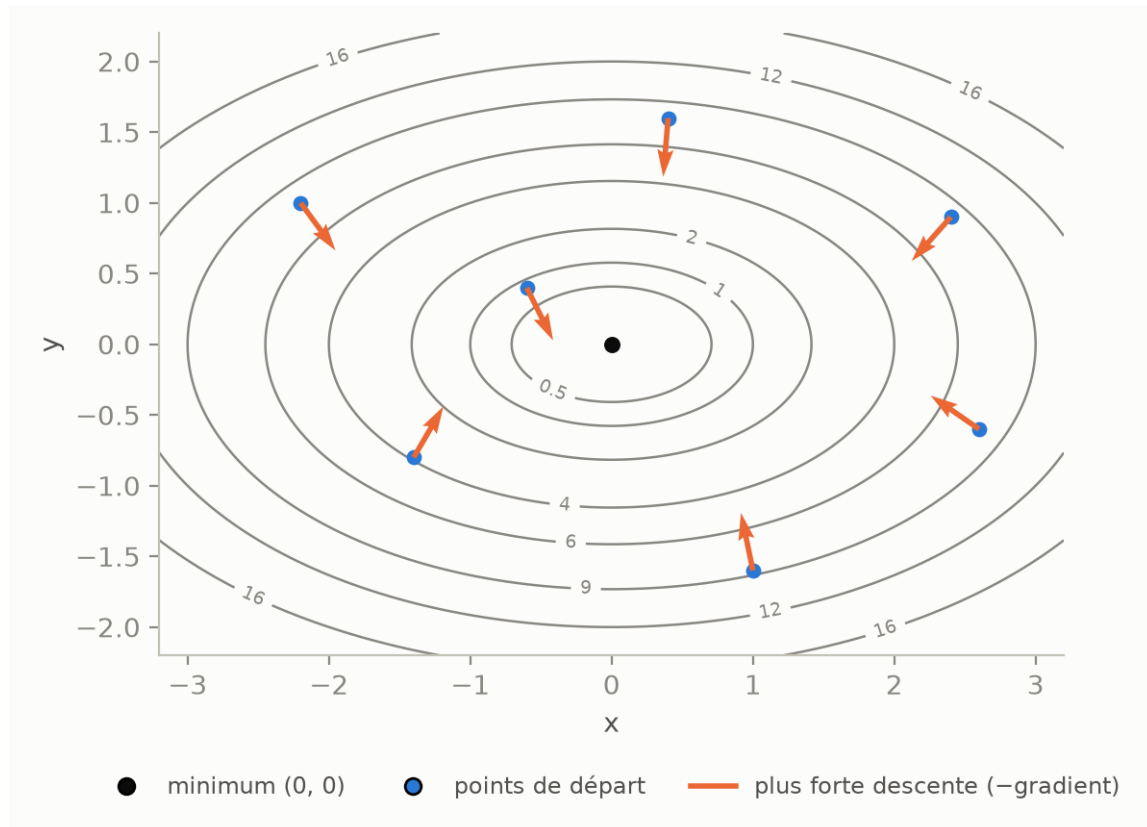


Figure 5: Courbes de niveau de $f(x,y) = x^2 + 3y^2$ (chaque ellipse est une « altitude » constante) et, en chaque point, la direction de la plus forte descente ($-\text{gradient}$). Les flèches sont perpendiculaires aux courbes de niveau et pointent vers le fond du bol.

Deux faits à retenir sur cette figure : le gradient est **perpendiculaire aux courbes de niveau**, et **$-\text{gradient}$ pointe vers le bas**. C'est exactement ce que nous utiliserons en 1.3 pour descendre vers le minimum.

Un exemple : le gradient de l'erreur d'une droite

La gérante veut ajuster une droite $y = ax + b$ à trois mesures (x_i, y_i) : (1,2), (2,3) et (3,5). Pour mesurer la qualité d'une droite candidate, on utilise la **somme des carrés des erreurs** :

$$L(a,b) = \sum_{i=1}^3 (y_i - (ax_i + b))^2.$$

C'est une fonction de **deux** variables, a et b . Notons $r_i = y_i - (ax_i + b)$ l'erreur du point i . Par la règle de la composée :

$$\frac{\partial L}{\partial a} = -2 \sum_i x_i r_i, \quad \frac{\partial L}{\partial b} = -2 \sum_i r_i.$$

Calcul à la main en $(a, b) = (0, 0)$ (la droite $y = 0$). Les erreurs sont $r = (2, 3, 5)$.

- $L(0, 0) = 4 + 9 + 25 = 38$.
- $\partial L / \partial a = -2 (1 \cdot 2 + 2 \cdot 3 + 3 \cdot 5) = -2 \times 23 = -46$.
- $\partial L / \partial b = -2 (2 + 3 + 5) = -20$.

Le gradient $(-46, -20)$ est très négatif : en augmentant a et b , l'erreur diminue fortement.

Au point $(a, b) = (1,5 ; 1/3)$, le gradient est **nul** : on est au fond du bol. À la main : les erreurs valent alors $r = (2 - \frac{11}{6}, 3 - \frac{10}{3}, 5 - \frac{29}{6}) = (\frac{1}{6}, -\frac{1}{3}, \frac{1}{6})$, donc $\sum r_i = 0$ et $\sum x_i r_i = \frac{1}{6} - \frac{2}{3} + \frac{1}{2} = 0$, et la perte vaut $L = \frac{1}{36} + \frac{1}{9} + \frac{1}{36} = \frac{1}{6} \approx 0,167$. Nous montrerons en 1.3 comment y arriver automatiquement.

À retenir (gradient).

- Le gradient ∇f rassemble les dérivées partielles ; il pointe vers la **plus forte montée** et sa norme mesure la pente.
- $-\nabla f$ pointe vers la plus forte **descente**.
- Au minimum (ou au maximum, ou au col), le gradient est nul.
- La règle de la composée rend le calcul du gradient d'une « somme de carrés d'erreurs » mécanique : c'est le cœur de l'entraînement des modèles.

Pour s'entraîner. Cahier, chapitre 1 : application 1.6, exercice 1.5.

1.2.3 Intégrales : accumuler les petits changements

Intuition. L'intégrale est l'opération inverse de la dérivée. Si la dérivée donne « la vitesse à chaque instant », l'**intégrale** donne « la distance parcourue sur un intervalle » : elle **additionne une infinité de petits morceaux**. Graphiquement, c'est l'**aire sous la courbe**.

En data science, l'intégrale sert surtout à une chose : calculer des **probabilités**. Quand une quantité est décrite par une courbe de densité, la probabilité d'un événement est l'aire sous la courbe sur la zone concernée.

Calculer une aire avec des rectangles

L'aire sous une courbe f entre a et b se note $\int_a^b f(x) dx$. L'idée de **Riemann** : découper l'intervalle en n bandes étroites, approcher chaque bande par un rectangle de hauteur $f(x)$, et additionner. Plus n est grand, meilleure est l'approximation.

Exemple à la main. Aire sous $f(x) = x^2$ entre 0 et 3, avec $n = 3$ rectangles de largeur 1 (hauteur mesurée au bord droit) : $1 \cdot 1^2 + 1 \cdot 2^2 + 1 \cdot 3^2 = 1 + 4 + 9 = 14$. C'est une approximation grossière (les rectangles dépassent la courbe). Avec $n = 6$ rectangles de largeur 0,5 : $0,5 (0,5^2 + 1^2 + 1,5^2 + 2^2 + 2,5^2 + 3^2) = 0,5 \times 22,75 = 11,375$. Mieux ! En continuant avec de plus en plus de rectangles, on obtient :

n	3	6	30	300	3000
aire approchée	14,000	11,375	9,455	9,045	9,005

Les approximations convergent vers **9** (la valeur exacte). L'intégrale *est* cette limite.

Le théorème fondamental : la dérivée à l'envers

Faut-il toujours découper en rectangles ? Heureusement non. Le **théorème fondamental de l'analyse** relie intégrale et dérivée :

Si F est une **primitive** de f (c'est-à-dire $F' = f$), alors

$$\int_a^b f(x) dx = F(b) - F(a).$$

Exemple. Pour $f(x) = x^2$, une primitive est $F(x) = x^3/3$ (car $(x^3/3)' = x^2$). Donc $\int_0^3 x^2 dx = F(3) - F(0) = 9 - 0 = 9$. ✓ C'est la valeur vers laquelle convergeaient nos rectangles.

Quelques primitives à connaître :

$f(x)$	une primitive $F(x)$
x^n ($n \neq -1$)	$\frac{x^{n+1}}{n+1}$
e^{kx}	$\frac{e^{kx}}{k}$
$\frac{1}{x}$	$\ln x $

Pourquoi ça marche : l'idée de la preuve. Notons $A(x) = \int_a^x f(t) dt$ l'aire accumulée jusqu'en x . Quand x avance de h , l'aire gagne une fine bande de largeur h et de hauteur à peu près $f(x)$: $A(x+h) - A(x) \approx f(x)h$. En divisant par h et en faisant tendre h vers 0, on obtient $A'(x) = f(x)$. L'aire accumulée est donc **une** primitive de f ; deux primitives ne diffèrent que d'une constante, qui disparaît dans la différence $F(b) - F(a)$. ■ (Preuve simplifiée, suffisante pour l'intuition.)

Un exemple : une commande dans les trois prochaines minutes ?

Sur le site de la boutique, en heure de pointe, une commande arrive en moyenne toutes les 2 minutes. On montrera au chapitre 2 que le temps d'attente T (en minutes) avant la prochaine commande suit une **loi exponentielle** de taux $\lambda = 0,5$ par minute, dont la densité est

$$f(t) = \lambda e^{-\lambda t} = 0,5 e^{-0,5t}, \quad t \geq 0.$$

La probabilité qu'une commande arrive dans les 3 prochaines minutes est l'aire sous cette courbe entre 0 et 3.

À la main. Une primitive de $0,5 e^{-0,5t}$ est $-e^{-0,5t}$ (on dérive : $-(-0,5)e^{-0,5t} = 0,5 e^{-0,5t}$ ✓). Donc

$$P(T \leq 3) = [-e^{-0,5t}]_0^3 = -e^{-1,5} - (-e^0) = 1 - e^{-1,5} \approx 0,777.$$

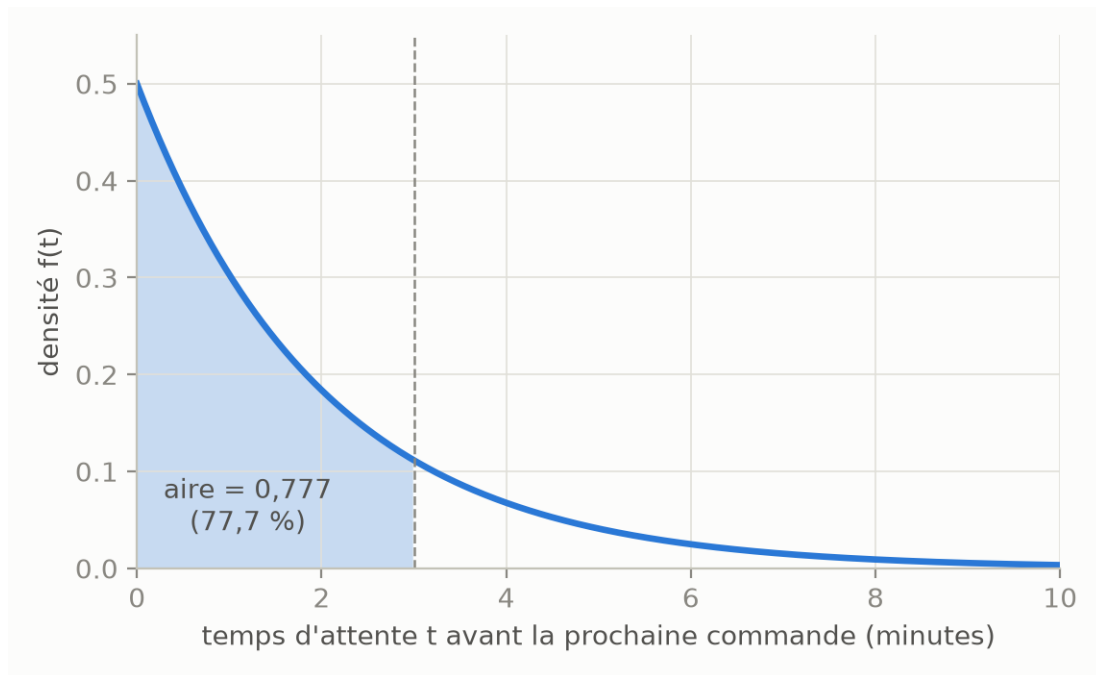


Figure 6: Densité du temps d'attente avant la prochaine commande. L'aire grisée entre 0 et 3 minutes vaut 0,777 : il y a environ 78 % de chances qu'une commande arrive dans les trois prochaines minutes.

Lecture. Le calcul exact et une intégration numérique concordent : environ 77,7 % de chances. Et l'aire **totale** sous la courbe vaut 1 : c'est une exigence de toute densité de probabilité (la probabilité que *quelque chose* arrive est de 100 %). Nous reverrons ces idées en détail au chapitre 2.

Piège : les aires sous l'axe comptent négativement. Si f est négative sur une partie de l'intervalle, l'intégrale en tient compte avec un signe moins. L'intégrale mesure une quantité **algébrique** accumulée, pas toujours une surface géométrique.

À retenir (intégrale).

- $\int_a^b f(x) dx$ est l'aire (algébrique) sous la courbe entre a et b : une limite de sommes de rectangles.
- Théorème fondamental : $\int_a^b f = F(b) - F(a)$ où $F' = f$.
- Pour une densité de probabilité, l'aire sous la courbe sur une zone **est** la probabilité de cette zone ; l'aire totale vaut 1.

Pour s'entraîner. Cahier, chapitre 1 : application 1.7.

1.3 Optimisation : trouver le meilleur choix

Presque tout problème de data science se ramène à la même question : **parmi toutes les valeurs possibles des paramètres de mon modèle, laquelle est la meilleure ?** Et « meilleure » veut dire : celle qui **minimise une erreur** (ou maximise un gain). C'est la troisième phrase de notre épigraphe : *apprendre, c'est minimiser une erreur*. Tout ce que nous avons vu dans ce chapitre (vecteurs, matrices, dérivées, gradients) converge ici.

1.3.1 Le problème d'optimisation

Un **problème d'optimisation** consiste à trouver les paramètres θ qui minimisent une **fonction objectif** $f(\theta)$ (on dit aussi *fonction de coût* ou *fonction de perte*) :

$$\theta^* = \arg \min_{\theta} f(\theta).$$

La notation $\arg \min$ se lit « l'argument qui minimise » : on cherche **où** le minimum est atteint, pas seulement sa valeur. Maximiser f revient à minimiser $-f$: on peut donc toujours se ramener à une minimisation.

Nous avons déjà rencontré deux problèmes de ce type :

- le bénéfice $P(q)$ de la section 1.2 (une variable, maximisation) ;
- la perte $L(a, b)$ d'une droite ajustée à des points (deux variables, minimisation).

Rappel utile. Aux points qui annulent le gradient ($\nabla f = 0$), la fonction est « à plat ». Ces points sont des **candidats** : minimum local, maximum local, ou col. La question est de savoir lequel, et surtout de savoir si c'est le **meilleur de tous** (minimum *global*) ou seulement le meilleur dans son voisinage (minimum *local*).

1.3.2 La convexité : quand il n'y a qu'une vallée

Intuition. Imaginez un bol : où que vous posiez une bille, elle roule vers l'unique point le plus bas. Maintenant imaginez un paysage de montagnes avec plusieurs vallées : la bille peut se retrouver coincée dans une petite vallée alors qu'une plus profonde existe ailleurs. Une fonction **convexe** est un bol : **pas de piège possible**.

Formellement, f est **convexe** si, pour deux points quelconques $\mathbf{x}$ et $\mathbf{y}$ et pour tout $t \in [0, 1]$,

$$f(t\mathbf{x} + (1-t)\mathbf{y}) \leq tf(\mathbf{x}) + (1-t)f(\mathbf{y}).$$

Graphiquement : **la corde qui relie deux points de la courbe est toujours au-dessus de la courbe**. Pour une fonction d'une variable deux fois dérivable, c'est équivalent à $f''(x) \geq 0$ partout.

Fonction	f''	Convexe ?
x^2	2	oui
$ x $	(pas dérivable en 0, mais la corde est au-dessus)	oui
e^x	$e^x > 0$	oui
x^3	$6x$ (change de signe)	non
$\sin x$	$-\sin x$ (change de signe)	non

La propriété qui rend la convexité précieuse :

Théorème. Si f est convexe, **tout minimum local est un minimum global**.

Démonstration. Raisonnons par l'absurde. Soit $\mathbf{x}^*$ un minimum local de f , et supposons qu'il existe un point $\mathbf{y}$ avec $f(\mathbf{y}) < f(\mathbf{x}^*)$. Pour $t \in (0, 1)$, considérons le point $\mathbf{z}_t = (1-t)\mathbf{x}^* + t\mathbf{y}$,

sur le segment qui joint $\mathbf{x}^*$ à $\mathbf{y}$. Par convexité,

$$f(\mathbf{z}_t) \leq (1-t)f(\mathbf{x}^*) + tf(\mathbf{y}) < (1-t)f(\mathbf{x}^*) + tf(\mathbf{x}^*) = f(\mathbf{x}^*).$$

Or, quand $t \rightarrow 0$, le point $\mathbf{z}_t$ se rapproche de $\mathbf{x}^*$ tout en gardant $f(\mathbf{z}_t) < f(\mathbf{x}^*)$: il existe donc des points arbitrairement proches de $\mathbf{x}^*$ où f est plus petite. Cela contredit le fait que $\mathbf{x}^*$ est un minimum local. ■

La perte des moindres carrés est convexe

Voici le résultat qui justifie pourquoi la régression linéaire est « facile » à optimiser. Pour un modèle linéaire, la perte s'écrit avec une matrice de données $\mathbf{X}$ (une ligne par individu), un vecteur de paramètres θ et un vecteur de cibles $\mathbf{y}$:

$$L(\theta) = \|\mathbf{y} - \mathbf{X}\theta\|^2.$$

Son gradient est $\nabla L(\theta) = -2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\theta)$ (c'est la forme matricielle de ce que nous avons calculé à la main en 1.2.2), et sa dérivée seconde, la **matrice hessienne**, est $\mathbf{H} = 2\mathbf{X}^\top\mathbf{X}$.

Démonstration : la hessienne est « positive », donc L est convexe. Pour tout vecteur $\mathbf{v}$,

$$\mathbf{v}^\top \mathbf{H} \mathbf{v} = 2\mathbf{v}^\top \mathbf{X}^\top \mathbf{X} \mathbf{v} = 2\|\mathbf{X}\mathbf{v}\|^2 \geq 0.$$

Une hessienne dont toutes les valeurs propres sont positives ou nulles correspond à une fonction convexe (c'est l'analogue en plusieurs variables de $f'' \geq 0$). ■

De plus, si les colonnes de $\mathbf{X}$ sont **indépendantes** (rang plein), alors $\mathbf{X}\mathbf{v} = \mathbf{0}$ n'est possible que pour $\mathbf{v} = \mathbf{0}$: la hessienne est strictement positive, le bol a un **unique** fond.

On retrouve ici le rang de la section 1.1. Reprenons nos colonnes redondantes (prix HT et prix TTC) : si $\mathbf{X}$ contient les deux, alors $\theta = (1, 0)$ (« on utilise le HT ») et $\theta = (0, 1/1, 19)$ (« on utilise le TTC divisé par 1,19 ») produisent **exactement les mêmes prédictions** et donc la même perte : le fond du bol est une *rigole*, pas un point. Le minimum existe mais n'est pas unique.

Pour notre petit exemple de trois points, la hessienne est $2\mathbf{X}^\top\mathbf{X}$ avec $\mathbf{X}$ constituée de la colonne $x = (1, 2, 3)$ et d'une colonne de 1. On calcule $\mathbf{X}^\top\mathbf{X} = \begin{pmatrix} 14 & 6 \\ 6 & 3 \end{pmatrix}$, donc

$$\mathbf{H} = \begin{pmatrix} 28 & 12 \\ 12 & 6 \end{pmatrix}, \quad \text{tr } \mathbf{H} = 34, \quad \det \mathbf{H} = 168 - 144 = 24,$$

de sorte que ses valeurs propres sont $\lambda = 17 \pm \sqrt{265}$, soit environ 33,28 et 0,72.

Les deux valeurs propres (0,72 et 33,28) sont positives : L est convexe, avec un unique minimum. Remarquez toutefois qu'elles sont **très inégales** (un rapport de 46). Cela signifie que le bol est très allongé : raide dans une direction, presque plat dans l'autre. Nous allons voir que cela a des conséquences concrètes.

1.3.3 La descente de gradient

Intuition. Vous êtes un randonneur dans le brouillard ; vous ne voyez pas la vallée, mais vous sentez la pente sous vos pieds. La stratégie : **faire un pas dans la direction où le sol descend le plus, puis recommencer**. Nous savons déjà que cette direction est $-\nabla f$.

L'algorithme

On part d'un point initial θ_0 et on répète :

$$\theta_{k+1} = \theta_k - \eta \nabla f(\theta_k)$$

Le nombre $\eta > 0$ est le **pas d'apprentissage** (*learning rate*) : la taille de nos enjambées. On s'arrête quand le gradient est presque nul, ou après un nombre fixé d'itérations.

Un exemple à une variable, calculé à la main

Minimisons $f(x) = (x - 3)^2$. Sa dérivée est $f'(x) = 2(x - 3)$ et son minimum est évidemment en $x = 3$. Partons de $x_0 = 0$ avec $\eta = 0,1$.

Pas à pas.

- $x_1 = 0 - 0,1 \times 2(0 - 3) = 0 + 0,6 = 0,6$
- $x_2 = 0,6 - 0,1 \times 2(0,6 - 3) = 0,6 + 0,48 = 1,08$
- $x_3 = 1,08 - 0,1 \times 2(1,08 - 3) = 1,08 + 0,384 = 1,464$

On avance vers 3, avec des pas qui raccourcissent (la pente s'adoucit à mesure qu'on approche du fond).

Peut-on prévoir ce comportement ? Oui, et c'est instructif :

Démonstration : quand la descente converge-t-elle ? Écrivons l'itération : $x_{k+1} = x_k - 2\eta(x_k - 3)$. Soustrayons 3 des deux côtés :

$$x_{k+1} - 3 = (x_k - 3) - 2\eta(x_k - 3) = (1 - 2\eta)(x_k - 3).$$

L'écart au minimum est donc **multiplié à chaque pas par le même facteur** $(1 - 2\eta)$. Par récurrence, $x_k - 3 = (1 - 2\eta)^k (x_0 - 3)$. Cet écart tend vers zéro si et seulement si $|1 - 2\eta| < 1$, c'est-à-dire

$$0 < \eta < 1.$$

Avec $\eta = 0,1$, le facteur vaut 0,8 : on réduit l'erreur de 20 % à chaque pas ($x_1 - 3 = 0,8 \times (-3) = -2,4$, soit $x_1 = 0,6$ ✓). Le cas $\eta = 0,5$ donne un facteur 0 : convergence **en un seul pas** (c'est le pas idéal, égal à l'inverse de la courbure $f'' = 2$). Pour $\eta > 1$, le facteur dépasse 1 en valeur absolue : l'erreur **grandit** à chaque pas. ■

Voici les cinq premiers pas, pour cinq valeurs de η (départ $x_0 = 0$) :

η	x_0	x_1	x_2	x_3	x_4	x_5	x_6
0,1	0	0,6	1,08	1,464	1,771	2,017	2,214
0,5	0	3	3	3	3	3	3
0,9	0	5,4	1,08	4,536	1,771	3,983	2,214
1,0	0	6	0	6	0	6	0

η	x_0	x_1	x_2	x_3	x_4	x_5	x_6
1,1	0	6,6	-1,32	8,184	-3,221	10,465	-5,958

On y voit : $\eta = 0,1$ converge doucement ; $\eta = 0,5$ tombe sur 3 dès le premier pas ; $\eta = 0,9$ converge **en oscillant** de part et d'autre du minimum ; $\eta = 1$ oscille éternellement entre 0 et 6 sans jamais converger ; $\eta = 1,1$ **diverge** (les valeurs s'éloignent de plus en plus).

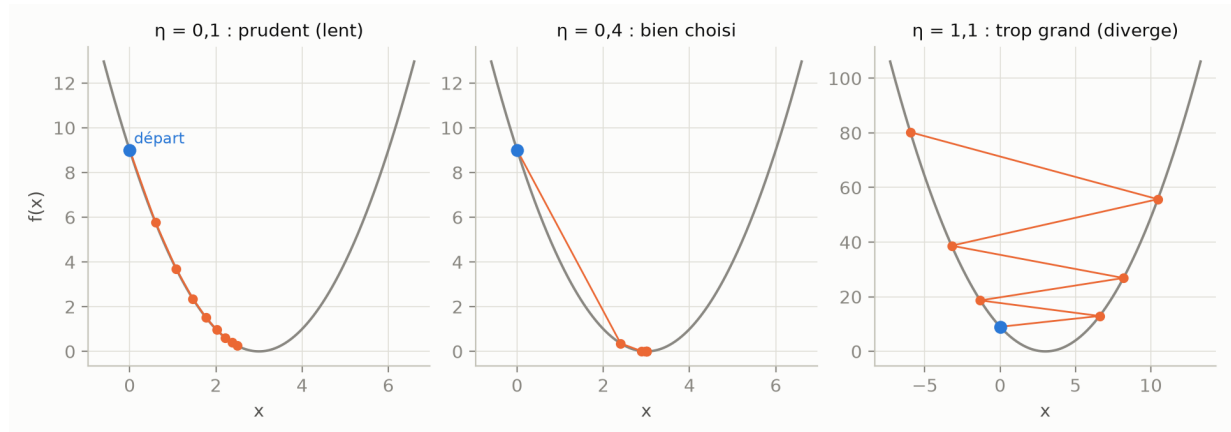


Figure 7: Descente de gradient sur $f(x) = (x-3)^2$ pour trois pas d'apprentissage : prudent (lent), bien choisi (rapide), trop grand (divergence).

Piège : le choix du pas. Un pas trop petit donne un calcul lent ; un pas trop grand fait diverger l'algorithme. En pratique, le pas se règle à l'essai, ou avec des méthodes adaptatives (que nous verrons au volume III).

Descente de gradient en deux dimensions : ajuster une droite

Passons au problème de la droite ajustée aux points $(1,2)$, $(2,3)$, $(3,5)$. Pour la perte $L(\theta) = \|\mathbf{y} - \mathbf{X}\theta\|^2$, le gradient est $-2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\theta)$. L'algorithme complet tient en cinq lignes :

1. choisir un point de départ θ_0 et un pas η ;
2. calculer le gradient $\mathbf{g} = \nabla L(\theta)$;
3. si $\|\mathbf{g}\|$ est presque nul, s'arrêter ;
4. sinon, remplacer θ par $\theta - \eta \mathbf{g}$;
5. recommencer en 2 (au plus un nombre fixé de fois).

Le code complet est dans l'application 1.8 du cahier.

On a vu que la hessienne a pour valeurs propres 0,72 et 33,28. Pour que la descente converge, le pas doit être inférieur à $2/\lambda_{\max} = 2/33,28 \approx 0,060$ (même raisonnement que $0 < \eta < 1$ en une dimension, où la courbure était 2). Essayons un pas de 0,05 (acceptable), puis de 0,07 (trop grand). Voici ce que l'on observe.

Avec $\eta = 0,05$, l'algorithme converge vers $(1,5 ; 0,3333)$, la droite $y = 1,5x + 1/3$, mais il lui faut **335 itérations** pour un problème à trois points. Avec $\eta = 0,07$ (juste au-dessus de la limite 0,060), la perte **explose**.

Pourquoi tant d'itérations ? À cause du bol allongé : le pas est limité par la direction raide (valeur propre 33), mais la progression dans la direction presque plate (valeur propre 0,72) est minuscule. Il existe un remède classique et très simple : **centrer** la variable. Au lieu de $x = (1,2,3)$, on utilise $x - \bar{x} = (-1,0,1)$.

Pourquoi ça aide ? Avec la colonne centrée, le produit $\mathbf{X}^\top \mathbf{X}$ devient **diagonal** : $\begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$. Les deux directions deviennent indépendantes et de courbures comparables (4 et 6 pour la hessienne). Le bol est presque rond.

18 itérations au lieu de 335, pour exactement le même résultat. (On retrouve la droite d'origine par $b = b' - a \bar{x}$.) C'est la raison pour laquelle on **centre et standardise** presque toujours les variables avant d'entraîner un modèle par descente de gradient.

Enfin, pour ce problème précis, une solution **exacte** existe, sans itérer. Le gradient s'annule quand $-2\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\theta) = \mathbf{0}$, c'est-à-dire pour

$$\mathbf{X}^\top \mathbf{X} \theta = \mathbf{X}^\top \mathbf{y},$$

un système linéaire (les « équations normales ») que l'on sait résoudre avec ce que nous avons vu en 1.1.2. Ici $\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} 14 & 6 \\ 6 & 3 \end{pmatrix}$ (de déterminant 6) et $\mathbf{X}^\top \mathbf{y} = (23, 10)^\top$, donc

$$\theta = \frac{1}{6} \begin{pmatrix} 3 & -6 \\ -6 & 14 \end{pmatrix} \begin{pmatrix} 23 \\ 10 \end{pmatrix} = \frac{1}{6} \begin{pmatrix} 9 \\ 2 \end{pmatrix} = \begin{pmatrix} 1,5 \\ 1/3 \end{pmatrix},$$

la même droite $y = 1,5x + 1/3$ que la descente de gradient, trouvée sans itérer.

Pourquoi alors se servir de la descente de gradient ? Parce que **la plupart des modèles n'ont pas de solution exacte** : réseaux de neurones, régression logistique, etc. La descente de gradient fonctionne partout où l'on sait calculer un gradient. La régression linéaire est notre terrain d'entraînement, car on connaît la bonne réponse.

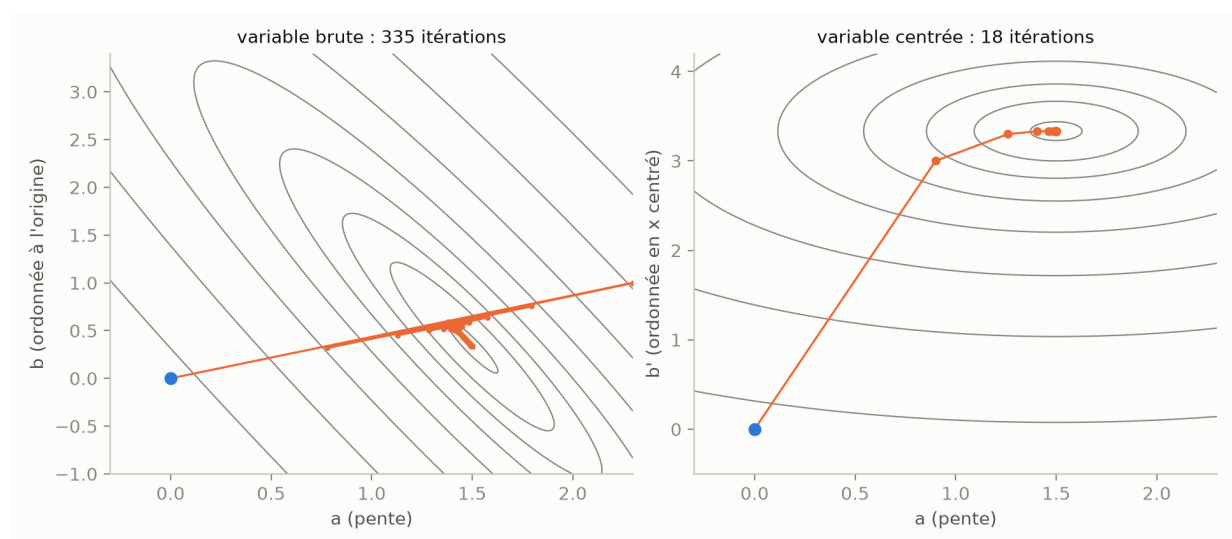


Figure 8: Chemin de la descente de gradient sur les courbes de niveau de la perte, avec la variable brute (à gauche) et la variable centrée (à droite). Le bol allongé force à zigzaguer ; le bol presque rond permet d'aller droit au but.

Pour s'entraîner. Cahier, chapitre 1 : application 1.8, exercices 1.6 et 1.8.

1.3.4 Un exemple de décision : le prix qui maximise les recettes

Voici un petit problème de décision réaliste, qui combine tout le chapitre. L'application 1.9 du cahier le refait pas à pas avec du code. La gérante a testé huit prix pour un même bol en céramique, chacun pendant une semaine :

Prix p (€)	25	28	31	34	37	40	43	46
Ventes q (pièces)	93	82	82	69	67	56	56	47

Étape 1 : modéliser la demande. On suppose une relation linéaire $q = \alpha + \beta p$ et on cherche α et β par descente de gradient. Ici les prix valent environ 35 et les ventes environ 70 : les variables n'ont pas la même échelle. Nous appliquons donc ce que nous venons d'apprendre : **standardiser le prix** avant de descendre.

Treize itérations seulement. Le modèle trouvé est $q \approx 143,9 - 2,11 p$: **chaque euro de hausse fait perdre environ 2,1 ventes par semaine.** (Un ajustement direct par les équations normales redonne exactement les mêmes valeurs.)

Étape 2 : exprimer les recettes. Les recettes sont le prix multiplié par les quantités vendues :

$$R(p) = p \cdot q(p) = p(\alpha + \beta p) = \alpha p + \beta p^2.$$

Étape 3 : maximiser. On dérive et on annule : $R'(p) = \alpha + 2\beta p = 0$, d'où

$$p^* = -\frac{\alpha}{2\beta}.$$

Comme $\beta < 0$, on a $R'' = 2\beta < 0$: c'est bien un maximum.

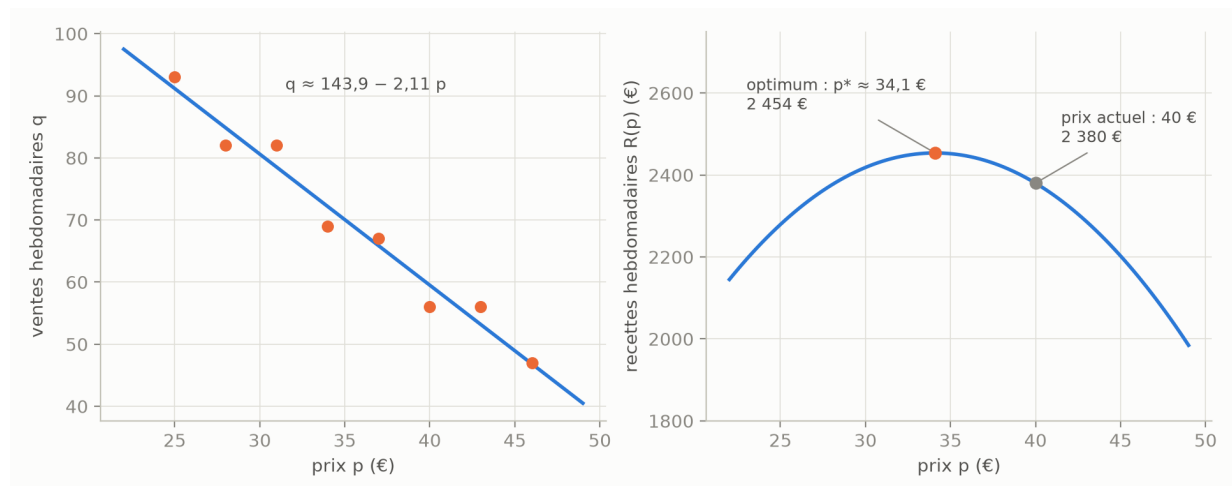


Figure 9: À gauche : les huit mesures et la droite de demande ajustée. À droite : les recettes prévues selon le prix, avec leur maximum vers 34 €.

Résultat. Le prix qui maximise les recettes est d'environ 34 €, avec 2 454 € de recettes hebdomadaires prévues. Au prix actuel de 40 €, elles seraient de 2 380 € : baisser le prix de 6 euros rapporterait un peu plus de 70 € par semaine, soit environ 3 % de mieux.

Prudence. Ce résultat est obtenu avec **huit** points et un modèle très simple. Il ne dit rien de l'incertitude (de combien p^* pourrait-il se tromper ?), ni du bénéfice (ici nous avons maximisé les *recettes*, sans tenir compte des coûts), ni de l'extrapolation hors de la plage de prix testée. Ces questions sont l'objet du chapitre 3 (statistique) et du volume II (régression). Retenez la démarche : **modéliser, écrire la fonction objectif, la dériver, l'annuler.**

Pour s'entraîner. Cahier, chapitre 1 : application 1.9.

1.3.5 Optimisation sous contraintes : les multiplicateurs de Lagrange

Dans la vraie vie, on n'optimise presque jamais librement : on a un **budget**, une capacité, un poids maximal. On cherche alors le meilleur choix **parmi ceux qui respectent une contrainte**.

Intuition. Vous voulez atteindre le point le plus haut d'une colline, mais vous devez rester sur un sentier. Au meilleur point du sentier, vous ne pouvez plus monter en suivant le sentier : celui-ci est **tangent à une courbe de niveau** de la colline. À cet endroit, la direction de plus forte montée de la colline (le gradient de f) est **perpendiculaire au sentier**, donc **parallèle au gradient de la contrainte** (qui, lui aussi, est perpendiculaire au sentier).

Un exemple concret

La gérante dispose de 1 000 € de budget publicitaire à répartir entre deux canaux de publicité, le canal A (x euros) et le canal B (y euros). Elle estime les recettes générées par :

$$R(x, y) = 80\sqrt{x} + 120\sqrt{y}.$$

(La racine carrée traduit des **rendements décroissants** : les premiers euros investis rapportent plus que les derniers.) Elle veut maximiser R sous la contrainte $x + y = 1000$.

La méthode de Lagrange. On introduit un nombre λ (le *multiplicateur*) et on forme le **lagrangien** :

$$\mathcal{L}(x, y, \lambda) = 80\sqrt{x} + 120\sqrt{y} - \lambda(x + y - 1000).$$

On annule toutes ses dérivées partielles :

- $\frac{\partial \mathcal{L}}{\partial x} = \frac{40}{\sqrt{x}} - \lambda = 0$
- $\frac{\partial \mathcal{L}}{\partial y} = \frac{60}{\sqrt{y}} - \lambda = 0$
- $\frac{\partial \mathcal{L}}{\partial \lambda} = -(x + y - 1000) = 0$ (qui redonne la contrainte).

Résolution à la main. Les deux premières équations donnent $\frac{40}{\sqrt{x}} = \frac{60}{\sqrt{y}}$, donc $\sqrt{y} = 1,5\sqrt{x}$, soit $y = 2,25x$. En reportant dans la contrainte : $x + 2,25x = 1000$, donc

$$x = \frac{1000}{3,25} \approx 307,7 \text{ €}, \quad y = 2,25x \approx 692,3 \text{ €}.$$

Recettes : $80\sqrt{307,7} + 120\sqrt{692,3} \approx 1\,403,3 + 3\,157,4 = 4\,560,7 \text{ €}.$

Pourquoi cette méthode marche. Le long de la contrainte, on peut paramétrer $y = 1000 - x$ et regarder R comme fonction d'une seule variable. Au maximum, sa dérivée s'annule, ce qui s'écrit $\nabla R \cdot \mathbf{t} = 0$ où $\mathbf{t}$ est la direction du sentier : ∇R est perpendiculaire au sentier. Or le gradient de $g(x, y) = x + y - 1000$ l'est aussi. Deux vecteurs perpendiculaires à la même direction (en dimension 2) sont parallèles : $\nabla R = \lambda \nabla g$. C'est exactement ce que disent les équations $\partial \mathcal{L} / \partial x = \partial \mathcal{L} / \partial y = 0$. ■

Un solveur numérique, puis une recherche exhaustive le long de la contrainte, donnent le même résultat que la formule : $x = 307,7$, $y = 692,3$ et des recettes de 4 560,70 €.

Le multiplicateur λ a une signification concrète. Au point optimal, $\lambda = 40 / \sqrt{x} \approx 40 / 17,54 \approx 2,28$. C'est le **prix de l'ombre** (*shadow price*) de la contrainte : **un euro de budget supplémentaire rapporterait environ 2,28 € de recettes en plus**, si l'on réoptimise la répartition. Vérification : porter le budget à 1 001 € fait gagner en réalité 2,2798 € de recettes, contre $\lambda \approx 2,2804$.

C'est une information précieuse pour fixer un budget : 1 € de publicité en plus rapporte environ 2,28 € de recettes. Il n'est donc rentable d'augmenter le budget que si la marge brute de la gérante dépasse $1/2,28 \approx 44\%$ (sinon la dépense supplémentaire coûte plus qu'elle ne rapporte).

À retenir (optimisation).

- Apprendre = minimiser une fonction de perte. $\arg \min$ désigne le point où le minimum est atteint.
- Si la fonction est **convexe**, tout minimum local est global. La perte des moindres carrés est convexe (hessienne $2\mathbf{X}^T \mathbf{X}$ positive) ; elle a un fond unique si les colonnes de $\mathbf{X}$ sont indépendantes.
- Descente de gradient : $\theta_{k+1} = \theta_k - \eta \nabla f(\theta_k)$. Le pas η ne doit être ni trop petit (lent) ni trop grand (divergence : $\eta < 2/\lambda_{\max}$ pour une fonction quadratique).
- **Centrer et standardiser** les variables arrondit le bol et accélère considérablement la descente.
- Sous contrainte, on annule les dérivées du lagrangien ; λ mesure la valeur marginale de la contrainte.

Pour s'entraîner. Cahier, chapitre 1 : application 1.10, exercice 1.7.

1.4 La fiche de notations

À quoi sert cette fiche ? Les livres de data science sont pleins de symboles. Le but n'est pas de les apprendre par cœur, mais de **savoir où regarder** quand l'un d'eux vous bloque. Gardez cette page ouverte pendant la lecture de tous les volumes.

1.4.1 Conventions typographiques

Ce que vous voyez	Ce que cela désigne	Exemple
Lettre minuscule italique : x, a, η	un nombre (scalaire)	$x = 3,5$
Minuscule grasse : $\mathbf{x}, \mathbf{y}, \boldsymbol{\theta}$	un vecteur (colonne par défaut)	$\mathbf{x} = (1, 2, 3)^T$
Majuscule grasse : $\mathbf{A}, \mathbf{X}$	une matrice	$\mathbf{X} \in \mathbb{R}^{n \times p}$

Ce que vous voyez	Ce que cela désigne	Exemple
Majuscule italique : X, Y	(au chapitre 2) une variable aléatoire	X = montant d'un panier
Chapeau : $\hat{y}, \hat{\theta}$	une valeur estimée ou prédite	$\hat{y}_i$ = prédiction pour i
Barre : $\bar{x}$	la moyenne	$\bar{x} = \frac{1}{n} \sum x_i$
Étoile : p^*, θ^*	la valeur optimale	$p^* = 34,09$
Exposant T : A^T	la transposée (lignes $\leftrightarrow$ colonnes)	
Exposant -1 : A^{-1}	l' inverse d'une matrice	$AA^{-1} = I$

Attention aux indices. x_i est le i -ème **élément** d'un vecteur, ou la i -ème **observation**. x_{ij} est l'élément de la ligne i , colonne j d'une matrice. Mathématiquement on compte à partir de **1** ; en Python, à partir de **0** : x_1 s'écrit $x[0]$. C'est la source d'erreur n°1 quand on traduit une formule en code.

1.4.2 Ensembles et logique

Symbole	Se lit	Exemple et sens
$\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}$	entiers naturels, relatifs, rationnels, réels	$\mathbb{R}$ = tous les nombres de la droite
$\mathbb{R}^n$	vecteurs de n nombres réels	$\mathbb{R}^3$: l'espace usuel
$\mathbb{R}^{n \times p}$	matrices à n lignes et p colonnes	un tableau de données
$\in$	« appartient à »	$3 \in \mathbb{N}$
$\subset$	« est inclus dans »	$\mathbb{N} \subset \mathbb{R}$
$\cup, \cap$	union, intersection	$A \cup B$: dans A ou B
$\emptyset$	ensemble vide	
$\forall$	« pour tout »	$\forall x \in \mathbb{R}, x^2 \geq 0$
$\exists$	« il existe »	$\exists x, x^2 = 2$
$\Rightarrow$	« implique »	$x > 2 \Rightarrow x > 1$
$\Leftrightarrow$	« si et seulement si »	
$:=$	« est défini comme »	$f(x) := x^2$
$\approx, \propto$	environ égal ; proportionnel à	

1.4.3 Sommes, produits, fonctions

Symbole	Sens	Équivalent NumPy
$\sum_{i=1}^n x_i$	$x_1 + x_2 + \dots + x_n$	<code>x.sum()</code>
$\prod_{i=1}^n x_i$	$x_1 \times x_2 \times \dots \times x_n$	<code>x.prod()</code>
$\bar{x} = \frac{1}{n} \sum x_i$	moyenne	<code>x.mean()</code>
$\max, \min$	plus grande, plus petite valeur	<code>x.max(), x.min()</code>
$\arg \min_{\theta} f(\theta)$	l'argument θ qui rend f minimale	<code>np.argmin(f_values)</code>
$\arg \max$	idem pour le maximum	<code>np.argmax</code>
$ x $	valeur absolue	<code>np.abs(x)</code>
$\lfloor x \rfloor, \lceil x \rceil$	partie entière inférieure, supérieure	<code>np.floor, np.ceil</code>

Symbole	Sens	Équivalent NumPy
$\exp(x) = e^x, \ln x$	exponentielle, log népérien	<code>np.exp, np.log</code>
$1[\text{cond}]$	vaut 1 si la condition est vraie, 0 sinon	<code>(cond).astype(int)</code>
$f : A \rightarrow B$	f prend ses valeurs de A vers B	
$f \circ g$	composition : $(f \circ g)(x) = f(g(x))$	
$\lim_{x \rightarrow a} f(x)$	limite	
$\binom{n}{k}$	« k parmi n »	<code>math.comb(n, k)</code>
$n!$	factorielle	<code>math.factorial(n)</code>

1.4.4 Algèbre linéaire et analyse

Symbole	Sens	Équivalent NumPy
$\mathbf{u} \cdot \mathbf{v} = \mathbf{u}^\top \mathbf{v}$	produit scalaire	<code>u @ v</code>
$\ \mathbf{v}\ $ ou $\ \mathbf{v}\ _2$	norme euclidienne $\sqrt{\sum v_i^2}$	<code>np.linalg.norm(v)</code>
$\ \mathbf{v}\ _1$	norme 1 : $\sum v_i $	<code>np.linalg.norm(v, 1)</code>
$\mathbf{AB}$	produit matriciel	<code>A @ B</code>
$\mathbf{A}^\top$	transposée	<code>A.T</code>
$\mathbf{A}^{-1}$	inverse	<code>np.linalg.inv(A)</code> (ou mieux : <code>solve</code>)
$\mathbf{I}$	matrice identité	<code>np.eye(n)</code>
$\det \mathbf{A}$	déterminant	<code>np.linalg.det(A)</code>
$\text{rang } \mathbf{A}$	nombre de colonnes indépendantes	<code>np.linalg.matrix_rank(A)</code>
$\text{tr } \mathbf{A}$	trace (somme de la diagonale)	<code>np.trace(A)</code>
$\lambda, \mathbf{v}$	valeur propre, vecteur propre : $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$	<code>np.linalg.eig(A)</code>
$\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$	décomposition en valeurs singulières	<code>np.linalg.svd(A)</code>
$f'(x)$ ou $\frac{df}{dx}$	dérivée	
$\frac{\partial f}{\partial x_j}$	dérivée partielle	
∇f	gradient (vecteur des dérivées partielles)	
$\nabla^2 f$ ou $\mathbf{H}$	hessienne (matrice des dérivées secondes)	
$\int_a^b f(x) dx$	intégrale (aire sous la courbe)	<code>scipy.integrate.quad</code>

1.4.5 L'alphabet grec en data science

Les lettres grecques reviennent partout. Voici celles que vous rencontrerez, avec leur usage **habituel** (pas une règle absolue).

Lettre	Nom	Usage courant
α	alpha	niveau de risque d'un test ; ordonnée à l'origine ; paramètre de régularisation
β	bêta	coefficients d'une régression
γ	gamma	pas d'apprentissage (parfois) ; fonction Gamma
δ, Δ	delta	une petite variation ; Δx = écart
ε	epsilon	une très petite quantité ; l' erreur (bruit) d'un modèle
η	êta	pas d'apprentissage (learning rate)
θ	thêta	paramètres d'un modèle (ce que l'on apprend)
λ	lambda	valeur propre ; multiplicateur de Lagrange ; taux d'une loi exponentielle ; force de la régularisation
μ	mu	moyenne d'une loi (théorique)
ν	nu	degrés de liberté
π	pi	3,14159... ; ou une probabilité
ρ	rhô	coefficient de corrélation
σ, Σ	sigma	écart-type (σ) ; matrice de covariance (Σ) ; $\Delta \Sigma$ est la somme, pas la même lettre
τ	tau	un seuil
ϕ, φ	phi	densité de la loi normale (φ) ; une fonction de transformation
Φ	Phi majuscule	fonction de répartition de la loi normale
χ^2	khi-deux	loi et test du khi-deux
ω, Ω	oméga	l'ensemble des issues possibles (Ω)

Astuce pour lire une formule inconnue. (1) Repérez d'abord ce qui est un nombre, un vecteur, une matrice (grâce à la typographie). (2) Cherchez ce qui est *sommé* ou *minimisé*. (3) Remplacez les symboles par un exemple chiffré minuscule ($n = 3$). Une formule devient presque toujours claire sur un exemple à trois éléments.

1.4.6 Exemple : lire une formule du début à la fin

Voici la fonction de perte des moindres carrés, que vous connaissez maintenant :

$$\hat{\theta} = \arg \min_{\theta} \sum_{i=1}^n (y_i - \mathbf{x}_i^{\top} \theta)^2.$$

Lecture mot à mot : « le vecteur de paramètres estimé $\hat{\theta}$ est l'**argument** θ qui **minimise** la somme, sur les n observations, du **carré de l'écart** entre la valeur observée y_i et la prédiction $\mathbf{x}_i^{\top} \theta$ ». Sur nos trois points (1,2), (2,3), (3,5) (section 1.3), cette formule donne $\hat{\theta} = (1,5 ; 1/3)$ et une somme des carrés de $1/6 \approx 0,1667$.

Une formule, une phrase, un exemple chiffré : c'est cette triple lecture qui vous rendra autonome.

1.5 ⊕ Pour aller plus loin : analyse numérique

Section optionnelle. Vous pouvez la sauter sans perdre le fil du livre. Mais si un jour votre code donne un résultat « presque juste » et que vous ne comprenez pas pourquoi, c'est ici qu'est la réponse.

Les mathématiques des sections précédentes travaillent avec des nombres **exacts**. Un ordinateur, lui, n'en a pas : il stocke des nombres avec un nombre **fini** de chiffres. L'**analyse numérique** étudie ce que cela change. Bonne nouvelle : on peut tout comprendre avec quelques expériences.

1.5.1 Pourquoi $0,1 + 0,2$ n'est pas égal à $0,3$

Intuition. En base 10, le nombre $1/3 = 0,3333 \dots$ ne s'écrit pas avec un nombre fini de chiffres. Il en va de même en base 2 pour $0,1$: l'ordinateur en garde une **approximation**, très bonne mais pas exacte.

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
print(f"{0.1:.25f}")
print(f"{0.3:.25f}")
```

```
0.30000000000000004
False
0.1000000000000000055511151
0.2999999999999999888977698
```

Les nombres décimaux de Python (type `float`) suivent la norme IEEE 754 en **double précision** : 64 bits, soit environ **16 chiffres significatifs**. L'erreur relative maximale d'un arrondi est appelée **epsilon machine** ; elle vaut environ $2,2 \times 10^{-16}$.

Règle d'or. On ne compare **jamais** deux flottants avec `==`. On teste s'ils sont *proches* :

```
import math
print(math.isclose(0.1 + 0.2, 0.3))
print(np.isclose(0.1 + 0.2, 0.3))
```

```
True
True
```

Autre piège du même type, que nous avons rencontré en préparant ce livre : demander le logarithme de 1000 en base 10.

```
print(math.log(1000, 10))      # calcule log(1000)/log(10) : deux arrondis
print(math.log10(1000))       # fonction dédiée : exacte ici
```

```
2.9999999999999996
3.0
```

Morale : quand une fonction **dédiée** existe (`log10`, `expm1`, `hypot`...), elle est plus précise que la formule naïve.

1.5.2 Les grands nombres avalent les petits

Additionner un très petit nombre à un très grand peut ne **rien changer** :

```
print(1e16 + 1 == 1e16)
print(1e15 + 1 == 1e15)
```

```
True
False
```

Conséquence pratique : l'**ordre** des additions compte. Additionner un million de fois 0,1 ne donne pas exactement 100 000 : une boucle d'additions successives renvoie 100 000,000 001 33 ..., alors qu'une somme « compensée » (la fonction `math.fsum`) renvoie 100 000,0, exacte à l'arrondi final.

1.5.3 L'annulation catastrophique

C'est le piège le plus important pour un data scientist. Quand on **soustrait deux nombres presque égaux**, les premiers chiffres s'annulent et il ne reste que... du bruit d'arrondi.

Intuition. Vous mesurez deux immeubles de 300,00 m et 300,01 m, avec une règle précise au mètre près. Leur différence (0,01 m) est noyée dans l'imprécision.

Exemple réel : le calcul de la **variance**. Il existe deux formules équivalentes en mathématiques :

- formule stable : $\text{Var} = \frac{1}{n} \sum (x_i - \bar{x})^2$;
- formule « du calcul à la main » : $\text{Var} = \overline{x^2} - \bar{x}^2$ (moyenne des carrés moins carré de la moyenne).

Appliquons les deux à des chiffres d'affaires de l'ordre du milliard, avec une petite dispersion :

```
rng = np.random.default_rng(0)
x = 1e9 + rng.normal(0, 1, size=1000)      # moyenne ~ 1 milliard, écart-type ~ 1

var_stable = np.mean((x - x.mean()) ** 2)
var_naive = np.mean(x ** 2) - x.mean() ** 2

print("formule stable :", round(var_stable, 6))
print("formule naive  :", var_naive)
```

```
formule stable : 0.954046
formule naive  : -128.0
```

La vraie variance vaut environ 1. La formule stable la retrouve (0,95) ; la naïve donne **-128**, une variance *négative*, ce qui est impossible ! Elle est fautive, car $\overline{x^2} \approx 10^{18}$ et $\bar{x}^2 \approx 10^{18}$ sont deux nombres presque égaux dont la différence est de l'ordre de 1 : on soustrait des nombres de 18 chiffres avec seulement 16 chiffres de précision.

Conséquence. NumPy et pandas utilisent des algorithmes stables, pas la formule naïve. Moralité : **faites confiance aux fonctions des bibliothèques** plutôt que de recoder des formules vues en cours.

Pourquoi mathématiquement ? Si a et b sont connus avec une erreur relative ε , leur différence $a - b$ a une erreur **absolue** d'environ $\varepsilon(|a| + |b|)$, donc une erreur **relative** de $\varepsilon \frac{|a| + |b|}{|a - b|}$. Quand $a \approx b$, le quotient est gigantesque : l'erreur relative explose.

1.5.4 Le conditionnement : quand un problème est « fragile »

Certains problèmes sont intrinsèquement sensibles : un minuscule changement des données change beaucoup la réponse. On mesure cette sensibilité par le **nombre de conditionnement** d'une matrice,

$$\kappa(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min}},$$

le rapport entre sa plus grande et sa plus petite valeur singulière (section 1.1.4). Si κ est grand, la matrice est « presque singulière » : c'est exactement le cas de la **multicollinéarité** vue au 1.1.

Voici un système $\mathbf{Ax} = \mathbf{b}$ dont les deux équations sont presque identiques :

```
A = np.array([[1.0, 1.0],
              [1.0, 1.0001]])
b = np.array([2.0, 2.0001])
print("solution :", np.linalg.solve(A, b))
print("conditionnement :", round(np.linalg.cond(A)))
```

```
solution : [1. 1.]
conditionnement : 40002
```

Perturbons légèrement le second membre (changement de 10^{-4}) :

```
b2 = np.array([2.0, 2.0002])
print("solution perturbée :", np.linalg.solve(A, b2))
```

```
solution perturbée : [0. 2.]
```

Un changement de 0,0001 sur les données a **complètement changé la solution** : de (1,1) à (0,2). Le conditionnement est de l'ordre de 40 000 : on peut perdre jusqu'à 4 à 5 chiffres de précision.

Règle empirique. Avec $\kappa \approx 10^k$, on perd environ k chiffres significatifs sur les 16 disponibles.

Deux conséquences pratiques :

1. Pour résoudre $\mathbf{Ax} = \mathbf{b}$, on utilise `np.linalg.solve(A, b)`, **pas** `np.linalg.inv(A) @ b` : c'est plus rapide **et** plus précis.
2. Centrer et standardiser les variables (1.3.3) diminue le conditionnement, ce qui explique en partie le gain de 335 à 18 itérations : sur notre exemple, $\kappa(\mathbf{X}^\top \mathbf{X})$ passe de 46,1 à 1,5.

1.5.5 Deux algorithmes classiques

La méthode de Newton pour résoudre $f(x) = 0$. Idée : on remplace la courbe par sa **tangente** (1.2.1) et on prend l'endroit où la tangente coupe l'axe des abscisses :

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Pour calculer $\sqrt{2}$, on cherche le zéro de $f(x) = x^2 - 2$, avec $f'(x) = 2x$, donc $x_{k+1} = x_k - \frac{x_k^2 - 2}{2x_k}$.

En partant de $x_0 = 1$:

étape	x	erreur
0	1,0000000000000000	$4,1 \times 10^{-1}$
1	1,5000000000000000	$8,6 \times 10^{-2}$
2	1,4166666666666667	$2,5 \times 10^{-3}$
3	1,414215686274510	$2,1 \times 10^{-6}$
4	1,414213562374690	$1,6 \times 10^{-12}$
5	1,414213562373095	0

Le nombre de chiffres justes **double** à chaque étape (on parle de convergence quadratique). C'est ainsi que votre calculatrice calcule vraiment les racines carrées.

La dichotomie. Plus lente mais beaucoup plus robuste : si f est continue et change de signe entre a et b , il existe un zéro entre les deux. On coupe l'intervalle en deux, on garde la moitié qui change de signe, et on recommence. L'erreur est divisée par 2 à chaque tour.

Pour $\sqrt{2}$ sur l'intervalle $[0, 2]$ avec une tolérance de 10^{-10} , il faut 35 étapes à la dichotomie, contre 5 à Newton, pour une précision comparable. Newton est rapide mais peut diverger si on part mal ; la dichotomie est lente mais ne rate jamais. Les bibliothèques (`scipy.optimize.brentq`) combinent les deux.

À retenir (analyse numérique).

- Un flottant a ~ 16 chiffres significatifs ; on compare avec `isclose`, jamais avec `==`.
- Soustraire deux nombres proches détruit la précision (annulation) : préférez les fonctions de bibliothèque.
- Le conditionnement κ mesure la fragilité d'un problème ; on perd environ $\log_{10} \kappa$ chiffres.
- `solve` plutôt que `inv`. Centrer et standardiser améliore le conditionnement.
- Newton : rapide mais capricieux ; dichotomie : lente mais sûre.

Pour s'entraîner. Cahier, chapitre 1 : exercice 1.9.

1.6 ⊕ Pour aller plus loin : mathématiques discrètes et graphes

Section optionnelle. Tout ce qui précède traitait de quantités qui varient de façon *continue* (prix, temps, pentes). Ici, on compte et on relie : des objets distincts, des choix, des liens. Ces outils servent pour les probabilités (chapitre 2), les algorithmes (chapitre 4), les bases de données (chapitre 5) et les systèmes de recommandation.

1.6.1 Ensembles : le langage de base

Un **ensemble** est une collection d'objets distincts, sans ordre. La gérante propose quatre produits : $P = \{\text{bol, tapis, lampe, plateau}\}$. Ses clients de la semaine ont acheté des sous-ensembles de P .

Les opérations de la fiche de notations (1.4.2) se calculent à la main. Prenons deux paniers : $A = \{\text{bol, tapis, lampe}\}$ et $B = \{\text{bol, plateau}\}$. Alors

- l'**union** $A \cup B = \{\text{bol, tapis, lampe, plateau}\}$;
- l'**intersection** $A \cap B = \{\text{bol}\}$;

- la **différence** $A \setminus B = \{\text{tapis, lampe}\}$;
- la **taille** $|A| = 3$.

La similarité de Jaccard. Comment mesurer à quel point deux paniers se ressemblent ? On divise la taille de ce qu'ils ont **en commun** par la taille de ce qu'ils ont **au total** :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Elle vaut 1 si les paniers sont identiques, 0 s'ils n'ont rien en commun. C'est l'équivalent « ensembliste » de la similarité cosinus du 1.1.1.

Un seul produit en commun (le bol), sur quatre produits au total : $1/4 = 0,25$.

1.6.2 Compter : le principe multiplicatif

Intuition. Si vous avez 3 pulls et 4 pantalons, vous avez $3 \times 4 = 12$ tenues. Quand on enchaîne des choix indépendants, on **multiplie** le nombre d'options.

La gérante veut proposer un **coffret cadeau** : un produit principal (4 choix), un emballage (3 choix), une carte message (2 choix). Le nombre de coffrets différents est $4 \times 3 \times 2 = 24$.

Permutations. De combien de façons peut-on **ranger** n objets distincts dans l'ordre ? n choix pour la première place, $n - 1$ pour la suivante, etc. :

$$n! = n \times (n - 1) \times \dots \times 2 \times 1.$$

Les 4 produits peuvent être alignés en vitrine de $4! = 24$ façons.

Arrangements et combinaisons. Choisir k objets parmi n :

- quand l'ordre compte (podium : 1er, 2e, 3e) : $\frac{n!}{(n-k)!}$;
- quand l'ordre ne compte pas (un panier de 3 produits) :

$$\binom{n}{k} = \frac{n!}{k! (n-k)!}.$$

Pourquoi cette formule ? Il y a $\frac{n!}{(n-k)!}$ façons de choisir k objets *dans l'ordre*. Mais chaque groupe de k objets a été compté $k!$ fois (une fois par ordre possible). On divise donc par $k!$.

Quelques valeurs : $\binom{4}{2} = 6$; un podium de 3 parmi 10 compte $10 \cdot 9 \cdot 8 = 720$ possibilités ; et $\binom{40}{3} = \frac{40 \cdot 39 \cdot 38}{3 \cdot 2 \cdot 1} = 9880$.

Attention à l'explosion. Si le catalogue contient 40 produits, il y a déjà 9 880 paniers de trois produits. Pour des paniers de dix produits parmi 40, on dépasse les 847 millions :

C'est pourquoi les algorithmes de recommandation ne peuvent pas **énumérer** tous les cas : on y reviendra avec la complexité (section ⊕ du chapitre 4).

Le triangle de Pascal et le binôme. Les nombres $\binom{n}{k}$ se calculent par la règle $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ et vérifient $(a + b)^n = \sum_k \binom{n}{k} a^k b^{n-k}$. Nous les retrouverons au chapitre 2 dans la **loi binomiale**.

1.6.3 Les graphes : modéliser des relations

Un **graphe** est un ensemble de **sommets** (des objets) et d'**arêtes** (des liens entre eux). Tout ce qui est « réseau » est un graphe : amis sur un réseau social, routes entre villes, produits souvent achetés ensemble, pages web reliées par des liens.

Voici le graphe des produits **achetés ensemble** dans la boutique : deux produits sont reliés si au moins un client les a pris dans le même panier. Les cinq produits sont le bol, le tapis, la lampe, le plateau et le coussin ; les liens sont bol—plateau, bol—tapis, tapis—lampe, tapis—coussin et lampe—coussin. On range un graphe en **liste d'adjacence** : pour chaque sommet, la liste de ses voisins.

Sommet	Voisins	Degré
bol	plateau, tapis	2
tapis	bol, lampe, coussin	3
lampe	tapis, coussin	2
plateau	bol	1
coussin	tapis, lampe	2

Le **degré** d'un sommet est son nombre de voisins : le tapis (degré 3) est le produit le plus « connecté », donc un bon candidat pour une promotion croisée.

La matrice d'adjacence. On peut ranger le graphe dans une matrice $\mathbf{A}$: $A_{ij} = 1$ si i et j sont reliés, 0 sinon. Avec l'ordre bol, tapis, lampe, plateau, coussin :

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{pmatrix}.$$

Elle est **symétrique** : si le bol est lié au tapis, le tapis est lié au bol.

Propriété remarquable : les puissances de $\mathbf{A}$ comptent les chemins. L'élément $(\mathbf{A}^k)_{ij}$ est le **nombre de chemins de longueur k** entre i et j .

Preuve pour $k = 2$. Par définition du produit matriciel, $(\mathbf{A}^2)_{ij} = \sum_m A_{im}A_{mj}$. Le terme $A_{im}A_{mj}$ vaut 1 exactement quand $i—m$ et $m—j$ sont deux arêtes, c'est-à-dire quand m est un intermédiaire possible. La somme compte donc les intermédiaires, c'est-à-dire les chemins en deux pas. Le cas général se démontre par récurrence sur k avec le même argument. ■

Entre le bol et la lampe, il y a un seul chemin en deux pas (bol → tapis → lampe). Le tapis a trois chemins de longueur 2 vers lui-même : c'est son degré (il part vers un voisin et revient). Et la trace de $\mathbf{A}^3$ compte les triangles (chaque triangle est compté 6 fois : 3 sommets de départ × 2 sens) ; ici on trouve 1 triangle : tapis—lampe—coussin. Un triangle dans un graphe de co-achat signale un trio de produits qui se vendent bien ensemble.

1.6.4 Parcourir un graphe : la recherche en largeur

Intuition. Vous lancez une pierre dans l'eau : les ondes atteignent d'abord les voisins directs, puis les voisins des voisins, etc. La **recherche en largeur** (*BFS, breadth-first search*) explore un graphe de la même manière, « cercle par cercle ». Elle trouve le **plus court chemin** (en nombre d'arêtes) entre deux sommets.

L'algorithme utilise une **file** (premier arrivé, premier servi) :

1. Mettre le sommet de départ dans la file, avec la distance 0.
2. Tant que la file n'est pas vide : retirer le premier sommet ; pour chacun de ses voisins non encore vus, noter sa distance (celle du sommet + 1) et le mettre en fin de file.

Depuis le plateau : le bol est à distance 1, le tapis à 2, la lampe et le coussin à 3. Si un client achète un plateau, la « chaîne de recommandations » la plus courte vers une lampe passe par le bol puis le tapis.

À retenir (discret et graphes).

- Ensembles : $\cup, \cap, \setminus$; Jaccard = $|A \cap B| / |A \cup B|$.
- Principe multiplicatif ; $n!$ ordres ; $\binom{n}{k}$ choix sans ordre ; ça explose très vite.
- Un graphe = sommets + arêtes ; le stocker en **liste d'adjacence** ou en **matrice d'adjacence**.
- $(A^k)_{ij}$ = nombre de chemins de longueur k de i à j .
- BFS : exploration par cercles, plus courts chemins sur graphe non pondéré.

Pour s'entraîner. Cahier, chapitre 1 : application 1.11, exercice 1.10.

Bilan du chapitre 1

Vous savez maintenant :

- **représenter** des données comme des vecteurs et des matrices, mesurer des ressemblances (produit scalaire, cosinus), résoudre des systèmes et comprendre le rang ;
- **décomposer** une matrice (valeurs propres, SVD) pour en extraire la structure — l'idée derrière l'ACP ;
- **dériver** et calculer un gradient pour savoir comment une quantité réagit à ses paramètres ;
- **optimiser** : écrire une fonction de perte, la minimiser par descente de gradient, gérer les contraintes avec Lagrange ;
- **lire** une formule grâce à la fiche de notations, et **se méfier** des arrondis de l'ordinateur.

Pour s'entraîner. Le chapitre 1 du *Cahier d'exercices et d'applications* rassemble onze applications guidées et dix exercices corrigés sur tout ce chapitre.

Le chapitre 2 change de point de vue : jusqu'ici, nos nombres étaient certains. Désormais ils seront **aléatoires**, et il faudra apprendre à raisonner dans l'incertitude.

Chapitre 2 : Probabilités

« Le hasard n'est pas le désordre : c'est un ordre qui apparaît quand on regarde **assez de fois**. »

Dans le chapitre 1, tous nos nombres étaient connus avec certitude. Mais les données réelles ne le sont jamais : demain, la gérante vendra peut-être 12 articles, peut-être 25. Un client cliquera ou non sur la publicité. Un paiement sera légitime ou frauduleux. Les **probabilités** sont le langage mathématique de cette incertitude, et la **statistique** (chapitre 3) en est la réciproque : à partir de ce qu'on observe, remonter à ce qui se passe « derrière ».

Le chemin de ce chapitre

- **2.1 Probabilités et formule de Bayes** : calculer des chances, mettre à jour ses croyances quand on apprend quelque chose. C'est le cœur du raisonnement en incertitude, et il est plus subtil qu'il n'y paraît.
- **2.2 Variables aléatoires et lois usuelles** : donner un « visage » au hasard avec les lois de Bernoulli, binomiale, de Poisson, uniforme, exponentielle et normale, et savoir laquelle choisir.
- **2.3 Espérance, variance, covariance** : résumer une loi par quelques nombres, et mesurer comment deux variables bougent ensemble.
- **2.4 Loi des grands nombres et théorème central limite** : les deux résultats qui rendent la statistique possible. Pourquoi une moyenne sur beaucoup d'observations devient fiable, et pourquoi la courbe en cloche est partout.
- **⊕ Pour aller plus loin** : la théorie de la mesure (ce que « probabilité » veut vraiment dire) et les processus stochastiques (le hasard qui évolue dans le temps).

Deux manières de voir une probabilité.

- **Fréquentiste** : $P(\text{pile}) = 0,5$ signifie que, sur un très grand nombre de lancers, environ la moitié donne pile.
- **Bayésienne** : $P(\text{la livraison arrivera demain}) = 0,8$ exprime un **degré de confiance**, même pour un événement qui n'arrivera qu'une fois.

Les règles de calcul sont les mêmes dans les deux cas. Nous utiliserons les deux interprétations selon les situations.

Voir pour croire : la simulation. Beaucoup de résultats de ce chapitre se démontrent. Mais on peut aussi **les voir** en faisant « jouer » l'ordinateur des milliers de fois : la démonstration sert à *comprendre pourquoi*, la simulation à *y croire*. Les figures et les ordres de grandeur de ce chapitre viennent de telles simulations, tirées avec une **graine** (*seed*) fixe : avec la même graine, on obtient exactement les mêmes nombres. Le livre en montre les résultats ; le code corres-

pondant est proposé dans le **cahier d'exercices et d'applications**, qui accompagne chaque chapitre.

Pour s'entraîner. Le chapitre 2 du cahier contient six applications guidées (classer des avis avec Bayes, voir les paradoxes par simulation, dimensionner un échantillon, chaînes de Markov, diversification, dépenses « à zéros ») et dix exercices corrigés. Chaque section de ce chapitre indique en fin de page ce qui s'y rapporte.

2.1 Probabilités et formule de Bayes

Cette section pose le vocabulaire et les règles de calcul des probabilités, puis aboutit à la formule de Bayes, l'outil qui permet de **renverser** une condition : passer de « si le paiement est frauduleux, l'alerte se déclenche » à « puisque l'alerte s'est déclenchée, le paiement est-il frauduleux ? ». Chaque notion est illustrée par un calcul fait à la main.

2.1.1 Le vocabulaire : univers, événements, probabilité

Intuition. Une **expérience aléatoire** est une situation dont l'issue est incertaine : lancer un dé, observer si un visiteur achète, mesurer le temps avant la prochaine commande. On liste d'abord **tout ce qui peut arriver**, puis on attribue à chaque possibilité un nombre entre 0 et 1 qui mesure sa chance.

- L'**univers** Ω (oméga) est l'ensemble de toutes les issues possibles. Pour un dé : $\Omega = \{1, 2, 3, 4, 5, 6\}$.
- Un **événement** est une partie de Ω : « obtenir un nombre pair » est $A = \{2, 4, 6\}$.
- La **probabilité** $P(A)$ est un nombre entre 0 (impossible) et 1 (certain).

Quand toutes les issues sont **équiprobables** (un dé non truqué), on compte :

$$P(A) = \frac{\text{nombre d'issues favorables}}{\text{nombre d'issues possibles}} = \frac{|A|}{|\Omega|}.$$

Ici, la section 1.6 sur le dénombrement nous sert directement : $P(\text{pair}) = 3/6 = 0,5$.

Les trois règles du jeu. Toute la théorie des probabilités découle de trois règles (les *axiomes de Kolmogorov*) :

1. $P(A) \geq 0$ pour tout événement A .
2. $P(\Omega) = 1$: il arrive forcément *quelque chose*.
3. Si A et B sont **incompatibles** (jamais ensemble), alors $P(A \cup B) = P(A) + P(B)$.

Trois conséquences immédiates.

- **Complémentaire** : $P(\bar{A}) = 1 - P(A)$. *Preuve* : A et $\bar{A}$ sont incompatibles et leur union est Ω , donc $P(A) + P(\bar{A}) = P(\Omega) = 1$. ■
- **Union quelconque** : $P(A \cup B) = P(A) + P(B) - P(A \cap B)$. *Preuve* : en additionnant $P(A)$ et $P(B)$ on compte deux fois $A \cap B$; on retire donc une fois ce qui est en trop. (Formellement, on écrit $A \cup B$ comme union des trois morceaux incompatibles $A \setminus B$, $A \cap B$, $B \setminus A$.) ■
- **Monotonie** : si $A \subset B$ alors $P(A) \leq P(B)$.

Exemple chiffré. La gérante envoie une promotion. La chance qu'un client ouvre l'e-mail est $P(O) = 0,4$, celle qu'il visite le site est $P(V) = 0,3$, et celle qu'il fasse les deux est $P(O \cap V) = 0,2$. Quelle est la probabilité qu'il fasse **au moins une** des deux choses ?

$$P(O \cup V) = 0,4 + 0,3 - 0,2 = 0,5.$$

Sans retrancher 0,2, on aurait trouvé 0,7 : on aurait compté deux fois les gens qui font les deux.

2.1.2 Voir une probabilité : la simulation

Une probabilité est la valeur vers laquelle tend la **fréquence** quand on répète l'expérience. Vérifions-le sur un dé, en lançant 10, 100, 1 000, 100 000 fois et en notant la fréquence de « 6 » (qui devrait être proche de $1/6 \approx 0,1667$) :

```
import numpy as np
rng = np.random.default_rng(1)          # la graine fixe la suite de tirages
for n in (10, 100, 1_000, 100_000):
    freq = (rng.integers(1, 7, size=n) == 6).mean() # proportion de 6 parmi n lancers
    print(f"{n:>7} lancers : fréquence de 6 = {freq:.4f}")
```

```
10 lancers : fréquence de 6 = 0.2000
100 lancers : fréquence de 6 = 0.1500
1000 lancers : fréquence de 6 = 0.1650
100000 lancers : fréquence de 6 = 0.1665
```

L'écart à $1/6$ se réduit à mesure que n grandit. Ce phénomène porte un nom, la **loi des grands nombres** ; nous le démontrerons en 2.4. Notez l'idiome de programmation : `(lancers == 6)` crée un tableau de vrai/faux dont la **moyenne** est la proportion de « vrai », car vrai compte pour 1 et faux pour 0. C'est la manière standard d'estimer une probabilité par simulation.

2.1.3 Probabilité conditionnelle : changer d'information

Intuition. Les probabilités dépendent de ce que l'on sait. La probabilité qu'une personne prise au hasard achète est de 20 %. Mais si l'on **sait** qu'elle vient des réseaux sociaux, ce n'est plus la même question : on se restreint à la population de ce canal.

Voici les 1 000 dernières visites de la boutique, classées par canal d'arrivée et par résultat (achat ou non) :

	Achat	Pas d'achat	Total
Réseaux sociaux	60	340	400
Site (recherche)	70	280	350
Boutique (passage)	75	175	250
Total	205	795	1 000

- $P(\text{achat}) = 205/1000 = 0,205$.
- $P(\text{achat et réseaux sociaux}) = 60/1000 = 0,06$.
- Parmi les 400 visiteurs venus des réseaux sociaux, 60 achètent : $P(\text{achat} \mid \text{réseaux}) = 60/400 = 0,15$.

Cette dernière quantité se note $P(A \mid B)$, « probabilité de A sachant B ». Remarquez qu'on peut la retrouver à partir des probabilités globales :

$$P(A | B) = \frac{P(A \cap B)}{P(B)} = \frac{0,06}{0,40} = 0,15.$$

C'est la **définition** de la probabilité conditionnelle (valable si $P(B) > 0$) : on divise par $P(B)$ parce qu'on a **réduit l'univers** à B . Les deux autres canaux donnent $70/350 = 0,20$ (site) et $75/250 = 0,30$ (boutique). Le canal « Boutique » convertit donc deux fois mieux que les réseaux sociaux. Mais attention à ne pas confondre :

$P(A | B) \neq P(B | A)$. Ici, $P(\text{achat} | \text{réseaux}) = 0,15$; mais $P(\text{réseaux} | \text{achat}) = 60/205 \approx 0,29$. Parmi les acheteurs, 29 % viennent des réseaux sociaux, alors que parmi les visiteurs de ce canal, seuls 15 % achètent. Ce sont deux questions différentes. Cette confusion est l'erreur de raisonnement la plus répandue, y compris chez les professionnels (nous en verrons un cas spectaculaire plus bas).

La règle du produit. En réarrangeant la définition :

$$P(A \cap B) = P(A | B) P(B).$$

Pour qu'un visiteur soit **à la fois** issu des réseaux sociaux **et** acheteur, il faut d'abord qu'il vienne de ce canal (0,40), puis qu'il achète sachant cela (0,15) : $0,40 \times 0,15 = 0,06$ ✓.

2.1.4 Indépendance

Deux événements sont **indépendants** si en connaître un ne change rien à la probabilité de l'autre : $P(A | B) = P(A)$, ce qui équivaut à

$$P(A \cap B) = P(A) P(B).$$

Dans le tableau, achat et canal sont-ils indépendants ? Si oui, on aurait $P(\text{achat} | \text{réseaux}) = P(\text{achat})$. Or $0,15 \neq 0,205$: **ils ne sont pas indépendants**. Le canal d'arrivée *informe* sur la probabilité d'achat, et c'est précisément ce qu'on cherche en analyse de données : repérer les variables qui informent sur d'autres.

Indépendance $\neq$ incompatibilité. Deux événements incompatibles ($A \cap B = \emptyset$) de probabilités non nulles sont au contraire **très dépendants** : si A arrive, on est *certain* que B n'arrive pas.

Exemple d'événements indépendants. Deux lancers de pièce successifs : $P(\text{pile puis pile}) = 0,5 \times 0,5 = 0,25$. Plus généralement, pour n événements indépendants, on multiplie.

Exemple : « au moins un ». Chaque colis a 2 % de chances d'être endommagé, indépendamment des autres. Sur 30 colis, quelle est la probabilité qu'**au moins un** soit endommagé ? Le complémentaire est « aucun n'est endommagé », de probabilité $0,98^{30}$. Donc

$$P(\text{au moins un}) = 1 - 0,98^{30} \approx 0,4545.$$

Environ 45 % : avec 30 colis, il est presque *aussi probable qu'improbable* d'avoir au moins un problème, bien que chaque colis soit sûr à 98 %. Passer par le complémentaire est le réflexe à avoir pour tout « au moins un ».

2.1.5 La formule des probabilités totales

Intuition. Pour trouver la probabilité d'un événement, on peut **découper** la population en groupes qui ne se chevauchent pas et dont l'union est tout l'univers (une **partition**), calculer la probabilité dans chaque groupe, puis faire la moyenne **pondérée** par la taille des groupes.

Si $B_1, \dots, B_k$ forment une partition de Ω :

$$P(A) = \sum_{j=1}^k P(A | B_j) P(B_j).$$

Preuve. Les événements $A \cap B_j$ sont deux à deux incompatibles et leur union est A . Par l'axiome 3, $P(A) = \sum_j P(A \cap B_j)$. On applique ensuite la règle du produit à chaque terme : $P(A \cap B_j) = P(A | B_j)P(B_j)$. ■

Exemple. Reprenons le tableau : $P(\text{achat}) = 0,15 \times 0,40 + 0,20 \times 0,35 + 0,30 \times 0,25 = 0,06 + 0,07 + 0,075 = 0,205$. On retrouve bien 205/1000.

2.1.6 La formule de Bayes

Voici le résultat le plus célèbre de ce chapitre. On connaît $P(A | B)$ et on veut **renverser** la condition pour obtenir $P(B | A)$.

Formule de Bayes. Pour $P(A) > 0$ et $P(B) > 0$,

$$P(B | A) = \frac{P(A | B) P(B)}{P(A)}.$$

Preuve. La règle du produit s'écrit de deux façons : $P(A \cap B) = P(A | B)P(B) = P(B | A)P(A)$. En divisant la seconde égalité par $P(A)$, on obtient la formule. ■

Avec la partition $B_1, \dots, B_k$ et la formule des probabilités totales au dénominateur :

$$P(B_i | A) = \frac{P(A | B_i) P(B_i)}{\sum_j P(A | B_j) P(B_j)}.$$

Les trois ingrédients ont des noms qu'on retrouvera tout au long de la data science :

Terme	Nom	Signification
$P(B)$	a priori (<i>prior</i>)	ce que l'on croyait <i>avant</i> d'observer
$P(A B)$	vraisemblance (<i>likelihood</i>)	à quel point l'observation est probable <i>si B</i> est vrai
$P(B A)$	a posteriori (<i>posterior</i>)	ce que l'on croit <i>après</i> avoir observé

Exemple fondateur : l'alerte antifraude. La boutique reçoit des paiements en ligne. Parmi eux, 1 % sont frauduleux. Le système de détection se déclenche pour 95 % des paiements frauduleux (c'est sa *sensibilité*), mais aussi pour 5 % des paiements légitimes (ses *fausses alertes*). Un paiement déclenche l'alerte. **Quelle est la probabilité qu'il soit réellement frauduleux ?**

Réfléchissez avant de lire : beaucoup de gens répondent « environ 95 % ». Calculons.

Méthode des « fréquences naturelles » (la plus intuitive) : imaginons **10 000 paiements**.

- 1 % de fraudes : **100** paiements frauduleux. Le système en repère 95 % : **95** alertes justifiées.
- 99 % de légitimes : **9 900** paiements. Le système se trompe pour 5 % d'entre eux : **495** fausses alertes.
- Au total : $95 + 495 = 590$ alertes, dont seulement 95 sont justifiées.

$$P(\text{fraude} \mid \text{alerte}) = \frac{95}{590} \approx 0,161.$$

Moins de 16 %. Une alerte sur six seulement est une vraie fraude. La formule de Bayes donne exactement la même chose :

$$P(F \mid A) = \frac{P(A \mid F) P(F)}{P(A \mid F) P(F) + P(A \mid \bar{F}) P(\bar{F})} = \frac{0,95 \times 0,01}{0,95 \times 0,01 + 0,05 \times 0,99} = \frac{0,0095}{0,0590} \approx 0,161.$$

Pourquoi est-ce si contre-intuitif ? Parce que la fraude est **rare** (1 %). Même un détecteur très bon produit, sur l'immense masse de paiements honnêtes, beaucoup plus de fausses alertes que de vraies. On appelle cela l'**erreur du taux de base** (*base rate fallacy*). Elle explique pourquoi un test médical « fiable à 95 % » pour une maladie rare donne surtout de faux positifs, et pourquoi il est si difficile de repérer des événements rares (fraude, panne, défaut).

Mettre à jour en continu. Supposons que le **même paiement** soit examiné par un second système indépendant, qui se déclenche aussi. La probabilité *a posteriori* d'hier devient l'*a priori* d'aujourd'hui : on refait le même calcul avec 0,161 à la place de 0,01.

$$P(F \mid 2 \text{ alertes}) = \frac{0,95 \times 0,161}{0,95 \times 0,161 + 0,05 \times 0,839} \approx 0,785.$$

Une troisième alerte, de la même façon, porte la probabilité à environ 0,986. Après une alerte : 16 %. Après deux : 78,5 %. Après trois : 98,6 %. Chaque nouvelle information **déplace** la croyance, et c'est exactement ce que fait un modèle bayésien. Ce schéma « prior → vraisemblance → posterior » est le fondement de l'inférence bayésienne.

2.1.7 Un classifieur à la main : l'idée du filtre naïf de Bayes

La gérante reçoit trop d'avis clients à lire un par un. Elle veut les classer automatiquement en « positif » ou « négatif ». Voici un échantillon de 20 avis déjà étiquetés, où l'on note si le mot « cassé » y apparaît :

	Avis positif	Avis négatif
Nombre d'avis	14	6
... dont contenant « cassé »	1	5

Un nouvel avis contient « cassé ». Est-il positif ou négatif ?

- A priori : $P(\text{pos}) = 14/20 = 0,7$ et $P(\text{nég}) = 0,3$.
- Vraisemblances : $P(\text{cassé} \mid \text{pos}) = 1/14$, $P(\text{cassé} \mid \text{nég}) = 5/6$.

$$P(\text{nég} | \text{cassé}) = \frac{\frac{5}{6} \times 0,3}{\frac{5}{6} \times 0,3 + \frac{1}{14} \times 0,7} = \frac{0,25}{0,25 + 0,05} \approx 0,833.$$

L'avis est négatif avec 83 % de confiance. Le **classifieur naïf de Bayes** étend cette idée à des dizaines de mots à la fois, en supposant (naïvement) que les mots sont indépendants entre eux *sachant la classe*, ce qui permet de **multiplier** leurs vraisemblances. Malgré cette hypothèse fautive, il marche étonnamment bien pour le texte ; nous le retrouverons au volume II.

2.1.8 Trois paradoxes pour s'entraîner à se méfier de l'intuition

Le problème de Monty Hall. Dans un jeu télévisé, trois portes : derrière l'une, une voiture ; derrière les deux autres, une chèvre. Vous choisissez la porte 1. L'animateur, qui **sait** où est la voiture, ouvre une porte où il y a une chèvre (disons la 3) et vous propose de **changer** pour la porte 2. Faut-il changer ?

Beaucoup pensent « c'est du 50/50, donc peu importe ». Une simulation de 100 000 parties donne pourtant environ **1/3** de victoires en restant et **2/3** en changeant. L'explication : votre premier choix a 1/3 de chances d'être le bon, et l'animateur ne change pas cela. Les 2/3 restants se « concentrent » sur l'unique autre porte fermée. On gagne donc en changeant exactement quand le premier choix était *mauvais*, ce qui arrive 2 fois sur 3. (L'animateur apporte de l'information *parce qu'il ne choisit pas au hasard* : il évite la voiture.)

Le paradoxe des anniversaires. Dans une salle de 23 personnes, quelle est la probabilité que deux d'entre elles au moins aient le même anniversaire ? Dites un chiffre avant de continuer. Par le complémentaire (et le principe multiplicatif de 1.6) :

$$P(\text{coïncidence}) = 1 - \frac{365}{365} \cdot \frac{364}{365} \cdots \frac{365 - n + 1}{365}.$$

Personnes	10	23	40	70
$P(\text{au moins une coïncidence})$	0,117	0,507	0,891	0,999

Dès **23** personnes, la probabilité dépasse **50 %** ; à 70, elle est quasi certaine. Contre-intuitif parce qu'on pense à *notre* anniversaire alors que ce sont les $\binom{23}{2} = 253$ **paires** qui comptent. Pour la data science, c'est la clé pour comprendre les **collisions** (deux clients avec le même identifiant haché, deux enregistrements en double) : elles arrivent bien plus tôt qu'on ne l'imagine.

Le paradoxe de Simpson. Une tendance observée dans plusieurs groupes peut s'**inverser** quand on les fusionne. Exemple : la gérante teste deux versions d'une page de paiement, A et B, auprès de clients sur mobile et sur ordinateur.

	Mobile	Ordinateur	Total
Page A	20 / 100 = 20 %	210 / 700 = 30 %	230 / 800 = 28,75 %
Page B	150 / 700 = 21,4 %	40 / 100 = 40 %	190 / 800 = 23,75 %

B est meilleure sur mobile (21,4 % contre 20 %) **et** sur ordinateur (40 % contre 30 %), pourtant A est meilleure au total (28,75 % contre 23,75 %).

L'explication est que la page A a reçu surtout des clients sur ordinateur (qui achètent beaucoup, quelle que soit la page), alors que B a reçu surtout des clients sur mobile (qui achètent moins).

Le total mélange l'effet de la page avec l'effet de l'appareil. La leçon est cruciale : **une moyenne globale peut cacher une variable qui explique tout**. Avant de conclure, on se demande toujours : « quels sous-groupes cachés se mélangent ici ? ». Nous creuserons cette idée de *confusion* au volume II (chapitre 7, facultatif, consacré à l'inférence causale).

À retenir (probabilités et Bayes).

- Une probabilité est un nombre entre 0 et 1 respectant 3 axiomes ; on en déduit $P(\bar{A}) = 1 - P(A)$ et $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.
- Pour un « au moins un », passez par le **complémentaire**.
- $P(A | B) = P(A \cap B) / P(B)$; règle du produit $P(A \cap B) = P(A | B)P(B)$; indépendance $\Leftrightarrow P(A \cap B) = P(A)P(B)$.
- **Probabilités totales** : $P(A) = \sum_j P(A | B_j)P(B_j)$.
- **Bayes** : $P(B | A) = \frac{P(A | B)P(B)}{P(A)}$; *posterior* $\propto$ *vraisemblance* $\times$ *prior*.
- $P(A | B) \neq P(B | A)$, et méfiez-vous de l'**erreur du taux de base** et du **paradoxe de Simpson**.

Pour s'entraîner. Cahier, chapitre 2 : applications 2.1 et 2.2 (paradoxes par simulation), exercices 2.1 à 2.3.

2.2 Variables aléatoires et lois usuelles

Les données sont des **nombres** (un montant, un nombre de commandes, un temps d'attente), pas des événements. Cette section introduit la notion de variable aléatoire, puis passe en revue les six lois qui couvrent l'essentiel des situations pratiques, avec pour chacune une question à se poser pour la reconnaître.

2.2.1 Qu'est-ce qu'une variable aléatoire ?

Intuition. Jusqu'ici nous parlions d'**événements** (« le client achète »). Mais les données sont des **nombres** : le montant du panier, le nombre de commandes, le temps d'attente. Une **variable aléatoire** (v.a.) est simplement **un nombre dont la valeur dépend du hasard**. On la note par une majuscule, X , et ses valeurs possibles par des minuscules, x .

Exemples dans la boutique :

- X = nombre de commandes reçues entre 14 h et 15 h (0, 1, 2, 3, ...) ;
- Y = montant en euros du prochain panier (n'importe quel nombre positif) ;
- $B = 1$ si le prochain visiteur achète, 0 sinon.

On distingue deux familles :

	Variable discrète	Variable continue
Valeurs	une liste dénombrable : 0, 1, 2, ...	tout un intervalle de réels
Exemple	nombre de commandes	temps d'attente, montant exact
Description	fonction de masse $P(X = k)$	densité $f(x)$
Probabilité d'un intervalle	somme des $P(X = k)$	aire sous la densité (intégrale du chapitre 1)

Piège à connaître dès maintenant. Pour une variable continue, la probabilité d'une valeur **exacte** est **zéro** : $P(X = 2,5 \text{ min}) = 0$. Avez-vous déjà mesuré un temps d'attente de *exactement* 2,500000... minutes ? Seuls des **intervalles** ont une probabilité : $P(2 \leq X \leq 3)$. La densité $f(x)$ n'est donc *pas* une probabilité : c'est une probabilité *par unité de longueur* (comme une densité de population se mesure en habitants par km²).

La fonction de répartition. Pour les deux familles, on peut définir

$$F(x) = P(X \leq x),$$

la probabilité de rester **sous** le seuil x . Elle croît de 0 à 1. Elle sert à tout : $P(a < X \leq b) = F(b) - F(a)$, et la **probabilité d'excéder** un seuil est $P(X > x) = 1 - F(x)$. Pour une variable continue, $F(x) = \int_{-\infty}^x f(t) dt$, et inversement $f = F'$: c'est le théorème fondamental du 1.2.4.

Pour faire ces calculs sans effort, on utilise la bibliothèque **SciPy** (`scipy.stats`), qui fournit pour chaque loi les mêmes méthodes :

Méthode	Rôle
<code>.pmf(k)</code> / <code>.pdf(x)</code>	masse (discret) ou densité (continu)
<code>.cdf(x)</code>	fonction de répartition $F(x) = P(X \leq x)$
<code>.sf(x)</code>	« survie » $P(X > x) = 1 - F(x)$ (plus précis que <code>1 - cdf</code>)
<code>.ppf(q)</code>	quantile : la valeur x telle que $F(x) = q$ (inverse de <code>cdf</code>)
<code>.rvs(size, random_state)</code>	simuler des tirages
<code>.mean()</code> , <code>.var()</code> , <code>.std()</code>	espérance, variance, écart-type (section 2.3)

2.2.2 Loi de Bernoulli : le oui/non

C'est la plus simple : une expérience à **deux issues**, « succès » (1) avec probabilité p , « échec » (0) avec probabilité $1 - p$.

$$P(B = 1) = p, \quad P(B = 0) = 1 - p.$$

Le visiteur venu des réseaux sociaux qui achète avec probabilité $p = 0,15$ est une Bernoulli(0,15). On la note $B \sim \text{Bern}(p)$ (le symbole $\sim$ se lit « suit la loi »). C'est la brique de base de toutes les prédictions « oui/non » (clic, achat, fraude, désabonnement).

Exemple. Un visiteur sur cinq achète : $p = 0,2$. Alors $P(B = 1) = 0,2$ et $P(B = 0) = 0,8$. Remarquez deux propriétés qui serviront en 2.3 : comme B ne vaut que 0 ou 1, on a $B^2 = B$, et la moyenne de B sur un grand nombre de visiteurs est simplement la **proportion d'acheteurs**, c'est-à-dire p . C'est pourquoi une probabilité se lit aussi comme un taux observé (par exemple un taux de conversion), et pourquoi les décisions de la boutique reposent en grande partie sur l'estimation d'un seul nombre, p .

2.2.3 Loi binomiale : compter les succès

Intuition. Vous répétez n fois la même expérience de Bernoulli, **indépendamment** ; la loi binomiale décrit le **nombre de succès**.

La gérante reçoit 20 visiteurs venant de la publicité ; chacun achète avec la probabilité $p = 0,2$, indépendamment des autres. Soit X le nombre d'acheteurs. Quelle est la probabilité d'avoir **exactement 4 acheteurs** ?

Construction de la formule. Prenons une configuration précise, par exemple : les 4 premiers visiteurs achètent, les 16 suivants non. Par indépendance, sa probabilité est $p^4(1-p)^{16}$. Mais il y a d'autres configurations avec 4 acheteurs : on choisit **lesquels** des 20 visiteurs achètent, soit $\binom{20}{4}$ façons (section 1.6). Chacune a la **même** probabilité $p^4(1-p)^{16}$, et les configurations sont incompatibles : on additionne.

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad k = 0, 1, \dots, n.$$

Pour $k = 4$: $\binom{20}{4} = 4845$, donc $P(X = 4) = 4845 \times 0,2^4 \times 0,8^{16} \approx 0,218$. Avec SciPy, la même loi s'écrit en une ligne, et chaque méthode du tableau répond à une question :

```
from scipy import stats
X = stats.binom(20, 0.2)           # nombre d'acheteurs parmi 20 visiteurs
print("P(X = 4) =", round(X.pmf(4), 4))
print("P(X >= 6) =", round(X.sf(5), 4))  # sf(5) = P(X > 5)
print("P(X <= 2) =", round(X.cdf(2), 4))
```

```
P(X = 4) = 0.2182
P(X >= 6) = 0.1958
P(X <= 2) = 0.2061
```

Quatre acheteurs est le résultat le plus fréquent (21,8 %), ce qui est logique car $20 \times 0,2 = 4$. Mais **six acheteurs ou plus** arrive avec une probabilité de presque 20 % : un jour « exceptionnel » n'est pas si exceptionnel. Voyez la forme complète :

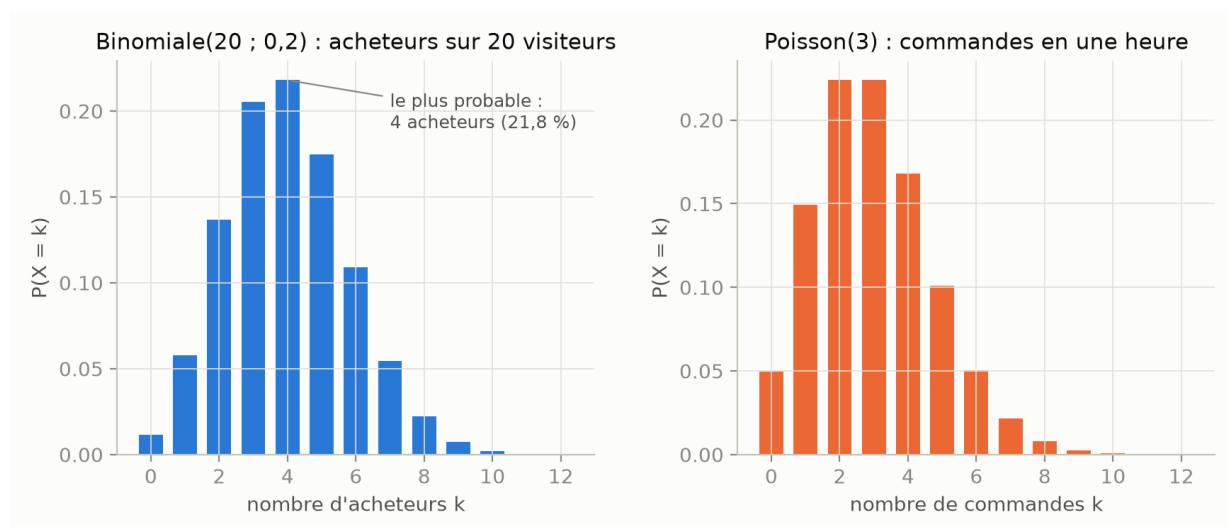


Figure 10: Loi binomiale et loi de Poisson : les probabilités de chaque valeur.

Si l'on simule 100 000 journées de 20 visiteurs, les fréquences observées retombent sur ces valeurs théoriques à quelques millièmes près (21,95 % de journées à exactement 4 acheteurs, 19,56 % à 6

ou plus).

2.2.4 Loi de Poisson : compter des événements rares

Intuition. On compte les **événements qui surviennent au hasard dans le temps ou l'espace** : appels à un standard, commandes par heure, fautes de frappe par page, pannes par mois. On connaît seulement la **cadence moyenne** λ (« en moyenne 3 commandes par heure »).

$$P(X = k) = e^{-\lambda} \frac{\lambda^k}{k!}, \quad k = 0, 1, 2, \dots$$

Si la gérante reçoit en moyenne **3 commandes par heure**, la probabilité de ne **recevoir aucune** commande pendant une heure est $e^{-3} \approx 0,0498$, soit 5 % : environ une heure sur vingt est complètement vide. La probabilité d'en recevoir **exactement 3** est $e^{-3} \cdot 3^3/3! \approx 0,224$, et celle d'en recevoir **six ou plus** vaut environ 8,4 % : utile pour dimensionner le personnel d'emballage.

D'où vient cette formule : Poisson comme limite de la binomiale. Découpons l'heure en n très petits intervalles (disons $n = 3600$ secondes). Dans chacun, une commande arrive avec une probabilité minuscule $p = \lambda/n$ (pour que la moyenne np reste égale à λ). Le nombre de commandes est alors $\text{binomial}(n, \lambda/n)$ et

$$P(X = k) = \binom{n}{k} \left(\frac{\lambda}{n}\right)^k \left(1 - \frac{\lambda}{n}\right)^{n-k} = \frac{\lambda^k}{k!} \cdot \frac{n(n-1) \cdots (n-k+1)}{n^k} \cdot \left(1 - \frac{\lambda}{n}\right)^n \left(1 - \frac{\lambda}{n}\right)^{-k}.$$

Quand $n \rightarrow \infty$: la fraction $\frac{n(n-1) \cdots (n-k+1)}{n^k} \rightarrow 1$ (il y a k facteurs, chacun tend vers 1) ; $(1 - \lambda/n)^n \rightarrow e^{-\lambda}$ (la définition de l'exponentielle) ; $(1 - \lambda/n)^{-k} \rightarrow 1$. Il reste $e^{-\lambda} \lambda^k / k!$.

■

Le calcul numérique confirme que la convergence est rapide : pour $n = 1000$ et $p = 0,003$, la binomiale et la loi de Poisson de paramètre $\lambda = 3$ sont presque indiscernables.

k	0	1	2	3	4	5
Binomiale(1000; 0,003)	0,04956	0,14914	0,22415	0,22438	0,16828	0,10087
Poisson(3)	0,04979	0,14936	0,22404	0,22404	0,16803	0,10082

Règle pratique : quand n est grand et p petit, on peut remplacer $\text{Binomiale}(n, p)$ par $\text{Poisson}(np)$.

2.2.5 Les variables continues : la densité

Pour une variable continue, la loi est donnée par une **densité** f : une fonction positive dont l'aire totale vaut 1, avec

$$P(a \leq X \leq b) = \int_a^b f(x) dx.$$

C'est ici que l'intégrale du chapitre 1 trouve son usage : une probabilité est une **aire**. Trois lois continues essentielles :

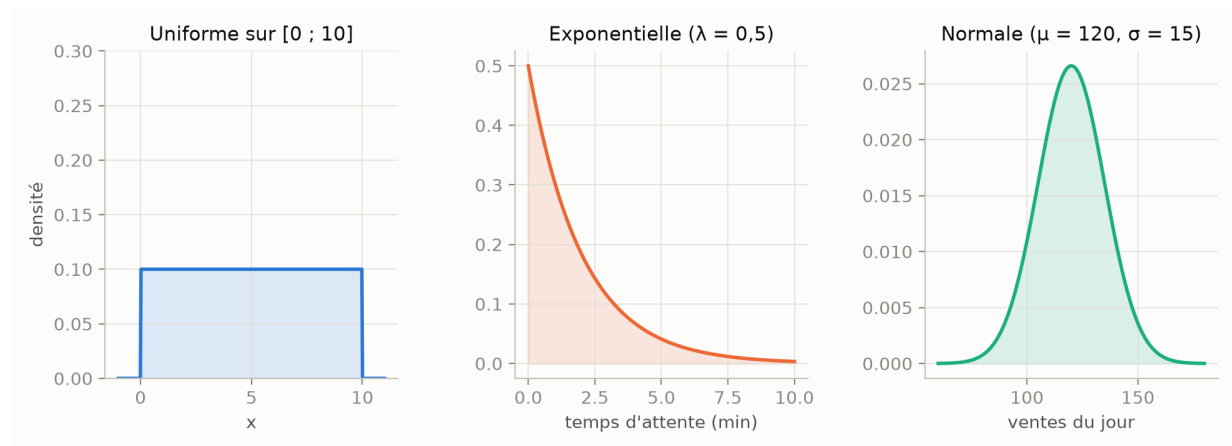


Figure 11: Trois lois continues : uniforme, exponentielle, normale.

Loi uniforme : « aucune préférence »

$X \sim \mathcal{U}(a, b)$: toutes les valeurs de $[a, b]$ sont également probables, de densité $\frac{1}{b-a}$. Un client appelle à une heure uniformément répartie entre 0 et 10 minutes après le début de l'heure : $P(X \leq 3) = 3/10$. Le générateur aléatoire de l'ordinateur produit des $\mathcal{U}(0, 1)$; toutes les autres lois en sont fabriquées.

Loi exponentielle : le temps d'attente

$X \sim \text{Exp}(\lambda)$ avec densité $f(x) = \lambda e^{-\lambda x}$ pour $x \geq 0$. C'est le **temps d'attente** entre deux événements d'un processus de Poisson (nous l'avons rencontrée en 1.2.4). Sa fonction de répartition est $F(x) = 1 - e^{-\lambda x}$, donc

$$P(X > x) = e^{-\lambda x}.$$

Si les clients arrivent à la cadence de $\lambda = 0,5$ par minute (un toutes les 2 minutes en moyenne), la probabilité d'attendre **plus de 3 minutes** le prochain client est $e^{-0,5 \times 3} = e^{-1,5} \approx 0,223$. La **médiane** du temps d'attente est $\ln 2 / \lambda \approx 1,39$ min : inférieure à la moyenne (2 min), car la loi exponentielle est **asymétrique**, avec quelques attentes très longues qui tirent la moyenne vers le haut. Neuf attentes sur dix durent moins de $\ln 10 / \lambda \approx 4,61$ min.

La propriété d'absence de mémoire. Si vous avez déjà attendu 2 minutes sans voir de client, la probabilité d'attendre encore 3 minutes est **la même** que si vous veniez d'arriver. Preuve :

$$P(X > s + t \mid X > s) = \frac{P(X > s + t)}{P(X > s)} = \frac{e^{-\lambda(s+t)}}{e^{-\lambda s}} = e^{-\lambda t} = P(X > t). \blacksquare$$

La première égalité est la définition du conditionnel (car $\{X > s + t\} \subset \{X > s\}$). Le processus « ne se souvient pas » de ce qui s'est passé : il n'y a pas de « retard » à rattraper. (Contre-intuitif pour un bus supposé passer toutes les 10 minutes, mais exact pour des arrivées vraiment aléatoires.)

Loi normale : la courbe en cloche

$X \sim \mathcal{N}(\mu, \sigma^2)$, de densité

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Deux paramètres : μ **centre** la cloche, σ (l'écart-type) mesure son **étalement**. On la rencontre partout : tailles, erreurs de mesure, moyennes d'échantillons (on verra pourquoi en 2.4).

Les ventes quotidiennes de la boutique suivent à peu près $\mathcal{N}(\mu = 120, \sigma = 15)$. Aucune formule fermée n'existe pour la fonction de répartition : on utilise l'ordinateur (`stats.norm(120, 15).sf(150)`, par exemple). Il donne $P(X > 150) \approx 0,0228$ (environ deux jours sur cent), $P(105 < X < 135) \approx 0,683$, et 95 % des jours restent sous 144,7 ventes.

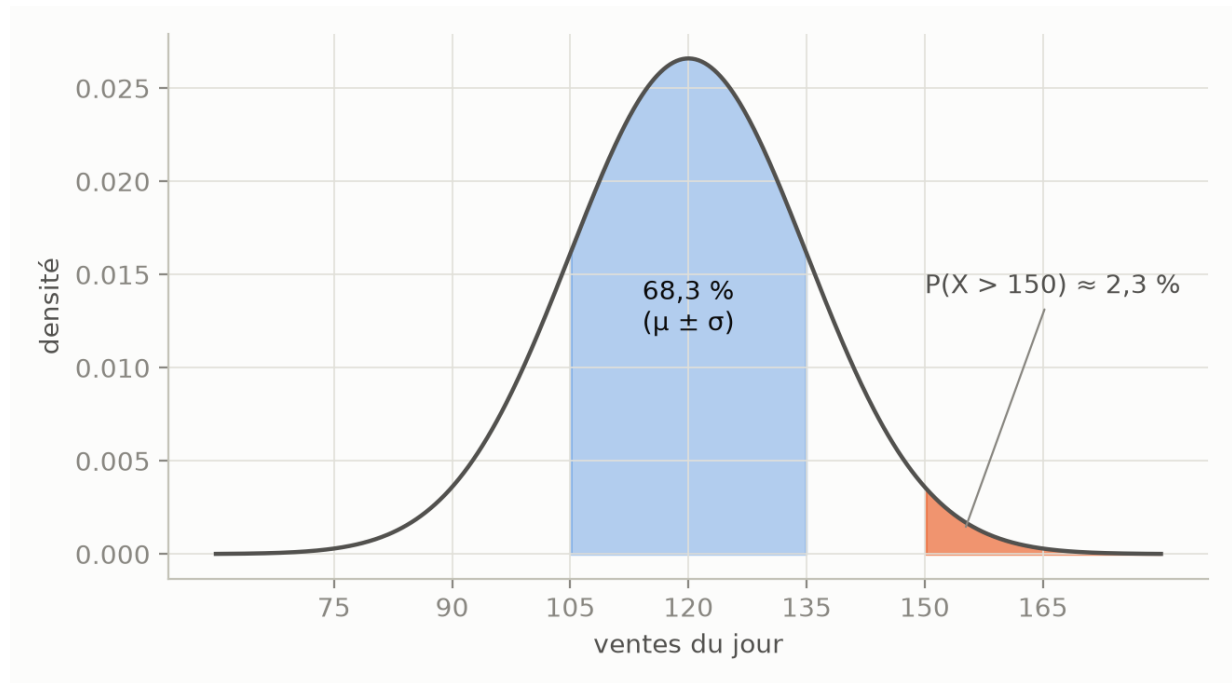


Figure 12: Les ventes quotidiennes suivent $\mathcal{N}(120 ; 15^2)$. Un jour à plus de 150 ventes survient environ 2 fois sur 100.

Centrer et réduire : le score z . Comment comparer des valeurs de lois différentes ? On mesure **combien d'écarts-types** on est du centre :

$$z = \frac{x - \mu}{\sigma}.$$

Si $X \sim \mathcal{N}(\mu, \sigma^2)$, alors $Z = (X - \mu)/\sigma \sim \mathcal{N}(0, 1)$, la **loi normale centrée réduite**. Un jour à 150 ventes correspond à $z = (150 - 120)/15 = 2$: deux écarts-types au-dessus de la moyenne. Toutes les probabilités normales se ramènent à celles de $\mathcal{N}(0, 1)$, dont on retient trois repères :

Intervalle	Probabilité
$\mu \pm 1\sigma$	environ 68 % (0,6827)
$\mu \pm 2\sigma$	environ 95 % (0,9545)
$\mu \pm 3\sigma$	environ 99,7 % (0,9973)

Le quantile qui laisse 2,5 % de probabilité de chaque côté vaut **1,96** : ce nombre apparaîtra sans cesse dans les intervalles de confiance du chapitre 3.

Test express « est-ce normal ? ». Un jour à 190 ventes ($z = 4,67$) serait extraordinairement rare si la loi était vraiment normale (de l'ordre de 1 chance sur 650 000 d'être aussi haut). Si cela arrive, la loi n'est probablement pas la bonne ou un événement particulier s'est produit

(promotion, fête). Détecter les valeurs avec $|z|$ très grand est la méthode de base de détection d'anomalies.

2.2.6 Quelle loi choisir ?

Situation	Loi	Paramètres
un oui/non	Bernoulli	p
nombre de succès sur n essais indépendants	Binomiale	n, p
nombre d'événements dans un intervalle, cadence λ	Poisson	λ
temps d'attente entre deux événements	Exponentielle	λ
aucune préférence sur un intervalle	Uniforme	a, b
somme de nombreux petits effets, mesures	Normale	μ, σ

À retenir (variables aléatoires et lois).

- Une variable aléatoire est un nombre issu du hasard. Discrète : on décrit $P(X = k)$. Continue : on décrit une densité, et les probabilités sont des **aires**.
- La fonction de répartition $F(x) = P(X \leq x)$; avec `scipy.stats` : `pmf/pdf`, `cdf`, `sf`, `ppf`, `rvs`.
- Bernoulli (oui/non), binomiale (compter les succès, $\binom{n}{k}p^k(1-p)^{n-k}$), Poisson (événements rares, $e^{-\lambda}\lambda^k/k!$).
- Exponentielle : temps d'attente, sans mémoire. Normale : cloche, score z , règle 68-95-99,7.
- Simulez pour vérifier vos formules (voir le cahier).

Pour s'entraîner. Cahier, chapitre 2 : exercices 2.4 à 2.6.

2.3 Espérance, variance, covariance

Une loi complète (une courbe, un tableau) est riche mais encombrante. Dans la pratique, on la résume par **quelques nombres** : où est son centre ? De combien s'étale-t-elle ? Comment deux variables évoluent-elles ensemble ? Cette section définit ces trois résumés, démontre leurs propriétés et les applique à des décisions chiffrées.

2.3.1 L'espérance : la moyenne « à long terme »

Intuition. L'**espérance** $E[X]$ est la valeur moyenne que prendrait X si on répétait l'expérience un très grand nombre de fois. C'est une **moyenne pondérée** : chaque valeur est comptée proportionnellement à sa probabilité.

Pour une variable discrète et pour une variable continue (de densité f) :

$$E[X] = \sum_k k P(X = k) \quad \text{et} \quad E[X] = \int_{-\infty}^{+\infty} x f(x) dx.$$

Exemple 1 : le dé. $E[X] = 1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + \dots + 6 \cdot \frac{1}{6} = \frac{21}{6} = 3,5$. Remarquez que l'espérance n'est pas une valeur possible : on n'obtiendra jamais 3,5. C'est un centre de gravité, pas un résultat.

Exemple 2 : une décision. La gérante hésite à lancer une nouvelle lampe. Elle envisage trois scénarios pour le bénéfice du premier trimestre :

Scénario	Probabilité	Bénéfice (€)
Grand succès	0,3	+5 000
Succès moyen	0,5	+1 000
Échec	0,2	−3 000

$$E[X] = 0,3 \times 5000 + 0,5 \times 1000 + 0,2 \times (-3000) = 1500 + 500 - 600 = 1400.$$

Le lancement rapporte **en moyenne** 1 400 €. Une option alternative sûre rapporterait 1 200 €. Faut-il lancer ? L'espérance seule dit oui, mais elle ne dit rien du **risque** : il y a 20 % de chances de perdre de l'argent. C'est exactement le rôle de la variance, ci-dessous.

Propriétés de l'espérance

Linéarité. Pour toutes variables X, Y et constantes a, b :

$$E[aX + b] = aE[X] + b, \quad E[X + Y] = E[X] + E[Y].$$

Preuve de la première (cas discret). $E[aX + b] = \sum_k (ak + b)P(X = k) = a \sum_k kP(X = k) + b \sum_k P(X = k) = aE[X] + b \cdot 1$. ■

La seconde se démontre de la même façon avec une somme double. Elle est **toujours vraie, même si X et Y sont dépendantes** : c'est ce qui la rend si puissante.

Exemple d'usage. Si les ventes du jour ont une espérance de 120 articles à 25 € l'unité, les recettes $25X$ ont pour espérance $25 \times 120 = 3000$ € : on multiplie simplement.

Application élégante : l'espérance d'une binomiale. Une binomiale $X \sim \text{Bin}(n, p)$ est la somme de n Bernoulli : $X = B_1 + \dots + B_n$, avec $E[B_i] = 1 \cdot p + 0 \cdot (1 - p) = p$. Par linéarité,

$$E[X] = E[B_1] + \dots + E[B_n] = np.$$

Sans aucun calcul avec $\binom{n}{k}$! Pour nos 20 visiteurs à 20 % : $E[X] = 4$, ce qu'on avait deviné.

Loi	Espérance
Bernoulli(p)	p
Binomiale(n, p)	np
Poisson(λ)	λ
Exponentielle(λ)	$1/\lambda$
Uniforme(a, b)	$(a + b)/2$
Normale(μ, σ^2)	μ

Attention : $E[XY] \neq E[X]E[Y]$ en général, et $E[f(X)] \neq f(E[X])$ en général. Exemple : pour le dé, $E[X^2] = \frac{91}{6} \approx 15,17$, alors que $(E[X])^2 = 12,25$. L'écart entre ces deux nombres est justement la **variance**.

2.3.2 La variance : mesurer l'étalement

Deux commerçants ont chacun un bénéfice moyen de 1 000 € par mois. Chez l'un, c'est toujours entre 950 et 1 050. Chez l'autre, ça varie de -2 000 à +4 000. Même espérance, risques très différents. La **variance** mesure l'écart typique au centre :

$$\text{Var}(X) = E[(X - \mu)^2], \quad \mu = E[X].$$

On prend le **carré** de l'écart pour que les écarts positifs et négatifs ne se compensent pas. L'**écart-type** $\sigma = \sqrt{\text{Var}(X)}$ ramène le résultat à l'unité d'origine (des euros, et non des euros²).

Formule de calcul (« moyenne des carrés moins carré de la moyenne »).

$$\text{Var}(X) = E[X^2] - (E[X])^2.$$

Preuve. On développe le carré : $(X - \mu)^2 = X^2 - 2\mu X + \mu^2$. Par linéarité, $E[(X - \mu)^2] = E[X^2] - 2\mu E[X] + \mu^2 = E[X^2] - 2\mu^2 + \mu^2 = E[X^2] - \mu^2$. ■

(Rappel de la section $\oplus$ analyse numérique : cette formule est parfaite sur le papier, mais dangereuse sur ordinateur si μ est grand devant σ .)

Retour à la lampe. Calculons $E[X^2]$ puis la variance :

$$E[X^2] = 0,3 \times 5000^2 + 0,5 \times 1000^2 + 0,2 \times 3000^2 = 7,5 \cdot 10^6 + 0,5 \cdot 10^6 + 1,8 \cdot 10^6 = 9,8 \cdot 10^6,$$

$$\text{Var}(X) = 9,8 \cdot 10^6 - 1400^2 = 9,8 \cdot 10^6 - 1,96 \cdot 10^6 = 7,84 \cdot 10^6, \quad \sigma = 2800 \text{ €}.$$

L'écart-type (2 800 €) est **deux fois plus grand** que l'espérance (1 400 €) : l'option est très risquée. Face à l'option sûre à 1 200 €, le gain moyen n'est supérieur que de 200 € alors que le risque est considérable. Beaucoup de gens (et de gérants) préféreraient l'option sûre. Il n'y a pas de « bonne » réponse mathématique : la variance **quantifie** le risque pour que la décision soit éclairée.

Propriétés de la variance

Effet d'un changement d'échelle. $\text{Var}(aX + b) = a^2 \text{Var}(X)$.

Preuve. $E[aX + b] = a\mu + b$, donc $(aX + b) - E[aX + b] = a(X - \mu)$ et $\text{Var}(aX + b) = E[a^2(X - \mu)^2] = a^2 \text{Var}(X)$. ■

Conséquences : ajouter une constante ($+b$) ne change pas l'étalement ; multiplier par a multiplie l'écart-type par $|a|$. Convertir des euros dans une autre monnaie multiplie l'écart-type par le taux de change, et c'est tout.

Somme de variables indépendantes. Si X et Y sont **indépendantes**, $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$. (Nous démontrons le cas général au 2.3.3.)

Variance d'une binomiale. B_i de Bernoulli : $E[B_i^2] = p$ (car $B_i^2 = B_i$), donc $\text{Var}(B_i) = p - p^2 = p(1 - p)$. Pour $X = \sum B_i$ avec des B_i indépendantes : $\text{Var}(X) = np(1 - p)$.

Loi	Variance
Bernoulli(p)	$p(1 - p)$
Binomiale(n, p)	$np(1 - p)$
Poisson(λ)	λ (variance = espérance !)
Exponentielle(λ)	$1/\lambda^2$
Uniforme(a, b)	$(b - a)^2/12$
Normale(μ, σ^2)	σ^2

Une simulation de 200 000 tirages pour chacune de ces lois retrouve les valeurs de ces deux tableaux à moins de 1 % près (par exemple, 4,004 pour l'espérance d'une binomiale(20 ; 0,2) contre 4 en théorie, et 3,209 pour sa variance contre 3,2).

La **loi de Poisson** a la propriété particulière que sa variance est égale à son espérance. Si les commandes de la gérante varient *beaucoup plus* que leur moyenne (on parle de **sur-dispersion**), c'est un signe que le modèle de Poisson est trop simple.

2.3.3 Covariance et corrélation : bouger ensemble

Intuition. Les jours où la gérante dépense plus en publicité, vend-elle plus ? On cherche à mesurer si deux variables **varient dans le même sens**. Chaque jour, on regarde si X est au-dessus de sa moyenne et si Y l'est aussi : si les deux écarts ont le **même signe** la plupart du temps, la covariance est positive.

$$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)] = E[XY] - E[X]E[Y].$$

Exemple à la main. Quatre semaines : dépenses publicitaires $x = (10, 20, 30, 40)$ € et ventes $y = (12, 18, 26, 32)$.

- Moyennes : $\bar{x} = 25, \bar{y} = 22$.
- Écarts à la moyenne : $x - \bar{x} = (-15, -5, 5, 15)$ et $y - \bar{y} = (-10, -4, 4, 10)$.
- Produits : (150, 20, 20, 150), de somme 340, donc covariance = $340/4 = 85$.

Un piège de programmation guette ici : les bibliothèques ne divisent pas toutes par le même nombre.

```
import numpy as np
x = np.array([10, 20, 30, 40.0]); y = np.array([12, 18, 26, 32.0])
print("à la main (+ n) :", ((x - x.mean()) * (y - y.mean())).mean())
print("np.cov (+ n-1):", np.cov(x, y)[0, 1].round(2))
```

```
à la main (+ n) : 85.0
np.cov (+ n-1): 113.33
```

n ou $n - 1$? `np.cov` divise par $n - 1$ (estimateur sans biais, section 3.2) et donne 113,33 ; la formule de la **loi** divise par n . Pour de grands échantillons la différence disparaît ; ici avec $n = 4$ elle est visible. On y reviendra.

Problème : la covariance dépend des unités. Si on mesure les dépenses en centimes, la covariance est multipliée par 100 sans que la relation change ! On **normalise** en divisant par les écarts-types, ce qui donne la **corrélation** de Pearson :

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} \in [-1, 1].$$

Pourquoi ρ est toujours entre -1 et 1 . C'est exactement l'inégalité de Cauchy-Schwarz du chapitre 1 ! Rangez les écarts à la moyenne $(x_i - \bar{x})$ dans un vecteur $\mathbf{u}$ et $(y_i - \bar{y})$ dans $\mathbf{v}$. Alors $\text{Cov} \propto \mathbf{u} \cdot \mathbf{v}$, $\sigma_X \propto \|\mathbf{u}\|$, $\sigma_Y \propto \|\mathbf{v}\|$ (avec le même facteur $1/n$), et

$$\rho = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \cos \theta \in [-1, 1].$$

La corrélation est le cosinus de l'angle entre les deux vecteurs d'écarts. $\rho = 1$: même direction ; $\rho = -1$: directions opposées ; $\rho = 0$: vecteurs perpendiculaires (orthogonaux). ■

Ici : $\rho = \frac{85}{\sqrt{125 \times 58}} \approx 0,998$, une relation presque parfaitement linéaire.

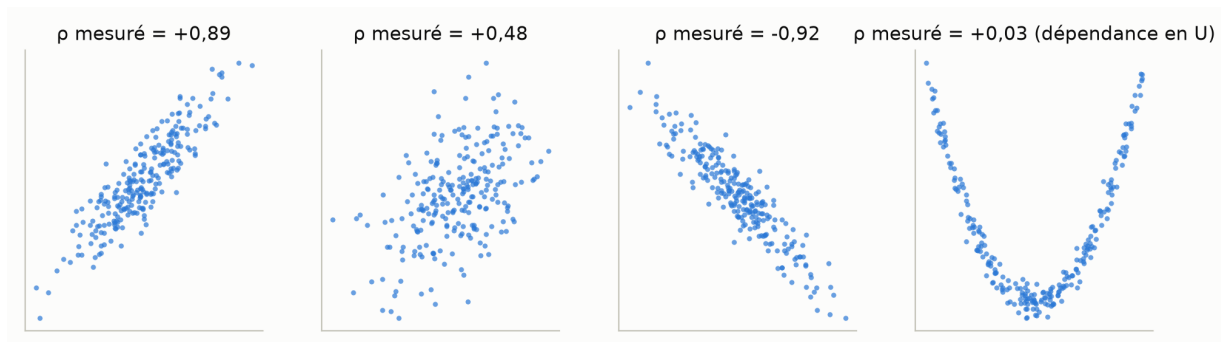


Figure 13: Quatre nuages de points et leur corrélation. Le dernier montre qu'une corrélation nulle n'implique pas l'indépendance.

Trois mises en garde essentielles.

1. **Corrélation n'est pas causalité.** Les glaces et les coups de soleil sont corrélés ; ni l'un ne cause l'autre : c'est la chaleur qui explique les deux. (Le volume II y consacre un chapitre facultatif, l'inférence causale.)
2. **Indépendantes $\Rightarrow$ non corrélées, mais pas l'inverse.** Le dernier nuage de la figure est une parabole : Y est une **fonction exacte** de X (à un peu de bruit près), donc très dépendante, pourtant $\rho \approx 0$. Cela arrive parce que ρ ne détecte que les relations **linéaires**.
3. **Regardez toujours le nuage de points.** Un chiffre unique peut cacher une structure très différente (nous le montrerons avec les « quartets » de la section 3.1).

Variance d'une somme (cas général). $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y)$.

Preuve. Posons $\tilde{X} = X - \mu_X$ et $\tilde{Y} = Y - \mu_Y$. Alors $\text{Var}(X + Y) = E[(\tilde{X} + \tilde{Y})^2] = E[\tilde{X}^2] + 2E[\tilde{X}\tilde{Y}] + E[\tilde{Y}^2]$, ce qui est bien $\text{Var} X + 2\text{Cov}(X, Y) + \text{Var} Y$. ■

Si X et Y sont indépendantes, $\text{Cov} = 0$ et on retrouve l'additivité des variances.

Exemple : la diversification. Les ventes quotidiennes de deux produits, X et Y , ont chacune une moyenne de 50 et un écart-type de 10. Quelle est la variabilité des ventes **totales** $X + Y$?

- Si les deux produits sont **corrélés positivement** ($\rho = +0,5$) : $\text{Var} = 100 + 100 + 2 \times 0,5 \times 100 = 300$, soit $\sigma \approx 17,3$.
- Si les deux produits sont **corrélés négativement** ($\rho = -0,5$: quand l'un se vend mal, l'autre

se vend bien) : $\text{Var} = 100 + 100 - 100 = 100$, soit $\sigma = 10$.

Mêmes moyennes, mêmes écarts-types individuels, mais un total **bien moins variable** (écart-type de 10 au lieu de 17) quand les produits se compensent. C'est le principe de la **diversification** : on réunit des produits (ou des placements) dont les hauts et les bas ne coïncident pas, pour stabiliser le tout. Une simulation de 100 000 jours confirme ces deux valeurs (17,29 et 9,97).

La matrice de covariance

Avec plusieurs variables, on range toutes les variances et covariances dans une **matrice de covariance** Σ : variances sur la diagonale, covariances ailleurs. Elle est **symétrique** et ses valeurs propres sont **positives** ; c'est celle dont nous avons calculé les vecteurs propres au 1.1.3 (aperçu de l'ACP).

La matrice de **corrélation** en est la version « normalisée » : diagonale de 1 et tout entre -1 et 1 . Prenons trois variables mesurées sur 365 jours : la température, le nombre de visites et les ventes. On obtient des corrélations de 0,76 entre température et visites, 0,84 entre visites et ventes, et 0,67 entre température et ventes. La corrélation température-ventes, un peu plus faible, est un effet **indirect** : la température agit sur les visites, qui agissent sur les ventes.

À retenir (espérance, variance, covariance).

- $E[X]$ = moyenne pondérée à long terme ; **linéaire** : $E[aX + b] = aE[X] + b$, $E[X + Y] = E[X] + E[Y]$ toujours.
- $\text{Var}(X) = E[(X - \mu)^2] = E[X^2] - \mu^2$; $\sigma = \sqrt{\text{Var}}$; $\text{Var}(aX + b) = a^2 \text{Var}(X)$.
- $\text{Var}(X + Y) = \text{Var } X + \text{Var } Y + 2 \text{Cov}(X, Y)$: la covariance mesure comment les variables se combinent.
- $\rho = \text{Cov} / (\sigma_X \sigma_Y)$ est un **cosinus** : entre -1 et 1 , sans unité. Il ne mesure que le lien **linéaire**, et ne prouve jamais une causalité.
- La matrice de covariance range toutes les covariances ; c'est l'objet central de l'ACP.

Pour s'entraîner. Cahier, chapitre 2 : application 2.5 (diversification et matrice de covariance), exercices 2.7 et 2.8.

2.4 Loi des grands nombres et théorème central limite

Ces deux résultats sont la **raison d'être de la statistique**. Le premier dit : *avec assez de données, la moyenne observée se rapproche de la vraie valeur*. Le second dit : *et l'erreur qui reste suit presque toujours la même courbe en cloche*. Ensemble, ils permettent de transformer un échantillon en conclusion chiffrée avec une marge d'erreur.

2.4.1 La moyenne d'échantillon est elle-même une variable aléatoire

Observons n clients et notons $X_1, \dots, X_n$ leurs paniers. Ces variables sont **indépendantes et identiquement distribuées** (on écrit **i.i.d.**) : indépendantes entre elles, et issues de la même loi, d'espérance μ et de variance σ^2 . Leur **moyenne d'échantillon** est

$$\bar{X}_n = \frac{X_1 + \dots + X_n}{n}.$$

Un point capital, que beaucoup de débutants manquent : $\bar{X}_n$ **est elle-même aléatoire**. Si on reprend un autre échantillon de n clients, on obtient une autre moyenne. Quelle est sa loi ? Par les propriétés de la section 2.3 :

$$E[\bar{X}_n] = \frac{1}{n} \sum E[X_i] = \mu, \quad \text{Var}(\bar{X}_n) = \frac{1}{n^2} \sum \text{Var}(X_i) = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n}.$$

- La moyenne d'échantillon est **centrée sur la vraie moyenne** μ (on dit qu'elle est **sans biais**).
- Sa dispersion vaut $\sigma/\sqrt{n}$, appelée **erreur-type** (*standard error*). Elle **diminue** quand n augmente, mais seulement comme $1/\sqrt{n}$: pour diviser l'erreur par 2, il faut **4 fois** plus de données ; par 10, il en faut 100 fois plus.

Pourquoi la variance se divise par n ? Quand on moyenne, les écarts positifs d'un client compensent partiellement les écarts négatifs d'un autre. L'aléa « se dilue » : c'est le même phénomène que la diversification vue en 2.3.3.

2.4.2 La loi des grands nombres

Énoncé (loi faible des grands nombres). Si $X_1, X_2, \dots$ sont i.i.d. d'espérance μ et de variance finie, alors pour tout $\varepsilon > 0$,

$$P(|\bar{X}_n - \mu| \geq \varepsilon) \xrightarrow{n \rightarrow \infty} 0.$$

En mots : la probabilité que la moyenne observée s'écarte de μ de plus de ε **tend vers 0**.

Voyons-le en action sur le taux de conversion de la boutique (vraie valeur $p = 0,205$). Quatre « expériences » indépendantes observent les visiteurs un à un et notent la proportion d'acheteurs au fil du temps :

Au début, les courbes sont chaotiques (avec 3 visiteurs, la proportion vaut 0, 33 % ou 67 %...) ; puis elles s'**écrasent** autour de 0,205. C'est la loi des grands nombres.

Démonstration. Elle repose sur une inégalité très utile.

Inégalité de Markov. Pour une variable $Y \geq 0$ et $a > 0$: $P(Y \geq a) \leq E[Y]/a$. *Preuve.* $E[Y] \geq E[Y \cdot \mathbb{1}_{Y \geq a}] \geq a P(Y \geq a)$. ■

Inégalité de Tchebychev. On applique Markov à $Y = (X - \mu)^2$ avec $a = \varepsilon^2$:

$$P(|X - \mu| \geq \varepsilon) \leq \frac{\text{Var}(X)}{\varepsilon^2}.$$

Conclusion. Appliquée à $\bar{X}_n$, dont la variance vaut σ^2/n :

$$P(|\bar{X}_n - \mu| \geq \varepsilon) \leq \frac{\sigma^2}{n \varepsilon^2} \xrightarrow{n \rightarrow \infty} 0. \quad \blacksquare$$

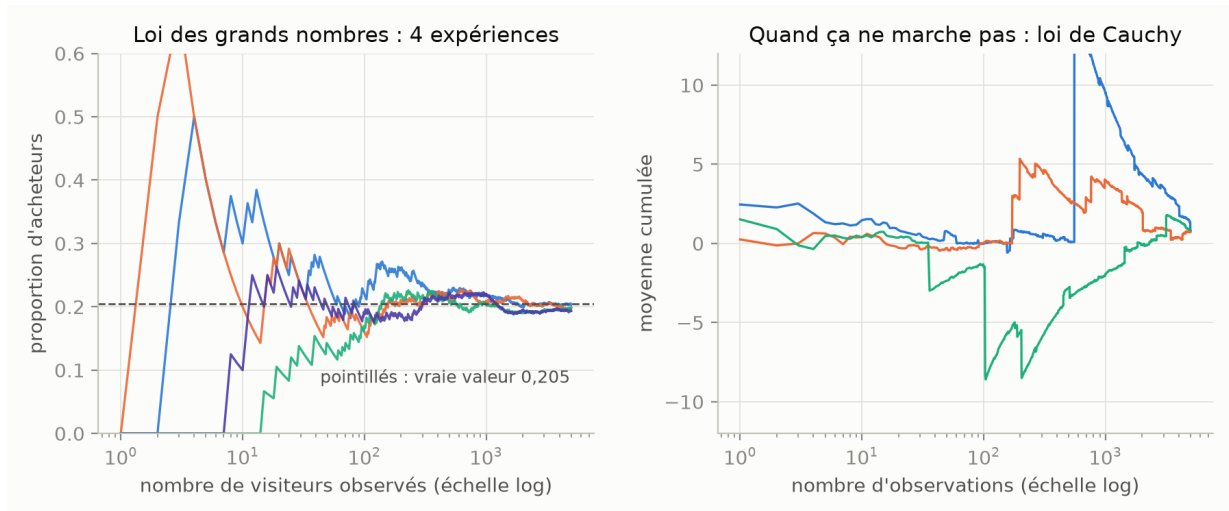


Figure 14: À gauche : la proportion observée se stabilise sur la vraie valeur, quel que soit le départ. À droite : avec une loi de Cauchy, la moyenne ne converge jamais.

La preuve donne même une information **quantitative** : la borne décroît comme $1/n$. Appliquons-la. Combien de visiteurs faut-il observer pour que la proportion mesurée soit à ± 2 points de la vérité avec une probabilité d'au moins 95 % ? Ici $\sigma^2 = p(1 - p) = 0,163$ et $\varepsilon = 0,02$; on veut

$$\frac{0,163}{n \times 0,0004} \leq 0,05, \text{ soit}$$

$$n \geq \frac{0,163}{0,05 \times 0,0004} \approx 8\,149.$$

Tchebychev garantit donc le résultat à partir d'environ **8 150 visiteurs**. C'est une borne **sûre mais très pessimiste** (elle marche pour *n'importe quelle* loi). Le théorème central limite, ci-dessous, donnera beaucoup mieux.

L'erreur du joueur. « La roulette est tombée 5 fois sur rouge, le noir est *dû*. » Faux : la loi des grands nombres ne dit **pas** que le hasard « compense » le passé. Chaque tirage est indépendant. Elle dit que la **proportion** se stabilise parce que les premiers tirages sont **dilués** dans une masse de tirages futurs, pas parce que le futur corrige le passé.

Quand elle échoue : la loi de Cauchy. Le graphique de droite montre la moyenne cumulée de tirages d'une loi de Cauchy, une loi aux queues si lourdes qu'elle **n'a pas d'espérance** : des valeurs gigantesques surviennent régulièrement et ruinent la moyenne. La moyenne ne se stabilise *jamais*. Moralité : les hypothèses d'un théorème comptent. Dans les données réelles (revenus, tailles de fichiers, populations de villes), les **valeurs extrêmes** peuvent rendre la moyenne instable ; on utilise alors la **médiane**, plus robuste.

2.4.3 Le théorème central limite

La loi des grands nombres dit *où* va la moyenne ; le **théorème central limite** (TCL) dit *comment* elle fluctue autour.

Énoncé. Soient $X_1, \dots, X_n$ i.i.d. d'espérance μ et de variance σ^2 finie. Alors, quand n est grand,

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \approx \mathcal{N}(0, 1), \quad \text{c'est-à-dire} \quad \bar{X}_n \approx \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right).$$

La portée est stupéfiante : **quelle que soit la loi d'origine** (asymétrique, discrète, bizarre), la moyenne d'un grand nombre d'observations est **approximativement normale**. C'est la raison pour laquelle la courbe en cloche est partout : *beaucoup de phénomènes sont la somme de nombreux petits effets indépendants*.

Voyons-le. Partons de la loi exponentielle (très asymétrique, voir 2.2.5) et regardons la distribution de la moyenne de $n = 1, 2, 10, 50$ observations, sur 20 000 échantillons. La courbe orange est la loi normale prédite par le TCL.

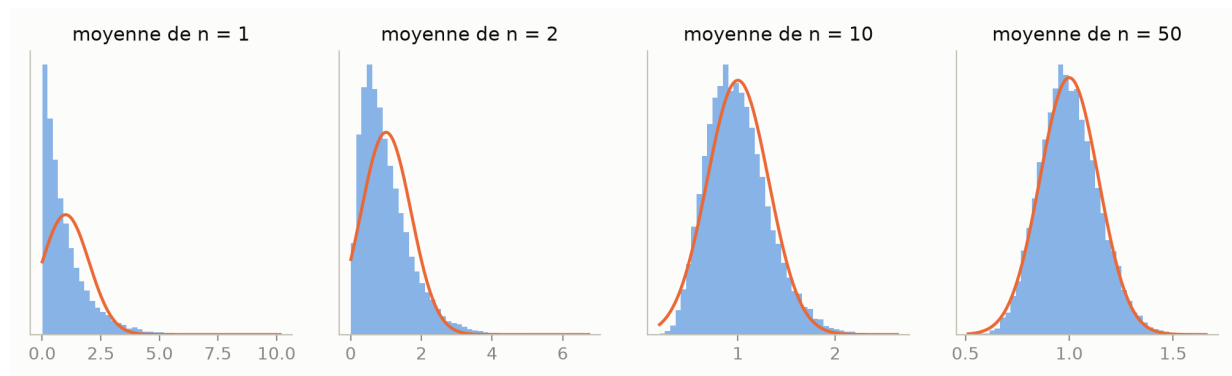


Figure 15: Distribution de la moyenne d'échantillon pour des tirages exponentiels. À $n = 1$ on voit la loi d'origine ; dès $n = 10$ la cloche apparaît ; à $n = 50$ elle est quasi parfaite. La courbe orange est la loi normale prédite par le TCL.

Le même résultat se mesure avec un seul chiffre, l'**asymétrie** (*skewness*) de la distribution, qui vaut 0 pour une cloche parfaite. Sur les mêmes simulations (exponentielle de moyenne 1, donc d'écart-type 1) :

n	1	2	10	50	500
Moyenne des moyennes	0,997	1,005	0,999	1,000	0,999
Écart-type des moyennes	1,004	0,706	0,316	0,141	0,045
Théorie $1/\sqrt{n}$	1,000	0,707	0,316	0,141	0,045
Asymétrie	+2,02	+1,41	+0,59	+0,29	+0,12

On observe trois choses : la moyenne reste à 1 (sans biais) ; l'écart-type suit la loi $1/\sqrt{n}$; l'asymétrie s'efface (de 2 pour $n = 1$ vers 0).

Idée de la preuve (esquisse). On étudie la **fonction génératrice des moments** $M(t) = E[e^{tZ}]$ de la variable centrée réduite $Z_n = \sqrt{n}(\bar{X}_n - \mu)/\sigma$. Par indépendance, $M_{Z_n}(t) = [M(t/\sqrt{n})]^n$ où M est celle d'une variable centrée réduite. Un développement de Taylor donne $M(s) = 1 + \frac{s^2}{2} + o(s^2)$ (le terme en s disparaît car l'espérance est nulle, et le coefficient de s^2 est $\text{Var}/2 = \frac{1}{2}$). Donc $M_{Z_n}(t) = (1 + \frac{t^2}{2n} + o(1/n))^n \rightarrow e^{t^2/2}$, qui est précisément la fonction génératrice de $\mathcal{N}(0, 1)$. Une démonstration complète, avec fonctions caractéristiques, relève de la $\oplus$ théorie de la mesure (section 2.5).

2.4.4 Trois usages du théorème central limite

Usage 1 : la probabilité sur une moyenne. Les paniers de la boutique sont très asymétriques (beaucoup de petits achats, quelques gros) ; supposons-les exponentiels de moyenne 60 €, donc d'écart-type 60 €. La gérante regarde les 40 prochains paniers. Quelle est la probabilité que leur **moyenne dépasse 70 €** ?

Par le TCL, $\bar{X}_{40} \approx \mathcal{N}(60, 60^2/40)$, d'erreur-type $60/\sqrt{40} \approx 9,49$. Le score z est $(70-60)/9,49 \approx 1,05$, d'où $P \approx 0,146$. Une simulation de 200 000 échantillons de 40 paniers donne 0,148 : l'approximation normale est très proche (elle sous-estime très légèrement la queue de droite, à cause de l'asymétrie de la loi d'origine). Observez ce que le TCL a fait : **sans connaître la loi des paniers**, seulement leur moyenne et leur écart-type, on a répondu à une question de probabilité.

Usage 2 : de combien de visiteurs a-t-on besoin ? Reprenons la question du 2.4.2 avec le TCL. La proportion observée $\hat{p} \approx \mathcal{N}(p, p(1-p)/n)$. Avec probabilité 95 %, $\hat{p}$ est à moins de 1,96 erreurs-types de p (le fameux 1,96 du 2.2.5). On veut donc

$$1,96\sqrt{\frac{p(1-p)}{n}} \leq \varepsilon \iff n \geq \left(\frac{1,96}{\varepsilon}\right)^2 p(1-p) = \left(\frac{1,96}{0,02}\right)^2 \times 0,163 \approx 1\,565.$$

Le TCL demande environ **1 565 visiteurs** au lieu de 8 150 : **5 fois moins**. Cette formule est celle des **tailles d'échantillon** des sondages et des tests A/B (chapitre 3). Une simulation vérifie que 1 570 visiteurs suffisent bien : la proportion observée tombe à moins de 2 points de la vérité dans 95,1 % des échantillons.

Usage 3 : la normale approche la binomiale. Une binomiale est une somme de n Bernoulli ; le TCL dit donc que pour n grand, $\text{Bin}(n, p) \approx \mathcal{N}(np, np(1-p))$. Sur 100 visiteurs à 20 % de conversion, quelle est la probabilité d'avoir **au moins 30 acheteurs** ? La binomiale donne exactement 0,0112. L'approximation normale $\mathcal{N}(20, 4^2)$ donne 0,0062 si l'on coupe à 30, mais 0,0088 avec la **correction de continuité** (couper à 29,5 plutôt qu'à 30) : on remplace des barres discrètes par une courbe continue, et la barre « 30 » occupe l'intervalle $[29,5; 30,5]$, ce qui améliore sensiblement l'approximation.

2.4.5 Un mot de prudence

Le TCL est un résultat **asymptotique** : « $\approx$ » devient exact quand $n \rightarrow \infty$. À partir de quelle taille est-ce valable ? Cela dépend de la loi d'origine :

Loi d'origine	n suffisant (règle empirique)
symétrique (uniforme, normale)	quelques unités à 10
modérément asymétrique (exponentielle)	une trentaine
très asymétrique, queues lourdes	des centaines, voire jamais (si la variance est infinie)

À retenir (LGN et TCL).

- $\bar{X}_n$ est une variable aléatoire : $E[\bar{X}_n] = \mu$, $\text{Var}(\bar{X}_n) = \sigma^2/n$, **erreur-type** = $\sigma/\sqrt{n}$. L'erreur diminue en $1/\sqrt{n}$: quatre fois plus de données pour deux fois moins d'erreur.
- **LGN** : $\bar{X}_n \rightarrow \mu$ (preuve par Tchebychev). Elle ne dit rien d'un « rattrapage » du hasard.
- **TCL** : $\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \approx \mathcal{N}(0, 1)$ pour *toute* loi de variance finie. C'est le pont entre les probabilités et la statistique.
- Utilisations : probabilités sur des moyennes, taille d'échantillon $n \geq (1,96/\varepsilon)^2 p(1-p)$,

approximation normale de la binomiale.

- Les hypothèses comptent : avec des queues très lourdes (Cauchy), rien de tout cela ne marche.

Pour s'entraîner. Cahier, chapitre 2 : application 2.3 (dimensionner un échantillon), exercice 2.9.

2.5 ⊕ Pour aller plus loin : la théorie de la mesure (ce que « probabilité » veut vraiment dire)

Section optionnelle, plus abstraite. Elle n'est pas nécessaire pour la suite du livre. Elle répond à une question que se posent les lecteurs curieux : « *jusqu'ici, on a dit « une probabilité est un nombre entre 0 et 1 qui vérifie des règles », mais pour quels événements est-elle définie, et pourquoi ces règles ?* »

2.5.1 Le problème : on ne peut pas mesurer tous les ensembles

Pour un dé, tout est simple : Ω a six éléments et on peut attribuer une probabilité à **n'importe quelle** partie. Mais prenons une variable **uniforme** sur $[0, 1]$: on voudrait que la probabilité d'un intervalle soit sa **longueur** ($P([a, b]) = b - a$). Peut-on étendre cette idée à **tous** les sous-ensembles de $[0, 1]$ en gardant des règles raisonnables (invariance par translation, additivité) ?

Non. En 1905, Giuseppe Vitali a construit des ensembles « pathologiques » auxquels aucune longueur cohérente ne peut être attribuée (la construction utilise l'axiome du choix). Ce n'est pas une curiosité : cela oblige à **limiter** les événements auxquels on attribue une probabilité.

2.5.2 Le cadre : un espace probabilisé $(\Omega, \mathcal{F}, P)$

La théorie moderne (Kolmogorov, 1933) est fondée sur trois objets.

- Ω : l'**univers** (toutes les issues).
- $\mathcal{F}$: une **tribu** (ou σ -algèbre), c'est-à-dire la famille des événements **dont on sait parler**. Elle doit contenir Ω , être stable par complémentaire et par **union dénombrable**. (Pour $[0, 1]$, on prend la tribu **borélienne**, engendrée par les intervalles.)
- P : une **mesure de probabilité** sur $\mathcal{F}$, avec $P(\Omega) = 1$ et la **σ -additivité** : pour des événements $A_1, A_2, \dots$ deux à deux incompatibles,

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i).$$

Notez la différence avec l'axiome 3 du 2.1.1 : on demande l'additivité pour des unions **infinies dénombrables**, pas seulement finies. C'est elle qui rend possibles les passages à la limite (comme la loi des grands nombres).

Intuition. La tribu est la liste des « questions autorisées » : « la variable tombe-t-elle entre 0,3 et 0,4 ? », « est-elle rationnelle ? », « est-elle dans l'union d'une suite d'intervalles ? ». La mesure P répond à chacune par un nombre.

2.5.3 Une conséquence surprenante : événements de probabilité 0 qui arrivent

Soit X uniforme sur $[0, 1]$. Pour tout réel x , $P(X = x) = 0$ (déjà vu en 2.2.1). Et pourtant X prend bien *une* valeur. Un événement de probabilité nulle n'est donc **pas impossible**.

Les rationnels ont une probabilité nulle. Les nombres rationnels de $[0, 1]$ forment un ensemble **dénombrable** : $q_1, q_2, q_3, \dots$. Par σ -additivité,

$$P(X \in \mathbb{Q}) = \sum_{i=1}^{\infty} P(X = q_i) = \sum_{i=1}^{\infty} 0 = 0.$$

Presque sûrement, un nombre tiré au hasard dans $[0, 1]$ est irrationnel ! (On dit qu'un événement est vrai **presque sûrement** (p.s.) quand sa probabilité vaut 1.)

2.5.4 L'espérance comme intégrale

En théorie de la mesure, une **variable aléatoire** est une fonction $X : \Omega \rightarrow \mathbb{R}$ **mesurable** (pour toute question « $X \leq x$? », la réponse est un événement de la tribu). Son espérance est l'**intégrale de Lebesgue** :

$$E[X] = \int_{\Omega} X dP.$$

Cette seule définition recouvre les cas discret ($\sum$) et continu ($\int f$) du 2.3.1, mais aussi les cas **mixtes** que ni l'un ni l'autre ne gère. Voici un exemple pratique.

Un cas réel : les dépenses « à zéros ». Un client visitant la boutique dépense 0 € avec une probabilité de 70 % (il regarde sans acheter) ; sinon sa dépense suit une loi exponentielle de moyenne 80 €. Cette variable n'a **ni** fonction de masse (car elle prend un continuum de valeurs) **ni** densité (car elle a un « atome » en 0 : $P(X = 0) = 0,7 > 0$). C'est une loi **mixte**. Mais son espérance se calcule sans difficulté : on décompose selon le cas.

$$E[X] = 0,7 \times 0 + 0,3 \times 80 = 24 \text{ €}.$$

Une simulation de 500 000 clients confirme ces chiffres : 70,0 % de dépenses nulles, une dépense moyenne de 24,03 € (théorie : 24), et une **médiane égale à 0** (70 % des clients ne dépensent rien).

Les données réelles de commerce sont très souvent de ce type (« zero-inflated »), et c'est la raison pour laquelle on ne peut pas toujours plaquer une loi normale ou exponentielle sans réfléchir.

Un conseil pratique. Pour une loi mixte, la moyenne (24) et la médiane (0) racontent des histoires totalement différentes. Résumer par un seul nombre est trompeur ; il faut présenter la part de zéros et la moyenne conditionnelle aux achats.

2.5.5 Les modes de convergence

Quand on dit « $\bar{X}_n$ converge vers μ », encore faut-il dire **en quel sens**. Pour des variables aléatoires, il existe plusieurs façons, de la plus forte à la plus faible :

Mode	Notation	Signification	Exemple
Presque sûre	$X_n \xrightarrow{p.s.} X$	pour (presque) chaque « histoire » ω , la suite de nombres $X_n(\omega)$ converge au sens usuel	loi forte des grands nombres
En probabilité	$X_n \xrightarrow{P} X$	$P(X_n - X > \varepsilon) \rightarrow 0$	loi faible (démontrée en 2.4.2)
En loi	$X_n \xrightarrow{\mathcal{L}} X$	les fonctions de répartition convergent	TCL

On a : p.s. $\Rightarrow$ en probabilité $\Rightarrow$ en loi (et jamais l'inverse en général). La **loi forte** des grands nombres, plus difficile à démontrer, affirme que, pour *presque* chaque suite d'observations, la moyenne cumulée converge. C'est ce que montre chacune des courbes de la figure de la section 2.4.2 : chaque trajectoire se stabilise.

Pourquoi se soucier de cela ? Pour un praticien, la différence compte surtout dans la formulation des garanties : « avec une probabilité 95 %, mon estimation est à ± 2 points » est un énoncé **en probabilité/en loi** sur *un* échantillon ; « mon estimateur finira par donner la bonne valeur » est un énoncé **presque sûr** sur une suite infinie de données.

2.5.6 La densité comme dérivée d'une mesure

Quand dit-on qu'une variable « a une densité » ? Réponse : quand sa loi P_X est **absolument continue** par rapport à la mesure de Lebesgue (la « longueur »), c'est-à-dire que tout ensemble de longueur nulle a probabilité nulle. Le **théorème de Radon–Nikodym** garantit alors l'existence d'une densité f telle que $P_X(A) = \int_A f(x) dx$. Dans l'exemple des dépenses à zéros, $P_X(\{0\}) = 0,7$ alors que $\{0\}$ est de longueur nulle : pas de densité, ce que nous avons remarqué.

À retenir (théorie de la mesure).

- On ne peut pas assigner de probabilité à *tous* les sous-ensembles : on se limite à une tribu $\mathcal{F}$.
- Un espace probabilisé est $(\Omega, \mathcal{F}, P)$ avec P **σ -additive**.
- Un événement de probabilité 0 n'est pas impossible ; « presque sûrement » = avec probabilité 1.
- Espérance = intégrale de Lebesgue ; elle gère les lois mixtes (atome + densité).
- Trois convergences : presque sûre $\Rightarrow$ en probabilité $\Rightarrow$ en loi (LGN forte, LGN faible, TCL).

Pour s'entraîner. Cahier, chapitre 2 : application 2.6 (dépenses « à zéros »).

2.6 ⊕ Pour aller plus loin : les processus stochastiques

Section optionnelle. Jusqu'ici, une variable aléatoire était un nombre tiré **une fois**. Un **processus stochastique** est une variable aléatoire qui **évolue dans le temps** : $X_0, X_1, X_2, \dots$ Les stocks, les clients actifs, le cours d'une action, le nombre d'appels : tout cela est une suite de

variables aléatoires dépendantes les unes des autres.

2.6.1 La marche aléatoire

Intuition. Chaque jour, le stock d'un produit varie de $+1$ ou -1 de façon aléatoire. Où sera-t-il après 100 jours ?

On part de $S_0 = 0$ et on pose $S_n = S_{n-1} + \xi_n$ où chaque pas ξ_n vaut $+1$ ou -1 avec probabilité $\frac{1}{2}$. Comme $E[\xi_n] = 0$ et $\text{Var}(\xi_n) = 1$, les propriétés du 2.3 (somme de variables indépendantes) donnent

$$E[S_n] = 0, \quad \text{Var}(S_n) = n, \quad \sigma(S_n) = \sqrt{n}.$$

Une marche aléatoire n'a donc **pas de tendance**, mais elle s'écarte de 0 de l'ordre de $\sqrt{n}$. Après 100 pas, on s'attend à être à environ 10 de l'origine, pas à 100 ! C'est la même loi en $\sqrt{n}$ que celle de l'erreur-type.

Une simulation de 20 000 marches de 100 pas confirme la théorie : la moyenne des positions finales est proche de 0 (0,08) et leur écart-type vaut 10,08, pour $\sqrt{100} = 10$ en théorie.

Par le TCL, $S_n / \sqrt{n} \approx \mathcal{N}(0, 1)$: environ 68 % des marches finissent à moins de $\sqrt{n}$ de l'origine (ici, 72 % : un peu plus que 68 %, car la borne ± 10 est incluse et la marche ne prend que des valeurs paires).

La marche aléatoire est partout. Le cours d'une action est souvent modélisé comme une marche aléatoire (le **mouvement brownien**, limite continue de la marche quand les pas deviennent infiniment petits). Elle explique aussi pourquoi les prévisions à long terme sont si incertaines : l'incertitude grandit en $\sqrt{\text{temps}}$.

2.6.2 Les chaînes de Markov : le futur ne dépend que du présent

Intuition. Dans une **chaîne de Markov**, la probabilité de passer à l'état suivant ne dépend que de l'état actuel, pas de la manière dont on y est arrivé :

$$P(X_{n+1} = j \mid X_n = i, X_{n-1}, \dots, X_0) = P(X_{n+1} = j \mid X_n = i) = P_{ij}.$$

La gérante classe chaque mois ses clients en trois états : **Actif** (A : au moins 2 achats ce mois), **Occasionnel** (O : 1 achat) et **Inactif** (I : aucun achat). Elle a estimé les transitions d'un mois au suivant :

de ↓ / vers →	Actif	Occasionnel	Inactif
Actif	0,80	0,15	0,05
Occasionnel	0,30	0,50	0,20
Inactif	0,10	0,20	0,70

On range ces nombres dans la **matrice de transition P** : chaque **ligne** est une loi de probabilité (somme égale à 1).

Où sera un client dans 2 mois ? Un client Actif aujourd'hui peut être Actif dans 2 mois de plusieurs façons : $A \rightarrow A \rightarrow A$, $A \rightarrow O \rightarrow A$, $A \rightarrow I \rightarrow A$. La probabilité totale est $0,8 \times 0,8 + 0,15 \times 0,3 + 0,05 \times 0,1 = 0,64 + 0,045 + 0,005 = 0,69$. Mais c'est **exactement** le produit matriciel de la ligne A par la colonne A de $\mathbf{P}$! En général :

Les probabilités de transition en n pas sont les éléments de $\mathbf{P}^n$. (Même mécanisme que pour compter les chemins d'un graphe au 1.6.3 : la formule des probabilités totales fait apparaître le produit matriciel.)

Avec NumPy, la puissance d'une matrice est un appel de fonction :

```
import numpy as np
P = np.array([[0.80, 0.15, 0.05],      # lignes : état de départ (Actif, Occasionnel, Inactif)
              [0.30, 0.50, 0.20],      # colonnes : état d'arrivée
              [0.10, 0.20, 0.70]])
print(np.linalg.matrix_power(P, 2).round(3))

[[0.69  0.205 0.105]
 [0.41  0.335 0.255]
 [0.21  0.255 0.535]]
```

On retrouve 0,69 en haut à gauche : la probabilité d'être Actif dans 2 mois quand on l'est aujourd'hui.

Et dans 12 mois, ou 2 ans ? On calcule des puissances plus élevées :

Voici la ligne « Actif » de $\mathbf{P}^n$, c'est-à-dire la loi de l'état d'un client **Actif aujourd'hui**, après n mois, puis celle d'un client **Inactif** aujourd'hui :

n (mois)	1	3	6	12	24
Départ Actif : (A, O, I)	(0,800 ; 0,150 ; 0,050)	(0,624 ; 0,227 ; 0,149)	(0,537 ; 0,245 ; 0,218)	(0,503 ; 0,250 ; 0,247)	(0,500 ; 0,250 ; 0,250)
Départ Inactif : (A, O, I)	(0,100 ; 0,200 ; 0,700)	(0,298 ; 0,266 ; 0,436)	(0,437 ; 0,258 ; 0,305)	(0,494 ; 0,251 ; 0,255)	(0,500 ; 0,250 ; 0,250)

Observez : à mesure que n grandit, **toutes les lignes deviennent identiques**. Le système « oublie » son point de départ : qu'un client ait commencé Actif ou Inactif, sa probabilité d'être dans chaque état dans 2 ans est la même. Cette loi limite π s'appelle la **distribution stationnaire**.

La calculer exactement : valeurs propres, encore ! La loi stationnaire vérifie $\pi \mathbf{P} = \pi$: elle ne change plus après une transition. En transposant, $\mathbf{P}^T \pi^T = \pi^T$: c'est un **vecteur propre de $\mathbf{P}^T$ pour la valeur propre 1** (section 1.1.3).

On peut **vérifier à la main** que $\pi = (0,5 ; 0,25 ; 0,25)$ convient. Colonne Actif : $0,5 \times 0,8 + 0,25 \times 0,3 + 0,25 \times 0,1 = 0,4 + 0,075 + 0,025 = 0,5$ ✓. Colonne Occasionnel : $0,5 \times 0,15 + 0,25 \times 0,5 + 0,25 \times 0,2 = 0,075 + 0,125 + 0,05 = 0,25$ ✓. Colonne Inactif : $0,5 \times 0,05 + 0,25 \times 0,2 + 0,25 \times 0,7 = 0,025 + 0,05 + 0,175 = 0,25$ ✓. Pour la **trouver** quand on ne la connaît pas, on résout ce système linéaire (avec $\pi_A + \pi_O + \pi_I = 1$), ou on demande à l'ordinateur le vecteur propre de $\mathbf{P}^T$ pour la valeur propre 1.

À long terme, exactement **50 %** des clients sont Actifs, **25 %** Occasionnels et **25 %** Inactifs. Les deux autres valeurs propres de $\mathbf{P}$ (0,673 et 0,327, de module < 1) pilotent la **vitesse** de convergence : plus elles sont petites, plus vite le système oublie son passé.

Une application économique : la valeur à long terme d'un client. Supposons qu'un client Actif rapporte en moyenne 30 € par mois, un Occasionnel 10 €, un Inactif 0 €. À long terme, le revenu

moyen mensuel par client est $\pi \cdot \mathbf{v}$:

Soit **17,5 € par client et par mois** : $\pi \cdot \mathbf{v} = 0,5 \times 30 + 0,25 \times 10 + 0,25 \times 0 = 15 + 2,5 = 17,5$. Cette quantité permet de **chiffrer** l'effet d'une campagne : si une relance fait passer la probabilité Inactif→Actif de 0,10 à 0,20, il suffit de modifier $\mathbf{P}$ et de recalculer π . C'est un modèle simple, mais l'idée (états, transitions, régime permanent) est utilisée en analyse de la fidélité, en marketing, et à la base de l'algorithme PageRank (le web est une chaîne de Markov dont les états sont les pages).

Pour s'entraîner. L'application 2.4 du cahier mesure l'effet de cette relance sur le revenu à long terme.

2.6.3 Le processus de Poisson : des arrivées au hasard

Un **processus de Poisson** de cadence λ modélise des événements arrivant au hasard et indépendamment : appels, commandes, pannes. Il a deux visages **équivalents**, que nous avons déjà croisés :

- le **nombre** d'événements dans une durée t suit une loi de **Poisson**(λt) (2.2.4) ;
- les **temps d'attente** entre événements successifs sont **exponentiels**(λ), indépendants (2.2.5).

Vérifions que ces deux visages coïncident : on construit le processus **uniquement** par ses temps d'attente exponentiels (de cadence 3 par heure), puis on compte les arrivées dans chaque heure. Sur 50 000 heures simulées, la moyenne des comptes vaut 3,005 et leur variance 2,98 (théorie : 3 et 3, comme il se doit pour une loi de Poisson). Les fréquences observées collent à la loi de Poisson(3) :

Nombre de com- mandes k	0	1	2	3	4	5	6
Fréquence simulée	0,0492	0,1491	0,2228	0,2241	0,1687	0,1023	0,0511
Poisson(3)	0,0498	0,1494	0,2240	0,2240	0,1680	0,1008	0,0504

Les deux descriptions sont bien le même objet. C'est pourquoi les files d'attente (guichets, serveurs, centres d'appels) se modélisent presque toujours avec ce processus.

À retenir (processus stochastiques).

- Un processus stochastique est une famille (X_t) de variables aléatoires indexée par le temps.
- **Marche aléatoire** : $E[S_n] = 0$, $\sigma(S_n) = \sqrt{n}$; l'incertitude croît en $\sqrt{\text{temps}}$.
- **Chaîne de Markov** : le futur ne dépend que du présent ; probabilités en n pas = $\mathbf{P}^n$; **loi stationnaire** = vecteur propre de $\mathbf{P}^T$ pour la valeur propre 1.
- **Processus de Poisson** : nombres de Poisson $\Leftrightarrow$ temps d'attente exponentiels.

Pour s'entraîner. Cahier, chapitre 2 : applications 2.2 (processus de Poisson simulé) et 2.4 (chaîne de Markov), exercice 2.10.

Bilan du chapitre 2

Vous savez maintenant :

- **calculer** des probabilités (complémentaire, union, conditionnel) et **inverser** un conditionnement avec Bayes, en vous méfiant de l'erreur du taux de base ;
- **modéliser** un phénomène par la bonne loi (Bernoulli, binomiale, Poisson, exponentielle, normale) et calculer des probabilités avec `scipy.stats` ;
- **résumer** une loi par son espérance et sa variance, et mesurer le lien entre deux variables par la covariance et la corrélation ;
- **comprendre** pourquoi une moyenne devient fiable (LGN) et pourquoi son erreur est normale (TCL), avec une erreur-type en $\sigma/\sqrt{n}$;
- (en option) **situer** tout cela dans le cadre de la théorie de la mesure et des processus stochastiques.

Pour s'entraîner. Le chapitre 2 du cahier rassemble six applications guidées et dix exercices corrigés, classés par difficulté (★, ★★, ★★★).

Le chapitre 3 retourne le problème : on ne **connaît** plus la loi, on a seulement des **données**, et il faut en déduire la loi. C'est la statistique.

Chapitre 3 : Statistique

« Les probabilités vont de la **cause** vers les **données**. La statistique fait le chemin inverse : des **données** vers la cause. »

Au chapitre 2, nous *connaissions* la loi (par exemple « le taux de conversion est 20,5 % ») et nous calculions la probabilité d'observer certaines données. Dans la vie réelle, c'est l'inverse : on **observe** des données (400 commandes) et on veut deviner la loi (« quel est le vrai panier moyen ? Le canal Réseaux est-il vraiment moins rentable que la boutique ? »). C'est le travail de la **statistique**.

Le chemin de ce chapitre

- **3.1 Statistique descriptive** : résumer et regarder les données (moyenne, médiane, quantiles, graphiques) avant toute chose.
- **3.2 Estimation** : déduire un paramètre inconnu d'un échantillon (méthode des moments, maximum de vraisemblance), et juger la qualité d'un estimateur.
- **3.3 Intervalles de confiance** : ne pas donner un seul chiffre, mais une **fourchette** honnête.
- **3.4 Tests d'hypothèses** : décider, avec un risque maîtrisé, si un effet observé est réel ou dû au hasard (tests de moyenne, de proportion, du khi-deux, A/B).
- **3.5 p-valeurs, puissance et tests multiples** : bien interpréter les résultats, dimensionner une expérience, éviter les faux positifs.
- **⊕ Pour aller plus loin** : les sondages (comment échantillonner), et les méthodes non paramétriques (quand on ne veut pas supposer de loi).
- **Bilan du chapitre**, puis, dans le **cahier d'exercices**, des applications guidées et des exercices corrigés.

Le fil conducteur : un jeu de 400 commandes. Tout au long du chapitre nous travaillons sur un même tableau de 400 commandes de la boutique (canal, montant, délai de livraison, satisfaction), fourni dans `donnees/commandes.csv`. Il est **simulé** (graine fixe) pour que vous puissiez reproduire chaque calcul, et vous verrez qu'on y retrouve des phénomènes tout à fait réalistes : montants asymétriques, différences entre canaux, lien entre délai et satisfaction.

Peu de code dans ce chapitre. Les formules, les démonstrations et les exemples calculés à la main portent le contenu ; les résultats chiffrés viennent de calculs réalisés sur le jeu de données (reproductibles, fournis avec le livre). Le code n'apparaît que lorsqu'un appel court est instructif : nous utilisons `pandas` pour les tableaux (étudié en détail à la section 4.4) et `scipy.stats` pour les tests. Les simulations complètes et les applications guidées sont dans le **cahier d'exercices**.

3.1 Statistique descriptive

Intuition. Avant de modéliser, de tester ou de prédire quoi que ce soit, on **regarde** les données. La statistique descriptive, c'est l'art de résumer un tableau de centaines de lignes en quelques nombres et quelques graphiques *fidèles*. C'est aussi la meilleure façon de repérer des erreurs de saisie, des valeurs aberrantes et des surprises, **avant** qu'elles ne faussent une analyse.

3.1.1 Population, échantillon, variables

Deux mots que nous utiliserons constamment :

- La **population** est l'ensemble complet qui nous intéresse (*toutes* les commandes passées et à venir de la boutique).
- L'**échantillon** est la partie que l'on a effectivement observée (nos 400 commandes).

On calcule des **statistiques** sur l'échantillon pour apprendre des choses sur les **paramètres** de la population, qui eux restent inconnus. (On notera $\bar{x}$ la moyenne de l'échantillon et μ celle de la population.)

Chaque colonne d'un tableau est une **variable**. Son type décide des calculs et des graphiques qui ont un sens :

Type	Exemple	Résumés adaptés
Quantitative continue	montant (€)	moyenne, médiane, écart-type, histogramme
Quantitative discrète	nombre d'articles, délai en jours	idem, ou fréquences de chaque valeur
Qualitative nominale	canal (Réseaux / Site / Boutique)	effectifs, proportions, diagramme en barres
Qualitative ordinale	satisfaction (1 à 5)	effectifs, médiane, quantiles (la moyenne est discutable)

La moyenne d'une variable ordinale (satisfaction de 1 à 5) est très répandue mais n'a pas de sens strict : l'écart entre 1 et 2 est-il le même qu'entre 4 et 5 ? On la calcule quand même par convention, mais en gardant ceci en tête.

3.1.2 Le jeu de données du chapitre

Tout le chapitre repose sur **un seul tableau de 400 commandes**. Il est **simulé** avec une graine fixe (de sorte que chaque nombre du chapitre est reproductible) et il est fourni dans le fichier `donnees/commandes.csv`. Chaque ligne est une commande :

Variable	Type	Contenu
canal	qualitative nominale	canal de vente : Boutique, Site ou Réseaux (réseaux sociaux), tirés avec les probabilités 25 %, 35 %, 40 %

Variable	Type	Contenu
montant	quantitative continue	montant de la commande en € (loi log-normale, plus élevé en boutique)
livraison	quantitative discrète	délai de livraison en jours (0 pour un retrait en boutique)
satisfaction	qualitative ordinale	note de 1 à 5, qui baisse d'environ 0,18 point par jour de retard

Les trois premiers gestes à faire sur n'importe quel tableau : regarder ses dimensions (400 lignes, 4 colonnes), vérifier les types et les valeurs manquantes (il n'y en a aucune), puis demander un résumé numérique.

```
import pandas as pd

df = pd.read_csv("donnees/commandes.csv")
print(df.describe().round(2))
```

	montant	livraison	satisfaction
count	400.00	400.00	400.00
mean	60.25	3.29	3.96
std	38.02	2.56	0.80
min	8.60	0.00	1.00
25%	34.18	0.00	3.00
50%	51.00	3.00	4.00
75%	75.82	5.00	5.00
max	255.70	13.00	5.00

La méthode `describe()` donne d'un coup : effectif (count), moyenne (mean), écart-type (std), minimum, quartiles (25 %, 50 %, 75 %) et maximum. Aucune valeur manquante. Le montant moyen est d'environ 60 €, mais le **maximum dépasse 250 €** alors que la **médiane (50 %)** est d'environ 51 € : la distribution est probablement **asymétrique**. Regardons cela de plus près.

3.1.3 Mesures de position : où est le centre ?

La **moyenne** $\bar{x} = \frac{1}{n} \sum x_i$ est le centre de gravité. La **médiane** est la valeur qui partage l'échantillon en deux moitiés égales : 50 % des commandes sont en dessous. Le **mode** est la valeur la plus fréquente.

Voici un petit exemple à la main pour sentir la différence. Cinq commandes : 20, 25, 30, 35, 400 € (la dernière est un gros achat professionnel). La moyenne vaut $(20 + 25 + 30 + 35 + 400)/5 = 102$ € : aucune commande n'est proche de ce chiffre ! La médiane (la valeur du milieu après tri) vaut 30 €, bien plus représentative d'une commande « typique ».

Règle de base. La moyenne est sensible aux valeurs extrêmes ; la médiane est **robuste**. Quand la distribution est asymétrique, la moyenne est tirée vers la queue longue : **moyenne > médiane** pour une asymétrie à droite (cas des montants, revenus, durées).

La moyenne (60 €) dépasse nettement la médiane (51 €) : quelques grosses commandes tirent la moyenne vers le haut. La **moyenne tronquée**, qui écarte les 10 % de valeurs les plus extrêmes de chaque côté, se situe entre les deux.

Les **quantiles** généralisent la médiane : le quantile à q % est la valeur sous laquelle se trouvent q % des observations. Les **quartiles** (25 %, 50 %, 75 %) découpent l'échantillon en quatre parts

égales.

Les quantiles à 5, 25, 50, 75 et 95 % valent respectivement 19,0 ; 34,2 ; 51,0 ; 75,8 et 128,6 €. On lit par exemple : « 95 % des commandes font moins de ~130 € », une information très utile pour dimensionner un seuil de livraison gratuite.

3.1.4 Mesures de dispersion : de combien ça varie ?

Deux boutiques dont le panier moyen est de 60 € peuvent être très différentes si l'une a des paniers tous compris entre 55 et 65 et l'autre entre 5 et 300.

- **L'étendue** : $\max - \min$. Simple, mais entièrement dictée par deux valeurs extrêmes.
- **La variance** $s^2 = \frac{1}{n-1} \sum (x_i - \bar{x})^2$ et l'**écart-type** $s = \sqrt{s^2}$.
- **L'écart interquartile** (IQR) : $Q_3 - Q_1$, l'étalement des 50 % du milieu. Robuste.
- **Le coefficient de variation** $s/\bar{x}$: l'écart-type en proportion de la moyenne, sans unité, utile pour comparer des échelles différentes.

Pourquoi $n-1$ et pas n ? Pandas et NumPy ne donnent pas toujours la même chose : pandas divise par $n-1$ par défaut, `np.var` par n . Nous démontrons au 3.2 que $n-1$ est la bonne version pour **estimer** la variance de la population à partir d'un échantillon. Pour $n = 400$ la différence est minime, mais pour $n = 5$ elle est de 25 %.

Sur nos montants : écart-type de 38,02 € (avec $n-1$) contre 37,97 € (avec n), étendue de 247,1 €, écart interquartile de 41,6 €. Un écart-type de 38 € pour une moyenne de 60 € donne un coefficient de variation de 0,63 (63 %), ce qui est **très dispersé** (typique des montants).

3.1.5 La forme : histogrammes, asymétrie, boîtes à moustaches

Les nombres ne disent pas tout. L'**histogramme** découpe l'axe en classes et compte les observations dans chacune. La **boîte à moustaches** (*boxplot*) résume la distribution en cinq nombres : la boîte va du premier au troisième quartile, le trait central est la médiane, et les « moustaches » s'étendent jusqu'aux dernières valeurs situées à moins de $1,5 \times \text{IQR}$ de la boîte ; les points au-delà sont signalés comme **valeurs atypiques** (*outliers*).

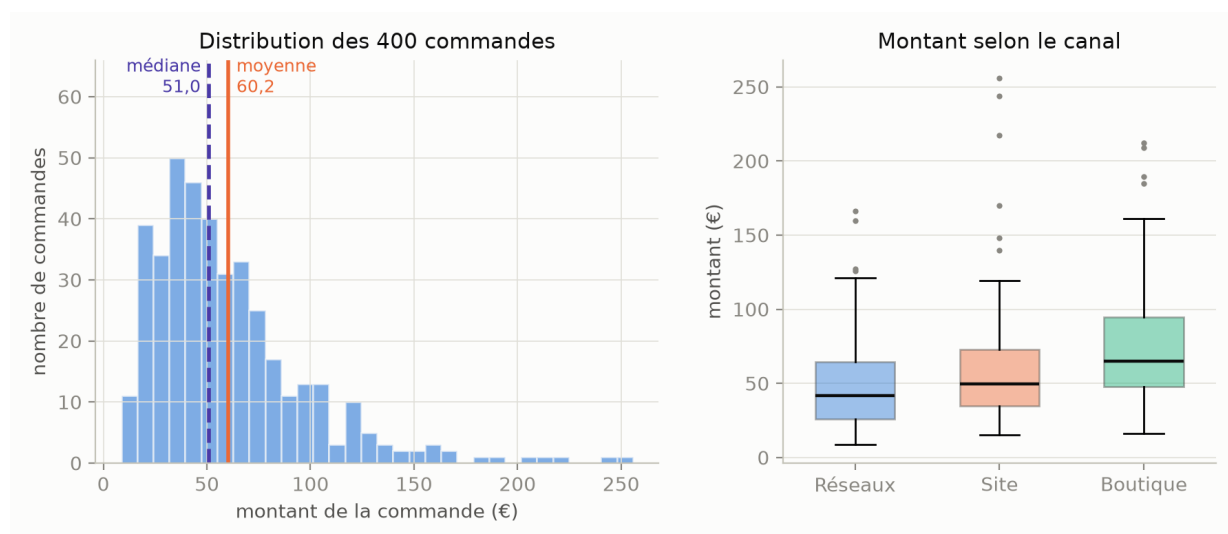


Figure 16: À gauche : histogramme des 400 montants, avec la moyenne (orange) tirée vers la droite de la médiane (violet). À droite : boîtes à moustaches du montant selon le canal.

On lit sur l'histogramme une **asymétrie à droite** : beaucoup de petites commandes, quelques très grosses. Sur le boxplot, la boutique a des commandes plus élevées que le site, lui-même plus élevé que le canal Réseaux. (Est-ce une vraie différence ou du hasard d'échantillonnage ? C'est la question du 3.4.)

L'**asymétrie** (*skewness*) se mesure par un nombre : nulle pour une courbe symétrique, positive à droite. Pour nos montants, elle vaut **1,75** : nettement à droite. Le seuil haut des valeurs atypiques ($Q_3 + 1,5 \times \text{IQR}$) est de 138,3 €, et 17 commandes le dépassent.

Que faire d'une valeur atypique ? Surtout ne pas la supprimer automatiquement. Se demander : est-ce une *erreur* (saisie : 2 500 au lieu de 25,00) ? Alors on corrige. Est-ce une valeur *légitime* mais rare (gros client professionnel) ? Alors on la garde et on utilise des résumés robustes. Les valeurs atypiques sont souvent l'information la plus intéressante : un fraudeur, une panne, une opportunité.

Transformer pour symétriser. Pour des variables positives très asymétriques, le **logarithme** rapproche la forme d'une cloche.

Sur nos données, l'asymétrie passe de 1,75 à $-0,07$, et la moyenne et la médiane du logarithme valent 3,92 et 3,93. Après transformation, l'asymétrie est donc presque nulle et moyenne $\approx$ médiane : le montant suit approximativement une loi **log-normale** (le logarithme est normal). Beaucoup de grandeurs économiques sont dans ce cas, car elles résultent de **multiplications** d'effets (là où la loi normale vient d'additions, TCL du 2.4).

3.1.6 Variables qualitatives et comparaison de groupes

Pour une variable catégorielle, on compte (**effectifs**) et on calcule des **proportions** (fréquences). Pour comparer des groupes, on utilise groupby.

```
print(df.groupby("canal")["montant"].agg(["count", "mean", "median", "std"]).round(1))
```

	count	mean	median	std
canal				
Boutique	114	74.8	64.8	40.6
Réseaux	138	49.0	41.5	31.1
Site	148	59.5	49.5	38.3

Les trois canaux apportent respectivement 138, 148 et 114 commandes (Réseaux, Site, Boutique). Les paniers moyens sont d'environ 49, 60 et 75 € : la boutique domine en valeur par commande. Mais attention :

Une différence observée n'est pas forcément une différence réelle. Avec 114 à 148 commandes par canal, le hasard seul peut créer des écarts de quelques euros. Pour savoir si l'écart entre 49 et 75 € est « assez grand » pour être crédible, il faut un **test** (3.4) ou un **intervalle de confiance** (3.3). La statistique descriptive décrit ; elle ne *conclut* pas.

3.1.7 Relier deux variables : corrélation, et pourquoi il faut toujours dessiner

La **corrélation de Pearson** (2.3.3), calculée sur l'échantillon, mesure le lien *linéaire* entre deux variables quantitatives.

```
print(df[["montant", "livraison", "satisfaction"]].corr().round(2))
```

	montant	livraison	satisfaction
montant	1.00	-0.20	0.07
livraison	-0.20	1.00	-0.53
satisfaction	0.07	-0.53	1.00

La corrélation entre le délai de livraison et la satisfaction est d'environ $-0,53$ (corrélation de Spearman : $-0,51$, voir ci-dessous) : plus la livraison est lente, moins les clients sont satisfaits ; c'est cohérent avec le bon sens. En revanche, le montant n'est que faiblement lié à la satisfaction ($+0,07$).

La corrélation de Spearman est la corrélation de Pearson calculée sur les **rangs** (1er, 2e, 3e...). Elle détecte toute relation **monotone** (pas seulement linéaire) et résiste aux valeurs extrêmes.

Le quartet d'Anscombe : pourquoi on dessine toujours. En 1973, le statisticien Francis Anscombe a construit **quatre jeux de données** qui ont exactement les mêmes moyennes, variances, corrélation et droite de régression... mais des formes totalement différentes.

Les quatre jeux ont la même moyenne en x (9,00), la même moyenne en y (7,50), la même corrélation (0,816) et la même droite de régression ($y = 3 + 0,5x$). Voici pourtant les quatre jeux :

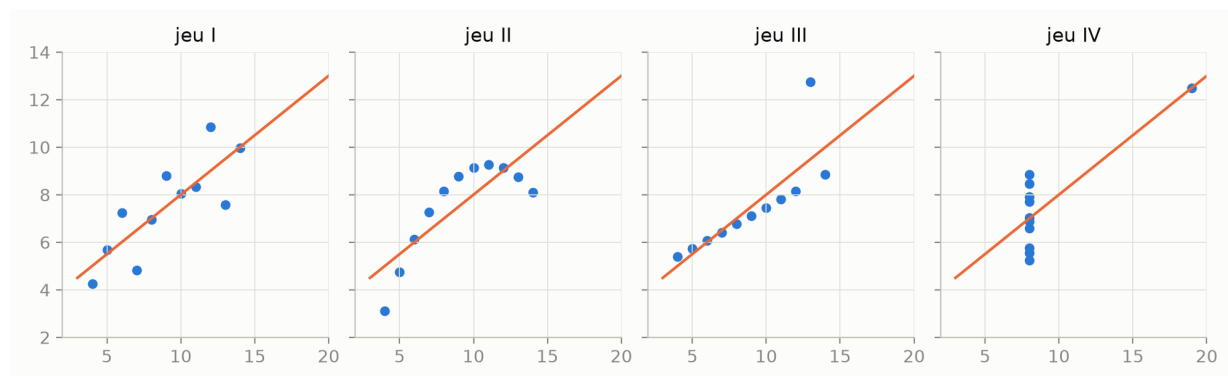


Figure 17: Le quartet d'Anscombe : mêmes statistiques, formes radicalement différentes. I : relation linéaire ordinaire. II : relation courbe. III : une valeur atypique fausse la droite. IV : un seul point (à droite) crée toute la corrélation.

Règle d'or : on dessine d'abord, on calcule ensuite. Un résumé numérique est une compression de l'information, avec perte. Seul un graphique révèle ce qui a été perdu.

À retenir (statistique descriptive).

- Population (inconnue) vs échantillon (observé) ; paramètres vs statistiques.
- Position : moyenne (sensible aux extrêmes), médiane (robuste), quantiles. Dispersion : écart-type ($n - 1$), IQR, coefficient de variation.
- Montants, revenus, durées : souvent asymétriques à droite ; le logarithme symétrise. Ne supprimez pas les valeurs atypiques sans réfléchir.
- Histogramme et boxplot pour la forme ; groupby pour comparer des groupes ; Pearson (linéaire) et Spearman (monotone) pour deux variables.
- **Toujours tracer** (Anscombe). Une différence observée n'est pas encore une différence prouvée.

■ **Pour s'entraîner.** Cahier, chapitre 3 : application 3.1, exercice 3.1.

3.2 Estimation

Intuition. La population a des paramètres (un panier moyen μ , un taux de conversion p , un taux d'arrivée λ) que personne ne connaît. À partir d'un **échantillon**, on fabrique une **estimation**. La formule qui transforme l'échantillon en estimation s'appelle un **estimateur**. Il y a toujours plusieurs estimateurs possibles ; la question est : **lequel choisir, et à quel point peut-on lui faire confiance ?**

3.2.1 Un estimateur est une variable aléatoire

Notons θ un paramètre inconnu (n'importe lequel) et $\hat{\theta}$ (« θ chapeau ») son estimateur. Comme l'échantillon est aléatoire, $\hat{\theta}$ **l'est aussi** : un autre échantillon donnerait une autre estimation. C'est exactement l'idée de la moyenne d'échantillon $\bar{X}_n$ au 2.4.1.

On peut le **voir** en jouant à Dieu : traitons nos 400 commandes comme *toute* la population (de moyenne connue) et tirons dedans des échantillons de 30 commandes, en recalculant la moyenne à chaque fois.

Chaque échantillon de 30 commandes donne une moyenne différente (ci-dessus : 54,2 ; 59,5 ; 60,3), mais **en moyenne** ces moyennes tombent sur la vraie valeur (60,25), avec une dispersion de l'ordre de 7 €. Cette dispersion est l'**erreur-type** : elle mesure la précision de l'estimateur. L'erreur-type observée (6,59 €) est proche de la valeur théorique $\sigma / \sqrt{n} = 6,93$ €. (Le petit écart vient du tirage **sans remise** dans une population finie.) Sur 10 000 échantillons, la moyenne des moyennes est 60,30 € : l'estimateur est bien centré.

3.2.2 Qu'est-ce qu'un bon estimateur ?

On juge un estimateur sur trois critères, que l'on comprend très bien avec l'image d'un tir à la cible :

- **Le biais** : $\text{Biais}(\hat{\theta}) = E[\hat{\theta}] - \theta$. Les tirs sont-ils centrés sur la cible, ou systématiquement décalés ? Un estimateur est **sans biais** si ce biais vaut 0.
- **La variance** : $\text{Var}(\hat{\theta})$. Les tirs sont-ils groupés ou dispersés ?
- **La cohérence** (ou *consistance*) : l'estimateur converge-t-il vers θ quand $n \rightarrow \infty$? (La loi des grands nombres l'assure pour la moyenne.)

Un bon estimateur est à la fois **peu biaisé** et **peu variable**. On les combine dans l'**erreur quadratique moyenne** :

Décomposition biais-variance.

$$\text{EQM}(\hat{\theta}) = E[(\hat{\theta} - \theta)^2] = \text{Var}(\hat{\theta}) + \text{Biais}(\hat{\theta})^2.$$

Preuve. Notons $m = E[\hat{\theta}]$. On écrit $\hat{\theta} - \theta = (\hat{\theta} - m) + (m - \theta)$ et on développe le carré :

$$E[(\hat{\theta} - \theta)^2] = E[(\hat{\theta} - m)^2] + 2(m - \theta) E[\hat{\theta} - m] + (m - \theta)^2.$$

Le terme du milieu est nul car $E[\hat{\theta} - m] = 0$. Il reste $\text{Var}(\hat{\theta}) + (m - \theta)^2$. ■

Cette formule a une conséquence profonde, qui reviendra tout au long du volume II : **accepter un petit biais peut réduire l'erreur totale si cela réduit beaucoup la variance**. C'est le principe de la régularisation en apprentissage automatique.

3.2.3 Pourquoi divise-t-on par $n - 1$ pour la variance ?

Nous avons promis cette démonstration (3.1.4 et 2.3.3). Soit $X_1, \dots, X_n$ i.i.d. de moyenne μ et de variance σ^2 . L'estimateur « naturel » de σ^2 est $\hat{\sigma}_n^2 = \frac{1}{n} \sum (X_i - \bar{X})^2$. Est-il sans biais ?

Calcul. On insère μ : $X_i - \bar{X} = (X_i - \mu) - (\bar{X} - \mu)$. Alors

$$\sum_i (X_i - \bar{X})^2 = \sum_i (X_i - \mu)^2 - n(\bar{X} - \mu)^2.$$

(En développant : $\sum (X_i - \mu)^2 - 2(\bar{X} - \mu) \sum (X_i - \mu) + n(\bar{X} - \mu)^2$ et $\sum (X_i - \mu) = n(\bar{X} - \mu)$, d'où le résultat.) Prenons l'espérance : $E[(X_i - \mu)^2] = \sigma^2$ et $E[(\bar{X} - \mu)^2] = \text{Var}(\bar{X}) = \sigma^2/n$. Donc

$$E\left[\sum_i (X_i - \bar{X})^2\right] = n\sigma^2 - n \cdot \frac{\sigma^2}{n} = (n-1)\sigma^2.$$

Par conséquent $E[\hat{\sigma}_n^2] = \frac{n-1}{n}\sigma^2 < \sigma^2$: l'estimateur « divisé par n » **sous-estime** la variance. Pour corriger, on divise par $n - 1$:

$$S^2 = \frac{1}{n-1} \sum_i (X_i - \bar{X})^2, \quad E[S^2] = \sigma^2. \blacksquare$$

Pourquoi intuitivement ? Les données sont toujours plus proches de **leur propre** moyenne $\bar{X}$ que de la vraie moyenne μ (car $\bar{X}$ est justement construite pour être au centre des données). Mesurer les écarts à $\bar{X}$ sous-estime donc légèrement les écarts à μ . L'échantillon a aussi « perdu un degré de liberté » : une fois $\bar{X}$ calculée, seules $n - 1$ valeurs sont libres, la dernière est imposée.

Vérifions-le par simulation sur de **tout petits** échantillons ($n = 5$, vraie variance égale à 1), là où l'effet est maximal : sur 100 000 échantillons, la division par n donne en moyenne **0,801** (la théorie prévoit $(n-1)/n = 0,8$), la division par $n - 1$ donne **1,001**.

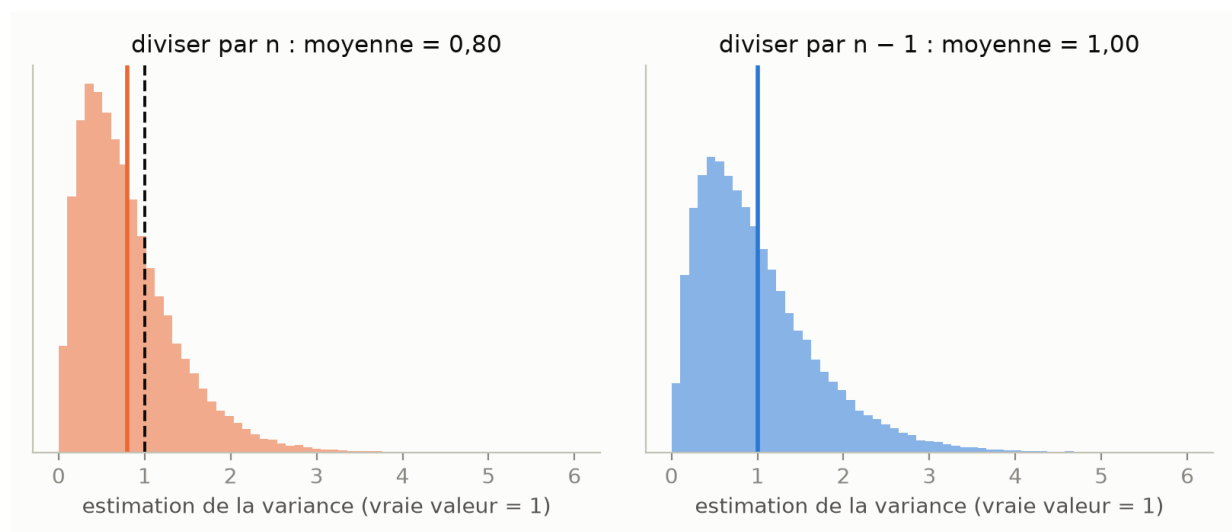


Figure 18: Distribution des estimations de la variance sur 100 000 échantillons de taille 5 : en divisant par n , on sous-estime en moyenne (0,80) ; en divisant par $n-1$, on est centré sur la vraie valeur (1).

Subtilité. S^2 est sans biais pour la **variance**, mais $S = \sqrt{S^2}$ reste (très légèrement) biaisé pour l'écart-type, car la racine carrée n'est pas linéaire. Ce biais est négligeable en pratique.

3.2.4 La méthode des moments

Idée. Les paramètres d'une loi s'expriment à l'aide de ses moments (espérance, variance...). La **méthode des moments** consiste à **égaler les moments théoriques aux moments observés** et à résoudre. C'est simple et intuitif.

Exemple 1 : le taux d'arrivée. Les temps (en minutes) séparant 25 commandes successives suivent une loi exponentielle de paramètre λ inconnu. On sait que $E[T] = 1/\lambda$. On égale à la moyenne observée $\bar{t}$: $1/\hat{\lambda} = \bar{t}$, donc $\hat{\lambda} = 1/\bar{t}$. Sur 25 attentes simulées avec une vraie valeur $\lambda = 0,5$ (moyenne 2 minutes), la moyenne observée est 1,988 minute, d'où $\hat{\lambda} = 1/1,988 \approx 0,503$: l'estimation est proche de la vérité.

Exemple 2 : une loi à deux paramètres. Les montants sont modélisés par une loi **Gamma** de forme k et d'échelle θ , avec $E[X] = k\theta$ et $\text{Var}(X) = k\theta^2$. En égalant à la moyenne $\bar{x}$ et à la variance s^2 observées, on obtient deux équations : $\hat{\theta} = s^2/\bar{x}$ et $\hat{k} = \bar{x}^2/s^2$. Avec $\bar{x} = 60,25$ et la variance observée, on trouve $\hat{k} \approx 2,51$ et $\hat{\theta} \approx 23,99$.

La méthode des moments est rapide, mais elle n'est pas toujours la plus précise. La suivante est la référence.

3.2.5 Le maximum de vraisemblance

Intuition. La gérante lance une nouvelle promotion et observe 7 achats sur 20 visiteurs. Quelle valeur du taux de conversion p rend ces données **les plus plausibles** ? Si p valait 0,05, observer 7 acheteurs sur 20 serait très improbable ; si p valait 0,9, tout autant. Il existe une valeur intermédiaire pour laquelle ce résultat est **le moins surprenant possible** : c'est l'estimation du maximum de vraisemblance.

La **vraisemblance** $L(\theta)$ est la probabilité (ou la densité) d'observer **les données effectivement observées**, vue comme une fonction du paramètre θ . Pour des observations indépendantes $x_1, \dots, x_n$:

$$L(\theta) = \prod_{i=1}^n f(x_i; \theta).$$

L'estimateur du maximum de vraisemblance (EMV, MLE) est la valeur $\hat{\theta}$ qui **maximise** $L(\theta)$. Comme les produits sont pénibles à dériver et numériquement instables (le produit de nombreuses probabilités minuscules s'écroule vers 0), on maximise plutôt le **logarithme** de la vraisemblance, qui transforme le produit en somme et a le même maximum (le logarithme est croissant) :

$$\ell(\theta) = \ln L(\theta) = \sum_{i=1}^n \ln f(x_i; \theta).$$

C'est de l'optimisation (section 1.3) : on dérive et on annule.

Exemple complet : le taux de conversion. On observe k achats sur n visiteurs. Le modèle est binomial : $L(p) = \binom{n}{k} p^k (1-p)^{n-k}$, donc

$$\ell(p) = \ln \binom{n}{k} + k \ln p + (n-k) \ln(1-p).$$

On dérive : $\ell'(p) = \frac{k}{p} - \frac{n-k}{1-p}$. En annulant : $k(1-p) = (n-k)p \iff k = np$, d'où

$$\hat{p} = \frac{k}{n}.$$

La dérivée seconde $\ell''(p) = -\frac{k}{p^2} - \frac{n-k}{(1-p)^2} < 0$: c'est bien un **maximum**. ■

L'estimateur du maximum de vraisemblance est donc tout simplement la **proportion observée**, ce qui rassure : la méthode retrouve le bon sens.

Pour $k = 7$, $n = 20$: $\hat{p} = 0,35$. La figure montre la vraisemblance et la log-vraisemblance en fonction de p ; le maximum est atteint en 0,35, et la log-vraisemblance est une courbe en cloche inversée, bien plus facile à optimiser.

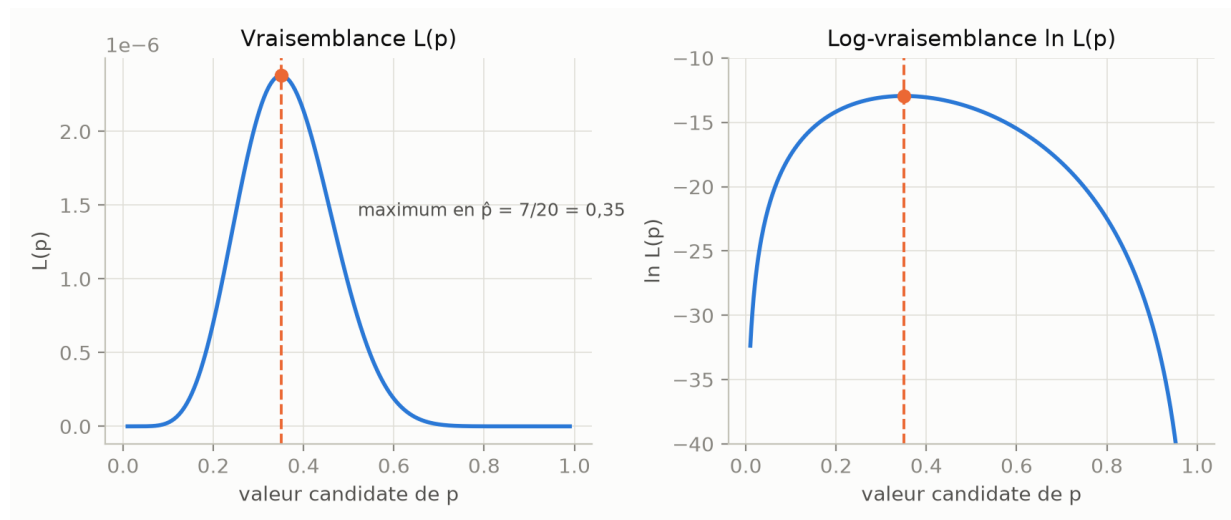


Figure 19: Vraisemblance (à gauche) et log-vraisemblance (à droite) pour 7 achats sur 20 visiteurs. Le maximum est en $p = 0,35$.

Vérifions par calcul numérique : un balayage de p sur une grille, puis un optimiseur (la méthode générale quand il n'y a pas de formule), trouvent tous deux 0,35.

D'autres exemples classiques (mêmes calculs, à retrouver en exercice) :

Modèle	Estimateur du maximum de vraisemblance
Bernoulli / binomiale	$\hat{p} = \bar{x}$ (proportion observée)
Poisson(λ)	$\hat{\lambda} = \bar{x}$
Exponentielle(λ)	$\hat{\lambda} = 1/\bar{x}$
Normale(μ, σ^2)	$\hat{\mu} = \bar{x}$; $\hat{\sigma}^2 = \frac{1}{n} \sum (x_i - \bar{x})^2$ (Δ divisé par n : biaisé !)

On retrouve pour l'exponentielle la même formule qu'avec les moments. Remarquez la dernière ligne : l'EMV de la variance divisé par n , donc est **légèrement biaisé**. Le maximum de vraisemblance n'est pas toujours sans biais ; il a d'autres qualités.

Quand il n'y a pas de formule : l'optimisation numérique. Pour la loi Gamma des montants,

on ne peut pas résoudre à la main. On confie la log-vraisemblance à un optimiseur numérique, exactement comme au chapitre 1 (la descente de gradient en est l'ancêtre) : on lui fournit $-\ell(k, \theta)$ à minimiser, en partant de l'estimation par les moments. Résultat : $\hat{k} = 3,001$ et $\hat{\theta} = 20,07$, contre 2,511 et 23,99 par les moments. La log-vraisemblance atteinte est $-1\,938,9$, contre $-1\,942,2$ pour les paramètres des moments.

Le maximum de vraisemblance trouve des paramètres de log-vraisemblance **plus élevée** que ceux des moments (c'est sa définition : il est le meilleur *pour les données observées*). Ici les deux approches donnent des valeurs du même ordre (la forme passe de 2,5 à 3,0). N'oubliez pas qu'une loi Gamma n'est qu'un **modèle** des montants parmi d'autres (on a vu au 3.1.5 qu'une loi log-normale convient aussi bien) : estimer les paramètres ne dit pas si le modèle est juste. Notez aussi que la bibliothèque sait faire tout cela d'un coup :

```
k, loc, theta = stats.gamma.fit(df["montant"], floc=0) # floc=0 : borne inférieure fixée à 0
print(round(k, 3), round(theta, 2))
```

```
3.001 20.07
```

3.2.6 Pourquoi le maximum de vraisemblance est la référence

Sous des conditions de régularité raisonnables, l'EMV a quatre propriétés remarquables, que l'on admettra :

1. **Cohérent** : $\hat{\theta} \rightarrow \theta$ quand $n \rightarrow \infty$.
2. **Asymptotiquement normal** : $\hat{\theta} \approx \mathcal{N}(\theta, 1/(nI(\theta)))$ pour n grand, où $I(\theta)$ est l'**information de Fisher** (la courbure moyenne de la log-vraisemblance autour du maximum : plus le pic est pointu, plus on est précis).
3. **Asymptotiquement efficace** : parmi les estimateurs cohérents, il a (presque) la plus petite variance possible.
4. **Invariant par reparamétrisation** : si $\hat{\theta}$ est l'EMV de θ , alors $g(\hat{\theta})$ est l'EMV de $g(\theta)$.

Le point 2 donne directement l'**erreur-type** : pour la proportion, $I(p) = \frac{1}{p(1-p)}$, donc

$$SE(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}.$$

Avec 20 visiteurs ($\hat{p} = 0,35$), l'erreur-type est $\sqrt{0,35 \times 0,65/20} \approx 0,107$ (près de 11 points !) : l'estimation est très imprécise. Avec 200 visiteurs (70 achats), elle tombe à 0,034. Estimer, c'est bien ; **chiffrer l'incertitude de l'estimation** est mieux : c'est l'objet de la section suivante.

À retenir (estimation).

- Un **estimateur** est une formule appliquée à l'échantillon ; c'est une variable aléatoire dont on étudie le **biais**, la **variance**, la **cohérence**. $EQM = Var + Biais^2$.
- $\bar{X}$ est sans biais pour μ , et S^2 (**divisé par $n-1$**) est sans biais pour σ^2 (preuve en 3.2.3).
- **Moments** : on égale moments théoriques et observés. **Maximum de vraisemblance** : on maximise $\ell(\theta) = \sum \ln f(x_i; \theta)$; formule fermée si possible, optimiseur sinon.
- Pour une proportion, l'EMV est la fréquence observée, d'erreur-type $\sqrt{\hat{p}(1-\hat{p})/n}$.
- L'EMV est cohérent, asymptotiquement normal et efficace : c'est l'outil standard.

■ **Pour s'entraîner.** Cahier, chapitre 3 : application 3.2, exercices 3.2 et 3.3.

3.3 Intervalles de confiance

Intuition. Dire « le panier moyen est de 60,25 € » est trompeur : cela suggère une précision que l'on n'a pas. Un meilleur énoncé est : « le panier moyen se situe, avec une confiance de 95 %, entre 56,5 et 64,0 € ». L'**intervalle de confiance** (IC) transforme une estimation ponctuelle en une **fourchette honnête**, dont la largeur reflète l'incertitude.

3.3.1 Construire un intervalle pour une moyenne

Rappelons ce que nous savons (2.4) : par le théorème central limite, $\bar{X}_n \approx \mathcal{N}(\mu, \sigma^2/n)$. Donc le score centré réduit $\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}}$ suit à peu près $\mathcal{N}(0, 1)$, et comme $P(-1,96 \leq Z \leq 1,96) = 0,95$:

Construction.

$$P\left(-1,96 \leq \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \leq 1,96\right) = 0,95.$$

On isole μ au milieu de l'encadrement : multiplier par $\sigma/\sqrt{n}$, puis soustraire $\bar{X}_n$ et multiplier par -1 (ce qui renverse les inégalités) :

$$P\left(\bar{X}_n - 1,96 \frac{\sigma}{\sqrt{n}} \leq \mu \leq \bar{X}_n + 1,96 \frac{\sigma}{\sqrt{n}}\right) = 0,95.$$

L'**intervalle de confiance à 95 %** est donc $\bar{x} \pm 1,96 \frac{\sigma}{\sqrt{n}}$. ■

Il a une structure à retenir absolument, qui se retrouvera partout :

$$\text{estimation} \pm (\text{valeur critique}) \times (\text{erreur-type}).$$

Exemple à la main. Sur 400 commandes, $\bar{x} = 60,25$ € et $s = 38,02$. L'erreur-type est $38,02/\sqrt{400} = 1,90$. L'IC à 95 % est $60,25 \pm 1,96 \times 1,90 = 60,25 \pm 3,73$, soit **[56,5 ; 64,0]** €.

3.3.2 Que veut dire « 95 % de confiance » ?

C'est la phrase la plus mal comprise de la statistique. Elle ne signifie **pas** « il y a 95 % de chances que la vraie moyenne soit dans cet intervalle-ci ». La vraie moyenne μ est un nombre **fixe** (inconnu) : elle y est ou elle n'y est pas. Ce qui est aléatoire, c'est l'**intervalle** (il change à chaque échantillon).

La bonne lecture : si l'on répétait l'expérience un très grand nombre de fois, avec un nouvel échantillon à chaque fois, 95 % des intervalles ainsi construits contiendraient la vraie valeur. La confiance porte sur la **méthode**, pas sur un intervalle particulier.

Voyons-le. Soixante échantillons de 40 commandes, un intervalle à 95 % pour chacun, et la vraie moyenne (connue ici car on traite nos 400 commandes comme la population) en pointillés :

Mesurons-le précisément : sur 20 000 échantillons de 40 commandes, **94,5 %** des intervalles contiennent la vraie moyenne, avec une largeur moyenne de 23,9 €.

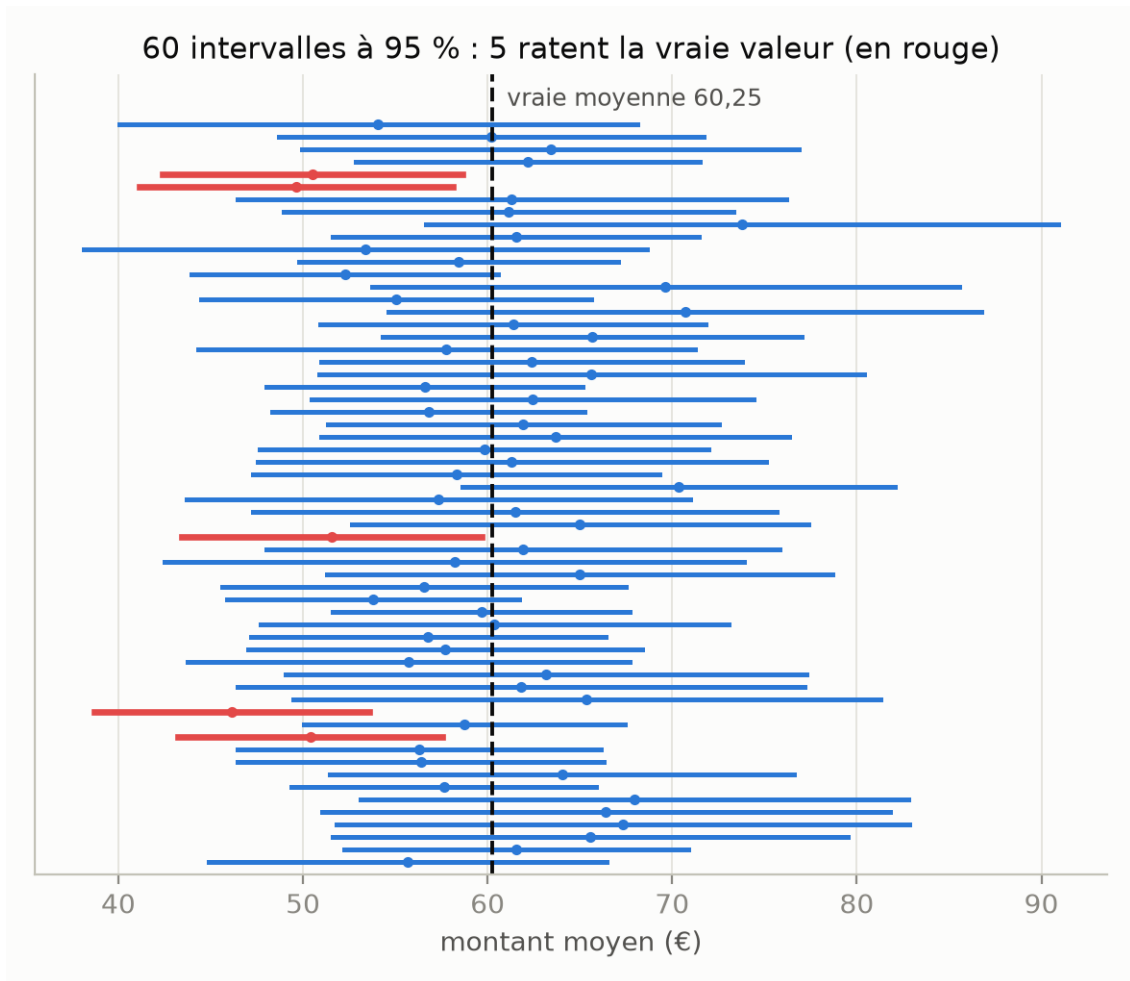


Figure 20: 60 intervalles de confiance à 95 % construits sur 60 échantillons différents de 40 commandes. Les intervalles en rouge n'atteignent pas la vraie moyenne : environ 1 sur 20 en moyenne (5 sur 60 ici, une fluctuation normale).

Cette couverture est proche de 95 % (un peu moins : la loi des montants est asymétrique et $n = 40$ est modeste ; nous reviendrons sur ces limites). L'idée est donc validée.

Deux erreurs d'interprétation à éviter.

1. « La vraie valeur a 95 % de chances d'être dans $[56,5 ; 64,0]$ » : formulation courante, rigoureusement fautive dans l'approche fréquentiste (dans l'approche bayésienne, elle est correcte pour un *intervalle de crédibilité*).
2. « 95 % des **commandes** sont dans cet intervalle » : confusion entre la précision de la **moyenne** et la dispersion des **données**. L'IC de la moyenne est étroit ($[56,5 ; 64,0]$), alors que 95 % des commandes sont entre 19 et 129 € (3.1.3).

3.3.3 Quand l'écart-type est inconnu : la loi de Student

En pratique, on ne connaît **pas** σ ; on le remplace par son estimation s . Mais s est elle-même aléatoire, ce qui ajoute de l'incertitude : pour de petits échantillons, on tomberait trop souvent à côté avec la valeur 1,96. William Gosset (qui signait « Student » en 1908, alors qu'il travaillait pour une grande brasserie) a montré que la bonne loi pour $\frac{\bar{X} - \mu}{S/\sqrt{n}}$ est la **loi de Student à $n - 1$ degrés de liberté** (si les données sont à peu près normales).

C'est une cloche comme la normale, mais avec des **queues plus lourdes** (plus de prudence), qui tend vers $\mathcal{N}(0, 1)$ quand n augmente. La valeur critique à 95 % :

Degrés de liberté	2	5	10	30	100	1 000	loi normale
Valeur critique	4,303	2,571	2,228	2,042	1,984	1,962	1,960

Avec 2 degrés de liberté (3 observations), la valeur critique est 4,30 : l'intervalle est plus de deux fois plus large qu'avec 1,96. Dès 30 degrés de liberté, on est proche de 2,04 ; avec 400 observations, la différence avec 1,96 est imperceptible.

L'intervalle devient $\bar{x} \pm t_{n-1, 0,975} \frac{s}{\sqrt{n}}$. Pour nos 400 commandes : $\bar{x} = 60,25$, erreur-type = 1,90, $t_{399, 0,975} = 1,966$, d'où $60,25 \pm 1,966 \times 1,90$, soit **[56,51 ; 63,98]** €. Une ligne de scipy donne le même résultat :

```
xbar, s, n = m.mean(), m.std(ddof=1), len(m)
print(np.round(stats.t.interval(0.95, n - 1, loc=xbar, scale=s / np.sqrt(n)), 2))
```

```
[56.51 63.98]
```

Et pour chaque canal ? On répète le calcul par groupe :

Canal	n	Moyenne	IC à 95 %
Réseaux	138	49,0 €	[43,8 ; 54,2]
Site	148	59,5 €	[53,3 ; 65,7]
Boutique	114	74,8 €	[67,3 ; 82,4]

Les intervalles du canal Réseaux ([43,8 ; 54,2]) et de la boutique ([67,3 ; 82,4]) **sont très éloignés** : c'est un indice sérieux que ces deux canaux diffèrent vraiment. L'intervalle du site ([53,3 ; 65,7]) chevauche légèrement celui de Réseaux mais pas celui de la boutique. Attention : « les intervalles se chevauchent » ne prouve **pas** que les moyennes sont égales, et même des intervalles qui se

touchent peuvent cacher une différence significative. La bonne méthode est de construire un intervalle (ou un test) pour la **différence** elle-même, ce que nous ferons au 3.4.

Ce qui fait varier la largeur. La demi-largeur est $t \times s / \sqrt{n}$. Elle **diminue** quand n augmente (en $1/\sqrt{n}$), **augmente** quand la dispersion s augmente, et **augmente** quand on exige plus de confiance (99 % donne un intervalle plus large que 95 %). Il n'y a pas de gratuité : plus de certitude coûte en précision. Pour les 400 commandes :

Confiance	80 %	90 %	95 %	99 %
Intervalle (€)	[57,81 ; 62,69]	[57,11 ; 63,38]	[56,51 ; 63,98]	[55,33 ; 65,17]
Largeur (€)	4,88	6,27	7,47	9,84

3.3.4 Intervalle pour une proportion

Pour un taux de conversion $\hat{p} = k/n$, l'erreur-type vue au 3.2.6 est $\sqrt{\hat{p}(1-\hat{p})/n}$, et l'intervalle approché (dit de **Wald**) est

$$\hat{p} \pm 1,96 \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}.$$

Sur 1 000 visiteurs dont 205 achètent : $0,205 \pm 1,96 \times 0,0128 = 0,205 \pm 0,025$, soit **[18,0 % ; 23,0 %]**.

Mais cet intervalle devient **mauvais** pour de petits échantillons ou des proportions proches de 0 ou 1. Par exemple, avec 0 achat sur 20 visiteurs, $\hat{p} = 0$ et l'intervalle de Wald est $[0 ; 0]$: « on est certain que le taux de conversion est exactement nul » ! Absurde. L'**intervalle de Wilson** corrige cela : il est centré non pas sur $\hat{p}$ mais sur une valeur légèrement « tirée vers 1/2 », et ne sort jamais de $[0, 1]$. Comparaison sur quatre cas :

Observé	$\hat{p}$	Wald	Wilson
205 sur 1 000	0,205	[0,180 ; 0,230]	[0,181 ; 0,231]
7 sur 20	0,350	[0,141 ; 0,559]	[0,181 ; 0,567]
0 sur 20	0,000	[0,000 ; 0,000]	[0,000 ; 0,161]
2 sur 15	0,133	[0,000 ; 0,305]	[0,037 ; 0,379]

Pour 1 000 visiteurs, les deux méthodes coïncident. Pour 7/20, l'intervalle est très large : **[0,18 ; 0,57]** pour Wilson, c'est-à-dire que 20 visiteurs ne permettent quasiment rien de conclure. Pour 0/20, Wilson dit que le taux réel peut aller jusqu'à environ 16 % (la fameuse « règle de trois » : avec 0 événement sur n , la borne haute à 95 % est environ $3/n = 15\%$).

Conseil pratique. Pour une proportion, utilisez **Wilson** (ou une méthode exacte) plutôt que Wald, sauf si n est très grand et p loin de 0 et 1.

3.3.5 Quand il n'y a pas de formule : le bootstrap

Et si l'on veut un intervalle pour la **médiane**, un quantile, un rapport, ou toute autre statistique pour laquelle on ne connaît pas de formule d'erreur-type ? Le **bootstrap** (Efron, 1979) offre une solution étonnamment simple et générale.

Idée. On ne peut pas retirer de nouveaux échantillons dans la vraie population, mais on peut **rééchantillonner dans l'échantillon lui-même**. On tire n valeurs **avec remise** dans nos n observations (certaines apparaissent plusieurs fois, d'autres pas), on recalcule la statistique, et on recommence des milliers de fois. La dispersion des valeurs obtenues imite la dispersion qu'on aurait observée en échantillonnant la vraie population.

L'algorithme (intervalle « percentile »). Il se programme en quelques lignes, mais l'idée suffit :

1. Répéter B fois (par exemple 10 000) : tirer un échantillon de taille n avec remise ; calculer la statistique.
2. L'intervalle à 95 % est formé des percentiles 2,5 % et 97,5 % des B valeurs obtenues.

Sur nos 400 commandes (10 000 rééchantillonnages), le bootstrap donne pour la moyenne l'intervalle $[56,7 ; 64,0]$, pratiquement celui de la formule de Student $([56,5 ; 64,0])$: rassurant. Pour la **médiane**, qui n'a pas de formule simple, il fournit un intervalle (la médiane observée est 51,0 € ; IC à 95 % : $[47,3 ; 55,1]$ €) que l'on n'aurait pas pu obtenir à la main.

Limites. Le bootstrap suppose que l'échantillon est représentatif de la population ; il marche mal pour des statistiques « extrêmes » (le maximum), avec de très petits échantillons, ou en présence de très fortes dépendances entre observations. Il reste un outil de base du data scientist, car il généralise à **n'importe quelle** statistique sans calcul mathématique.

3.3.6 Dimensionner un échantillon

On peut renverser le raisonnement : *quelle précision veut-on ?* Si la gérante veut estimer le panier moyen à ± 2 € près, avec une confiance de 95 % et en supposant $s \approx 38$ €, il faut $1,96 \times 38 / \sqrt{n} \leq 2$, soit

$$n \geq \left(\frac{1,96 s}{\varepsilon} \right)^2 = \left(\frac{1,96 \times 38}{2} \right)^2 \approx 1\,387 \text{ commandes.}$$

Pour une marge de ± 1 €, le même calcul donne 5 548 commandes.

Diviser la marge par 2 demande **4 fois plus de données** (la loi en $1/\sqrt{n}$ du 2.4). C'est pourquoi gagner de la précision devient vite très coûteux.

À retenir (intervalles de confiance).

- Structure universelle : **estimation $\pm$ valeur critique $\times$ erreur-type**.
- IC de la moyenne : $\bar{x} \pm t_{n-1} s / \sqrt{n}$ (Student). IC d'une proportion : préférez **Wilson** à Wald.
- « Confiance à 95 % » signifie : **la méthode** capture la vraie valeur dans 95 % des échantillons possibles. Ce n'est pas une probabilité sur la vraie valeur.
- Largeur : $\downarrow$ avec n (en $1/\sqrt{n}$) ; $\uparrow$ avec la dispersion et avec le niveau de confiance.
- **Bootstrap** : rééchantillonner avec remise pour obtenir un IC de n'importe quelle statistique.
- $n \geq (zs/\varepsilon)^2$ pour viser une marge ε .

■ **Pour s'entraîner.** Cahier, chapitre 3 : application 3.3, exercices 3.4 et 3.5.

3.4 Tests d'hypothèses

Intuition : le tribunal. Un test d'hypothèses fonctionne comme un procès. On part de la **présomption d'innocence** (l'hypothèse « il ne se passe rien », appelée H_0). On examine les **preuves** (les données). Si les preuves sont **très improbables** dans un monde où l'accusé est innocent, on le **condamne** (on rejette H_0). Sinon, on **acquitte** : cela ne prouve pas son innocence, cela veut seulement dire que les preuves sont insuffisantes.

3.4.1 Le vocabulaire et la méthode

- **Hypothèse nulle** H_0 : l'état de référence, « pas d'effet, pas de différence ». Par exemple : « le panier moyen vaut 55 € ».
- **Hypothèse alternative** H_1 : ce que l'on cherche à montrer. « Le panier moyen est différent de 55 € » (test **bilatéral**), ou « supérieur à 55 € » (test **unilatéral**).
- **Statistique de test** T : un nombre calculé sur l'échantillon qui mesure l'écart entre les données et ce que prédit H_0 .
- **Niveau de signification** α (souvent 5 %) : le risque que l'on accepte de se tromper en rejetant H_0 alors qu'elle est vraie.
- **p-valeur** : la probabilité, si H_0 est vraie, d'obtenir un résultat **au moins aussi extrême** que celui observé. Petite p-valeur = les données sont surprenantes sous H_0 .

Les deux erreurs possibles.

	H_0 est vraie	H_0 est fausse
On rejette H_0	✗ Erreur de type I (faux positif), probabilité α	✓ bonne décision (probabilité $1 - \beta =$ puissance)
On ne rejette pas H_0	✓ bonne décision	✗ Erreur de type II (faux négatif), probabilité β

On fixe α à l'avance, et on cherche à garder β petit (c'est la question de la puissance, 3.5).

La recette en 5 étapes, valable pour tous les tests de ce chapitre :

1. Poser H_0 et H_1 .
2. Choisir α (avant de regarder les données !).
3. Calculer la statistique de test.
4. Calculer la p-valeur (ou comparer à la valeur critique).
5. Conclure : si $p < \alpha$, **rejeter** H_0 ; sinon, **ne pas rejeter** (et jamais « accepter »).

3.4.2 Test de Student sur une moyenne

Question de la gérante. Son objectif de panier moyen était de 55 €. Les 400 commandes confirment-elles que le panier moyen **diffère** de 55 € ?

1. $H_0 : \mu = 55 ; H_1 : \mu \neq 55$.
2. $\alpha = 0,05$.
3. Statistique de test (la même construction qu'au 3.3.3, centrée sur la valeur de H_0) :

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}} = \frac{60,25 - 55}{38,02/\sqrt{400}} = \frac{5,25}{1,90} \approx 2,76.$$

Sous H_0 , t suit une loi de Student à $n - 1 = 399$ degrés de liberté. 4. La p-valeur est la probabilité qu'une telle loi donne une valeur **au moins aussi éloignée de 0** que 2,76, des deux côtés : $p = 2P(T_{399} > 2,76)$.

Le calcul (`scipy.stats.ttest_1samp` le fait en une ligne) donne $t = 2,76$ et $p = 0,0061$, la valeur critique bilatérale à 5 % étant 1,966.

5. **Conclusion.** $p = 0,006 < 0,05$: on rejette H_0 . Le panier moyen est significativement supérieur à 55 € (il est en fait de 60,25 ; l'IC à 95 % du 3.3.3 était [56,5 ; 64,0], qui n'inclut pas 55 : **un test bilatéral à 5 % et un IC à 95 % disent la même chose**).

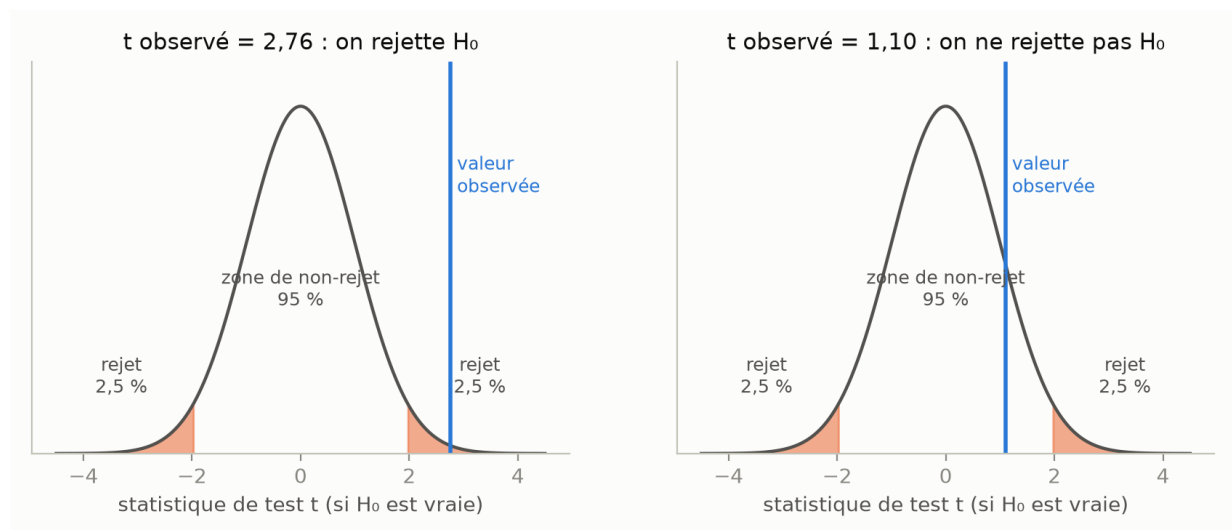


Figure 21: Statistique de test sous H_0 . Gauche : la valeur observée (2,76) tombe dans la zone de rejet (queues orange, 5 % au total). Droite : une valeur de 1,10 tomberait dans la zone de non-rejet.

« **Ne pas rejeter** » n'est pas « **accepter** ». Si p avait été de 0,30, on aurait dit « les données ne permettent pas de conclure que $\mu \neq 55$ », pas « $\mu = 55$ ». L'absence de preuve n'est pas la preuve de l'absence. (Un petit échantillon peut échouer à détecter un grand effet : c'est le problème de la puissance.)

3.4.3 Comparer deux groupes : le test de Welch

Question de la gérante. Les clients de la **boutique** dépensent-ils plus que ceux du canal **Réseaux** ? Les moyennes observées sont 74,8 et 49,0 €, soit un écart de 25,8 €. Cet écart est-il crédible ou dû au hasard ?

On teste $H_0 : \mu_B = \mu_I$ contre $H_1 : \mu_B \neq \mu_I$. On compare la différence des moyennes à son erreur-type. Comme les deux échantillons sont indépendants, les variances **s'additionnent** (2.3.3) :

$$t = \frac{\bar{x}_B - \bar{x}_I}{\sqrt{\frac{s_B^2}{n_B} + \frac{s_I^2}{n_I}}}.$$

C'est le **test de Welch**, qui n'exige pas que les deux groupes aient la même variance (c'est la version à utiliser par défaut ; l'ancien test de Student à variances égales est moins sûr). Ici, la différence des moyennes est 25,8 € et son erreur-type 4,64 €, d'où $t = 25,8/4,64 \approx 5,565$. En pratique, une seule instruction de scipy fait le calcul :

```
b = df.loc[df["canal"] == "Boutique", "montant"]
i = df.loc[df["canal"] == "Réseaux", "montant"]
res = stats.ttest_ind(b, i, equal_var=False) # equal_var=False : test de Welch
print(round(res.statistic, 3), f"{res.pvalue:.1e}", round(res.df, 1))
```

```
5.565 8.0e-08 208.4
```

La statistique est $t \approx 5,56$ et la p-valeur est de l'ordre de 10^{-7} : si les deux canaux avaient la même dépense moyenne, observer un écart aussi grand serait **quasi impossible**. On rejette H_0 .

Un test ne dit pas « de combien ». Une p-valeur minuscule dit que l'effet est *réel*, pas qu'il est *grand*. Il faut toujours accompagner un test d'un **intervalle de confiance de la différence** et d'une **taille d'effet**. Ici, l'IC à 95 % de la différence est [16,7 ; 34,9] € et le d de Cohen (écart divisé par l'écart-type commun) vaut 0,72.

Le client de la boutique dépense en moyenne entre 17 et 35 € de plus (IC à 95 %). Le **d de Cohen** (la différence en nombre d'écarts-types) vaut environ 0,7 : un effet « moyen à grand » selon les conventions usuelles (0,2 petit, 0,5 moyen, 0,8 grand).

Et la distribution asymétrique ? Le test de Student suppose des moyennes à peu près normales (ce que le TCL assure pour des groupes de plus d'une centaine d'observations) ; il reste correct ici. Pour de petits groupes très asymétriques, on teste plutôt $\log(\text{montant})$, ou on utilise un test non paramétrique (§ 3.7). Sur l'échelle logarithmique, le test de Welch donne $t = 6,46$ ($p \approx 5 \times 10^{-10}$) : la conclusion tient.

Même conclusion. Pour Site contre Réseaux, $p \approx 0,011$: l'écart (10,5 €) est aussi significatif au seuil de 5 %, mais bien moins fortement.

Le test apparié. Quand les deux séries concernent **les mêmes individus** (avant/après), on ne compare pas deux groupes indépendants : on calcule la **différence pour chaque individu** et on teste que sa moyenne est nulle. Exemple : 8 colis dont on a mesuré le délai avant (5, 4, 6, 7, 5, 6, 8, 5 jours) et après (4, 4, 5, 6, 5, 5, 6, 4 jours) un changement de transporteur. Les différences sont 1, 0, 1, 1, 0, 1, 2, 1 (moyenne 0,875) et le test apparié donne $t = 3,86$, $p = 0,0062$.

Le nouveau transporteur fait gagner en moyenne 0,875 jour ($p \approx 0,006$). Ignorer l'appariement (comme si les groupes étaient indépendants) donnerait un test beaucoup moins sensible, car on gaspillerait l'information que chaque colis est comparé à lui-même : le test non apparié donnerait ici $p = 0,128$, non significatif, au lieu de 0,006.

3.4.4 Test sur une proportion

Question de la gérante. Historiquement, le taux de conversion était de 18 %. Sur les 1 000 dernières visites, 205 ont acheté (20,5 %). Y a-t-il une amélioration réelle ?

$H_0 : p = 0,18$ contre $H_1 : p \neq 0,18$. Sous H_0 , l'erreur-type est $\sqrt{p_0(1-p_0)/n}$ (on utilise la valeur de H_0 , pas l'estimation) et la statistique

$$z = \frac{\hat{p} - p_0}{\sqrt{p_0(1-p_0)/n}} = \frac{0,205 - 0,18}{\sqrt{0,18 \times 0,82/1000}} = \frac{0,025}{0,01215} \approx 2,06.$$

On compare à la loi normale (TCL). Il existe aussi un test **exact** basé sur la loi binomiale, sans approximation. Le calcul donne $z = 2,058$ et $p = 0,0396$ avec l'approximation normale, $p = 0,0436$ avec le test exact.

Les deux p-valeurs (0,040 et 0,044) sont **juste en dessous** de 0,05. On rejette H_0 , mais de peu : c'est une preuve **modérée**, pas écrasante. Un intervalle de Wilson pour p , [18,1 % ; 23,1 %], inclut à peine 18 %. Lecture honnête : « il y a des indices d'amélioration, à confirmer avec davantage de données ».

3.4.5 Le test A/B : comparer deux proportions

C'est le test le plus utilisé en pratique dans le web et le marketing. La gérante essaie deux versions de sa page produit. La version A (1 000 visiteurs) donne 120 achats (12 %), la version B (1 000 visiteurs) donne 150 achats (15 %). B est-elle meilleure ?

$H_0 : p_A = p_B$. Sous H_0 , les deux groupes ont le même taux, estimé en **regroupant** les données : $\hat{p} = \frac{120+150}{2000} = 0,135$. L'erreur-type de la différence est $\sqrt{\hat{p}(1-\hat{p})(\frac{1}{n_A} + \frac{1}{n_B})}$ et

$$z = \frac{\hat{p}_B - \hat{p}_A}{\sqrt{\hat{p}(1-\hat{p})(\frac{1}{n_A} + \frac{1}{n_B})}} = \frac{0,03}{0,01528} \approx 1,96, \quad p = 0,0496.$$

L'intervalle de confiance de la différence, calculé avec l'erreur-type non regroupée, est [0,0001 ; 0,0599].

$p \approx 0,0496$: **tout juste** sous le seuil de 5 %, et l'intervalle de la différence ([0,0 ; 6 points]) frôle zéro. La conclusion « B est meilleure » est **fragile**. Ce cas, très courant, illustre pourquoi le seuil de 0,05 n'est pas une frontière magique : 0,0496 et 0,0504 ne sont pas deux mondes différents (nous y revenons en 3.5).

3.4.6 Le test du khi-deux : deux variables qualitatives sont-elles liées ?

Question de la gérante. La proportion de clients **satisfaits** (note ≥ 4) dépend-elle du canal de vente ?

On range les données dans un **tableau de contingence** (effectifs par canal et par satisfaction) :

```
df["satisfait"] = df["satisfaction"] >= 4
tableau = pd.crosstab(df["canal"], df["satisfait"])
print(tableau)
```

satisfait	False	True
canal		
Boutique	5	109
Réseaux	51	87
Site	45	103

H_0 : le canal et la satisfaction sont **indépendants**. Si c'était vrai, la proportion de satisfaits serait la même dans chaque canal (et égale à la proportion globale). On calcule alors, pour chaque case, l'**effectif attendu** sous H_0 :

$$E_{ij} = \frac{(\text{total de la ligne } i) \times (\text{total de la colonne } j)}{\text{total général}}.$$

La statistique du khi-deux mesure l'écart entre effectifs observés (O_{ij}) et attendus :

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}.$$

Sous H_0 , elle suit une loi du khi-deux à $(\text{lignes} - 1)(\text{colonnes} - 1)$ degrés de liberté. Plus les écarts sont grands, plus χ^2 est grand.

```
from scipy.stats import chi2_contingency

chi2, p, ddl, attendus = chi2_contingency(tableau)
print(round(chi2, 2), ddl, f"{p:.1e}")
```

```
38.4 2 4.6e-09
```

Les proportions de satisfaits sont de 95,6 % en boutique, 63,0 % sur Réseaux et 69,6 % sur le site. Dans la boutique, il y a **109 satisfaits sur 114** (96 %), alors que l'on en attendrait environ 85 si le canal n'avait aucun effet (soit 24 de plus) ; sur Réseaux, 63 % seulement (87 sur 138). La statistique est $\chi^2 \approx 38$ pour 2 degrés de liberté : $p \approx 5 \times 10^{-9}$. On rejette l'indépendance. Le **V de Cramér** (0 = indépendance, 1 = lien parfait) vaut 0,31 : un lien d'intensité moyenne. (Attention : la boutique n'a pas de délai de livraison, ce qui explique sans doute en grande partie l'écart ; l'association n'est pas une causalité.)

Condition de validité. L'approximation du khi-deux est fiable si **tous les effectifs attendus sont au moins 5**. Sinon, on utilise le test exact de Fisher (`scipy.stats.fisher_exact` pour un tableau 2×2).

3.4.7 Comment choisir son test ?

Question	Données	Test
La moyenne vaut-elle μ_0 ?	1 variable quantitative	Student à un échantillon
Deux groupes indépendants ont-ils la même moyenne ?	quantitative × 2 groupes	Welch
Avant/après sur les mêmes individus ?	quantitatives appariées	Student apparié
Plus de 2 groupes ?	quantitative × k groupes	ANOVA (<code>f_oneway</code>) ou Kruskal-Wallis ($\oplus$ 3.7)
La proportion vaut-elle p_0 ?	1 variable binaire	z (ou binomial exact)
Deux proportions égales ? (A/B)	binaire × 2 groupes	z à deux proportions, ou khi-deux
Deux variables qualitatives liées ?	catégorielle × catégorielle	khi-deux (ou Fisher)
Deux variables quantitatives liées ?	quantitative × quantitative	test de corrélation (<code>pearsonr</code> , <code>spearmanr</code>)

À retenir (tests d'hypothèses).

- On fixe H_0 (« rien ne se passe »), H_1 , α ; on calcule une statistique de test et sa **p-valeur** ; on rejette si $p < \alpha$.
- Deux erreurs : **type I** (faux positif, probabilité α) et **type II** (faux négatif, probabilité β). Puissance = $1 - \beta$.
- « Ne pas rejeter » $\neq$ « accepter ». Une p-valeur faible dit que l'effet est *réel*, pas qu'il est *grand* : donnez toujours l'**IC** de l'effet et une **taille d'effet**.

- Student (une moyenne), **Welch** (deux moyennes), apparié, **z** (proportions), **khi-deux** (deux variables qualitatives).
- Un test bilatéral à 5 % équivaut à regarder si 0 (ou la valeur de H_0) est dans l'IC à 95 %.

■ **Pour s'entraîner.** Cahier, chapitre 3 : application 3.4, exercices 3.6 à 3.8.

3.5 p-valeurs, puissance et tests multiples

Au 3.4, nous avons *utilisé* la p-valeur. Voici maintenant comment ne pas s'en servir de travers. Cette section est celle qui vous évitera le plus d'erreurs concrètes : une bonne partie des « découvertes » publiées qui ne se reproduisent pas viennent de ce que nous allons voir.

3.5.1 Ce que la p-valeur est vraiment

Définition. La **p-valeur** est la probabilité, calculée **en supposant H_0 vraie**, d'obtenir une statistique de test **au moins aussi extrême** que celle observée.

$$p = P(\text{résultat aussi extrême ou plus} \mid H_0).$$

Observez la direction du conditionnement : c'est $P(\text{données} \mid H_0)$ et **non** $P(H_0 \mid \text{données})$. Nous avons vu au 2.1 que **inverser un conditionnement est l'erreur classique**.

Pour la sentir, rien de mieux que de fabriquer un monde où H_0 est vraie et de regarder les p-valeurs qu'on obtient. Simulons 10 000 tests de Student de deux groupes de 30 individus **tirés dans la même loi** (donc aucune vraie différence). Résultat : 4,86 % des p-valeurs sont inférieures à 0,05, 1,03 % à 0,01, et 50,1 % à 0,5 ; leur moyenne vaut 0,499.

Quand H_0 est vraie, **la p-valeur suit une loi uniforme sur $[0, 1]$** : 5 % des tests donnent $p < 0,05$, 1 % donnent $p < 0,01$, etc. C'est exactement ce que signifie « niveau $\alpha = 5\%$ » : **un test sur vingt crie au loup à tort** quand il n'y a rien. Ce n'est pas un défaut du test, c'est sa définition.

Et quand H_0 est **fausse** ? Avec un vrai effet ($d = 0,8$), 86,5 % des tests donnent $p < 0,05$, et les p-valeurs se tassent vers 0 : en 10 classes de largeur 0,1, l'histogramme sous H_0 est plat (980, 1 008, 1 028, 1 006, 991, 985, 1 036, 964, 1 024, 978), alors que sous H_1 il est entassé à gauche (9 248, 419, 160, 62, 47, 25, 16, 9, 7, 7).

Sous H_0 , l'histogramme est **plat** ; sous H_1 , il est entassé à gauche. C'est un outil de diagnostic précieux : si vous testez des milliers de variables et que l'histogramme de vos p-valeurs est plat, il n'y a probablement **rien** à trouver.

3.5.2 Ce que la p-valeur n'est pas

Cinq contresens fréquents. Une p-valeur de 0,03 ne signifie **pas** :

1. que H_0 a 3 % de chances d'être vraie ;
2. que H_1 a 97 % de chances d'être vraie ;
3. que l'effet est **grand** ou **important** ;
4. que l'on obtiendrait de nouveau $p < 0,05$ en refaisant l'étude (la **reproductibilité** dépend de la puissance) ;
5. qu'on a 3 % de chances de se tromper en rejetant H_0 .

Le point 5 mérite une démonstration, car il est lourd de conséquences. Le risque réel de se tromper quand on rejette dépend de la **proportion d'hypothèses qui sont vraies** au départ : on retrouve la formule de Bayes de la section 2.1 et l'erreur du taux de base !

Exemple. La gérante teste 1 000 idées d'amélioration (couleur d'un bouton, texte d'une promotion, ordre des produits...). Réalistement, **10 %** seulement ont un vrai effet (100 vraies idées, 900 inutiles). Son test a un niveau $\alpha = 5\%$ et une puissance de 80 %.

- Vraies idées détectées : $100 \times 0,80 = 80$.
- Idées inutiles « détectées » à tort : $900 \times 0,05 = 45$.
- Au total, 125 résultats « significatifs », dont **45 sont des faux positifs**.

$$P(\text{fausse découverte} \mid \text{significatif}) = \frac{45}{125} = 36\%.$$

Plus d'un résultat « significatif » sur trois est faux, alors que α n'est que de 5 % ! Même mécanisme que l'alerte antifraude du 2.1.6. C'est pourquoi on exige des preuves plus fortes pour des hypothèses peu plausibles a priori (« des affirmations extraordinaires exigent des preuves extraordinaires »).

3.5.3 Signification statistique $\neq$ importance pratique

Avec assez de données, **n'importe quelle** différence, même ridicule, devient « significative ». Un exemple extrême : deux versions d'une page ont des taux de conversion de 20,00 % et 20,10 %, mesurés sur 10 millions de visiteurs chacune. Le test à deux proportions donne $z = 5,58$ et $p = 2,3 \times 10^{-8}$: l'écart est hautement « significatif », alors que le gain n'est que de **0,1 point** de pourcentage.

La p-valeur est minuscule, mais le gain est de **0,1 point** de conversion. Est-il utile ? Cela dépend du coût du changement, pas de la p-valeur. **Toujours rapporter la taille de l'effet et son intervalle de confiance**, jamais seulement « $p < 0,05$ ».

3.5.4 La puissance : savoir si l'on peut voir ce qu'on cherche

Intuition. Un test est comme un détecteur de métaux. Un détecteur peu sensible ne signale pas un petit objet enterré profondément : **l'absence de signal ne prouve pas l'absence d'objet**. La **puissance** $1 - \beta$ est la probabilité que le test détecte un effet **s'il existe vraiment** (de taille donnée).

La puissance dépend de quatre choses liées entre elles :

Facteur	Si...	...alors la puissance
Taille de l'effet	grandit	augmente
Taille d'échantillon n	grandit	augmente
Dispersion des données	diminue	augmente
Niveau α	on l'assouplit (0,10 au lieu de 0,05)	augmente (au prix de plus de faux positifs)

Retour sur notre test A/B du 3.4.5 (12 % contre 15 %, 1 000 visiteurs par version). Quelle était la puissance de cette expérience ? Sous H_1 avec $p_A = 0,12$ et $p_B = 0,15$, l'erreur-type de la différence

est $\sqrt{\frac{0,12 \times 0,88}{1000} + \frac{0,15 \times 0,85}{1000}} = 0,0154$, et la statistique de test est centrée sur $0,03/0,0154 = 1,95$. La puissance est donc la probabilité que cette statistique dépasse 1,96 :

$$\text{puissance} = P(Z > 1,96 - 1,95) \approx 0,50.$$

Le calcul exact donne 0,502, et une simulation de 10 000 expériences identiques retrouve 0,495 : la formule est bonne.

La puissance n'était que de **50 %** : même si B est réellement meilleure de 3 points, l'expérience n'avait qu'une **chance sur deux** de le détecter. Le résultat « tout juste significatif » obtenu était donc de la chance autant que de l'information. Un test sous-dimensionné est un pari.

Dimensionner l'expérience avant de la lancer. On fixe l'effet minimal intéressant (ici +3 points), $\alpha = 5\%$ et la puissance voulue (80 % est l'usage), puis on calcule n :

$$n \text{ par groupe} = \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 [p_1(1-p_1) + p_2(1-p_2)]}{(p_2 - p_1)^2}.$$

Effet à détecter	12 % → 17 %	12 % → 15 %	12 % → 14 %	12 % → 13 %
Visiteurs par version	775	2 033	4 435	17 166

Pour détecter 3 points avec 80 % de puissance, il faut **environ 2 000 visiteurs par version**, soit le double de ce que la gérante avait. Pour détecter 1 point, il en faut **près de 18 000**. La loi en $1/\text{effet}^2$ est impitoyable : diviser l'effet par 3 multiplie les besoins par 9.

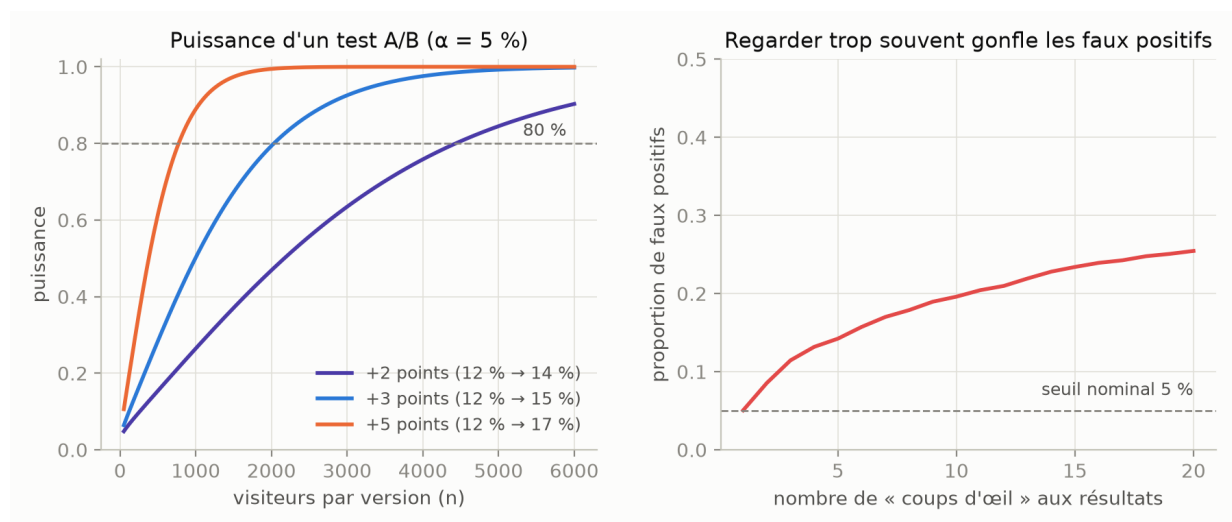


Figure 22: À gauche : puissance d'un test A/B en fonction du nombre de visiteurs, pour trois tailles d'effet. À droite : si l'on « jette un œil » aux résultats de plus en plus souvent et que l'on s'arrête dès que $p < 0,05$, le taux de faux positifs explose (sous H_0).

L'arrêt prématuré (peeking). Dans une expérience en ligne, la tentation est forte de regarder les résultats chaque jour et de s'arrêter dès que $p < 0,05$. La figure de droite montre le résultat : en regardant 20 fois, le **taux de faux positifs passe de 5 % à environ 25 %**, alors qu'il n'y a *aucun* effet réel. Règle : **fixer la taille d'échantillon à l'avance et ne conclure qu'à la fin** (ou

utiliser des méthodes séquentielles conçues pour cela).

3.5.5 Les tests multiples : le piège du « fouillis de comparaisons »

Intuition. Si vous lancez un dé 20 fois, vous obtiendrez presque sûrement un 6 quelque part. Si vous effectuez 20 tests à 5 %, il est presque sûr que l'un d'eux sera « significatif » par pur hasard.

Raisonnons comme au 2.1.4 (« au moins un ») : si les 20 tests sont indépendants et que toutes les hypothèses nulles sont vraies, la probabilité d'avoir **au moins un faux positif** est

$$1 - (1 - \alpha)^m = 1 - 0,95^{20} \approx 0,64.$$

Nombre de tests m	1	5	10	20	50	100
$P(\text{au moins un faux positif})$	0,050	0,226	0,401	0,642	0,923	0,994

Avec 100 tests, c'est quasi certain (99,4 %). Dès que l'on teste plusieurs variables, segments ou métriques, il faut en tenir compte. C'est exactement ce qui arrive quand on « fouille » un jeu de données : on regarde 40 sous-groupes et on rapporte celui qui sort (« les femmes de 25 à 34 ans achètent plus le jeudi »).

Les corrections classiques. On teste m hypothèses nulles, avec les p-valeurs $p_1, \dots, p_m$.

- **Bonferroni** : on rejette H_i si $p_i < \alpha/m$. Simple, très prudent (le **FWER**, probabilité d'au moins un faux positif, reste $\leq \alpha$) mais il perd beaucoup de puissance quand m est grand.
- **Holm** : trie les p-valeurs et applique des seuils $\alpha/m, \alpha/(m-1), \dots$; **toujours** au moins aussi puissant que Bonferroni, avec la même garantie. À préférer.
- **Benjamini-Hochberg (BH)** : contrôle non plus le risque d'un seul faux positif, mais la **proportion de fausses découvertes** parmi les rejets (le **FDR**, *false discovery rate*). Moins strict, beaucoup plus puissant. Idéal en exploration (criblage de centaines de variables).

Procédure de Benjamini-Hochberg. Trier les p-valeurs : $p_{(1)} \leq \dots \leq p_{(m)}$. Trouver le plus grand k tel que $p_{(k)} \leq \frac{k}{m} \alpha$. Rejeter les hypothèses correspondant à $p_{(1)}, \dots, p_{(k)}$.

Mettons-les à l'épreuve dans une simulation réaliste : la gérante compare 100 catégories de produits entre deux périodes. Parmi elles, **10** ont vraiment changé (effet $d = 1$) et **90** n'ont pas bougé. Chaque comparaison utilise 40 observations par période. Les résultats sont :

Méthode	Découvertes	Vraies	Fausse
Aucune correction ($p < 0,05$)	12	9	3
Bonferroni	7	7	0
Holm	7	7	0
Benjamini-Hochberg	9	9	0

Lecture (pour cette graine) : sans correction, on « découvre » 12 effets, dont **3 sont de fausses alertes** (sur 90 hypothèses nulles, on s'attend à environ 4,5 faux positifs à 5 %). Bonferroni et Holm n'en gardent que 7, **tous vrais**, mais au prix d'avoir **raté** 3 vrais effets. Benjamini-Hochberg

en retrouve 9 vrais sans aucun faux ici ; par construction, il garantit seulement qu'en moyenne la part de fausses découvertes reste sous 5 %. Le compromis est net : plus on corrige strictement, moins on se trompe, mais plus on rate de vrais effets.

Quel choix pratique ?

- Décision importante, peu d'hypothèses, un faux positif coûteux (lancer un produit, un traitement) : **Holm** (ou Bonferroni).
- Exploration de nombreuses hypothèses, où l'on vérifiera ensuite les candidats : **Benjamini-Hochberg**.
- Le mieux de tout : **décider à l'avance** de la ou des questions testées. Une analyse exploratoire est une source d'hypothèses, pas une preuve.

3.5.6 Les bonnes pratiques, en dix lignes

1. Écrire H_0 , H_1 , α et le plan d'analyse **avant** de regarder les données.
2. Dimensionner l'échantillon pour une puissance d'au moins 80 %.
3. Ne pas s'arrêter dès que $p < 0,05$ (*peeking*).
4. Compter **tous** les tests effectués, pas seulement ceux qui « marchent », et corriger si nécessaire.
5. Rapporter la **taille d'effet** et un **intervalle de confiance**, pas seulement la p-valeur.
6. Distinguer significativité statistique et importance pratique.
7. Ne pas dire « accepter H_0 » : dire « pas de preuve suffisante ».
8. Se méfier d'un résultat « juste significatif » (0,04) : il est fragile.
9. Marquer clairement ce qui est **exploratoire** et ce qui est **confirmatoire**.
10. Refaire l'expérience si la décision est importante : la **réplication** est la meilleure preuve.

À retenir (p-valeurs, puissance, tests multiples).

- $p = P(\text{données aussi extrêmes} \mid H_0)$. Sous H_0 , elle est **uniforme** sur $[0, 1]$; 5 % des tests donnent $p < 0,05$ par hasard.
- Ce n'est ni $P(H_0 \mid \text{données})$, ni la taille de l'effet. Le taux de fausses découvertes dépend de la proportion d'hypothèses vraies (Bayes).
- **Puissance** = $1 - \beta$: dépend de l'effet, de n , de la dispersion et de α . Dimensionner avant l'expérience : $n \propto 1/\text{effet}^2$.
- **Tests multiples** : $1 - (1 - \alpha)^m$; corriger par **Holm** (FWER) ou **Benjamini-Hochberg** (FDR). Pas de *peeking*, pas de *p-hacking*.

■ **Pour s'entraîner.** Cahier, chapitre 3 : application 3.5, exercices 3.9 et 3.10.

3.6 ⊕ Pour aller plus loin : les sondages et l'échantillonnage

Section optionnelle. Tout ce chapitre suppose que l'échantillon est « tiré au hasard dans la population ». Mais **comment** l'obtient-on, et que se passe-t-il quand ce n'est pas le cas ? La théorie des sondages répond à ces questions, essentielles pour une enquête de satisfaction, une étude de marché, ou tout jeu de données dont on ne maîtrise pas la collecte.

3.6.1 Le biais de sélection : le pire ennemi

Une leçon historique. En 1936, le magazine *Literary Digest* prédit la défaite de Roosevelt à l'élection américaine, d'après plus de **2 millions** de réponses reçues à son questionnaire. Roosevelt a été réélu largement. Au même moment, un jeune institut de sondage, avec un échantillon de quelques milliers de personnes seulement mais mieux choisi, avait prévu la victoire. Le magazine avait sollicité ses abonnés, des annuaires et des propriétaires de voitures : des personnes **plus aisées que la moyenne** des électeurs. Aucune quantité de données ne corrige un échantillon qui **ne représente pas** la population.

C'est la leçon centrale de cette section : **la taille de l'échantillon réduit la variance, pas le biais.** Un million d'observations mal choisies donne un résultat précis... et faux.

Les formes de biais les plus courantes :

Biais	Mécanisme	Exemple pour la boutique
Sélection	la méthode de recrutement favorise certains profils	enquête par e-mail : seuls les clients déjà inscrits à la newsletter répondent
Non-réponse	les répondants diffèrent des non-répondants	seuls les clients très contents (ou très fâchés) répondent
Survie	on n'observe que ceux « qui restent »	analyser uniquement les clients encore actifs surestime la satisfaction
Couverture	une partie de la population n'est pas dans la base	un sondage en ligne ignore les clients sans accès à Internet

3.6.2 L'échantillonnage aléatoire simple

Dans un **échantillon aléatoire simple** (EAS), chaque individu de la base de sondage a la **même probabilité** d'être choisi, et tous les groupes de n individus sont également probables. C'est le modèle de tout ce que nous avons fait. L'estimateur de la moyenne est $\bar{x}$; quand on tire **sans remise** dans une population de taille finie N , l'erreur-type est corrigée par le **facteur de population finie** :

$$SE(\bar{x}) = \frac{\sigma}{\sqrt{n}} \sqrt{1 - \frac{n}{N}}.$$

Si l'on interroge une grande part de la population (n/N non négligeable), l'incertitude diminue plus vite. Si $n \ll N$ (le cas habituel), le facteur vaut presque 1 : **ce qui compte, c'est n , pas la fraction interrogée**. Voilà pourquoi sonder 1 000 personnes suffit autant pour un pays de 10 millions d'habitants que pour une ville de 100 000.

La marge d'erreur d'un sondage. Pour une proportion estimée à $\hat{p}$ avec n personnes, la marge d'erreur à 95 % est $1,96 \sqrt{\hat{p}(1 - \hat{p})/n}$. Son maximum est atteint pour $\hat{p} = 0,5$, ce qui donne la **règle à retenir** : marge $\approx \frac{1}{\sqrt{n}}$.

Taille n	100	400	1 000	2 500	10 000
Marge d'erreur maximale	$\pm 9,8$ points	$\pm 4,9$ points	$\pm 3,1$ points	$\pm 2,0$ points	$\pm 1,0$ point
Règle $1/\sqrt{n}$	$\pm 10,0$	$\pm 5,0$	$\pm 3,2$	$\pm 2,0$	$\pm 1,0$

Avec 1 000 personnes : $\pm 3,1$ points. Pour obtenir ± 1 point, il en faut près de 10 000. C'est pourquoi les sondages nationaux s'arrêtent le plus souvent autour de 1 000 à 2 000 personnes. Et cette marge ne couvre que **l'erreur d'échantillonnage** : elle ne dit rien du biais de sélection ou de non-réponse, qui sont souvent plus grands.

3.6.3 L'échantillonnage stratifié

Intuition. Si la population est composée de groupes **homogènes en eux-mêmes mais différents entre eux** (les canaux de vente !), il est dommage de laisser le hasard décider combien de chaque groupe tombera dans l'échantillon. On **découpe** la population en **strates** et on tire un échantillon aléatoire **dans chaque strate**, en proportion de sa taille. On garantit ainsi une représentation fidèle, et l'on gagne en précision.

L'**estimateur stratifié** pondère les moyennes de strates par leur poids dans la population : $\bar{x}_{\text{strat}} = \sum_h W_h \bar{x}_h$ avec $W_h = N_h/N$.

Montrons le gain par simulation. La base clients de la boutique compte 10 000 personnes réparties en trois canaux (4 000, 3 500 et 2 500 clients), dont les dépenses moyennes diffèrent nettement. La vraie dépense moyenne de cette population est 56,4 €. On tire 5 000 fois un échantillon de 200 clients, d'abord au hasard dans toute la base (EAS), puis en stratifiant par canal (80 clients de Réseaux, 70 du site, 50 de la boutique). Dans les deux cas, la moyenne des estimations est 56,39 € ; l'erreur-type vaut 2,46 € pour l'EAS et 2,36 € pour l'estimateur stratifié (soit 8,1 % de variance en moins).

Les deux estimateurs sont **sans biais** (leur moyenne tombe sur la vraie valeur), mais l'estimateur stratifié est **plus précis** : son erreur-type (2,36 €) est inférieure d'environ 4 % à celle de l'EAS (2,46 €), soit 8 % de variance en moins. Le gain est modeste ici car les différences entre canaux, bien que réelles, restent petites comparées à la dispersion *à l'intérieur* de chaque canal. Il serait bien plus grand si les strates étaient très différentes entre elles.

Pourquoi ça marche : décomposition de la variance. La variance totale se décompose en variance **entre** strates et variance **à l'intérieur** des strates : $\sigma^2 = \sigma_{\text{entre}}^2 + \sigma_{\text{intra}}^2$. Dans un EAS, le hasard de la composition de l'échantillon introduit l'incertitude liée à la variance *entre* strates. La stratification **fixe** cette composition, et seule la variance *intra* demeure : l'erreur-type diminue exactement de la part « entre ».

L'allocation de Neyman. On peut aller plus loin : au lieu d'allouer proportionnellement à la taille, on interroge **davantage** les strates **plus hétérogènes** (de grand écart-type) : $n_h \propto N_h \sigma_h$. Ici, la boutique est la plus dispersée (écart-type de ses dépenses plus élevé en valeur absolue) : on gagnerait à en sur-échantillonner un peu.

3.6.4 Autres plans de sondage

Plan	Principe	Avantage	Inconvénient
Systématique	un individu tous les k dans la liste	simple	biais si la liste a une périodicité
Par grappes	on tire des groupes entiers (magasins, classes) puis on interroge tout le groupe	peu coûteux (déplacements)	moins précis (individus d'une grappe se ressemblent)
À plusieurs degrés	tirage de grappes puis d'individus dans les grappes	pratique pour de vastes populations	calcul d'erreur plus complexe
Par quotas	on remplit des quotas (âge, sexe...) sans tirage aléatoire	rapide, peu coûteux	pas de théorie d'erreur rigoureuse
De convenance	on prend ceux qui sont disponibles	très facile	biais incontrôlable

Les plans aléatoires (EAS, stratifié, grappes) permettent de **quantifier** l'incertitude ; les plans de quotas ou de convenance non. C'est une raison de plus de se méfier des « sondages » de réseaux sociaux.

3.6.5 Redresser un échantillon biaisé : la pondération

On ne choisit pas toujours son échantillon. La gérante envoie un questionnaire de satisfaction à tous ses clients ; les **réponses sont inégalement réparties** : les clients de la boutique répondent très peu (ils ne laissent pas d'e-mail), ceux venus des réseaux sociaux beaucoup. Parmi les 300 réponses : 150 du canal Réseaux, 120 du site et 30 de la boutique, alors que la clientèle réelle est répartie en 40 % / 35 % / 25 %.

Si l'on moyenne naïvement les 300 réponses, la boutique est **sous-représentée** (10 % au lieu de 25 %). Or les clients de la boutique sont aussi les plus satisfaits : on **sous-estime** donc la satisfaction globale. La solution est de **pondérer** chaque réponse par $w = \frac{\text{part dans la population}}{\text{part dans l'échantillon}}$ (une *post-stratification*). Les taux de satisfaits par canal sont ceux du 3.4.6 (63 %, 70 % et 96 %). Les poids valent $0,40/0,50 = 0,8$ pour Réseaux, $0,35/0,40 \approx 0,87$ pour le site et $0,25/0,10 = 2,5$ pour la boutique.

- **Moyenne naïve** : $\frac{150 \times 0,63 + 120 \times 0,70 + 30 \times 0,96}{300} = \frac{207,3}{300} \approx 0,691$.
- **Moyenne pondérée** (chaque canal compte pour sa vraie part) : $0,40 \times 0,63 + 0,35 \times 0,70 + 0,25 \times 0,96 = 0,737$.

La moyenne naïve (environ 69 %) sous-estime la vraie valeur (73,7 %) ; la pondération corrige l'erreur. Chaque réponse de la boutique « compte pour » 2,5 réponses (poids 2,5) et chaque réponse du canal Réseaux pour 0,8. (Cette correction n'est valable que si, **à l'intérieur de chaque canal**, répondants et non-répondants sont comparables : la pondération redresse les déséquilibres **observables**, pas ceux que l'on ne mesure pas.)

À retenir (sondages).

- **La taille ne corrige pas le biais** (*Literary Digest*) ; ce qui compte, c'est la qualité du tirage.
- EAS : marge d'erreur $\approx 1/\sqrt{n}$ (± 3 points pour 1 000 personnes), indépendante de la

taille de la population si elle est grande.

- **Stratification** : on tire dans chaque groupe, en proportion ; estimateur $\sum W_h \bar{x}_h$, plus précis que l'EAS quand les strates diffèrent.
- Plans par grappes, de quotas, de convenance : moins précis ou sans théorie d'erreur.
- On redresse un échantillon déséquilibré par **pondération** ($w = \text{part population} / \text{part échantillon}$), sous réserve de comparabilité à l'intérieur des groupes.

■ **Pour s'entraîner.** Cahier, chapitre 3 : application 3.6, exercice 3.11.

3.7 ⊕ Pour aller plus loin : les méthodes non paramétriques

Section optionnelle. Les tests du 3.4 (Student, Welch) supposent, au moins approximativement, une loi normale des moyennes. Les méthodes **non paramétriques** (ou *sans loi*) évitent de postuler une forme de distribution. Elles sont précieuses pour de petits échantillons, des variables ordinales (notes de 1 à 5) ou des données très asymétriques et pleines de valeurs extrêmes.

3.7.1 Remplacer les valeurs par leurs rangs

Intuition. Au lieu de travailler sur les valeurs, on les **range** du plus petit au plus grand et l'on travaille sur leurs **rangs** (1er, 2e, 3e...). Une valeur extrême de 1 000 000 n'a que le rang « dernier » : elle ne peut plus fausser le résultat. Les rangs perdent un peu d'information (l'ampleur des écarts), mais gagnent une **robustesse** considérable.

Pour six commandes de 12, 15, 14, 10, 13 et 40 €, les rangs sont 2, 5, 4, 1, 3 et 6 : l'extrême (40) reçoit simplement le rang 6, le même qu'il aurait eu en valant 16.

3.7.2 Le test de Mann-Whitney (deux groupes indépendants)

C'est l'équivalent non paramétrique du test de Welch. On mélange les deux groupes, on range toutes les valeurs, puis on regarde si les rangs d'un groupe sont systématiquement plus élevés que ceux de l'autre.

Interprétation très parlante. La statistique $U/(n_1 n_2)$ est la probabilité qu'une observation tirée au hasard dans le groupe A **dépasse** une observation tirée au hasard dans le groupe B. Valeur 0,5 : aucune différence ; proche de 1 : A domine B. (C'est aussi l'**AUC** du volume II.)

Un cas où le test de Student se trompe. Deux petits groupes de 6 commandes : 12, 15, 14, 10, 13 et 40 € d'un côté, 9, 8, 11, 10, 7 et 12 € de l'autre (moyennes 17,3 et 9,5 €). Dans le premier, un client dépense une somme exceptionnelle. Le test de Welch donne $p = 0,15$, celui de Mann-Whitney $p = 0,02$.

Presque **tous** les clients du premier groupe dépensent plus que ceux du second ; seul le cas de 40 gonfle la variance et noie l'effet dans le test de Student ($p \approx 0,15$, non significatif). Le test de Mann-Whitney, lui, voit la domination systématique du groupe A ($p \approx 0,02$, significatif). C'est le gain de puissance de la robustesse quand les données sont « sales ».

Sur nos 400 commandes (boutique contre Réseaux), une seule instruction de `scipy` suffit :

```
u = stats.mannwhitneyu(b, i)          # b, i : montants de la boutique et du canal Réseaux (3.4.3)
print(u.statistic, f"{u.pvalue:.1e}")
```

```
11246.0 4.4e-09
```

Une commande de la boutique dépasse une commande du canal Réseaux dans 71,5 % des paires comparées ($U/(n_1 n_2) = 11\,246/(114 \times 138) = 0,715$; $p \approx 4 \times 10^{-9}$). Conclusion identique à celle du test de Welch, avec une interprétation plus intuitive.

3.7.3 Autres tests de rangs

Situation	Test paramétrique	Équivalent non paramétrique
2 groupes indépendants	Welch	Mann-Whitney (<code>mannwhitneyu</code>)
2 séries appariées	Student apparié	Wilcoxon des rangs signés (<code>wilcoxon</code>)
$k > 2$ groupes	ANOVA (<code>f_oneway</code>)	Kruskal-Wallis (<code>kruskal</code>)
Corrélation	Pearson	Spearman / Kendall (<code>spearmanr</code> , <code>kendalltau</code>)

Sur nos 400 commandes, ces tests donnent : Wilcoxon sur les 8 colis du 3.4.3, $p = 0,031$; ANOVA sur les trois canaux, $p = 3,4 \times 10^{-7}$, et Kruskal-Wallis, $p = 9,4 \times 10^{-9}$; corrélation entre délai et satisfaction : Pearson $-0,532$, Spearman $-0,511$, Kendall $-0,437$ (p-valeurs arrondies à 0). Pour la satisfaction (note de 1 à 5, **ordinaire**), Spearman et Kendall sont plus appropriés que Pearson. Dans les trois cas, le lien entre délai et satisfaction est très significatif.

Quand choisir le non paramétrique ? Données ordinales ; petits échantillons ($n < 20$) d'allure non normale ; valeurs extrêmes qu'on ne veut pas supprimer. Si les données sont vraiment normales, le test de Student est un peu plus puissant (de l'ordre de 5 %) : le non paramétrique est une **assurance bon marché**.

3.7.4 Les tests de permutation : l'idée la plus simple de la statistique

Intuition. H_0 dit : « le canal n'a aucun effet sur le montant ». Si c'est vrai, l'étiquette « boutique » ou « Réseaux » collée sur une commande est **arbitraire** : on aurait pu l'échanger avec n'importe quelle autre. Alors **mélangeons** les étiquettes au hasard, recalculons la différence de moyennes, et recommençons des milliers de fois. On obtient ainsi la **distribution de la différence quand H_0 est vraie**, sans aucune hypothèse de loi. La p-valeur est la fréquence des mélanges qui donnent une différence au moins aussi grande que celle observée.

Sur nos données, avec 20 000 mélanges, aucun n'atteint la différence observée de 25,8 € : la différence maximale obtenue par hasard est bien plus petite. On majore donc la p-valeur par $1/20\,001 \approx 5 \times 10^{-5}$ (le « +1 » évite de déclarer $p = 0$). Faisons maintenant la même chose sur la petite expérience à 6 + 6 commandes, où l'on peut même énumérer toutes les permutations possibles : il y en a 924, et la p-valeur exacte est 0,0173.

Il y a $\binom{12}{6} = 924$ manières de répartir les 12 valeurs en deux groupes (clin d'œil au 1.6 !) ; la p-valeur est la proportion de ces 924 répartitions dont l'écart de moyennes est au moins aussi grand que celui observé. Le test est **exact** et n'a besoin d'aucune hypothèse. Il est très souple : on peut l'appliquer à **n'importe quelle statistique** (médiane, rapport, corrélation), comme le bootstrap.

3.7.5 Tester la normalité

Comment savoir si l'hypothèse de normalité du test de Student est raisonnable ? Deux outils.

Le diagramme quantile-quantile (QQ-plot) : on compare les quantiles des données à ceux d'une loi normale ; si les points suivent la droite, c'est normal. **Le test de Shapiro-Wilk** : H_0 = « les données sont normales ». Sur nos montants, il rejette nettement la normalité ($p = 3 \times 10^{-18}$) ; sur leur logarithme, il ne rejette pas ($p = 0,846$, et le test de Kolmogorov-Smirnov donne $p = 0,879$).

Le montant brut est **clairement non normal** ($p \approx 10^{-18}$), alors que son logarithme est tout à fait compatible avec la normalité (pas de rejet : $p \approx 0,85$), ce qui confirme la structure **log-normale** vue au 3.1.5. Le **test de Kolmogorov-Smirnov** compare la fonction de répartition observée à celle d'une loi donnée (ou deux échantillons entre eux).

Piège : tester la normalité n'est pas toujours utile. Avec beaucoup de données, ces tests rejettent la normalité pour des écarts infimes sans conséquence ; avec peu de données, ils ne détectent rien. On s'appuie surtout sur les **graphiques** et sur le **TCL** : la normalité de la **moyenne** (ce qui compte pour Student) est assurée dès que n est grand, même si les données ne sont pas normales.

À retenir (non paramétrique).

- Travailler sur les **rangs** rend robuste aux valeurs extrêmes et applicable aux données ordinales.
- **Mann-Whitney** (2 groupes), **Wilcoxon** (apparié), **Kruskal-Wallis** (k groupes), **Spearman/Kendall** (corrélation).
- **Test de permutation** : on mélange les étiquettes pour fabriquer la loi de la statistique sous H_0 ; valable pour toute statistique.
- **Shapiro-Wilk** et **Kolmogorov-Smirnov** testent une loi, mais préférez les graphiques et le TCL.

■ **Pour s'entraîner.** Cahier, chapitre 3 : application 3.7, exercice 3.12.

Bilan du chapitre 3

Vous savez maintenant :

- **décrire** un jeu de données (position, dispersion, forme, relations) et **toujours dessiner avant de calculer** ;
- **estimer** un paramètre (moments, maximum de vraisemblance), juger un estimateur par son biais et sa variance, et savoir pourquoi on divise par $n - 1$;
- **quantifier l'incertitude** par des intervalles de confiance (Student, Wilson, bootstrap), sans en faire une mauvaise lecture ;
- **tester** une hypothèse (Student, Welch, proportions, A/B, khi-deux) en distinguant significativité et importance pratique ;
- **dimensionner** une expérience (puissance), et **éviter les faux positifs** (tests multiples, *peeking*) ;
- (en option) **échantillonner** correctement et employer des méthodes **sans hypothèse de loi**.

Le chapitre 4 change de registre : après la théorie, la **pratique du code**. Python, R, algorithmes, NumPy, pandas, visualisations : ce sont les outils qui permettront d'appliquer tout ce que vous venez d'apprendre sur de vraies données.

Pour s'entraîner. Le cahier d'exercices du volume I (chapitre 3) rassemble sept applications guidées sur le jeu de 400 commandes et douze exercices corrigés, du calcul à la main à la synthèse.

Chapitre 4 : Programmation

« Les mathématiques vous disent **quoi** calculer. La programmation vous permet de le calculer sur **un million de lignes**. »

Jusqu'ici, nous avons utilisé du code comme un outil de vérification. Ce chapitre prend le code **au sérieux** : c'est lui qui transforme vos idées en résultats reproductibles. Pas besoin d'avoir jamais programmé : on part de zéro. Si vous programmez déjà, parcourez les premières sections en diagonale et attardez-vous sur NumPy, pandas et les visualisations.

Le chemin de ce chapitre

- **4.1 Python** : le langage de ce livre, des variables aux fonctions, avec un petit programme complet (un ticket de caisse).
- **4.2 R** : l'autre grand langage de la statistique ; on refait les mêmes analyses pour comparer.
- **4.3 Algorithmes et structures de données** : piles, files, dictionnaires, récursion, tris, recherche. Penser comme un informaticien.
- **4.4 NumPy et pandas** : les deux bibliothèques qui font de Python un outil d'analyse de données.
- **4.5 Visualisations** : choisir le bon graphique, le tracer proprement, avec matplotlib, pandas, seaborn et ggplot2.
- **⊕ Pour aller plus loin** : programmation orientée objet, code propre et tests (4.6) ; d'autres langages (4.7) ; complexité algorithmique (4.8).
- Le **bilan** du chapitre ; les **applications et exercices corrigés** sont dans le **cahier** du volume.

Comment travailler avec ce chapitre. Tapez le code vous-même plutôt que de le copier : c'est la seule façon d'apprendre à programmer. Modifiez-le, cassez-le, observez les messages d'erreur. Ils sont vos amis : un message d'erreur lu attentivement dit presque toujours où est le problème. Pour exécuter du code Python, vous pouvez utiliser un notebook Jupyter (section 6.2) ou simplement un terminal avec la commande `python`. Dans ce chapitre, le code est le sujet : il reste donc visible, mais **par petits morceaux**, toujours introduits et commentés. Les programmes complets et les études plus longues sont dans le cahier (chapitre 4).

Le fichier de données. Au chapitre 3, nous avons construit un tableau de 400 commandes. Il a été enregistré dans le fichier `donnees/commandes.csv`, fourni avec le livre (c'est la sortie de `df.to_csv("donnees/commandes.csv", index=False)` appliqué au tableau du 3.1.2). Nous l'utiliserons pour les sections 4.2, 4.4 et 4.5, ainsi qu'au chapitre 5 (SQL).

4.1 Les fondamentaux de Python

Intuition. Un programme est une **recette de cuisine** écrite pour un exécutant très docile mais totalement dépourvu de bon sens : il fait *exactement* ce que vous écrivez, à une vitesse folle, sans jamais se fatiguer, et sans jamais deviner ce que vous vouliez dire. Apprendre à programmer, c'est apprendre à écrire des recettes sans ambiguïté. Python est un excellent choix pour cela : ses recettes se lisent presque comme des phrases.

Dans cette section, nous partons de zéro et nous terminons par un **programme complet** : le ticket de caisse d'une boutique. Chaque notion suit le même rythme que dans le reste du livre : une image, un exemple fait à la main, puis un court morceau de code et sa sortie réelle.

Section à lire dans l'ordre. Si vous programmez déjà, lisez seulement les titres et les encadrés $\triangle$, puis passez au petit programme final (4.1.10) pour vérifier que tout vous semble familier.

4.1.1 Pourquoi Python, et comment l'exécuter

Python est gratuit, lisible, et il dispose de milliers de bibliothèques pour les données (NumPy, pandas, SciPy, matplotlib..., que nous rencontrerons en 4.4 et 4.5). C'est le langage le plus utilisé en data science, et c'est celui des exemples de ce livre.

Il y a trois façons d'exécuter du code Python :

1. **Interactivement**, en tapant python dans un terminal : on écrit une ligne, on voit le résultat tout de suite. Idéal pour essayer.
2. **Dans un fichier** `mon_programme.py`, lancé avec `python mon_programme.py`. Idéal pour garder et rejouer son travail.
3. **Dans un notebook Jupyter** (section 6.2) : des cellules de code mélangées à du texte. Idéal pour explorer et raconter.

Le tout premier programme du monde informatique :

```
print("Bonjour la boutique !")
print(2 + 3 * 4)
```

```
Bonjour la boutique !
14
```

La fonction `print` affiche ce qu'on lui donne. La deuxième ligne montre que Python respecte la priorité habituelle des opérations : 3×4 d'abord, puis $+2$, soit 14.

Les commentaires. Tout ce qui suit un `#` sur une ligne est ignoré par Python : c'est une note pour le lecteur humain (vous, dans six mois). Un bon commentaire explique **pourquoi**, pas **quoi**.

4.1.2 Variables et types

Une **variable** est une étiquette collée sur une valeur. L'instruction `prix = 12.5` se lit : « colle l'étiquette `prix` sur la valeur 12,5 ». Ce n'est **pas** une égalité mathématique : à droite on calcule, à gauche on nomme.

Les quatre types de base :

Type	Nom Python	Exemple	À quoi ça sert
Entier	<code>int</code>	3	compter (articles, jours)
Décimal	<code>float</code>	12.5	mesurer (prix, poids) ; séparateur : le point
Texte	<code>str</code>	"bol"	noms, étiquettes
Booléen	<code>bool</code>	True, False	vrai ou faux

Exemple à la main. La gérante vend 3 bols à 12,5 € hors taxe. Le total HT est $3 \times 12,5 = 37,5$ €. Avec 19 % de TVA : $37,5 \times 1,19 = 44,625$ €, soit 44,63 € si l'on arrondit « comme à l'école » (la moitié vers le haut). Faisons-le faire à Python :

```
quantite, prix_ht = 3, 12.5
total_ttc = quantite * prix_ht * 1.19
print("total TTC :", total_ttc)
print("arrondi   :", round(total_ttc, 2))
print(type(quantite), type(prix_ht), type("bol"), type(True))

total TTC : 44.625
arrondi   : 44.62
<class 'int'> <class 'float'> <class 'str'> <class 'bool'>
```

Le total correspond au calcul à la main, **sauf l'arrondi** : Python affiche 44.62 et non 44,63. La raison : 44,625 tombe pile à mi-chemin entre 44,62 et 44,63, et `round` arrondit ces cas vers le chiffre **pair** (« arrondi du banquier »), d'où 44,62. Dans d'autres cas, c'est l'approximation binaire des décimaux (1.5.1) qui fait pencher l'arrondi d'un côté ou de l'autre. Pour des centimes exacts, on utilise le module `decimal` ; pour un ticket de caisse, on peut aussi calculer en **millimes** (entiers). Gardez cet écart en tête : il illustre qu'un résultat de programme se **vérifie** toujours contre un calcul indépendant. Notez enfin que `type(...)` révèle le type d'une valeur : une commande que vous utiliserez souvent pour comprendre une erreur.

Les opérateurs arithmétiques sont + - * / et trois autres moins connus :

Opérateur	Sens	Exemple	Résultat
<code>**</code>	puissance	<code>2 ** 10</code>	1024
<code>//</code>	division entière	<code>17 // 5</code>	3
<code>%</code>	reste (modulo)	<code>17 % 5</code>	2

Par exemple, $17 = 3 \times 5 + 2$, d'où $17 // 5 = 3$ et $17 \% 5 = 2$; un nombre est pair si son reste modulo 2 vaut 0. Quant à la division /, elle donne toujours un `float` : $17 / 5$ vaut 3,4.

Les décimaux sont approchés. Python (comme tous les langages) stocke les `float` en binaire, ce qui explique les petites surprises du type $0,1 + 0,2 \neq 0,3$ expliquées en 1.5.1. Conséquence pratique : on **n'écrit jamais** `a == b` pour comparer deux décimaux calculés, on utilise `math.isclose(a, b)`. Et pour des montants d'argent, on arrondit explicitement avec `round(x, 2)` à l'affichage.

4.1.3 Le texte et les f-strings

Un texte (str) s'écrit entre guillemets. On peut le découper, le mettre en majuscules, le chercher :

```
nom = " Bol en céramique bleue "
print(nom.strip().upper())
print(nom.strip().replace("bleue", "verte"))
print(len(nom.strip()), "céramique" in nom)      # longueur, présence d'un morceau
print(nom.strip().split(" "))                  # découpe en liste de mots
```

```
BOL EN CÉRAMIQUE BLEUE
Bol en céramique verte
22 True
['Bol', 'en', 'céramique', 'bleue']
```

Pour **insérer des valeurs dans une phrase**, la méthode moderne est la **f-string** : on fait précéder le guillemet d'un f et on met les valeurs entre accolades. Après les deux-points, on peut régler le format.

```
produit, quantite, prix = "bol", 3, 12.5
print(f"{quantite} x {produit} à {prix} €")
print(f"total : {quantite * prix:.2f} € (TVA {0.19:.0%})")  # .2f : 2 décimales ; .0% :
↳ pourcentage
```

```
3 x bol à 12.5 €
total : 37.50 € (TVA 19%)
```

Les f-strings sont la base de tous les rapports et tickets de caisse que vous écrirez.

4.1.4 Les collections : listes, tuples, dictionnaires, ensembles

Une variable ne contient pas forcément une seule valeur. Python offre quatre « boîtes » pour en regrouper plusieurs. On les étudie plus en détail à la section 4.3 (algorithmes et structures de données) ; voici l'essentiel.

Collection	Notation	Ordonnée ?	Modifiable ?	Doublons ?	Cas typique
liste	[1, 2, 3]	oui	oui	oui	une suite de montants
tuple	(1, 2, 3)	oui	non	oui	un couple (code, quantité)
dictionnaire	{"bol": 12.5}	par insertion	oui	clés uniques	un catalogue : nom → prix
ensemble	{1, 2, 3}	non	oui	non	les clients distincts

Les listes. Les positions (**indices**) commencent à 0. Un indice négatif compte depuis la fin. Le **découpage** liste[a:b] prend de l'indice a inclus à b exclu.

```
montants = [44.8, 34.5, 88.2, 30.1, 110.1]
print(montants[0], montants[-1], montants[1:4])  # premier, dernier, indices 1 à 3
montants.append(39.8)                             # ajoute à la fin
print(len(montants), sum(montants), max(montants), min(montants))
print(sorted(montants))                             # copie triée ; l'original ne change pas
```

```
44.8 110.1 [34.5, 88.2, 30.1]
6 347.5 110.1 30.1
[30.1, 34.5, 39.8, 44.8, 88.2, 110.1]
```

Piège classique du débutant : le décalage de 1. Dans une liste de 6 éléments, les indices vont de 0 à 5 ; `montants[6]` provoque une erreur. Et `montants[1:4]` contient 3 éléments (1, 2, 3), pas 4. Règle : la longueur d'un découpage est `b - a`.

Les tuples sont des listes figées : une fois créés on ne les modifie plus. Parfaits pour des paires qui ne doivent pas bouger, comme (code produit, quantité).

Les dictionnaires associent une **clé** à une **valeur**, comme un annuaire : on cherche par le nom, pas par la position.

```
catalogue = {"bol": 12.5, "tasse": 8.0, "plateau": 45.0}
catalogue["bougie"] = 15.9 # ajout
print(catalogue["tasse"], catalogue.get("lampe", "inconnu")) # .get évite l'erreur
for nom, prix in catalogue.items():
    print(f" {nom:<8} {prix:>6.2f} €")
```

```
8.0 inconnu
bol      12.50 €
tasse    8.00 €
plateau  45.00 €
bougie   15.90 €
```

Les ensembles oublient l'ordre et éliminent les doublons : exactement ce qu'il faut pour compter des clients **distincts**.

```
acheteurs = ["Léa", "Hugo", "Léa", "Inès", "Hugo", "Léa"]
distincts = set(acheteurs)
print(len(acheteurs), "achats par", len(distincts), "clients distincts")
print(sorted(distincts)) # trié pour un affichage stable
```

```
6 achats par 3 clients distincts
['Hugo', 'Inès', 'Léa']
```

L'ordre d'affichage d'un ensemble peut changer d'une exécution à l'autre : n'y comptez jamais (c'est pourquoi nous l'avons passé par `sorted` pour l'afficher).

4.1.5 Décider : conditions

Un programme doit pouvoir **choisir**. L'instruction `if` exécute un bloc seulement si la condition est vraie. En Python, l'**indentation** (4 espaces) délimite le bloc : c'est la grammaire du langage, pas un détail de présentation.

Comparaisons : `==` (égal), `!=` (différent), `<`, `<=`, `>`, `>=`. Combinaisons : `and`, `or`, `not`.

Exemple à la main. Règle de livraison de la boutique : *gratuite à partir de 100 €, sinon 7 € ; et pour un retrait en boutique, toujours 0*. Pour un panier de 80 € livré : 7 €. Pour 120 € livré : 0. Pour 80 € en retrait : 0.

```
def frais_livraison(total, retrait_boutique):
    if retrait_boutique:
        return 0
    elif total >= 100:
        return 0
    else:
        return 7
```

```
print(frais_livraison(80, False), frais_livraison(120, False), frais_livraison(80, True))
```

```
7 0 0
```

Les trois cas retombent sur les valeurs trouvées à la main. (Nous n'avons pas encore parlé des **fonctions** : le mot `def` en 4.1.7 va tout expliquer.)

= et ==. Un seul signe égal **colle** une étiquette ; deux signes égaux **testent** l'égalité. Python refuse d'ailleurs `if x = 3`, ce qui vous évite un grand classique des autres langages.

4.1.6 Répéter : boucles

Une **boucle** répète un bloc. Deux formes :

- `for élément in collection` : une fois pour chaque élément (on sait combien).
- `while condition` : tant que la condition est vraie (on ne sait pas combien).

Exemple à la main. La gérante place 1 000 € à 5 % par an, intérêts composés. Au bout d'un an : $1000 \times 1,05 = 1050$. Deux ans : $1050 \times 1,05 = 1102,5$. Combien d'années pour **doubler** ? C'est une question « jusqu'à ce que » : une boucle `while`.

```
capital, annees = 1000.0, 0
while capital < 2000:
    capital = capital * 1.05
    annees += 1 # raccourci pour annees = annees + 1
print("doublé en", annees, "ans :", round(capital, 2), "€")
```

```
doublé en 15 ans : 2078.93 €
```

Vérification mathématique : on cherche le plus petit n tel que $1,05^n \geq 2$, soit $n \geq \ln 2 / \ln 1,05 \approx 14,21$.

Le résultat réel (14,21) est entre 14 et 15 : il faut donc 15 années entières, ce que la boucle a trouvé.

La boucle infinie. Si, dans un `while`, la condition ne devient jamais fausse (ici, si on oubliait la ligne `capital = ...`), le programme tourne éternellement. Dans un terminal, `Ctrl+C` l'arrête. Avant de lancer un `while`, demandez-vous : *qu'est-ce qui le fera s'arrêter ?* (La même question sera posée avec rigueur pour les algorithmes en 4.3.)

La boucle `for` parcourt n'importe quelle collection ; `range(n)` produit les entiers de 0 à $n - 1$, et `enumerate` donne en plus la position :

```
for i in range(3):
    print("i =", i)
for rang, nom in enumerate(["Léa", "Hugo", "Inès"], start=1):
    print(rang, nom)
```

```
i = 0
i = 1
i = 2
1 Léa
2 Hugo
3 Inès
```

Les compréhensions de liste sont une écriture compacte d'une boucle qui construit une liste : `[expression for x in collection if condition]`. Elles se lisent comme une phrase mathéma-

tique « l'ensemble des $f(x)$ pour x dans... tel que... ».

```
montants = [44.8, 34.5, 88.2, 30.1, 110.1, 39.8, 74.7]
ttc = [round(m * 1.19, 2) for m in montants]
gros = [m for m in montants if m > 60]
print(ttc)
print(gros)
print({n: n ** 2 for n in range(1, 6)})      # même idée pour un dictionnaire
```

```
[53.31, 41.05, 104.96, 35.82, 131.02, 47.36, 88.89]
[88.2, 110.1, 74.7]
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

4.1.7 Fonctions : écrire ses propres outils

Une **fonction** est une recette nommée qu'on peut réutiliser. Elle reçoit des **paramètres**, fait un calcul, et **renvoie** un résultat avec `return`. Sans fonctions, un programme devient vite un long ruban illisible ; avec elles, on le découpe en briques que l'on teste séparément.

Mathématiquement, c'est la même idée que $f(x) = 1,19x$: on la définit une fois, on l'utilise partout.

```
def ttc(prix_ht, taux_tva=0.19):
    """Renvoie le prix TTC arrondi au centime."""
    return round(prix_ht * (1 + taux_tva), 2)

print(ttc(100))          # utilise la TVA par défaut : 19 %
print(ttc(100, 0.07))    # TVA à 7 % : on remplace la valeur par défaut
print(ttc(prix_ht=50))   # appel par nom : plus lisible
```

```
119.0
107.0
59.5
```

Trois choses à retenir : (1) la ligne entre triples guillemets, la **docstring**, documente la fonction ; (2) `taux_tva=0.19` est un paramètre **par défaut** ; (3) une fonction peut renvoyer **plusieurs** valeurs, sous forme de tuple.

```
def resume(valeurs):
    """Renvoie (minimum, moyenne, maximum)."""
    return min(valeurs), sum(valeurs) / len(valeurs), max(valeurs)

mini, moy, maxi = resume([44.8, 34.5, 88.2, 30.1, 110.1])
print(f"min = {mini}, moyenne = {moy:.2f}, max = {maxi}")
```

```
min = 30.1, moyenne = 61.54, max = 110.1
```

La portée. Les variables créées *dans* une fonction n'existent que dans la fonction (on dit qu'elles sont **locales**). Cela évite les mélanges : deux fonctions peuvent utiliser chacune une variable `total` sans se gêner.

Piège : le paramètre par défaut modifiable. N'écrivez jamais `def f(x, liste=[])` : cette liste est créée **une seule fois** et partagée par tous les appels. Écrivez `liste=None` puis `if liste is None: liste = []`.

Les fonctions peuvent aussi être **passées** à d'autres fonctions, ce qui donne des écritures très concises. Exemple : trier des commandes selon un critère choisi avec l'argument `key`.

```
commandes = [("Léa", 44.8), ("Hugo", 110.1), ("Inès", 30.1)]
print(sorted(commandes, key=lambda c: c[1]))           # du plus petit au plus gros montant
print(sorted(commandes, key=lambda c: c[1], reverse=True)[0]) # la plus grosse
```

```
[('Inès', 30.1), ('Léa', 44.8), ('Hugo', 110.1)]
('Hugo', 110.1)
```

`lambda c: c[1]` est une mini-fonction sans nom qui renvoie le second élément du couple.

4.1.8 Les erreurs : vos meilleures amies

Quand quelque chose ne va pas, Python s'arrête et affiche un **message d'erreur** (*traceback*). Il se lit **de bas en haut** : la dernière ligne dit *quel type d'erreur* et *pourquoi* ; les lignes au-dessus disent *où*. Voici les erreurs que vous rencontrerez le plus souvent, provoquées volontairement et rattrapées avec `try / except` pour pouvoir les afficher :

```
def tenter(description, fonction):
    try:
        fonction()
    except Exception as e:
        print(f"{description:<20} -> {type(e).__name__}: {e}")
```

```
tenter("indice hors liste", lambda: [44.8, 34.5, 88.2][5])
tenter("clé absente", lambda: {"bol": 12.5}["lampe"])
tenter("division par zéro", lambda: 10 / 0)
tenter("texte + nombre", lambda: "total : " + 12.5)
tenter("texte vers entier", lambda: int("douze"))
tenter("nom inconnu", lambda: variable_inexistante)
```

```
indice hors liste    -> IndexError: list index out of range
clé absente         -> KeyError: 'lampe'
division par zéro   -> ZeroDivisionError: division by zero
texte + nombre      -> TypeError: can only concatenate str (not "float") to str
texte vers entier   -> ValueError: invalid literal for int() with base 10: 'douze'
nom inconnu         -> NameError: name 'variable_inexistante' is not defined
```

Lecture :

- `IndexError` : vous demandez un indice qui n'existe pas (rappelez-vous le décalage de 1).
- `KeyError` : la clé n'est pas dans le dictionnaire (pensez à `.get`).
- `ZeroDivisionError` : on ne divise pas par zéro, ni en Python ni ailleurs.
- `TypeError` : on a mélangé des types incompatibles (texte et nombre).
- `ValueError` : le bon type, mais une valeur impossible à convertir.
- `NameError` : le nom n'existe pas (faute de frappe ? variable pas encore créée ?).

Attraper une erreur à bon escient. Quand on lit des données saisies par des humains, certaines valeurs sont inexploitable. On préfère alors **prévoir** l'erreur plutôt que de planter :

```
saisies = ["12.5", "8", "abc", "", "15,9", "45.0"]
valides = []
for s in saisies:
    try:
        valides.append(float(s))
    except ValueError:
        print("rejetée :", repr(s))
print("valides :", valides)
```

```
rejetée : 'abc'
rejetée : ''
rejetée : '15,9'
```

```
valides : [12.5, 8.0, 45.0]
```

Remarquez que "15,9" (virgule française) est rejeté : Python attend le **point** décimal. C'est une cause fréquente de données « cassées » quand elles viennent d'un tableur configuré en français.

Méthode pour déboguer (à épingler au-dessus de votre écran). (1) Lisez la **dernière ligne** du message. (2) Repérez la ligne de code citée. (3) Affichez avec `print` les valeurs et les types utilisés à cet endroit. (4) Réduisez le problème au plus petit exemple qui échoue. (5) Seulement ensuite, cherchez le message sur Internet. Neuf fois sur dix, les étapes 1 à 3 suffisent.

4.1.9 Modules, fichiers et données réelles

Python ne contient pas tout, mais il sait **importer** du code déjà écrit. Un **module** est un fichier de fonctions ; la bibliothèque standard en fournit des dizaines (`math`, `random`, `statistics`, `csv`, `datetime`...), et on en installe d'autres avec `pip` (section 6.3).

```
import math, statistics
from collections import Counter

print(math.sqrt(144), statistics.mean([2, 4, 4, 4, 5, 5, 7, 9]))
print(Counter("abracadabra"))      # compte les occurrences
```

```
12.0 5
Counter({'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1})
```

Lisons maintenant le fichier de données du livre, `donnees/commandes.csv`, **sans aucune bibliothèque externe**, avec le module `csv`. Un fichier CSV est du texte brut : une ligne par commande, des valeurs séparées par des virgules, la première ligne contenant les noms des colonnes.

```
import csv

with open("donnees/commandes.csv", encoding="utf-8") as f:
    lignes = list(csv.DictReader(f))      # chaque ligne devient un dictionnaire
print(len(lignes), "commandes ; première :", lignes[0])
print("type du montant :", type(lignes[0]["montant"]))
```

```
400 commandes ; première : {'canal': 'Boutique', 'montant': '44.8', 'livraison': '0', 'satisfaction':
↳ '4'}
type du montant : <class 'str'>
```

Le mot-clé `with` ouvre le fichier **et le referme proprement** à la sortie du bloc, même en cas d'erreur. Remarquez que les valeurs sont lues comme du **texte** : "44.8" n'est pas un nombre ! Il faut convertir.

```
montants = [float(l["montant"]) for l in lignes]
print("montant moyen :", round(sum(montants) / len(montants), 2))

par_canal = {}                                # un dictionnaire de listes
for l in lignes:
    par_canal.setdefault(l["canal"], []).append(float(l["montant"]))
for canal, valeurs in par_canal.items():
    print(f" {canal:<10} n = {len(valeurs):3d}   moyenne = {sum(valeurs) / len(valeurs):6.2f} €")
```

```
montant moyen : 60.25
Boutique  n = 114   moyenne = 74.81 €
Site      n = 148   moyenne = 59.50 €
Réseaux   n = 138   moyenne = 49.01 €
```

On retrouve le montant moyen de 60,25 € calculé au chapitre 3. Nous avons tout fait à la main, avec des boucles et des dictionnaires : c'est précisément le travail que pandas fera en **une ligne** à la section 4.4. Savoir le faire « à la main » vous permet de comprendre ce que pandas fait pour vous.

4.1.10 Un petit programme complet : le ticket de caisse

Rassemblons tout. La gérante veut un petit programme qui, pour un panier, **édite un ticket de caisse** avec les règles suivantes :

- les prix du catalogue sont **hors taxe** ;
- une **remise fidélité de 10 %** s'applique si le sous-total HT dépasse 100 € ;
- la **TVA de 19 %** s'applique sur le montant après remise ;
- le ticket affiche chaque ligne, le sous-total, la remise, la TVA et le total TTC.

Calcul à la main pour le panier « 2 bols, 1 plateau, 3 bougies » (prix HT : bol 12,5 ; plateau 45 ; bougie 15,9) :

- bols : $2 \times 12,5 = 25,00$; plateau : 45,00 ; bougies : $3 \times 15,9 = 47,70$;
- sous-total HT : $25 + 45 + 47,7 = 117,70$ € ;
- le sous-total dépasse 100 €, donc remise de 10% : 11,77 €, soit 105,93 € après remise ;
- TVA : $105,93 \times 0,19 = 20,1267 \approx 20,13$ € ;
- total TTC : $105,93 + 20,13 = 126,06$ €.

Le programme se découpe en **petites fonctions** faciles à tester. Les deux premières suffisent à calculer les montants :

```
CATALOGUE = {"bol": 12.5, "tasse": 8.0, "plateau": 45.0, "bougie": 15.9}
TVA, SEUIL_REMISE, TAUX_REMISE = 0.19, 100, 0.10

def sous_total(panier):
    """panier : liste de couples (produit, quantité)."""
    return sum(CATALOGUE[nom] * qte for nom, qte in panier)

def remise(montant_ht):
    return montant_ht * TAUX_REMISE if montant_ht > SEUIL_REMISE else 0.0
```

Une troisième fonction, `ticket`, appelle les deux premières puis met le résultat en forme avec des f-strings (l'application 4.1 du cahier la construit pas à pas). Pour notre panier, elle produit :

```
=== BOUTIQUE ===
2 x bol      12.50    25.00
1 x plateau  45.00    45.00
3 x bougie   15.90    47.70
Sous-total HT      117.70
Remise fidélité    -11.77
TVA 19 %           20.13
TOTAL TTC          126.06
```

Le ticket affiche 117,70 € de sous-total, 11,77 de remise, 20,13 de TVA et 126,06 € au total : **exactement** nos valeurs à la main. Testons aussi les cas limites avec `assert`, une instruction qui ne dit rien quand la condition est vraie et **arrête** le programme avec une erreur quand elle est fausse :

```
assert remise(100) == 0.0 # pile au seuil : pas de remise (condition « > »)
assert ticket([("tasse", 2)])[1] == 19.04 # 2 tasses : 16,00 HT ; TVA 3,04
assert ticket([])[1] == 0.0 # panier vide
assert ticket([("bol", 2), ("plateau", 1), ("bougie", 3)])[1] == 126.06
print("tous les tests passent")
```

tous les tests passent

Ce qu'on vient de faire, c'est de la rigueur. Calculer à la main *avant* de coder fournit un « oracle » : si le programme et la main divergent, l'un des deux a tort, et on cherche lequel. Les `assert` transforment cette vérification en filet de sécurité automatique. Nous irons plus loin avec de vrais tests unitaires en 4.6.

Pour s'entraîner. Cahier, chapitre 4 : application 4.1 (le programme complet de la caisse) ; exercices 4.1 à 4.3 (dictionnaires et boucles, produit absent du catalogue, code promo).

À retenir

- Une **variable** est une étiquette sur une valeur ; les types de base sont `int`, `float`, `str`, `bool`.
- Les décimaux sont approchés : on les arrondit à l'affichage et on ne les compare pas avec `==`.
- **Listes** (ordonnées, modifiables), **tuples** (figés), **dictionnaires** (clé → valeur), **ensembles** (sans doublons). Les indices commencent à 0.
- `if/elif/else` pour décider, `for` et `while` pour répéter ; l'**indentation** délimite les blocs.
- Une **fonction** (`def`, `return`) est une brique réutilisable ; on la documente (`docstring`) et on la teste.
- Un message d'erreur se lit **par la dernière ligne** ; `try/except` permet de prévoir les erreurs attendues.
- Les fichiers CSV sont du texte : il faut convertir les nombres avant de calculer.
- Méthode d'or : **calculer à la main un petit cas, puis vérifier que le programme donne la même chose.**

4.2 Les fondamentaux de R

Intuition. Si Python est un couteau suisse qui sait tout faire, **R est le scalpel du statisticien** : il a été conçu dès l'origine (années 1990, par des statisticiens) pour manipuler des tableaux de données, calculer des statistiques et tracer des graphiques. Les tests, les modèles et les méthodes de recherche récentes y arrivent souvent *en premier*. Dans l'industrie comme à l'université, vous croiserez du R : savoir le lire est un vrai atout.

Cette section ne vous demande pas de choisir un camp. Nous allons **refaire dans R les analyses des sections 3.1, 3.3 et 3.4 et du 4.1**, sur le même fichier `donnees/commandes.csv`. Retrouver les mêmes nombres avec un autre langage est la meilleure des vérifications : si deux outils indépendants donnent la même réponse, vous pouvez avoir confiance.

Section à lire après 4.1. Nous supposons connues les notions de variable, fonction, boucle et condition vues en 4.1 ; ici on ne s'attarde que sur ce qui **change** en R.

4.2.1 Lancer R, et premières différences

On lance R avec la commande R (session interactive) ou Rscript mon_script.R (pour exécuter un fichier). Le confort d'un éditeur s'obtient avec **RStudio** ou VS Code. Tous les blocs R de ce livre ont été **réellement exécutés** avec R, et leur sortie est affichée juste en dessous, comme pour Python.

Voici un premier contact, à comparer avec Python :

```
x <- 12.5          # l'affectation s'écrit « <- » (le « = » marche aussi, mais on utilise « <- »)
x * 3
print("Bonjour la boutique !")
```

```
[1] 37.5
[1] "Bonjour la boutique !"
```

Les trois différences à connaître tout de suite :

	Python	R
Affectation	x = 12.5	x <- 12.5
Premier indice d'une liste	0	1
Fin de ligne	indentation	accolades { }

En R, on compte à partir de 1. x[1] est le **premier** élément (en Python : x[0]). Et le découpage x[2:4] donne les éléments 2, 3 **et** 4 : les deux bornes sont **incluses** (en Python, la borne de droite est exclue). C'est la source n°1 d'erreurs quand on passe d'un langage à l'autre.

4.2.2 Vecteurs : la brique de base

En R, il n'existe pas de « nombre seul » : 12.5 est un **vecteur de longueur 1**. La fonction c() (*combine*) assemble un vecteur, et **toutes les opérations s'appliquent élément par élément**, sans boucle. Cette idée de **vectorisation** est le cœur de R (et, nous le verrons, de NumPy en 4.4).

Exemple à la main. Quatre commandes de 44,8 ; 34,5 ; 88,2 et 30,1 €. Leur somme est 197,6, leur moyenne 49,4 €. Avec 19 % de TVA, chaque montant est multiplié par 1,19 : 44,8 → 53,31 (arrondi), etc.

```
montants <- c(44.8, 34.5, 88.2, 30.1)
c(sum(montants), mean(montants))
round(montants * 1.19, 2)      # la multiplication s'applique à chaque élément
montants[2:3]                 # éléments 2 et 3 (bornes incluses)
montants[-1]                  # indice négatif : tout SAUF le premier
montants[montants > 40]       # sélection par condition
```

```
[1] 197.6  49.4
[1] 53.31 41.06 104.96 35.82
[1] 34.5 88.2
[1] 34.5 88.2 30.1
[1] 44.8 88.2
```

Les valeurs 197,6 et 49,4 retombent sur le calcul à la main. Remarquez les deux emplois de [] :

- montants[-1] signifie « **tout sauf** le premier » (en Python, -1 désignait le **dernier** !)
- montants[montants > 40] : on donne au crochet un vecteur de vrai/faux et R ne garde que les TRUE. Ce « **filtrage logique** » est l'équivalent exact de la compréhension de liste de 4.1.6 ; on le retrouvera avec pandas.

Les autres types de base ressemblent à ceux de Python : `numeric` (décimaux), `integer`, `character` (texte), `logical` (TRUE/FALSE). Une particularité précieuse : R a une **valeur manquante intégrée**, NA (*not available*), qui contamine les calculs tant qu'on ne demande pas explicitement de l'ignorer :

```
ventes <- c(12, 15, NA, 9)
mean(ventes)                # une valeur manquante rend la moyenne inconnue
mean(ventes, na.rm = TRUE)  # on demande d'ignorer les NA
```

```
[1] NA
[1] 12
```

Pourquoi NA contamine-t-il ? Si une valeur est inconnue, la moyenne l'est aussi : R refuse de faire comme si de rien n'était. C'est un comportement *honnête*, qui évite d'oublier des données manquantes sans s'en rendre compte. Nous verrons au volume suivant comment les traiter avec soin.

4.2.3 Fonctions, conditions et boucles

La syntaxe change, pas les idées :

```
ttc <- function(prix_ht, taux_tva = 0.19) {
  round(prix_ht * (1 + taux_tva), 2)  # la dernière valeur calculée est renvoyée
}
ttc(100)
ttc(100, 0.07)
ttc(c(12.5, 45, 15.9))                # la fonction marche sur un vecteur entier, gratuitement
```

```
[1] 119
[1] 107
[1] 14.88 53.55 18.92
```

La dernière ligne est remarquable : parce que `*` et `round` sont vectorisés, notre fonction `ttc`, écrite pour **un** prix, marche telle quelle sur **une liste** de prix. En Python pur, il aurait fallu une boucle ou une compréhension.

Les conditions s'écrivent avec `if` / `else`, et la version vectorisée avec `ifelse` :

```
frais_livraison <- function(total, retrait_boutique) {
  if (retrait_boutique) 0 else if (total >= 100) 0 else 7
}
c(frais_livraison(80, FALSE), frais_livraison(120, FALSE), frais_livraison(80, TRUE))

ifelse(c(80, 120, 100, 35) >= 100, 0, 7)  # version vectorisée de « si... alors... sinon »
```

```
[1] 7 0 0
[1] 7 0 0 7
```

On retrouve 7, 0, 0 pour les frais ; la boucle `while (capital < 2000) { ... }` s'écrit presque comme en Python et trouve elle aussi 15 années pour doubler un capital placé à 5 % : exactement ce que Python a donné en 4.1. La fonction `ifelse` applique le test à **chaque élément** d'un vecteur, ce que `if` ne sait pas faire.

Piège de la vectorisation. `if (totaux >= 100)` appliqué à un vecteur de plusieurs éléments n'a pas de sens : `if` attend **un seul** vrai/faux. Utilisez `ifelse` pour les vecteurs.

Le dictionnaire de R. Un vecteur dont les éléments portent un **nom** joue le rôle du dictionnaire de Python : on retrouve un prix par son nom, ou plusieurs d'un coup. Le ticket de caisse de 4.1.10

se refait en quelques lignes grâce à la vectorisation (c'est l'application 4.2 du cahier).

```
catalogue <- c(bol = 12.5, tasse = 8.0, plateau = 45.0, bougie = 15.9) # un vecteur nommé joue le
↪ rôle du dictionnaire
catalogue[c("bol", "bougie")]
```

```
bol bougie
12.5 15.9
```

4.2.4 Le data.frame : le tableau de données

Le tableau est l'**objet central de R**. Un `data.frame` est un tableau dont chaque colonne est un vecteur (de types éventuellement différents). Chargeons les commandes, avec les trois gestes de 3.1.2 : forme, types, résumé.

```
df <- read.csv("donnees/commandes.csv") # lit le CSV directement en tableau
str(df) # structure : type de chaque colonne
```

```
'data.frame': 400 obs. of 4 variables:
 $ canal      : chr "Boutique" "Site" "Réseaux" "Réseaux" ...
 $ montant    : num 44.8 34.5 88.2 30.1 110.1 ...
 $ livraison  : int 0 2 5 4 0 5 0 0 5 6 ...
 $ satisfaction: int 4 4 4 4 5 3 5 4 4 3 ...
```

`read.csv` a deviné les types : `canal` est du texte (`chr`), les trois autres colonnes sont numériques. Contrairement au module `csv` de Python (4.1.9) qui lisait tout en texte, la conversion est faite pour nous. Le résumé numérique :

```
summary(df)
```

canal	montant	livraison	satisfaction
Length:400	Min. : 8.60	Min. : 0.000	Min. :1.000
Class :character	1st Qu.: 34.17	1st Qu.: 0.000	1st Qu.:3.000
Mode :character	Median : 51.00	Median : 3.000	Median :4.000
	Mean : 60.25	Mean : 3.288	Mean :3.965
	3rd Qu.: 75.83	3rd Qu.: 5.000	3rd Qu.:5.000
	Max. :255.70	Max. :13.000	Max. :5.000

La ligne `montant` donne un minimum de 8,6, une médiane de 51, une moyenne de 60,25 et un maximum de 255,7 : **les mêmes chiffres** que le `describe()` de `pandas` au 3.1.2. On accède à une colonne avec `$` :

```
m <- df$montant
c(moyenne = mean(m), mediane = median(m), ecart_type = sd(m), iqr = IQR(m))
quantile(m, c(0.05, 0.25, 0.50, 0.75, 0.95))
```

moyenne	mediane	ecart_type	iqr	
60.24575	51.00000	38.01791	41.65000	
5%	25%	50%	75%	95%
18.985	34.175	51.000	75.825	128.555

Les nombres sont ceux du 3.1 : moyenne 60,25 ; médiane 51 ; écart-type 38,02 (R divise par $n - 1$, comme `pandas`) ; quartiles 34,2 et 75,8. Les deux logiciels utilisent la même définition par défaut du quantile (interpolation linéaire), ce qui explique la concordance exacte. Pour résumer **par groupe** :

```
tapply(df$montant, df$canal, mean) # moyenne par canal
aggregate(montant ~ canal, data = df, FUN = mean) # même chose, avec la notation « formule »
```

```
Boutique Réseaux Site
74.80965 49.01087 59.50338
      canal montant
1 Boutique 74.80965
2 Réseaux 49.01087
3 Site 59.50338
```

La formule `montant ~ canal` se lit « le montant **en fonction du** canal » : cette notation en tilde est partout en R (nous la retrouverons pour les modèles de régression au volume suivant). Les moyennes par canal (74,81 ; 59,50 ; 49,01) sont celles que nous avons obtenues avec Python au 4.1.9, à la main.

4.2.5 Les statistiques « sortent de la boîte »

C'est ici que R brille : les procédures de la statistique classique sont **au catalogue de base**, sans rien installer.

Intervalle de confiance et test de Welch. Rappel du 3.3 et du 3.4 : IC à 95 % de la moyenne, [56,51 ; 63,98] ; test boutique contre Réseaux, $t = 5,565$, 208,4 degrés de liberté, $p \approx 8 \times 10^{-8}$.

```
b <- df$montant[df$canal == "Boutique"]
i <- df$montant[df$canal == "Réseaux"]
w <- t.test(b, i) # R fait le test de Welch par défaut
c(t = unname(w$statistic), ddl = unname(w$parameter), p = w$p.value)
round(w$conf.int, 1) # IC95 % de la différence des moyennes
```

```
      t      ddl      p
5.564672e+00 2.084304e+02 8.016366e-08
[1] 16.7 34.9
attr(,"conf.level")
[1] 0.95
```

On lit exactement $t = 5,565$, 208,4 degrés de liberté et une p-valeur $p \approx 8 \times 10^{-8}$: les valeurs obtenues avec `scipy` au 3.4.3. Un seul appel de fonction fait ce que nous avons codé en plusieurs lignes à la main.

R fait le test de Welch par défaut, ce qui est bien la bonne pratique recommandée en 3.4.3 (en Python, il faut penser à écrire `equal_var=False`). Chaque langage a ses défauts : **lisez la documentation** (`?t.test` en R, `help()` en Python) pour savoir ce que fait réellement une fonction.

Corrélation et régression, un avant-goût. Les mêmes outils donnent la corrélation entre délai de livraison et satisfaction, et une droite de régression, que le volume II étudiera en détail :

```
livres <- subset(df, canal != "Boutique") # on exclut les retraits en boutique (délai = 0)
cor(livres$livraison, livres$satisfaction)
modele <- lm(satisfaction ~ livraison, data = livres)
round(coef(modele), 3)
```

```
[1] -0.4069396
(Intercept) livraison
      4.581      -0.180
```

La corrélation vaut environ $-0,41$: elle est **négative**, plus le délai est long, moins les clients sont satisfaits. La pente estimée, $-0,18$, est la perte de satisfaction **par jour de retard supplémentaire**. Elle coïncide avec le coefficient $-0,18$ qui a servi à fabriquer les données au 3.1.2 (et l'ordonnée à l'origine, 4,58, est proche du 4,6 utilisé) : un joli moyen de vérifier que la machine retrouve bien ce qu'on y a mis.

4.2.6 Le tidyverse : manipuler les tableaux avec des « phrases »

Les fonctions de base de R sont puissantes mais leur syntaxe est parfois irrégulière. Un ensemble de packages cohérents, le **tidyverse** (dplyr, tidyr, readr, ggplot2...), a standardisé une façon de travailler : des **verbes** simples enchaînés par un **tube** (|>, qui se lit « puis »). Chaque verbe prend un tableau et rend un tableau.

Verbe	Rôle	Équivalent SQL (chapitre 5)
filter()	garder des lignes	WHERE
select()	garder des colonnes	SELECT
mutate()	créer ou modifier une colonne	colonne calculée
arrange()	trier	ORDER BY
group_by() + summarise()	résumer par groupe	GROUP BY

```
library(dplyr)
```

```
df |>
  group_by(canal) |>
  summarise(n = n(), panier_moyen = round(mean(montant), 2),
            satisfaction = round(mean(satisfaction), 2)) |>
  arrange(desc(panier_moyen))
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':
```

```
filter, lag
```

```
The following objects are masked from 'package:base':
```

```
intersect, setdiff, setequal, union
```

```
# A tibble: 3 × 4
```

```
  canal      n panier_moyen satisfaction
<chr> <int>    <dbl>      <dbl>
1 Boutique  114      74.8        4.49
2 Site      148      59.5        3.79
3 Réseaux   138      49.0        3.72
```

Lisez cette « phrase » à voix haute : *prends le tableau des commandes, puis groupe par canal, puis résume (effectif, panier moyen, écart-type, satisfaction), puis trie par panier moyen décroissant*. C'est presque du français, et c'est ce qui rend le tidyverse si agréable à lire. Une seconde phrase, avec filtre et colonne calculée :

```
df |>
  filter(canal != "Boutique", livraison > 8) |>      # livraisons lentes
  mutate(montant_ttc = round(montant * 1.19, 2)) |>
  select(canal, montant, montant_ttc, livraison, satisfaction) |>
  head(3)
```

```
  canal montant montant_ttc livraison satisfaction
1 Site    40.1      47.72         13           2
2 Réseaux 61.1      72.71          9           4
3 Site    65.4      77.83         10           3
```

Nous retrouverons exactement cette logique, en Python, avec **pandas** (section 4.4), puis en SQL au chapitre 5. Les trois langages expriment les mêmes idées : *filtrer, sélectionner, créer, trier, regrouper*. Apprendre l'un fait gagner du temps sur les deux autres.

Et les graphiques ? R possède aussi une bibliothèque de graphiques célèbre, **ggplot2**. Nous la présentons à la section 4.5, côte à côte avec matplotlib et seaborn.

4.2.7 Python ou R ? Un tableau de correspondance

Tâche	Python (pandas)	R
Lire un CSV	<code>pd.read_csv("f.csv")</code>	<code>read.csv("f.csv")</code>
Moyenne d'une colonne	<code>df["montant"].mean()</code>	<code>mean(df\$montant)</code>
Filtrer des lignes	<code>df[df["canal"] == "Site"]</code>	<code>df[df\$canal == "Site",]</code> ou <code>filter(df, canal == "Site")</code>
Moyenne par groupe	<code>df.groupby("canal")["montant"].mean()</code>	<code>df\$montant %>% group_by(df\$canal) %>% summarise(mean)</code>
Test de Welch	<code>stats.ttest_ind(a, b, equal_var=False)</code>	<code>t.test(a, b)</code>
Premier élément	<code>x[0]</code>	<code>x[1]</code>
Valeur manquante	NaN / None	NA

Comment choisir ? Python est un langage généraliste : il excelle dès qu'il faut **mettre en production**, automatiser, faire de l'apprentissage automatique (volume III) ou du traitement de texte. R excelle pour **l'analyse statistique exploratoire et les rapports**. Beaucoup de professionnels utilisent les deux. La stratégie de ce livre : **Python comme langage principal**, R en parallèle quand il éclaire un concept ou qu'il est l'outil naturel.

Pour s'entraîner. Cahier, chapitre 4 : application 4.2 (le ticket de caisse en R) ; exercice 4.4 (part de commandes satisfaites, en R de base, avec dplyr et avec pandas).

À retenir

- R est un langage pensé pour les statistiques ; les opérations y sont **vectorisées** (élément par élément, sans boucle).
- Attention aux différences de Python : on **compte à partir de 1**, les bornes d'un découpage sont **incluses**, `x[-1]` signifie « tout sauf le premier » et `<-` est l'affectation.
- Le `data.frame` est l'objet central ; `read.csv`, `summary`, `tapply`, `aggregate` couvrent l'essentiel de l'exploration.
- NA représente une valeur manquante et **contamine** les calculs sauf si l'on écrit `na.rm = TRUE`.
- Les tests classiques (`t.test`, `cor`, `lm`) sont fournis d'origine ; `t.test` fait le test de **Welch** par défaut.
- Le tidyverse (`filter`, `mutate`, `group_by`, `summarise`) enchaîne des « verbes » avec `|>`, comme pandas et SQL.
- Retrouver les mêmes nombres dans deux langages indépendants (moyenne 60,25 ; $t = 5,565$) est la meilleure des vérifications.

4.3 Algorithmes et structures de données

Intuition. Un **algorithme** est une méthode précise pour résoudre un problème, comme une recette. Une **structure de données** est la façon dont on range les ingrédients. Deux cuisiniers peuvent faire le même plat, l'un en dix minutes parce que ses ingrédients sont bien rangés, l'autre en deux heures parce qu'il fouille dans tous les tiroirs. En data science, la différence se joue entre un calcul qui prend une seconde et un calcul qui ne finit jamais.

Cette section est plus « informatique » que les autres : on y apprend à **penser** un calcul, à le **prouver** correct et à **compter** son coût. Rassurez-vous, tout part d'exemples faits à la main.

4.3.1 Qu'est-ce qu'un algorithme ?

Un algorithme est une suite d'instructions **non ambiguës** qui, pour toute entrée valide, **se termine** et produit la **bonne** sortie. Trois exigences, donc : être précis, **terminer**, être **correct**.

Exemple fil rouge : trouver la plus grosse commande d'une liste, sans utiliser max. Méthode : on retient « le plus grand vu jusqu'ici » et on parcourt la liste, en le mettant à jour chaque fois qu'on trouve mieux.

À la main, sur [44,8; 34,5; 88,2; 30,1; 110,1] :

Élément lu	Plus grand vu jusqu'ici
44,8 (on commence avec le premier)	44,8
34,5	44,8 (34,5 est plus petit)
88,2	88,2 (nouveau record)
30,1	88,2
110,1	110,1 (nouveau record)

Résultat : 110,1. En code :

```
def plus_grand(valeurs):
    record = valeurs[0]
    for v in valeurs[1:]:
        if v > record:
            record = v
    return record

print(plus_grand([44.8, 34.5, 88.2, 30.1, 110.1]))
```

110.1

Preuve de correction (par invariant de boucle). Un **invariant** est une propriété vraie *avant* chaque tour de boucle, qu'on prouve comme une récurrence.

Invariant : après avoir lu les k premiers éléments, record est le maximum de ces k éléments. *Initialisation* ($k = 1$) : record est le premier élément, qui est bien le maximum d'une liste d'un élément. *Conservation* : supposons l'invariant vrai pour k . On lit l'élément v n° $k + 1$. Si $v > \text{record}$, le nouveau record est v , qui est supérieur à tous les précédents ; sinon record reste le maximum. Dans les deux cas l'invariant est vrai pour $k + 1$. *Fin* : la boucle for parcourt une liste finie : elle **termine** après $n - 1$ tours. Pour $k = n$, l'invariant dit que record est le maximum de **toute** la liste. ■

Coût : exactement $n - 1$ comparaisons, quelle que soit la liste. On dit que l'algorithme est

linéaire (section 4.8).

Cette habitude (**invariant**, **terminaison**, **coût**) est la boîte à outils de base pour tout algorithme, y compris les plus sophistiqués.

4.3.2 Choisir sa structure de données

Les quatre collections de 4.1.4 ne sont pas interchangeables : chacune est **bonne** pour certaines opérations et **mauvaise** pour d'autres.

Besoin	Meilleure structure	Pourquoi
Garder un ordre, accéder par position	liste	<code>x[i]</code> est immédiat
Retrouver une valeur à partir d'un nom, d'un identifiant	dictionnaire	accès direct par clé
Savoir « est-ce que X est déjà vu ? », éliminer les doublons	ensemble	test d'appartenance immédiat
Traiter dans l'ordre d'arrivée	file (deque)	retrait au début immédiat
Revenir en arrière (annuler)	pile (une liste utilisée par la fin)	dernier entré, premier sorti

Pourquoi un dictionnaire ou un ensemble est-il si rapide ? Ils reposent sur une **table de hachage** : une fonction (le *hachage*) transforme la clé en un numéro de case, et l'on va **directement** à cette case, sans parcourir quoi que ce soit. Imaginez un vestiaire où chaque cintre porte le numéro calculé à partir du nom du client : pas besoin de passer en revue tous les manteaux. Une liste, elle, doit être parcourue de gauche à droite pour savoir si une valeur y figure : c'est la **recherche linéaire**, une boucle `for` qui compare chaque élément à la cible. Mesurons ce parcours en **nombre de comparaisons** (une mesure qui ne dépend pas de l'ordinateur) sur la liste montants des 400 montants lus en 4.1.9 : trouver 44,8, qui est en tête, coûte **1** comparaison ; trouver la plus grosse commande (255,7) en coûte **157** ; et constater qu'une valeur est **absente** (1,0) en coûte **400**.

Quand la valeur est **absente**, il faut lire les 400 éléments pour en être sûr : le pire cas est proportionnel à la taille de la liste. Un ensemble ou un dictionnaire, lui, répond en une seule opération, quel que soit le nombre d'éléments. Nous mesurerons l'écart en secondes à la section 4.8 ; retenez pour l'instant l'ordre de grandeur : pour un million d'éléments, une liste doit parcourir *jusqu'à un million* de cases, un ensemble en regarde *quelques-unes*.

Application directe : **compter les montants différents et les plus fréquents** (montants est la liste des 400 montants) :

```
from collections import Counter

print("montants distincts :", len(set(montants)), "sur", len(montants), "commandes")
print(Counter(montants).most_common(3))      # un Counter est un dictionnaire de comptage

montants distincts : 344 sur 400 commandes
[(37.5, 4), (53.5, 4), (65.8, 3)]
```

Un Counter est un dictionnaire spécialisé dans le comptage : la clé est la valeur, la valeur est son nombre d'occurrences. C'est l'outil idéal pour une table de fréquences (3.1) construite à la main.

4.3.3 Piles et files

Deux structures très simples, définies par l'ordre dans lequel on y entre et sort.

- La **pile** (*stack*, **LIFO** : *last in, first out*) : le dernier arrivé est le premier servi. Une pile d'assiettes : on pose et on reprend toujours en haut. C'est le mécanisme du bouton « annuler » de votre éditeur de texte, et de l'**historique** de votre navigateur.
- La **file** (*queue*, **FIFO** : *first in, first out*) : le premier arrivé est le premier servi. La file d'attente à la caisse.

En Python, une pile est une simple liste que l'on manipule **par la fin** (append pour empiler, pop pour dépiler). Pour une file, on utilise deque (prononcez « dèque »), car retirer un élément au **début** d'une liste est lent, alors qu'un deque le fait à coût constant.

```
from collections import deque

pile = ["ajouter bol", "ajouter tasse", "ajouter plateau"]      # une liste utilisée par la fin
print("annuler :", pile.pop(), "| il reste :", pile)
file = deque(["Léa", "Hugo", "Inès"])
print("servi :", file.popleft(), "| il reste :", list(file))
```

```
annuler : ajouter plateau | il reste : ['ajouter bol', 'ajouter tasse']
servi : Léa | il reste : ['Hugo', 'Inès']
```

Exemple : vérifier les parenthèses d'une formule (avec une pile). Un tableur doit rejeter $=(B2+B3)*(1-(C2/100))$ car il manque une parenthèse fermante. Comment un programme le détecte-t-il ?

Idée : à chaque parenthèse **ouvrante**, on empile ; à chaque parenthèse **fermante**, on dépile et on vérifie qu'elle correspond. La formule est correcte si, **à la fin**, la pile est vide et si nous n'avons jamais dû dépiler une pile vide.

À la main sur $(1+(2*3))$: (→ pile [(] ; 1, + ignorés ; (→ [(, (] ; $2*3$ ignorés ;) → on dépile : [(] ;) → on dépile : []. Pile vide à la fin : équilibrée. Sur $(1+2))$: après le premier) la pile est vide ; le second) ne trouve rien à dépiler : **erreur**.

```
def equilibre(formule):
    ouvrantes, pile = {"(": "(", ")": ")", "[": "[", "]": "]"}, []
    for c in formule:
        if c in "([{":
            pile.append(c)
        elif c in ")]}" and (not pile or pile.pop() != ouvrantes[c]):
            return False
    return not pile      # vrai seulement si tout a été refermé

print([equilibre(t) for t in ["(1+(2*3))", "(1+2))", "(1-(C2/100))", "[(1+2)*3)", ""]])

[True, False, False, False, True]
```

Le cas $[(1+2)*3]$ est instructif : il y a autant d'ouvrantes que de fermantes, mais elles sont **mal imbriquées** : une simple comptabilité ne suffirait pas, la pile, si. Voilà pourquoi tous les compilateurs et analyseurs de formules utilisent cette structure.

Avec une file, on peut aussi **simuler** une caisse ou un atelier d'emballage : les commandes arrivent, attendent leur tour, sont traitées une à une. C'est le premier pas vers la **théorie des files d'attente** vue au chapitre 2 (processus de Poisson et loi exponentielle). Le cahier en propose une simulation complète (application 4.3).

4.3.4 La récursion : une fonction qui s'appelle elle-même

Une fonction est **réursive** quand elle se résout en s'appelant sur un problème **plus petit**. Pour qu'elle marche, il faut toujours deux ingrédients :

1. un **cas de base** (le plus petit problème, résolu directement) ;
2. un **cas général** qui ramène le problème à un problème **strictement plus petit**.

Sans cela, la fonction s'appelle à l'infini (Python finit par s'arrêter avec une erreur `RecursionError`).

Premier exemple : la factorielle. $n! = n \times (n - 1)!$ avec $0! = 1$. Elle compte les façons d'ordonner n objets (1.6). **À la main** : $4! = 4 \times 3! = 4 \times 3 \times 2! = 4 \times 3 \times 2 \times 1! = 24$.

```
def factorielle(n):
    if n == 0:                # cas de base
        return 1
    return n * factorielle(n - 1)  # cas général : un problème plus petit

print([factorielle(n) for n in range(7)])
```

```
[1, 1, 2, 6, 24, 120, 720]
```

Preuve (par récurrence). *Terminaison* : à chaque appel, n diminue de 1 et reste un entier ≥ 0 : on atteint forcément le cas de base. *Correction* : `factorielle(0)` renvoie $1 = 0!$ (base). Si `factorielle(n-1)` renvoie $(n-1)!$ (hypothèse), alors `factorielle(n)` renvoie $n \times (n-1)! = n!$. ■ C'est la même récurrence qu'en mathématiques : écrire une fonction réursive *est* écrire une preuve par récurrence.

Une application naturelle : parcourir un arbre. Le catalogue de la boutique est un dictionnaire de dictionnaires : des catégories contenant des sous-catégories contenant des produits (prix, stock). On veut la **valeur totale du stock**. La difficulté : on ne sait pas combien de niveaux il y a. La récursion le fait tout naturellement : *la valeur d'une catégorie est la somme des valeurs de ses éléments, la valeur d'un produit est prix $\times$ stock*.

À la main : bol bleu $12,5 \times 10 = 125$; bol vert $12,5 \times 4 = 50$; tasse $8 \times 20 = 160$; bougie $15,9 \times 6 = 95,4$; plateau $45 \times 2 = 90$. Total : 520,4 €. La fonction ci-dessous le retrouve sur ce catalogue.

```
def valeur_stock(noeud):
    if isinstance(noeud, tuple):                # cas de base : un produit (prix, stock)
        prix, stock = noeud
        return prix * stock
    return sum(valeur_stock(enfant) for enfant in noeud.values())  # une catégorie

print(round(valeur_stock(catalogue), 2), "€")
```

```
520.4 €
```

Le danger : la récursion naïve peut être catastrophique. Les nombres de Fibonacci ($F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$) se codent en deux lignes récursives, mais le programme refait **sans cesse les mêmes calculs** : pour calculer `fib(5)`, on calcule deux fois `fib(3)`, trois fois `fib(2)`, etc. En comptant les appels de la version naïve, on trouve 177 appels pour $n = 10$, 21 891 pour $n = 20$ et **242 785** pour $n = 25$. Une seule ligne ajoutée, le décorateur `lru_cache`, qui retient chaque résultat déjà calculé, change tout :

```
from functools import lru_cache

@lru_cache(maxsize=None)    # « mémoïsation » : on retient les résultats déjà calculés
def fib(n):
```

```

    return n if n < 2 else fib(n - 1) + fib(n - 2)

print(fib(25))

```

```
75025
```

Le nombre d'appels de la version naïve **explose** (il est lui-même de l'ordre de F_n , donc il croît environ de 62 % à chaque pas), alors que la version qui **mémore** ne calcule chaque valeur qu'une seule fois (26 exécutions réelles pour $n = 25$). Même problème, même résultat, des ordres de grandeur d'écart : c'est tout l'enjeu de la complexité (section 4.8). La mémorisation est la première idée de la **programmation dynamique**, très utilisée en optimisation.

Récursion ou boucle ? Tout algorithme récursif peut s'écrire avec une boucle (et inversement). La récursion est souvent plus *lisible* pour les structures **arborescentes** (catalogue, dossiers, expressions) ; la boucle est plus économe en mémoire. Python limite la profondeur de récursion à environ 1 000 appels : au-delà, préférez une boucle.

4.3.5 Rechercher : de la liste à la dichotomie

Reprenons la **recherche linéaire** de 4.3.2 : sur une liste quelconque, elle est inévitable. Mais si la liste est **triée**, on peut faire beaucoup mieux, comme quand vous cherchez un mot dans un dictionnaire papier : vous ouvrez au milieu, vous voyez si le mot est avant ou après, et vous éliminez **la moitié** des pages d'un coup. C'est la **recherche dichotomique** (*binary search*).

À la main. Liste triée de 9 montants : [8; 15; 22; 31; 40; 47; 58; 66; 79] (indices 0 à 8). On cherche 47.

Étape	Zone d'indices [g,d]	Milieu $m = \lfloor (g + d) / 2 \rfloor$	Valeur	Décision
1	[0,8]	4	40	40 < 47 : on cherche à droite, g = 5
2	[5,8]	6	58	58 > 47 : on cherche à gauche, d = 5
3	[5,5]	5	47	trouvé !

Trois étapes au lieu de six pour la recherche linéaire. Le programme (il renvoie la position, ou -1) :

```

def recherche_dichotomique(triee, cible):
    """Renvoie la position de la cible, ou -1. La liste doit être triée."""
    g, d = 0, len(triee) - 1
    while g <= d:
        m = (g + d) // 2
        if trie[m] == cible:
            return m
        elif trie[m] < cible:
            g = m + 1
        else:
            d = m - 1
    return -1

print(recherche_dichotomique([8, 15, 22, 31, 40, 47, 58, 66, 79], 47))

```

Preuve de correction et de terminaison.

Invariant : si la cible est dans la liste, elle se trouve à un indice entre g et d inclus. *Initialisation :* $g = 0, d = n - 1$: toute la liste. *Conservation :* si $triee[m] < cible$, comme la liste est triée, tous les éléments d'indice $\leq m$ sont $< cible$: on peut les éliminer, d'où $g = m + 1$. Symétriquement si $triee[m] > cible$. L'invariant reste vrai. *Conclusion :* si la boucle s'arrête parce que $g > d$, la zone est vide, donc la cible est absente (on renvoie -1) ; si elle s'arrête sur $triee[m] == cible$, c'est gagné.

Terminaison et coût : appelons $s = d - g + 1$ la taille de la zone. À chaque tour, la nouvelle zone a au plus $\lfloor s/2 \rfloor$ éléments (on a éliminé le milieu et une moitié). Après k tours, la taille est au plus $\lfloor n/2^k \rfloor$, qui devient 0 dès que $2^k > n$. L'algorithme s'arrête donc en **au plus** $\lfloor \log_2 n \rfloor + 1$ tours. ■

Vérifions la borne théorique sur nos 400 montants triés, en cherchant **chacun** des 400 montants et en relevant le pire cas : on observe au plus **9** comparaisons (la borne vaut $\lfloor \log_2 400 \rfloor + 1 = 9$), **7,42** en moyenne, et 8 pour une valeur absente comme 1,0.

Le pire cas observé respecte bien la borne (au plus 9 comparaisons pour 400 éléments, et 8 pour une valeur absente comme 1,0, contre 400 pour la recherche linéaire de cette même valeur). Pour **un million** d'éléments : $\lfloor \log_2 10^6 \rfloor + 1 = 20$ comparaisons seulement. C'est le pouvoir du logarithme : chaque doublement de la taille ne coûte qu'**une** comparaison de plus.

Une application : le seuil de livraison gratuite. Une liste triée et la dichotomie répondent à « quelle part des commandes dépasse s € ? » : le module `bisect` contient la recherche dichotomique toute faite. Avec nos montants, un seuil de **70 €** concerne 29,2 % des commandes, à peu près les 30 % qu'une gérante pourrait viser ; le quantile à 70 % (68,9 €) pointe au même endroit. Nous avons retrouvé par un algorithme de recherche ce que les quantiles du 3.1.3 donnaient directement : un bon moyen de comprendre ce que « quantile » veut dire, *la position dans la liste triée*. Le cahier détaille ce calcul (application 4.4).

4.3.6 Trier

Trier est l'un des problèmes les plus étudiés. Il sert partout : classer les clients, calculer une médiane, préparer une recherche dichotomique. Voyons deux algorithmes : un simple, un efficace.

Le tri par insertion. On procède comme avec des cartes à jouer : on prend les éléments un à un et on **insère** chacun à sa place dans la partie déjà triée.

À la main sur [30; 12; 25; 8; 19] (la partie triée est à gauche de la double barre ||) :

Étape	Liste	Action
départ	[30 12, 25, 8, 19]	
1	[12, 30 25, 8, 19]	12 passe devant 30
2	[12, 25, 30 8, 19]	25 passe devant 30
3	[8, 12, 25, 30 19]	8 passe devant tous
4	[8, 12, 19, 25, 30]	19 s'insère entre 12 et 25

Le programme tient en une dizaine de lignes (deux boucles imbriquées : l'une parcourt les éléments, l'autre décale les plus grands vers la droite) ; la preuve ci-dessous en dit plus que le code.

Correction. *Invariant* : avant le tour i , les i premiers éléments de a sont triés (et ce sont les i premiers éléments d'origine). Au tour i , on décale vers la droite tous les éléments plus grands que x puis on place x juste avant eux : les $i + 1$ premiers éléments sont triés. À la fin ($i = n$), tout est trié. *Terminaison* : deux boucles bornées. *Coût* : au pire (liste triée à l'envers), le tour i fait i comparaisons, soit $1 + 2 + \dots + (n - 1) = n(n - 1)/2$ comparaisons au total, de l'ordre de n^2 . ■

Le **tri fusion** (*merge sort*) applique la stratégie « **diviser pour régner** » : on coupe la liste en deux, on trie chaque moitié (récursivement), puis on **fusionne** les deux moitiés triées. Fusionner est facile : on compare les deux premiers éléments des moitiés, on prend le plus petit, et on recommence, comme deux files de gens que l'on entrelace.

À la main sur $[44; 12; 30; 8; 25; 19]$: on coupe en $[44, 12, 30]$ et $[8, 25, 19]$; chacun est trié en $[12, 30, 44]$ et $[8, 19, 25]$; la fusion donne $8 < 12 \rightarrow 8$; $12 < 19 \rightarrow 12$; $19 < 30 \rightarrow 19$; $25 < 30 \rightarrow 25$; il reste 30, 44 : $[8, 12, 19, 25, 30, 44]$.

```
def tri_fusion(liste):
    if len(liste) <= 1:
        return liste
    g, d = tri_fusion(liste[:len(liste) // 2]), tri_fusion(liste[len(liste) // 2:])
    fusion = []
    while g and d:
        fusion.append(g.pop(0) if g[0] <= d[0] else d.pop(0))
    return fusion + g + d

print(tri_fusion([44, 12, 30, 8, 25, 19]))
```

```
[8, 12, 19, 25, 30, 44]
```

Pour **mesurer** les coûts, on ajoute un compteur de comparaisons aux deux algorithmes (versions de mesure, non reproduites ici) :

Comparons les deux sur nos 400 montants. Le tri par insertion fait **41 010** comparaisons, le tri fusion **2 972** ; les deux donnent le même résultat que `sorted()`, et les formules $n^2/4 = 40\,000$ et $n \log_2 n \approx 3\,458$ donnent le bon ordre de grandeur.

En moyenne, le tri par insertion fait de l'ordre de $n^2/4$ comparaisons (près de quatorze fois plus que le tri fusion ici), le tri fusion de l'ordre de $n \log_2 n$. Pour 400 éléments, cela fait la différence entre « bien » et « très bien » ; pour 10 millions, entre quelques secondes et plusieurs jours. Le tri intégré de Python, `sorted`, utilise un algorithme hybride très optimisé (*Timsort*) de coût $n \log n$: **en pratique, on utilise toujours `sorted()` ou `.sort()`**, mais comprendre ce qu'il fait permet de raisonner sur son coût.

Un tri est dit stable s'il laisse dans leur ordre d'origine les éléments « égaux » au regard du critère de tri. C'est ce que garantit `sorted` de Python, et cela permet d'enchaîner des tris successifs : trier d'abord par montant, puis par canal, donne les commandes **rangées par canal et, à l'intérieur de chaque canal, par montant croissant**.

```
cmds = [("Site", 40.1), ("Boutique", 65.8), ("Site", 19.6), ("Boutique", 17.4), ("Réseaux", 30.1)]
print(sorted(sorted(cmds, key=lambda c: c[1]), key=lambda c: c[0])) # stable : l'ordre par montant
↪ est conservé
```

```
[('Boutique', 17.4), ('Boutique', 65.8), ('Réseaux', 30.1), ('Site', 19.6), ('Site', 40.1)]
```

4.3.7 Une dernière application : les « top k »

La gérante demande : « Quelles sont mes **cinq plus grosses commandes** ? » On pourrait trier les 400 montants puis prendre les cinq derniers (coût de l'ordre de $n \log n$). Mais c'est du gaspillage : on n'a pas besoin que *tout* soit trié. Une structure appelée **tas** (*heap*) maintient efficacement les k plus grands vus jusqu'ici, avec un coût de l'ordre de $n \log k$. Le module `heapq` la fournit :

```
import heapq

print(heapq.nlargest(5, montants))
print(sorted(montants, reverse=True)[:5])      # même réponse, mais en triant tout
```

```
[255.7, 243.8, 217.1, 212.4, 208.8]
[255.7, 243.8, 217.1, 212.4, 208.8]
```

Les deux listes sont identiques, la version `heapq` coûtant moins cher quand $k \ll n$ (pensez aux dix meilleurs clients sur dix millions). Ce réflexe, **ne pas faire plus de travail que nécessaire**, est l'une des habitudes les plus rentables de la programmation scientifique.

Pour s'entraîner. Cahier, chapitre 4 : applications 4.3 (file d'attente à l'atelier) et 4.4 (seuil de livraison gratuite) ; exercices 4.5 à 4.8 (pile et notation polonaise inversée, file à deux emballeuses, dichotomie à variante, tri stable).

Pour aller plus loin. Les mêmes idées (hachage, tri, recherche dichotomique, arbres) sont au cœur des **bases de données** : l'**index** d'une table SQL (chapitre 5) est précisément un arbre trié qui permet une recherche dichotomique ; les **GROUP BY** s'appuient sur des tables de hachage ou des tris. Comprendre cette section, c'est comprendre pourquoi une requête est rapide ou lente.

À retenir

- Un algorithme doit être **précis, terminer, et être correct** ; on le prouve avec un **invariant** (vrai à chaque tour) et un argument de **terminaison** ; on mesure son coût en **nombre d'opérations**.
- **Liste** : ordre et position ; **dictionnaire / ensemble** : accès et test d'appartenance immédiats (table de hachage) ; **pile** : dernier entré, premier sorti ; **file** : premier entré, premier sorti.
- La **réursion** exige un cas de base et un problème strictement plus petit ; elle se prouve comme une récurrence. Sans précaution (mémoïsation), elle peut refaire des calculs en nombre exponentiel.
- La **recherche dichotomique** (liste triée) coûte au plus $\lfloor \log_2 n \rfloor + 1$ comparaisons : 9 pour 400 éléments, 20 pour un million.
- Le **tri par insertion** coûte de l'ordre de n^2 , le **tri fusion** de l'ordre de $n \log n$; en pratique on utilise `sorted()`, qui est stable.
- Ne trier que ce qui est nécessaire : pour les « top k », `heapq.nlargest` suffit.

4.4 NumPy et pandas : manipuler des données

Intuition. Python « de base » (section 4.1) sait faire des calculs, mais il est lent et verbeux dès qu'il faut traiter des milliers de nombres. Deux bibliothèques ont changé la donne et sont aujourd'hui le socle de la data science en Python :

- **NumPy** ajoute un nouveau type d'objet, le **tableau** (ndarray), qui permet de calculer sur toute une colonne de nombres d'un seul coup, à vitesse quasi native ;
- **pandas** pose par-dessus un **tableau étiqueté** (le DataFrame) : des colonnes qui ont un nom et un type, des lignes qui ont une étiquette, des dates, des valeurs manquantes... bref, un tableur programmable, sans souris et sans limite de taille.

La gérante a l'habitude d'Excel. Tout ce que nous allons faire ici, elle pourrait le faire à la main dans une feuille de calcul, mais pour 400 lignes ce serait pénible, et pour 400 000 lignes ce serait impossible. Surtout, **le code est rejouable** : on corrige une erreur, on relance, on retrouve toutes les analyses mises à jour.

Comment lire cette section. Elle est longue parce que pandas est l'outil que vous utiliserez tous les jours. Les trois premières parties (4.4.1 à 4.4.4) concernent NumPy ; la suite concerne pandas. Chaque notion est introduite par un **petit exemple à la main**, puis appliquée aux 400 commandes de la boutique. Si vous êtes pressé(e), lisez au moins 4.4.5 à 4.4.9, puis la conclusion de 4.4.13.

4.4.1 NumPy : pourquoi un tableau n'est pas une liste

La gérante veut afficher les prix TTC de cinq articles à partir de leurs prix hors taxe (TVA à 19 %). Avec une **liste** Python classique, on écrit une boucle :

```
import numpy as np

print([round(p * 1.19, 2) for p in [10, 20, 5, 40, 15]])    # avec une liste : une boucle
```

```
[11.9, 23.8, 5.95, 47.6, 17.85]
```

À la main : $10 \times 1,19 = 11,90$, $20 \times 1,19 = 23,80$, $5 \times 1,19 = 5,95$, $40 \times 1,19 = 47,60$ et $15 \times 1,19 = 17,85$. Le résultat est celui attendu. Avec un **tableau NumPy**, la boucle disparaît :

```
ht = np.array([10, 20, 5, 40, 15])
print(ht * 1.19)
print(type(ht), ht.dtype, ht.shape)
```

```
[11.9  23.8   5.95 47.6  17.85]
<class 'numpy.ndarray'> int64 (5,)
```

Une seule opération `ht * 1.19` s'applique à **chaque élément** : on dit qu'elle est **vectorisée**. Ce n'est pas qu'une commodité d'écriture : les listes Python sont des collections d'objets quelconques (un entier, puis un texte, puis une autre liste...), alors qu'un tableau NumPy est un **bloc de mémoire homogène** (ici des entiers de 64 bits, `dtype=int64`) traité par du code compilé en C. D'où deux différences que tout le monde rencontre un jour :

```
print([10, 20] * 2)          # liste : * 2 répète la liste
print(np.array([10, 20]) * 2) # tableau : * 2 multiplie chaque élément
```

```
[10, 20, 10, 20]
[20 40]
```

Piège classique. Sur une liste, `*` et `+` **répètent** et **concatènent**. Sur un tableau, ils **calculent**. Si vous voyez une liste de 10 000 nombres se mettre à « doubler de longueur » au lieu de doubler de valeur, c'est qu'un tableau a été oublié.

Mesurons l'écart de vitesse sur un million de prix : une boucle `[v * 1.19 for v in liste]` contre l'opération vectorisée `x * 1.19`. Les deux donnent exactement les mêmes résultats, et la version NumPy est **au moins 5 fois plus rapide** ; les durées exactes dépendent de votre machine, nous ne retenons donc que cette conclusion robuste. Sur du calcul plus lourd que cette simple multiplication, l'écart se compte en **dizaines, voire centaines de fois**. C'est la raison pour laquelle on cherche toujours à **vectoriser** : écrire `x * 1.19` plutôt qu'une boucle `for`.

À retenir. Un tableau NumPy = un bloc de nombres **du même type**, sur lequel les opérations s'appliquent **élément par élément** et vite. Règle d'or : *pas de boucle for sur des données, sauf si l'on n'a vraiment pas le choix*.

4.4.2 Créer, indexer, trancher

Quelques manières courantes de fabriquer des tableaux :

```
print(np.arange(0, 10, 2))      # de 0 à 10 (exclu), pas de 2
print(np.linspace(0, 1, 5))    # 5 points régulièrement espacés entre 0 et 1
print(np.zeros(3), np.ones(3)) # que des 0, que des 1
```

```
[0 2 4 6 8]
[0.  0.25 0.5  0.75 1.  ]
[0. 0. 0.] [1. 1. 1.]
```

Un tableau a trois caractéristiques à toujours avoir en tête : sa **forme** (shape), son nombre de dimensions (ndim) et son type (dtype). Prenons les ventes hebdomadaires (en €) des trois canaux de la boutique sur quatre semaines : une **matrice** de 3 lignes (canaux) et 4 colonnes (semaines), exactement comme celles du chapitre 1.

	Sem. 1	Sem. 2	Sem. 3	Sem. 4
Réseaux	120	150	90	140
Site	90	100	120	70
Boutique	60	50	90	105

```
A = np.array([[120, 150, 90, 140],
              [ 90, 100, 120, 70],
              [ 60, 50, 90, 105]])
print(A.shape, A.ndim, A.dtype, A.size)
```

```
(3, 4) 2 int64 12
```

Indexation. On désigne un élément par `A[ligne, colonne]`, en comptant à **partir de 0** (comme partout en Python). Quelques exemples à vérifier *sur le tableau ci-dessus* avant de regarder la sortie :

- `A[1, 2]` : ligne 1 (le Site), colonne 2 (semaine 3) → **120** ;
- `A[0]` : toute la ligne 0 (Réseaux) ;
- `A[:, 3]` : toute la colonne 3 (semaine 4) ; le « : » signifie « tout » ;
- `A[0:2, 1:3]` : lignes 0 et 1, colonnes 1 et 2 (la borne de fin est **exclue**).

```
print(A[1, 2])
print(A[0])
print(A[:, 3])
print(A[0:2, 1:3])
```

```
120
[120 150  90 140]
[140  70 105]
[[150  90]
 [100 120]]
```

Filtrer avec un masque booléen. Comparer un tableau à un nombre produit un tableau de True/False de même forme : le **masque**. Utilisé comme indice, il ne garde que les cases True.

```
masque = A > 100
print(A[masque])                # les ventes strictement supérieures à 100 €
print("combien ?", masque.sum()) # True compte pour 1 : on compte donc les cases vraies
```

```
[120 150 140 120 105]
combien ? 5
```

Dans le masque, on compte 5 cases vraies : 120, 150 et 140 (Réseaux), 120 (Site, semaine 3) et 105 (Boutique, semaine 4). Notez que le 100 du Site (semaine 2) n'est **pas** compté : la condition est « strictement supérieur à 100 ». La somme d'un masque est une astuce très utile : elle **compte** les True.

Pour combiner deux conditions, on utilise & (et), | (ou) et ~ (non), **avec des parenthèses** :

```
print(A[(A > 80) & (A < 130)])    # entre 80 et 130 exclus
```

```
[120  90  90 100 120  90 105]
```

Tranche = vue, pas copie. Tailler un morceau d'un tableau avec : ne copie pas les données : la tranche est une **fenêtre** sur le tableau d'origine. Modifier la fenêtre modifie l'original ! Pour obtenir une copie indépendante, écrivez explicitement `.copy()`.

```
C = A.copy()
fenetre = C[0, :]    # une vue sur la ligne de Réseaux
fenetre[0] = 999
print(C[0, 0], "<- modifié par la vue ; A[0, 0] =", A[0, 0], "(intact, car C est une copie)")
```

```
999 <- modifié par la vue ; A[0, 0] = 120 (intact, car C est une copie)
```

Enfin, un tableau a **un seul type** : si l'on mélange entiers et décimaux, tout devient décimal (`np.array([1, 2, 3.5])` donne `[1. 2. 3.5]`). Et un tableau d'entiers ne sait pas représenter une valeur manquante, ce qui sera une raison de plus d'aimer pandas (4.4.11).

4.4.3 Le broadcasting : calculer entre tableaux de formes différentes

Que se passe-t-il si l'on fait $A - m$ où A est une matrice 3×4 et m un vecteur de 4 nombres ? Mathématiquement, la soustraction n'est pas définie (les formes diffèrent) ; NumPy, lui, **étire** le plus petit tableau pour qu'il s'adapte : c'est le **broadcasting** (« diffusion »).

Reprenons l'exemple de la gérante. Elle veut savoir, pour chaque semaine, **de combien chaque canal s'écarte de la moyenne de cette semaine**. Moyenne de la semaine 1 : $(120 + 90 + 60)/3 = 90$. Semaine 2 : $(150 + 100 + 50)/3 = 100$. Semaine 3 : $(90 + 120 + 90)/3 = 100$. Semaine 4 : $(140 + 70 + 105)/3 = 105$. Donc $m = (90, 100, 100, 105)$ et, par exemple, Réseaux en semaine 1 s'écarte de $120 - 90 = +30$.

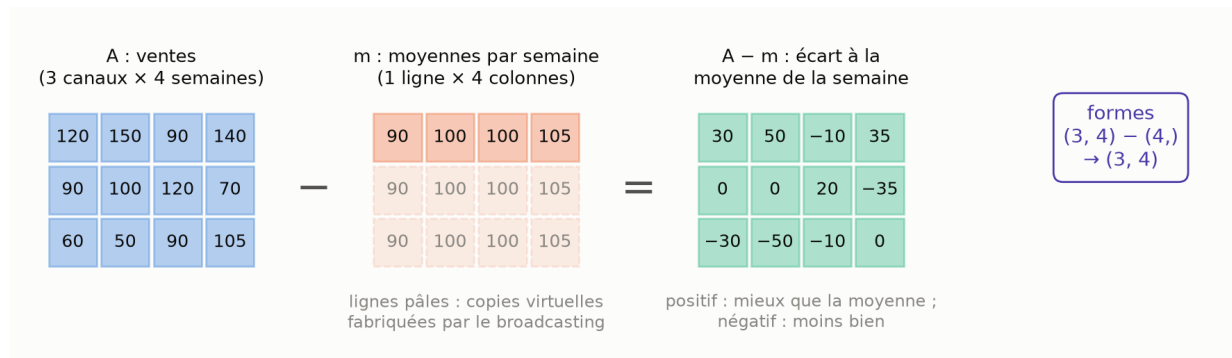


Figure 23: Le broadcasting : le vecteur m (4 nombres) est « étiré » sur les trois lignes de A , puis la soustraction se fait case par case.

```
m = A.mean(axis=0)      # moyenne de chaque colonne (axis=0 : on « écrase » les lignes)
print("moyennes par semaine :", m)
print(A - m)
```

```
moyennes par semaine : [ 90. 100. 100. 105.]
[[ 30.  50. -10.  35.]
 [  0.   0.  20. -35.]
 [-30. -50. -10.   0.]]
```

Le résultat reproduit les valeurs du dessin : 30 pour Réseaux en semaine 1, -35 pour le Site en semaine 4, etc.

La règle du broadcasting. NumPy compare les formes **en partant de la droite**. Deux dimensions sont compatibles si elles sont **égales** ou si l'une des deux vaut 1 (elle est alors étirée). Une dimension manquante à gauche est comptée comme 1.

- (3,4) et (4,) : $4 = 4$ ✓, puis le 3 n'a pas de vis-à-vis → compatible, résultat (3,4) ;
- (3,4) et (3,1) : 4 vs 1 ✓, $3 = 3$ ✓ → compatible, résultat (3,4) ;
- (3,4) et (3,) : 4 vs 3 ✗ → **erreur**.

Pour soustraire cette fois la **moyenne de chaque ligne** (la vente moyenne de chaque canal), il faut un vecteur *colonne* de forme (3,1). L'argument `keepdims=True` garde la dimension écrasée avec la taille 1, ce qui rend le broadcasting possible :

```
moy_canal = A.mean(axis=1, keepdims=True)
print(moy_canal.shape, "->", moy_canal.ravel())
print(A - moy_canal)
```

```
(3, 1) -> [125.   95.   76.25]
[[ -5.   25.  -35.   15. ]
 [ -5.    5.   25.  -25. ]
 [-16.25 -26.25  13.75  28.75]]
```

À la main : la moyenne de Réseaux est $(120 + 150 + 90 + 140)/4 = 125$, donc la première ligne devient $(-5, 25, -35, 15)$. Et si on oublie `keepdims` ? NumPy refuse et le dit clairement (operands could not be broadcast together with shapes (3,4) (3,)), ce qui rappelle la règle.

Standardiser des colonnes. Au chapitre 1, nous avons vu que les algorithmes de modélisation aiment que les variables aient la même échelle. Centrer-réduire chaque colonne (soustraire sa moyenne, diviser par son écart-type) s'écrit sans boucle grâce au broadcasting : $(X - X.mean(axis=0)) / X.std(axis=0, ddof=1)$. Vous le ferez avec pandas à la section 4.4.7.

4.4.4 Agréger, trier, tirer au hasard

Les fonctions de résumé (`sum`, `mean`, `std`, `min`, `max`...) prennent un argument **axis** qui choisit la direction du calcul. Une mnémotechnique : *axis est la dimension qui disparaît*.

Appel	Ce qui disparaît	Résultat pour notre A (3 × 4)
<code>A.sum()</code>	tout	un seul nombre (le chiffre d'affaires total)
<code>A.sum(axis=0)</code>	les lignes	un total par semaine (4 nombres)
<code>A.sum(axis=1)</code>	les colonnes	un total par canal (3 nombres)

```
print("total général :", A.sum())
print("par semaine  :", A.sum(axis=0))
print("par canal    :", A.sum(axis=1))
```

```
total général : 1185
par semaine  : [270 300 300 315]
par canal    : [500 380 305]
```

Vérification à la main : Réseaux $120 + 150 + 90 + 140 = 500$, Site $90 + 100 + 120 + 70 = 380$, Boutique $60 + 50 + 90 + 105 = 305$, soit 1 185 € au total, ce qui est aussi la somme des totaux par semaine ($270 + 300 + 300 + 315$).

Autres opérations fréquentes : `np.cumsum` (somme cumulée), `np.diff` (différences successives), `np.sort` et `np.argsort` (le tri, et l'ordre qui trierait).

ddof encore. Comme pour `np.std` à la section 3.1.4, NumPy divise par n par défaut. Pour estimer la variance d'une population à partir d'un échantillon, il faut **ddof=1** (division par $n - 1$). pandas, lui, utilise $n - 1$ par défaut : un écart entre `np.std(x)` et `serie.std()` n'est donc pas un bug.

Nombres aléatoires. On rencontre le générateur de nombres aléatoires depuis le chapitre 2 : on le crée une fois avec une **graine** (*seed*), puis on tire.

Avec 100 000 journées simulées (`rng.normal(120, 15, size=100_000)`), la moyenne simulée est 119,9 et la part des jours à plus de 150 ventes est 0,0233. La moyenne d'un masque booléen est la **proportion** de True : c'est la façon la plus économique d'estimer une probabilité par simulation. On retrouve (à très peu près) les 2,3 % calculés à la main au 2.2 pour un jour à plus de deux écarts-types au-dessus de la moyenne.

Enfin, NumPy sait faire l'algèbre linéaire du chapitre 1 : produit matriciel `@`, transposée `.T`, inverse, valeurs propres, résolution de système, etc.

```
M = np.array([[2.0, 1.0], [1.0, 3.0]])
b = np.array([5.0, 10.0])
print("solution de M x = b :", np.linalg.solve(M, b))
```

```
solution de M x = b : [1. 3.]
```

À la main : $2x + y = 5$ et $x + 3y = 10$ donnent $x = 1$, $y = 3$. (`np.linalg` sait aussi inverser, calculer des valeurs propres, etc. : voir le chapitre 1.)

À retenir (NumPy). (1) tableau = type unique + forme ; (2) indexer avec [ligne, colonne], tranches `a:b` (fin exclue) et masques booléens ; (3) le **broadcasting** étire les dimensions de taille 1 ; (4) `axis` = la dimension qui disparaît ; (5) une tranche est une **vue** : `.copy()` pour

travailler sans risque.

4.4.5 pandas : Series et DataFrame

NumPy ne connaît que des nombres rangés dans des cases numérotées. Mais une vraie table de données, ce sont des colonnes **nommées** (« montant », « canal ») de **types différents** (nombres, texte, dates), avec des lignes qu'on veut pouvoir **identifier**. C'est ce que fait pandas.

Deux objets suffisent pour commencer :

- la **Series** : une colonne, c'est-à-dire un tableau NumPy muni d'un **index** (une étiquette par valeur) et d'un nom ;
- le **DataFrame** : un tableau de plusieurs Series partageant le même index.

```
import pandas as pd

ventes = pd.Series([500, 380, 305], index=["Réseaux", "Site", "Boutique"], name="ventes")
print(ventes)
print("par étiquette :", ventes["Site"], "| par position :", ventes.iloc[2])
```

```
Réseaux    500
Site       380
Boutique   305
Name: ventes, dtype: int64
par étiquette : 380 | par position : 305
```

Un DataFrame s'obtient, par exemple, à partir d'un dictionnaire « nom de colonne → valeurs » :

```
mini = pd.DataFrame({"canal": ["Réseaux", "Site", "Boutique"], "ventes": [500, 380, 305],
                    "ouvert_le_dimanche": [True, True, False]})
print(mini.dtypes)
```

```
canal          str
ventes         int64
ouvert_le_dimanche  bool
dtype: object
```

Chaque colonne a son type : str (texte), int64 (entiers), bool (vrai/faux).

pandas 3.0 : le texte a désormais son propre type. Dans les versions de pandas antérieures à la 3.0, les colonnes de texte avaient le type object (« n'importe quoi »). Depuis la **version 3.0**, elles ont un type dédié, affiché str, plus rapide, plus économe en mémoire, et qui représente les valeurs manquantes de façon uniforme. Si vous lisez un vieux tutoriel qui affiche dtype : object pour une colonne de texte, ne vous inquiétez pas : seul l'affichage a changé. Ce livre a été exécuté avec pandas 3.0.

Charger le fichier de données. Voici le fichier donnees/commandes.csv du chapitre 3 :

```
df = pd.read_csv("donnees/commandes.csv")
print(df.shape)
print(df.head(3))
```

```
(400, 4)
   canal  montant  livraison  satisfaction
0  Boutique    44.8         0           4
1    Site     34.5         2           4
2  Réseaux    88.2         5           4
```

`info()` est le meilleur premier réflexe : nombre de lignes, nom et type de chaque colonne, nombre de valeurs **non nulles** (ici, 400 partout : aucune valeur manquante) et mémoire utilisée.

Le fichier ne contient pas de **date**, or la plupart des vraies données de vente en ont une. Pour illustrer le travail sur les dates (4.4.12) et sur plusieurs tables (4.4.10), nous allons **ajouter deux colonnes simulées** : la date de chaque commande (réparties sur 20 semaines à partir du lundi 5 janvier 2026) et un numéro de client (120 clients possibles). La simulation est reproductible grâce à la graine (le code est donné dans la préparation du cahier, chapitre 4) ; la colonne `date` a le type `datetime64`, et `id_commande` numérote les commandes par ordre chronologique :

```
print(df.head(4))
```

	id_commande	canal	montant	livraison	satisfaction	date	id_client
0	1	Site	49.9	8	2	2026-01-05	100
1	2	Réseaux	21.5	3	4	2026-01-05	46
2	3	Boutique	34.2	0	4	2026-01-05	26
3	4	Site	103.0	3	5	2026-01-05	97

Notez que pandas sait que `date` n'est pas du texte, mais de vraies dates, avec lesquelles on peut calculer. Dernier réflexe, le résumé statistique (ici, pour le montant) :

```
print(df["montant"].describe().round(2))
```

```
count    400.00
mean      60.25
std       38.02
min        8.60
25%       34.18
50%       51.00
75%       75.82
max      255.70
Name: montant, dtype: float64
```

4.4.6 Sélectionner : colonnes, lignes, conditions

Pour sélectionner, on dispose de plusieurs outils, résumés dans le tableau ci-dessous. Prenons une toute petite table pour voir clairement ce qui se passe :

```
petit = df[["id_commande", "canal", "montant"]].head(5)
print(petit)
```

	id_commande	canal	montant
0	1	Site	49.9
1	2	Réseaux	21.5
2	3	Boutique	34.2
3	4	Site	103.0
4	5	Réseaux	26.7

Je veux...	J'écris	Remarque
une colonne	<code>petit["montant"]</code>	donne une Series
plusieurs colonnes	<code>petit[["canal", "montant"]]</code>	doublets crochets : une liste de noms
des lignes par position	<code>petit.iloc[1:3]</code>	<code>iloc</code> = <i>integer location</i> ; fin exclue
des lignes par étiquette	<code>petit.loc[1:3]</code>	<code>loc</code> = <i>label</i> ; fin incluse !
des lignes par condition	<code>petit[petit["montant"] > 50]</code>	masque booléen, comme en NumPy

```
print(petit[["canal", "montant"]].iloc[1:3])      # par position : fin exclue
print(petit.loc[1:3, ["canal", "montant"]])      # par étiquette : fin INCLUDE
```

```
   canal  montant
1  Réseaux    21.5
2  Boutique    34.2
   canal  montant
1  Réseaux    21.5
2  Boutique    34.2
3    Site    103.0
```

iloc exclut la fin, loc l'inclut. `iloc[1:3]` renvoie les lignes 1 et 2 ; `loc[1:3]` renvoie les lignes 1, 2 et 3 (car on désigne des étiquettes, et on veut « de 1 jusqu'à 3 »). C'est l'une des étourderies les plus fréquentes.

Filtrer avec des conditions. Les opérateurs `&`, `|`, `~` demandent des parenthèses autour de chaque condition (car `&` s'évalue avant `>`) :

```
gros_reseaux = df[(df["canal"] == "Réseaux") & (df["montant"] > 100)]
print("commandes Réseaux de plus de 100 € :", len(gros_reseaux))
print(df.query("canal == 'Site' and livraison >= 7").shape[0])    # la même idée, écrite comme une
↪ phrase
```

```
commandes Réseaux de plus de 100 € : 11
21
```

La méthode `query` accepte une condition écrite comme une phrase : elle se lit mieux sur des conditions longues. Deux autres formes pratiques : `isin([...])` (appartient à une liste) et `between(a, b)` (bornes incluses). Pour **trier** et chercher les extrêmes, on a `sort_values` ; `nsmallest(3, "montant")` donne directement les trois plus petits :

```
print(df.sort_values("montant", ascending=False).head(3)[["id_commande", "canal", "montant"]])
```

```
   id_commande  canal  montant
380          381  Site    255.7
102          103  Site    243.8
198          199  Site    217.1
```

4.4.7 Créer et transformer des colonnes

Une nouvelle colonne s'obtient en l'assignant, avec des opérations **vectorisées** (comme dans NumPy, sans boucle) :

```
df["livraison_rapide"] = df["livraison"] <= 3
df["gros_panier"] = np.where(df["montant"] >= 100, "oui", "non")
print(df[["canal", "montant", "gros_panier", "livraison_rapide"]].head(3))
```

```
   canal  montant  gros_panier  livraison_rapide
0    Site    49.9          non             False
1  Réseaux    21.5          non             True
2  Boutique    34.2          non             True
```

Pour le texte, le préfixe `.str` donne accès à toutes les méthodes de Python (`upper`, `lower`, `contains`, `replace`, `split`...) appliquées à **chaque élément** de la colonne : par exemple `df["canal"].str.upper()`.

Découper une variable continue en classes. `pd.cut` fabrique des classes de bornes choisies, `pd.qcut` des classes d'effectifs égaux (par quantiles). Par exemple, quatre tranches de panier :

« petit » (moins de 30 €), « moyen » (30 à 60), « grand » (60 à 100) et « très grand » (plus de 100) :

```
noms = ["petit", "moyen", "grand", "très grand"]
df["tranche"] = pd.cut(df["montant"], bins=[0, 30, 60, 100, np.inf], labels=noms)
print(df["tranche"].value_counts().reindex(noms))
```

```
tranche
petit      76
moyen     160
grand     112
très grand  52
Name: count, dtype: int64
```

Remarquez la nuance : avec `cut`, les classes sont **définies par vous** et leurs effectifs sont inégaux ; avec `qcut`, ce sont les effectifs qui sont **égaux** ($400 / 4 = 100$) et les bornes qui s'adaptent aux données.

Remplacer des valeurs selon un dictionnaire se fait avec `map` : `df["satisfaction"].map({1: "très mécontent", 2: "mécontent", 3: "neutre", 4: "content", 5: "très content"})` remplace chaque note par son libellé (195 commandes sont « content » et 104 « très content »).

Standardiser (centrer-réduire), comme annoncé à la fin de 4.4.3, se fait sur une colonne entière :

Par construction, `z` a une moyenne nulle et un écart-type de 1. Sept commandes dépassent 3 écarts-types. Si les montants suivaient une loi normale, on n'en attendrait qu'**une seule** sur 400 environ (la probabilité d'être à plus de 3 écarts-types est de 0,27 %, d'après les repères du 2.2). En trouver sept confirme ce que nous avons vu au 3.1.5 : la distribution des montants a une **queue lourde à droite**.

apply : à garder en dernier recours. Si une transformation n'existe pas en version vectorisée, `df["col"].apply(ma_fonction)` appelle votre fonction **ligne par ligne** : pratique, mais lent (une boucle Python déguisée). Réflexe : chercher d'abord une opération vectorisée (`.str`, `np.where`, `cut`, `map`, opérateurs arithmétiques...). Sur nos 400 lignes la différence est invisible ; en répétant les montants 500 fois (200 000 lignes), `apply` donne le même résultat que la version vectorisée `np.where(gros > 50, gros * 1.19, gros)` mais s'avère **plus lent**.

4.4.8 Copie, vue et pièges de pandas 3.0

Une question revient sans cesse : *si je modifie un morceau de mon tableau, est-ce que je modifie aussi l'original ?* Dans les versions anciennes de pandas, la réponse dépendait de détails obscurs (c'était le fameux `SettingWithCopyWarning`). **Depuis pandas 3.0, la règle est simple : le *copy-on-write* (copie à l'écriture)**. Tout objet dérivé d'un autre se comporte comme **une copie indépendante** ; modifier l'un ne modifie jamais l'autre.

```
montants = df["montant"]           # une Series dérivée de df
montants.iloc[0] = -1             # on la modifie...
print(df['montant'] au premier rang : ", df["montant"].iloc[0], "(inchangé)")
```

```
df['montant'] au premier rang : 49.9 (inchangé)
```

La conséquence la plus importante : l'**assignation en chaîne** (`df[condition]["colonne"] = valeur`) **ne fonctionne plus**, car `df[condition]` fabrique une copie temporaire que l'on modifie puis que l'on jette. pandas 3.0 émet même un avertissement (`ChainedAssignmentError`). Vérifions-le :

```
copie = df.copy()
copie[copie["canal"] == "Site"]["montant"] = 0           # [X] assignation en chaîne : sans effet
```

```
copie.loc[copie["canal"] == "Site", "montant"] = 0      # [OK] la bonne écriture
print(int((copie["montant"] == 0).sum()), "montants mis à zéro")
```

```
148 montants mis à zéro
```

Avec la mauvaise écriture, **aucun** montant n'est modifié (0) ; avec la bonne, 148 le sont : les 148 commandes du Site.

La bonne écriture, toujours : `df.loc[condition, "colonne"] = valeur`. Une seule opération, qui désigne à la fois les lignes et la colonne. Et pour *vraiment* garder une copie intacte avant de modifier : `df2 = df.copy()`.

4.4.9 Regrouper : le « split-apply-combine » (groupby)

C'est l'outil le plus important de pandas pour répondre à des questions du type : « *quel est le montant moyen par canal ?* », « *combien de commandes par semaine ?* ». La méthode s'appelle **découper – appliquer – combiner** (*split-apply-combine*) :

1. **découper** le tableau en groupes (une valeur de canal = un groupe) ;
2. **appliquer** un calcul à chaque groupe (moyenne, somme, comptage...) ;
3. **combiner** les résultats dans un nouveau tableau.

Faisons-le **à la main** sur six commandes :

Commande	Canal	Montant
1	Réseaux	20
2	Site	30
3	Réseaux	40
4	Site	50
5	Boutique	100
6	Site	70

Groupes : Réseaux {20, 40}, Site {30, 50, 70}, Boutique {100}. Moyennes : 30, 50 et 100. Comptes : 2, 3, 1. Voici le même calcul en code (pandas range les groupes par ordre alphabétique) :

```
six = pd.DataFrame({"canal": ["Réseaux", "Site", "Réseaux", "Site", "Boutique", "Site"],
                      "montant": [20, 30, 40, 50, 100, 70]})
print(six.groupby("canal")["montant"].agg(["mean", "count"]))
```

```
      mean  count
canal
Boutique 100.0     1
Réseaux   30.0     2
Site      50.0     3
```

Passons aux 400 commandes. L'**agrégation nommée** donne des colonnes lisibles : `nom= ("colonne", "fonction")`.

```
resume = df.groupby("canal").agg(
    commandes= ("montant", "count"),
    ca= ("montant", "sum"),
    panier_moyen= ("montant", "mean"),
).round(2)
print(resume)
```

	commandes	ca	panier_moyen
canal			
Boutique	114	8528.3	74.81
Réseaux	138	6763.5	49.01
Site	148	8806.5	59.50

On peut regrouper selon **plusieurs** critères (`df.groupby(["canal", "gros_panier"]).size().unstack()` donne un tableau canal × panier) ou demander une répartition en proportions :

```
print(pd.crosstab(df["canal"], df["tranche"], normalize="index").round(2)) # proportions par ligne
```

tranche	petit	moyen	grand	très grand
canal				
Boutique	0.07	0.37	0.32	0.24
Réseaux	0.32	0.38	0.22	0.08
Site	0.16	0.44	0.30	0.09

`crosstab` (tableau croisé) est un raccourci pour compter les effectifs de deux variables qualitatives ; avec `normalize="index"` chaque ligne est ramenée à 1, ce qui répond à « *parmi* les commandes Réseaux, quelle part de petits paniers ? ». Pour des tableaux de synthèse à deux entrées sur une variable numérique, on a `pivot_table` :

```
print(df.pivot_table(index="canal", columns="gros_panier", values="satisfaction",
↳ aggfunc="mean").round(2))
```

gros_panier	non	oui
canal		
Boutique	4.49	4.48
Réseaux	3.72	3.64
Site	3.80	3.71

Enfin `transform` calcule un résultat **par groupe** mais le renvoie **aligné sur les lignes d'origine**, ce qui permet de comparer chaque commande à son groupe :

```
df["ecart_moy_canal"] = df["montant"] - df.groupby("canal")["montant"].transform("mean")
print(df[["canal", "montant", "ecart_moy_canal"]].head(3).round(1))
```

	canal	montant	ecart_moy_canal
0	Site	49.9	-9.6
1	Réseaux	21.5	-27.5
2	Boutique	34.2	-40.6

À retenir (groupby). `df.groupby(clé)[colonne].fonction()` : *découper, appliquer, combiner*. `agg` pour plusieurs résumés, `size` pour compter les lignes, `transform` pour ré-aligner un résultat de groupe sur chaque ligne, `pivot_table` et `crosstab` pour des tableaux croisés.

4.4.10 Combiner plusieurs tables : merge et concat

Dans la vraie vie, l'information est **répartie sur plusieurs tables** : la liste des commandes d'un côté, celle des clients de l'autre (nous verrons au chapitre 5 pourquoi, et comment SQL fait la même chose avec JOIN). Le travail s'appelle une **jointure** : on associe les lignes qui partagent la même **clé**.

Créons la table des clients : 125 clients, chacun avec une ville. (Les clients 121 à 125 n'ont, par construction, jamais commandé ; comme les commandes ont été attribuées au hasard à des clients de 1 à 120, quelques autres clients n'auront rien commandé non plus. Cela nous servira.)

```
print(clients.head(3))
print("clients :", len(clients), "| commandes :", len(df))
```

```
   id_client  ville
0         1  Ville H
1         2  Ville F
2         3  Ville G
clients : 125 | commandes : 400
```

Voici d'abord un exemple minuscule pour comprendre les types de jointure. Deux commandes (clients 1 et 9) et deux fiches clients (1 et 2) :

```
cmd = pd.DataFrame({"id_client": [1, 9], "montant": [50, 80]})
fiche = pd.DataFrame({"id_client": [1, 2], "ville": ["Ville H", "Ville F"]})
print(cmd.merge(fiche, on="id_client", how="left"))
```

```
   id_client  montant  ville
0         1      50  Ville H
1         9      80     NaN
```

(Avec `how="inner"`, on n'obtiendrait que la ligne du client 1 ; avec `how="outer"`, trois lignes : les clients 1, 2 et 9.)

- **inner** : seulement les clés présentes **des deux côtés** (client 1) ;
- **left** : toutes les lignes de la table de gauche ; on met NaN (valeur manquante) quand il n'y a pas de correspondance (client 9 sans fiche) ;
- **outer** : tout ce qui existe d'un côté **ou** de l'autre.

Quel how choisir ? Dans 90 % des cas : `left`, avec votre table « principale » (les commandes) à gauche. On garde alors *toutes* les commandes et on y ajoute des informations, sans en perdre en route. Et on **vérifie** toujours le nombre de lignes avant/après.

Sur nos données :

```
cmd_ville = df.merge(clients, on="id_client", how="left")
print("lignes avant/après la jointure :", len(df), len(cmd_ville))      # aucune ligne perdue ni
↪ dupliquée
print(cmd_ville.groupby("ville")["montant"].agg(["count", "mean"]).round(1))
```

```
lignes avant/après la jointure : 400 400
      count  mean
ville
Ville B    19  65.5
Ville E    42  56.0
Ville F    71  63.6
Ville G    92  59.7
Ville H   176  59.6
```

Quels sont les clients qui n'ont **jamais** commandé ? `indicator=True` ajoute une colonne `_merge`

qui dit d'où vient chaque ligne (both : présent des deux côtés ; left_only : seulement dans la table de gauche) :

```
test = clients.merge(df[["id_client"]].drop_duplicates(), on="id_client", how="left", indicator=True)
print(test["_merge"].value_counts())
```

```
_merge
both      114
left_only   11
right_only    0
Name: count, dtype: int64
```

Onze clients n'ont jamais commandé : les cinq prévus (121 à 125) et six clients de la plage 1 à 120 que le hasard n'a pas tirés (1, 6, 43, 50, 67 et 84). Le comptage both (114) + left_only (11) retombe bien sur nos 125 clients.

Le piège n°1 des jointures : l'explosion du nombre de lignes. Si la clé est **dupliquée** dans la table de droite, chaque ligne de gauche est recopiée autant de fois. Imaginez que la fiche du client 1 ait été saisie deux fois : sa commande apparaîtrait en double, et le chiffre d'affaires serait faux sans qu'aucune erreur ne soit signalée. Le paramètre **validate** déclenche une erreur dans ce cas : "m:1" signifie « plusieurs lignes à gauche pour une seule à droite ».

```
fiche_doublon = pd.DataFrame({"id_client": [1, 1], "ville": ["Ville H", "Ville H"]})
print("sans validate :", len(cmd.merge(fiche_doublon, on="id_client", how="left")), "lignes au lieu de 2")
try:
    cmd.merge(fiche_doublon, on="id_client", how="left", validate="m:1")
except Exception as e:
    print(type(e).__name__, ":", str(e).split("\n")[0])
```

```
sans validate : 3 lignes au lieu de 2
MergeError : Merge keys are not unique in right dataset; not a many-to-one merge
```

Pour **empiler** des tables de même structure (les commandes de janvier et celles de février, par exemple), on utilise `pd.concat([janvier, fevrier])` : 75 + 69 lignes donnent bien 144.

4.4.11 Valeurs manquantes

Dans les données réelles, il manque toujours quelque chose : un client n'a pas laissé de note, un capteur est tombé en panne, une case n'a pas été remplie. pandas représente ces trous par **NaN** (*not a number*) pour les nombres, et par **NaT** pour les dates. Créons (code non reproduit) une copie de nos données où 30 notes de satisfaction, tirées au hasard, sont perdues : 7 chez Boutique, 10 chez Réseaux, 13 chez Site.

D'abord, **comment les calculs se comportent-ils ?** Sur un exemple à la main : les notes 4, 5, NaN, 3 ont pour moyenne $(4 + 5 + 3)/3 = 4$ (pandas **ignore** le NaN, il ne le compte pas comme 0 ; sinon on trouverait $12/4 = 3$).

```
notes = pd.Series([4, 5, np.nan, 3])
print("moyenne :", notes.mean(), "| somme avec skipna=False :", notes.sum(skipna=False))
```

```
moyenne : 4.0 | somme avec skipna=False : nan
```

Piège. Comparer avec `==` ne détecte pas un NaN (`NaN == NaN` est faux, par convention). Utilisez `isna()` / `notna()`.

Que faire des trous ? Trois stratégies, à choisir selon le contexte : supprimer les lignes incomplètes

(dropna), remplacer par une valeur « raisonnable » comme la médiane (fillna), ou remplacer par la médiane **du groupe** (fillna avec groupby(...).transform("median")). Voici les deux premières :

```
supprimee = dfm.dropna(subset=["satisfaction"]) # 1. supprimer les lignes
↳ incomplètes
remplie = dfm["satisfaction"].fillna(dfm["satisfaction"].median()) # 2. remplacer par la médiane
print(len(supprimee), "lignes conservées ; moyenne après remplissage :", round(remplie.mean(), 3))
```

```
370 lignes conservées ; moyenne après remplissage : 3.985
```

Lisons les résultats. La vraie moyenne (calculée avant de perdre les 30 notes) est **3,965**. En ignorant simplement les trous, on trouve 3,984 ; avec un remplissage par la médiane globale, 3,985 ; avec la médiane de chaque canal, 4,003. Les écarts sont **petits** parce que nous avons effacé les notes **au hasard** (30 tirées au sort). Si les notes manquantes avaient été celles des clients mécontents, toutes les méthodes auraient surestimé la satisfaction, et d'autant plus que les trous seraient nombreux.

Ce qu'il faut comprendre. Il n'y a pas de bonne réponse universelle. Supprimer des lignes est simple mais perd de l'information, et **biaise** les résultats si les données manquantes ne sont pas dues au hasard (par exemple, si les clients mécontents répondent moins souvent). Remplir par la moyenne ou la médiane garde les effectifs mais **écrase la variabilité** (tous les trous prennent la même valeur). Retenez surtout un réflexe : **toujours compter les manquants (isna().sum()) avant d'analyser**, et se demander *pourquoi* ils manquent. Les méthodes d'imputation plus fines viendront dans la suite de la série.

Enfin, pandas propose des types à **valeurs manquantes natives** (Int64 avec un grand « I ») qui permettent de garder des entiers malgré les trous :

```
print(pd.Series([1, 2, None]).dtype, "|", pd.Series([1, 2, None], dtype="Int64").dtype)
```

```
float64 | Int64
```

4.4.12 Travailler avec des dates

Avec une colonne de vrais **dates** (type datetime64), pandas offre un accès **.dt** aux composantes (année, mois, jour de la semaine...), de l'arithmétique et des regroupements par période.

```
df["jour_semaine"] = df["date"].dt.dayofweek # 0 = lundi ... 6 = dimanche
df["mois"] = df["date"].dt.month
print(df[["date", "jour_semaine", "mois"]].head(3))
```

```
      date  jour_semaine  mois
0 2026-01-05           0     1
1 2026-01-05           0     1
2 2026-01-05           0     1
```

Si vos dates sont lues comme du **texte** (cas fréquent à l'import d'un fichier), on les convertit avec **pd.to_datetime**, en précisant le format pour éviter l'ambiguïté jour/mois :

```
dates = pd.to_datetime(pd.Series(["05/01/2026", "12/01/2026", "03/02/2026"]), format="%d/%m/%Y")
print(dates.dt.month.tolist(), "<- 3 février = mois 2, comme attendu")
```

```
[1, 1, 2] <- 3 février = mois 2, comme attendu
```

Pour **regrouper par semaine**, on convertit chaque date en période hebdomadaire (du lundi au dimanche) et on prend son premier jour :

```
df["semaine"] = df["date"].dt.to_period("W").dt.start_time      # le lundi de la semaine de chaque
↳ commande
print(df[["date", "semaine"]].head(3))
```

```
   date    semaine
0 2026-01-05 2026-01-05
1 2026-01-05 2026-01-05
2 2026-01-05 2026-01-05
```

Vérifions sur un exemple. Le 5 janvier 2026 est un lundi : la semaine du 5 au 11 janvier est donc repérée par le **5 janvier**. Une commande du jeudi 8 janvier doit avoir pour semaine le 5 janvier ; une commande du lundi 12 janvier, le 12. Le programme le confirme : le lundi 5, le jeudi 8 et le dimanche 11 tombent tous dans la semaine du 5 ; le lundi 12 ouvre celle du 12.

Dernier outil, la **moyenne mobile** (*rolling*), qui lisse une série en moyennant les k dernières valeurs. À la main : pour les ventes 10, 20, 30, 40, la moyenne mobile sur 2 périodes vaut NaN, 15, 25, 35 (la première valeur n'a pas assez d'historique).

```
print(pd.Series([10, 20, 30, 40]).rolling(2).mean().tolist())
```

```
[nan, 15.0, 25.0, 35.0]
```

4.4.13 Du tableau au rapport

Tout ce qui précède se combine naturellement. En regroupant les commandes par semaine (`groupby("semaine")`), on obtient un **tableau de bord hebdomadaire** : une ligne par semaine, avec le nombre de commandes, le chiffre d'affaires, le panier moyen, la part du canal Réseaux et une **moyenne mobile** sur quatre semaines qui lisse les à-coups. Une petite fonction qui rédige, pour une semaine donnée, le rapport que la gérante lit chaque lundi (chiffre d'affaires, variation par rapport à la semaine précédente, répartition par canal, meilleurs clients, satisfaction) n'est alors qu'un assemblage de filtres, de `groupby` et de formatage. Cette fonction est **réutilisable** : la semaine suivante, on change la date et on obtient le nouveau rapport sans rien refaire. C'est ce qui sépare une analyse « à la souris » d'une analyse reproductible. Vous la construirez pas à pas dans l'application 4.5 du cahier.

À retenir (pandas).

1. `read_csv` → `head()`, `info()`, `describe()` : toujours **regarder** avant de calculer ;
2. sélection : `df["col"]`, `df[["a", "b"]]`, `loc` (étiquettes, fin incluse), `iloc` (positions, fin exclue), masques avec `&`, `|`, `~` **et parenthèses** ;
3. écrire `df.loc[condition, "col"] = valeur`, jamais d'assignation en chaîne (copy-on-write de pandas 3.0) ;
4. `groupby(...).agg(...)` : découper, appliquer, combiner ;
5. `merge(..., how="left", validate="m:1")` : toujours vérifier le nombre de lignes avant/après ;
6. compter les manquants avec `isna().sum()` *avant* de choisir quoi en faire ;
7. des dates en type `datetime64` (`to_datetime`), puis `.dt`, `to_period`, `rolling`.

■ **Pour s'entraîner.** Cahier, chapitre 4 : application 4.5 (le rapport hebdomadaire) ; exercices 4.9 à 4.11 (NumPy et broadcasting, `groupby` et dates, jointure et clients dormants).

4.5 Premières visualisations

Intuition. Un tableau de 400 lignes, personne ne le « lit » vraiment ; un graphique bien choisi, on le comprend en trois secondes. Au 3.1 nous avons vu avec le quartet d'Anscombe que des données très différentes peuvent avoir **les mêmes statistiques** : seul le dessin révèle la différence. Visualiser n'est donc pas de la décoration, c'est un **outil de réflexion** (on dessine pour *comprendre*) et un **outil de communication** (on dessine pour *convaincre* sans tromper). Cette section a trois objectifs : savoir **quel graphique choisir** selon la question (4.5.1), savoir **le tracer** avec les trois bibliothèques les plus utilisées (matplotlib et pandas, seaborn, puis ggplot2 en R : 4.5.2 à 4.5.5), et savoir **éviter les graphiques trompeurs** (4.5.6).

Pour la suite du livre. Nous ne ferons ici que les graphiques fondamentaux. Les graphiques interactifs, les cartes ou les tableaux de bord complets sont abordés dans la série Data Analyst.

Les figures de cette section utilisent les données de 4.4 (mêmes graines, mêmes résultats) et la palette du livre : traits fins, grille discrète, pas de cadre inutile. Seuls les extraits utiles sont reproduits ; le code complet de chaque figure se trouve dans le fichier source de la section, exécuté à chaque construction du livre.

4.5.1 Quel graphique pour quelle question ?

La première erreur du débutant est de commencer par se demander « quel joli graphique ? ». La bonne démarche est inverse : on part de la **question**, puis du **type des variables** (revoyez le tableau du 3.1.1), et le graphique s'impose presque tout seul.

La question	Les variables	Le graphique	Pourquoi
Comment se répartissent les montants ?	1 quantitative	histogramme (ou boîte à moustaches)	montre la forme : symétrie, queue, valeurs atypiques
Quelle part de chaque canal ?	1 qualitative	diagramme en barres (trié)	on compare des longueurs alignées, ce que l'œil fait très bien
Quel canal vend le plus ?	1 qualitative + 1 quantitative	barres (une valeur par groupe) ou boîtes à moustaches (toute la distribution)	comparer des groupes
Comment évolue le chiffre d'affaires ?	1 quantitative + le temps	courbe	la ligne relie les points : on voit la tendance
Les retards de livraison font-ils baisser la satisfaction ?	2 quantitatives	nuage de points	montre la forme de la relation, pas seulement un nombre
Comment se croisent deux variables qualitatives ?	2 qualitatives	carte de chaleur (<i>heatmap</i>) d'un tableau croisé	couleur = fréquence

Trois questions avant de tracer. (1) *Quel message* veux-je faire passer en une phrase ? (2) *Quelles variables* sont en jeu, et de quel type ? (3) *Qui va lire* ce graphique, en combien de temps ? Un graphique pour votre propre exploration peut être brouillon ; un graphique

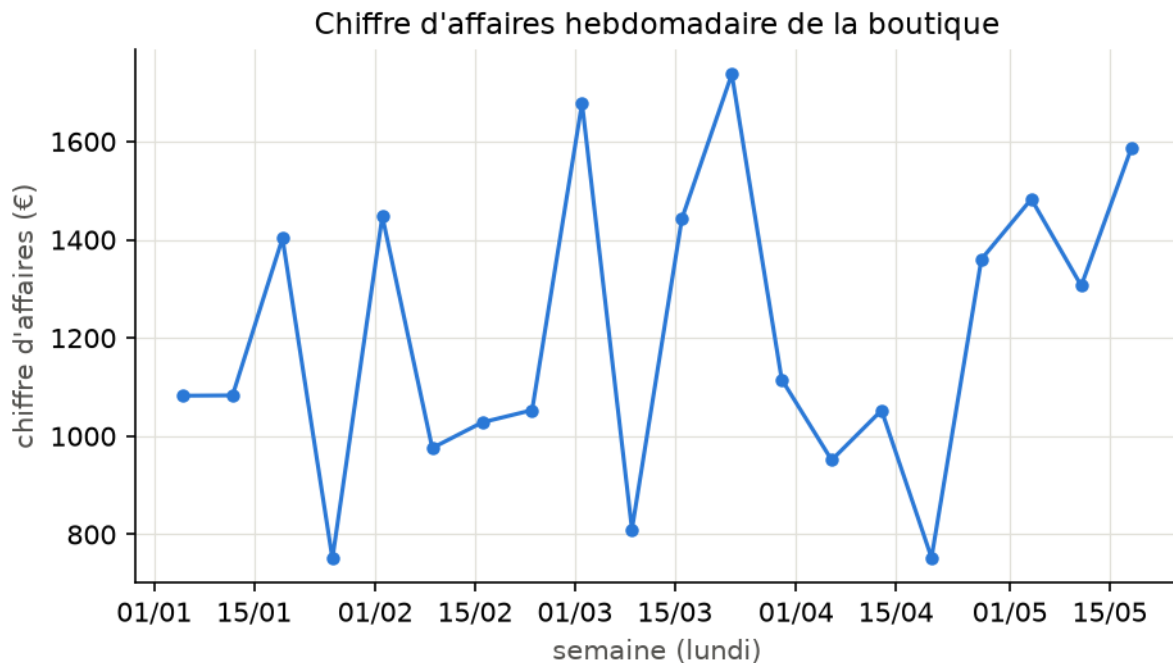


Figure 25: Notre premier graphique : une courbe du chiffre d'affaires par semaine.

Tout y est : un **titre**, des **axes nommés avec leur unité**, une ligne avec des **marqueurs** (un point par semaine, pour ne pas laisser croire que l'on connaît les valeurs entre les semaines). La courbe est irrégulière d'une semaine à l'autre, ce qui est normal : chaque semaine ne compte qu'entre 14 et 27 commandes, donc quelques gros paniers suffisent à faire bondir le total. Pour voir la **tendance**, on lisse avec une moyenne mobile, que nous ajouterons en 4.5.3.

Le patron de tous les graphiques matplotlib. (1) `fig, ax = plt.subplots(...)` ; (2) `ax.plot / ax.bar / ax.hist / ax.scatter(...)` ; (3) `ax.set_title / set_xlabel / set_ylabel` ; (4) `fig.savefig(...)`. Pour *plusieurs* panneaux : `fig, axes = plt.subplots(2, 2)` renvoie un tableau d'Axes que l'on indexe `axes[0, 0]`, `axes[0, 1]`, etc. (c'est un tableau NumPy, voir 4.4.2).

4.5.3 Les quatre graphiques essentiels, avec pandas

pandas offre une méthode `.plot` sur ses Series et DataFrames, qui appelle matplotlib en coulisses et **renvoie l'Axes** : on peut donc la combiner avec tout ce qui précède, en lui passant l'argument `ax=`. Voici les quatre graphiques que vous tracerez le plus souvent, dans une seule figure à quatre panneaux. En une ligne chacun, ils s'écrivent ainsi :

1. **histogramme** des montants ;
2. **barres horizontales** du chiffre d'affaires par canal (triées) ;
3. **courbe** du chiffre d'affaires hebdomadaire avec sa moyenne mobile ;
4. **nuage de points** livraison/satisfaction.

Le code complet de la figure ajoute les titres, les étiquettes et les annotations. Pour le quatrième graphique, un détail d'importance. La livraison est un nombre entier de jours et la satisfaction une note entière de 1 à 5 : beaucoup de commandes tombent *exactement au même point*, et un nuage de points normal en cacherait la plupart. On les **décale aléatoirement d'un tout petit peu** (« jitter », en français *jitter* ou *bruitage*) et on rend les points translucides : les zones denses apparaissent plus foncées.

```
df["montant"].plot.hist(bins=30, color=BLEU)
ca_canal = df.groupby("canal")["montant"].sum().sort_values()
ca_canal.plot.barh(color=BLEU)
hebdos["ca"].rolling(4).mean().plot()
df.plot.scatter(x="livraison", y="satisfaction", alpha=0.3)
```

histogramme
barres horizontales, triées
courbe lissée
nuage de points

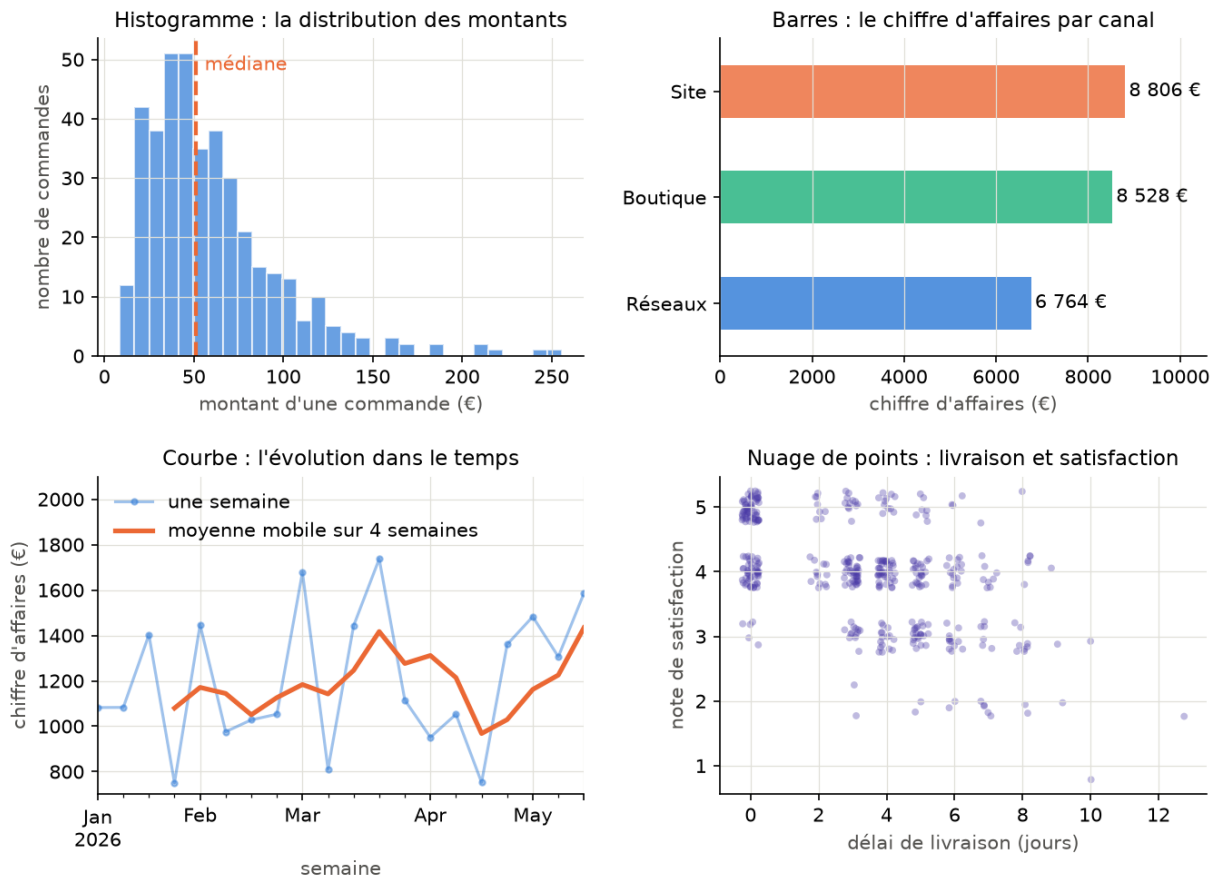


Figure 26: Les quatre graphiques essentiels : histogramme, barres triées, courbe avec moyenne mobile, nuage de points « bruité ».

Comment lire chaque panneau (c'est aussi comme cela qu'on doit *légender* un graphique dans un rapport) :

- **Histogramme** : la forme est asymétrique à droite, avec une longue queue de grosses commandes ; la médiane (trait orange, 51 €) est nettement sous la moyenne (60 €), tirée vers le haut par les grosses commandes (revoir 3.1.5).
- **Barres** : on lit au premier coup d'œil l'ordre des canaux, et les valeurs sont écrites au bout des barres : plus besoin de deviner sur l'axe. Les barres sont **triées** : un classement doit être lisible.
- **Courbe** : le trait pâle est le chiffre d'affaires de chaque semaine, très irrégulier ; le trait orange, plus lisse, montre la **tendance**.
- **Nuage de points** : plus le délai augmente, plus les notes basses apparaissent ; la corrélation affichée ($-0,53$) est négative et d'intensité moyenne : un retard fait *tendanciellement* baisser la note, sans que ce soit une règle absolue (beaucoup de commandes tardives ont quand même 4).

Pourquoi .plot renvoie-t-il l'Axes ? Parce que ainsi tout est modifiable après coup : titre, limites, annotations. Si vous écrivez `ax = df["montant"].plot.hist()`, vous pouvez ensuite faire `ax.set_title(...)`. pandas propose aussi `.plot.bar()`, `.plot.line()`, `.plot.box()`, `.plot.scatter(x=, y=)`, `.plot.pie()`, `.plot.area()`... Pour l'exploration rapide, c'est imbattable (une ligne par graphique).

4.5.4 seaborn : des graphiques statistiques en une ligne

seaborn est une couche au-dessus de matplotlib spécialisée dans les graphiques **statistiques**. Son atout : on lui donne le **tableau complet** (`data=df`) et on dit quelle colonne va où (`x=`, `y=`, `hue=` pour la couleur), et seaborn s'occupe des regroupements, des légendes et des couleurs.

Deux exemples. D'abord la distribution des montants **selon le canal**, en histogramme et en boîte à moustaches (on précise juste `hue=` pour la couleur) :

```
import seaborn as sns

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
sns.histplot(data=df, x="montant", hue="canal", hue_order=ordre, palette=COULEURS, element="step",
             alpha=0.3, ax=axes[0])
sns.boxplot(data=df, x="canal", y="montant", order=ordre, hue="canal", palette=COULEURS, ax=axes[1])
axes[0].set(title="Histogrammes superposés selon le canal", xlabel="montant (€)", ylabel="nombre de commandes")
axes[1].set(title="Boîtes à moustaches selon le canal", xlabel="", ylabel="montant (€)")
fig.savefig("figures/ch04-seaborn-distributions.png", dpi=150, bbox_inches="tight")
print(df.groupby("canal")["montant"].median().reindex(ordre).round(1).to_string())
```

canal	
Réseaux	41.5
Site	49.5
Boutique	64.8

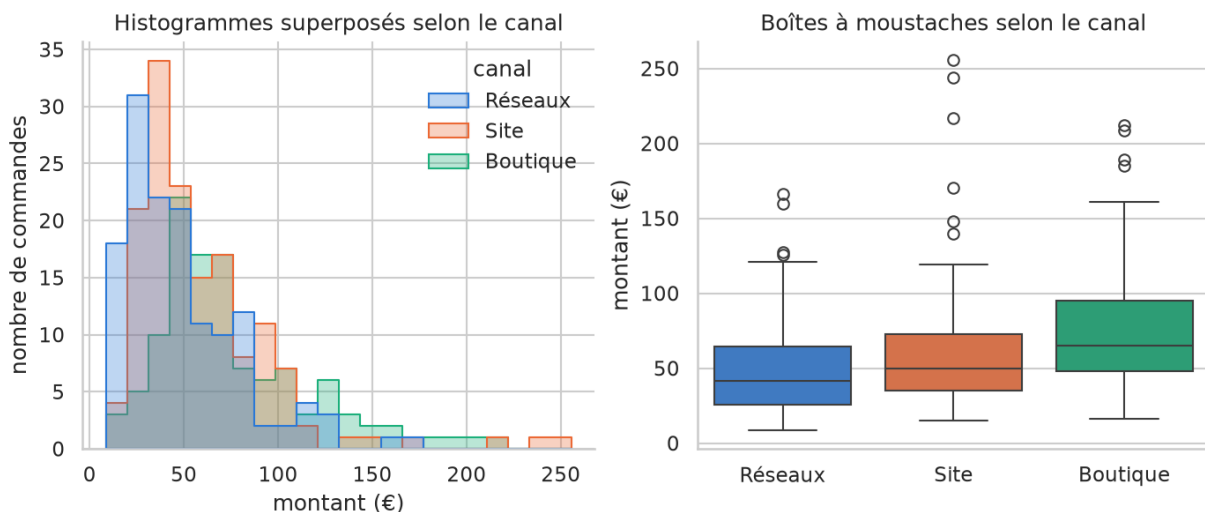


Figure 27: Avec seaborn, une ligne suffit pour séparer les données par canal : histogrammes superposés et boîtes à moustaches.

Les médianes imprimées confirment la lecture : 64,8 € pour la boutique, 49,5 € pour le site, 41,5 € pour Réseaux. (Cette différence est-elle réelle ou due au hasard ? C'est la question d'un test statistique, 3.4.)

Deuxième exemple : une **carte de chaleur** pour le croisement de deux variables qualitatives, le

canal et la note de satisfaction. On calcule d'abord le tableau croisé (en proportions par canal), puis on le colore :

```
tab = pd.crosstab(df["canal"], df["satisfaction"], normalize="index").loc[ordre]
print(tab.round(2))

fig, ax = plt.subplots(figsize=(6.5, 3))
sns.heatmap(tab, annot=True, fmt=".0%", cmap="Blues", cbar=False, linewidths=2, linecolor="white",
            ↪ ax=ax)
fig.savefig("figures/ch04-seaborn-heatmap.png", dpi=150, bbox_inches="tight")
```

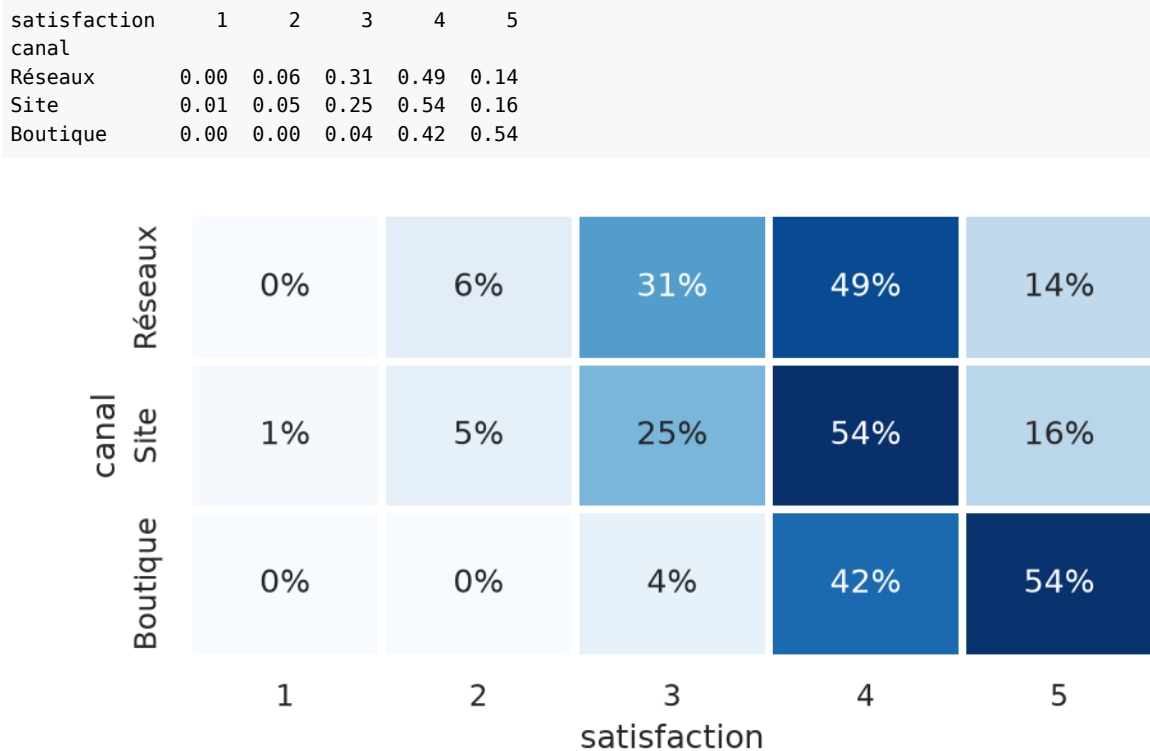


Figure 28: Carte de chaleur : chaque ligne (canal) somme à 100 %. Plus la case est foncée, plus la note est fréquente dans ce canal.

Chaque ligne du tableau somme à 1 (donc 100 %) : on lit « *parmi* les commandes de la boutique, 54 % ont donné la note 5 » (contre 14 % pour Réseaux et 16 % pour le Site). La case la plus foncée de la ligne Boutique est à droite (la note 5), celles de Réseaux et du Site sont sur la note 4, avec une part importante de 3 : la boutique, où la livraison est immédiate, a les clients les plus satisfaits.

matplotlib, pandas ou seaborn ? Ce n'est pas un choix exclusif : seaborn et pandas **dessinent dans des Axes matplotlib**, que l'on peut retoucher avec les méthodes de 4.5.2. Règle pratique : pandas `.plot` pour regarder vite, seaborn pour les graphiques statistiques avec groupes, matplotlib pour tout ce qui doit être personnalisé au pixel près.

4.5.5 ggplot2 : la grammaire des graphiques en R

En R, la bibliothèque de référence est **ggplot2**. Son idée, la « **grammaire des graphiques** » (*grammar of graphics*), est de **décrire** un graphique par couches plutôt que de le dessiner pas à pas :

- les **données** (`ggplot(data, ...)`) ;
- une **correspondance esthétique** `aes(x=, y=, fill=)` : quelle colonne va sur quel axe ou dans quelle couleur ;
- une ou plusieurs **géométries** `geom_...()` : histogramme, barres, points, lignes ;
- éventuellement des **facettes** (`facet_wrap`) pour faire un panneau par groupe, des **échelles** et un **thème**.

On additionne ces couches avec le signe +. Reproduisons l'histogramme par canal ; R lit le même fichier CSV (les bases du langage R sont présentées en 4.2) :

```
library(ggplot2)
commandes <- read.csv("donnees/commandes.csv")
commandes$canal <- factor(commandes$canal, levels = c("Réseaux", "Site", "Boutique"))

p <- ggplot(commandes, aes(x = montant, fill = canal)) +
  geom_histogram(bins = 30, colour = "white") + # une géométrie : l'histogramme
  facet_wrap(~ canal, ncol = 1) + # un panneau par canal
  scale_fill_manual(values = c(Réseaux = "#2a78d6", Site = "#eb6834", Boutique = "#1baf7a")) +
  labs(title = "Distribution des montants selon le canal", x = "montant (€)", y = "nombre de
  ↪ commandes") +
  theme_minimal(base_size = 11) + theme(legend.position = "none")
ggsave("figures/ch04-ggplot-montants.png", plot = p, width = 7, height = 5, dpi = 150)
```

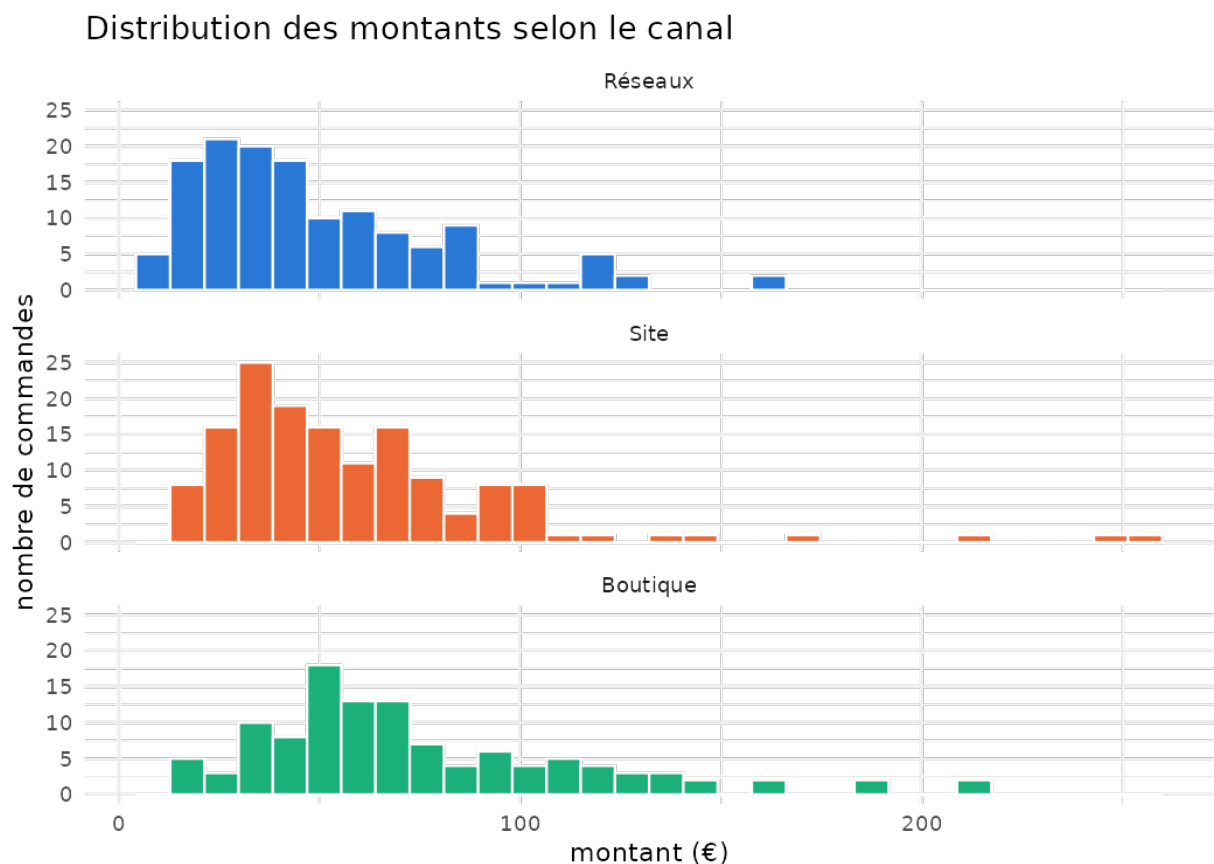


Figure 29: Le même type de graphique avec ggplot2 : un panneau par canal (facettes), même échelle horizontale.

On retrouve les mêmes médianes qu'en Python (41,5 € pour Réseaux, 49,5 € pour le Site, 64,8 € pour la Boutique) (confirmant que les deux outils lisent bien les mêmes données). Le tableau suivant résume la différence de philosophie :

	matplotlib / seaborn (Python)	ggplot2 (R)
Style	impératif : on construit pas à pas (figure → axes → éléments)	déclaratif : on décrit le résultat par couches
Séparer par groupe	hue= (seaborn) ou une boucle sur les groupes	fill=, colour=, ou facet_wrap()
Personnalisation fine	très grande, mais verbeuse	grande, via theme() et scale_*()
Combiner plusieurs couches	appels successifs sur le même ax	+ geom_...()

Honnêteté d'exécution. Les graphiques de cette section ont tous été produits par du code exécuté à chaque construction du livre (sauf trois dessins explicatifs, produits par build/fig_ch04.py : l'anatomie d'une figure, le broadcasting et l'axe tronqué) ; le bloc R a bien été exécuté (R avec ggplot2). Les versions de bibliothèques peuvent légèrement modifier l'aspect (polices, marges) sans changer l'information.

4.5.6 Bien faire, mal faire : les pièges du graphique trompeur

Un graphique peut mentir sans qu'une seule donnée soit fausse. Voici les six pièges les plus répandus. Le premier mérite une démonstration.

Piège n°1 : l'axe tronqué. Comparons la satisfaction moyenne du canal Réseaux et du Site. Les deux graphiques ci-dessous montrent **exactement les mêmes deux nombres** : 3,72 pour Réseaux et 3,79 pour le Site, soit un écart réel de 2 %.

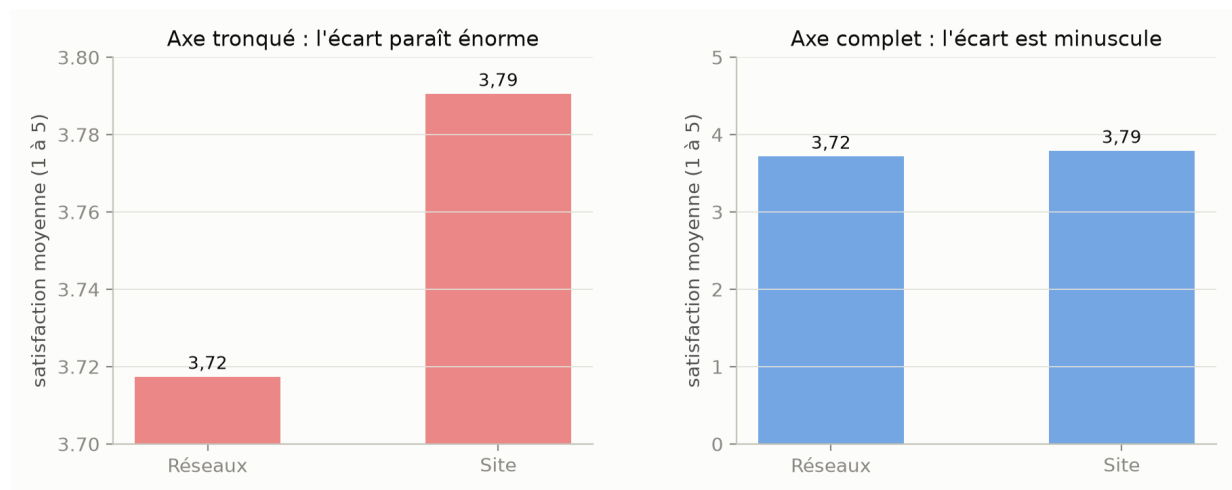


Figure 30: Mêmes données, deux impressions opposées : à gauche l'axe commence à 3,70, à droite à 0.

À gauche, avec un axe qui commence à 3,70, la barre du Site paraît **plus de 5 fois plus haute** que celle de Réseaux (exactement 5,2 fois : $(3,79 - 3,70) / (3,72 - 3,70)$) ; à droite, sur un axe complet, les deux barres sont quasiment identiques, ce qui correspond bien à l'écart réel de 2 %. **Règle : pour un diagramme en barres, l'axe doit commencer à 0**, car c'est la *longueur* de la barre qui porte l'information. (Pour une courbe ou un nuage de points, c'est la *position* qui compte : on peut zoomer, à condition de le signaler.)

Les autres pièges, et leurs remèdes :

Piège	Pourquoi c'est un problème	Remède
Camembert en 3D, ou à beaucoup de parts	la perspective déforme les aires ; l'œil compare mal les angles	barres triées, en 2D
Double axe vertical	on peut rendre n'importe quelle corrélation « visible » en choisissant les échelles	deux graphiques superposés, ou un seul axe
Trop de couleurs	10 couleurs sans ordre : personne ne retient la légende	3 à 5 couleurs, avec un sens (une couleur = un canal, partout dans le document)
Points superposés	400 observations qui se cachent les unes les autres (voir le jitter, 4.5.3)	transparence, bruitage, ou histogramme 2D
Titre vague (« Graphique 3 »)	le lecteur doit deviner la conclusion	un titre qui dit le message : « La boutique a les clients les plus satisfaits »
Axes sans nom ni unité	« 60 », mais de quoi ?	toujours nommer et donner l'unité (€, jours, %)

La liste de contrôle d'un bon graphique. (1) Un message, un titre qui le dit. (2) Le bon type de graphique pour la question (tableau 4.5.1). (3) Des axes nommés avec leurs unités ; **zéro pour les barres**. (4) Des barres **triées** quand elles représentent un classement. (5) Peu de couleurs, avec un sens constant. (6) Lisible en noir et blanc et pour un daltonien : ne pas reposer sur l'opposition rouge/vert seule. (7) La source des données et la date, si l'on communique à d'autres.

4.5.7 Enregistrer, réutiliser : de la figure au rapport

Un graphique n'est utile que s'il sort de votre ordinateur. `fig.savefig(chemin)` choisit le format d'après l'extension, avec trois réglages à connaître :

Format	Quand l'utiliser
PNG (.png)	pages web, diapositives, e-mails : image « pixels », léger
SVG / PDF (.svg, .pdf)	impression, articles, LaTeX : image vectorielle , nette à toute taille
<code>dpi=</code>	résolution en pixels par pouce : 150 pour l'écran, 300 pour l'impression
<code>bbox_inches="tight"</code>	rogne les marges blanches inutiles

En pratique, `fig.savefig("exemple.pdf", dpi=150, bbox_inches="tight")` suffit : l'extension choisit le format (nous avons vérifié que PNG, SVG et PDF s'enregistrent bien).

Mettons tout en commun dans une **fonction** qui produit, en une seule figure, le tableau de bord que la gérante joint à son rapport du lundi : la courbe lissée du chiffre d'affaires, la part de chaque canal, la répartition des notes. Chaque panneau a un **titre qui énonce sa conclusion, calculée à partir des données** (et non écrite à la main) : si les chiffres changent, le titre reste vrai. Tel que la gérante le lit, il dit que la tendance du chiffre d'affaires est à la hausse (de 1 080 à 1 434 € par semaine en moyenne mobile), que le Site et la Boutique pèsent chacun plus d'un tiers des ventes, et

que trois clients sur quatre sont satisfaits (4 ou 5). Voilà le chemin complet : des **données brutes** (fichier CSV) aux **tableaux** (pandas, 4.4) puis aux **figures** prêtes à insérer dans un rapport, le tout dans un script que l'on peut relancer chaque semaine. C'est l'esprit de la **recherche reproductible** que nous retrouverons au chapitre 6. La fonction complète est construite pas à pas dans l'application 4.6 du cahier.

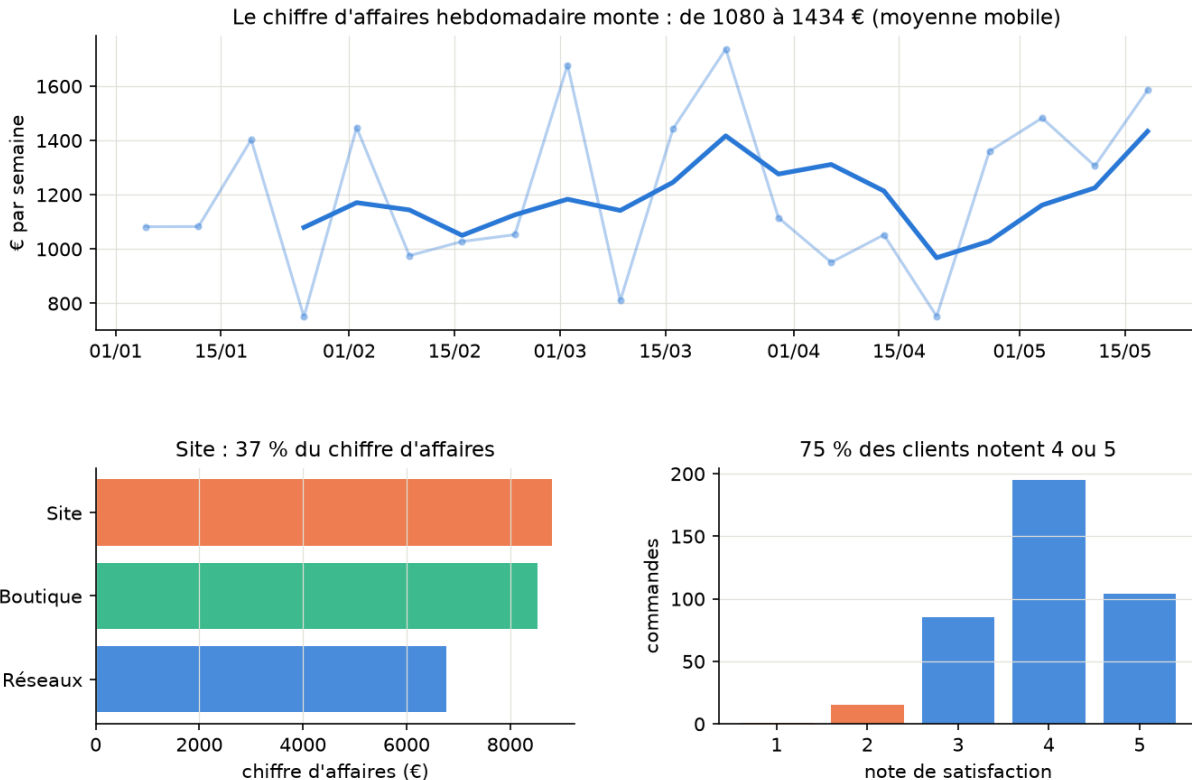


Figure 31: Le tableau de bord produit par la fonction : trois panneaux, chacun avec un titre qui énonce sa conclusion.

Pour s'entraîner. Cahier, chapitre 4 : application 4.6 (le tableau de bord) ; exercice 4.12 (un graphique honnête).

4.6 ⊕ Pour aller plus loin : programmation orientée objet, code propre et tests

Section optionnelle. Vous pouvez faire toute une carrière d'analyste avec des fonctions, des listes et des tableaux pandas. Mais dès que votre code dépasse quelques dizaines de lignes, ou qu'un collègue (ou vous-même, dans six mois) doit le relire, trois outils changent la vie : **regrouper** les données et les opérations qui vont ensemble (les *objets*), **écrire clairement** (le *code propre*), et **vérifier automatiquement** que le code fait ce qu'on croit (les *tests*). Cette section les présente sur un exemple concret : le panier d'une cliente de la boutique.

Pour les tests, nous écrivons de vrais **fichiers** Python que nous exécutons depuis un terminal, exactement comme vous le feriez sur votre machine : la commande `cat > fichier <<'FIN' ... FIN` crée un fichier avec le texte qui suit, et `python -m pytest` lance les tests. (Le chapitre 6.3

détaille le terminal.) Le module complet de la boutique et sa suite de tests sont donnés dans le cahier ; ici, nous n'en montrons que les passages utiles.

4.6.1 Pourquoi des objets ? Le problème des dictionnaires

Intuition. Jusqu'ici, un panier pouvait être une simple liste de prix. Mais un panier, ce n'est pas qu'une liste : c'est aussi « *savoir calculer son total, ajouter un article, refuser une quantité négative* ». Un **objet** est un petit paquet qui contient à la fois des **données** (les *attributs*) et les **opérations** qui vont avec (les *méthodes*). Sa **classe** est le moule qui fabrique ces objets, comme un patron de couture fabrique des robes.

Voyons pourquoi cela sert à quelque chose. Voici un panier représenté « à la main » par un dictionnaire, et une fonction qui calcule le total :

```
panier = {"savon": (20.0, 2), "plateau": (30.0, 1)}      # nom -> (prix HT, quantité)
total_ht = lambda p: sum(prix * qte for prix, qte in p.values())

print("total HT :", total_ht(panier))
panier["plateau"] = (30.0, -3)                        # une erreur de saisie...
print("total HT :", total_ht(panier), " <- personne ne nous a prévenus !")
```

```
total HT : 70.0
total HT : -50.0   <- personne ne nous a prévenus !
```

Rien ne protège le dictionnaire : une quantité négative passe sans bruit, et le total devient faux **sans aucune erreur**. C'est le pire cas possible pour un analyste (un résultat faux qui a l'air normal). Avec une classe, on décide à **un seul endroit** de ce qui est permis.

4.6.2 Votre première classe

Commençons par la version « longue », avec une classe ordinaire, pour comprendre les mécanismes :

```
class Article:
    def __init__(self, nom, prix_ht):                # appelée à la création de l'objet
        self.nom = nom                               # self = l'objet en cours de création
        self.prix_ht = prix_ht

    def prix_ttc(self):                               # une méthode : une fonction de l'objet
        return round(self.prix_ht * 1.19, 2)

savon = Article("savon", 20.0)
print(savon.nom, savon.prix_ht, savon.prix_ttc())
```

```
savon 20.0 23.8
```

Lisez ligne à ligne :

- `class Article:` définit le moule.
- `__init__` est le **constructeur** : Python l'appelle quand on écrit `Article("savon", 20.0)`. Le premier paramètre, `self`, désigne l'objet qu'on est en train de fabriquer ; on y accroche les attributs (`self.nom`, `self.prix_ht`).
- `prix_ttc` est une **méthode** : on l'appelle avec un point, `savon.prix_ttc()`, et `self` est passé automatiquement.
- Un défaut de cette version : si l'on écrit `print(savon)`, l'affichage `<__main__.Article object at 0x...>` ne dit rien d'utile (et l'adresse mémoire change à chaque exécution).

Écrire `__init__` et un affichage lisible pour chaque classe devient vite répétitif. C'est le rôle des **dataclasses** (`@dataclass`) : Python écrit pour vous le constructeur, un affichage lisible et la comparaison `==`.

```
from dataclasses import dataclass

@dataclass
class Article:
    nom: str
    prix_ht: float

a, b = Article("savon", 20.0), Article("savon", 20.0)
print(a)
print("a == b ?", a == b)
```

```
Article(nom='savon', prix_ht=20.0)
a == b ? True
```

Deux articles ayant les mêmes attributs sont égaux : c'est ce qu'on attend d'une « valeur ». Deux lignes de déclaration ont remplacé une quinzaine de lignes de code.

4.6.3 Le cahier des charges du module

Nous construirons le module de la boutique en 4.6.5, en deux temps : d'abord une fonction de remise, volontairement écrite **trop vite** (c'est l'occasion de découvrir les tests), puis les classes. Mais avant d'écrire du code, posons les règles.

Voici les règles métier, que nous vérifierons à **la main** avant de coder :

- la TVA est de 19 % ;
- un panier de 2 savons à 20 € et 1 plateau à 30 € vaut $2 \times 20 + 30 = 70$ € hors taxe, soit $70 \times 1,19 = 83,30$ € TTC ;
- avec une remise de 10 % sur le hors-taxé : $70 \times 0,90 = 63$ € HT, soit $63 \times 1,19 = 74,97$ € TTC ;
- sur le site, la livraison coûte 7 €, **offerte** si le panier TTC atteint 100 € ; en boutique, le retrait est gratuit.

4.6.4 Code propre : lisible avant tout

Intuition. Le code est lu bien plus souvent qu'il n'est écrit. L'objectif n'est pas de « faire marcher » un programme, mais de le rendre **compréhensible** par quelqu'un qui n'était pas là quand vous l'avez écrit (y compris vous, dans six mois). Cinq habitudes suffisent pour 90 % du résultat :

1. **Des noms qui parlent.** `total_ttc` plutôt que `t`, `remise` plutôt que `r`. Un nom long et clair vaut mieux qu'un commentaire.
2. **Des fonctions courtes qui font une seule chose.** Si vous devez écrire « et » pour décrire ce que fait une fonction, coupez-la en deux.
3. **Pas de nombres magiques.** Écrivez `TVA = 0.19` une fois en haut du fichier, pas `1.19` à quinze endroits (le jour où le taux change, vous n'en oublierez aucun).
4. **Une docstring** : une phrase entre triples guillemets sous la ligne `def` ou `class`, qui dit *ce que* fait la fonction. Elle s'affiche avec `help(...)`.
5. **Des annotations de type** (`nom: str, -> float`) : elles documentent ce que la fonction attend et renvoie.

Les annotations de type ne sont pas vérifiées à l'exécution. Python les lit, mais ne les impose pas. Ce sont des indications pour les humains et pour les outils de vérification (comme mypy). Regardez :

```
def double(x: int) -> int:
    return x * 2

print(double(21), double("ab"))      # une chaîne n'est pas un entier... et pourtant, aucune erreur

42 abab
```

Piège classique : l'argument par défaut modifiable. Une valeur par défaut comme [] est créée **une seule fois**, à la définition de la fonction, puis partagée entre tous les appels. C'est une des erreurs les plus fréquentes en Python :

```
def ajouter_mauvais(article, panier=[]):      # MAUVAIS : la liste est partagée entre les appels
    panier.append(article)
    return panier

print(ajouter_mauvais("savon"), ajouter_mauvais("plateau"))

['savon', 'plateau'] ['savon', 'plateau']
```

Avec la version fautive, le deuxième panier *contient aussi le savon* du premier : deux clientes se retrouvent avec le même panier. La version correcte écrit `panier=None` dans la signature, puis `if panier is None: panier = []` dans le corps de la fonction (on obtient alors ['savon'] puis ['plateau']). Vous retrouverez ce motif dans les dataclasses sous la forme `field(default_factory=list)`.

4.6.5 Tester son code : le filet de sécurité

Intuition. Un **test unitaire** est un petit programme qui appelle une fonction avec des entrées dont **vous connaissez la bonne réponse**, et vérifie qu'elle renvoie bien cette réponse. Vous les écrivez une fois ; ils se rejouent en une seconde après chaque modification. Si un test devient rouge, vous savez *quoi* vous venez de casser, *tout de suite*, et pas un mois plus tard en lisant un rapport faux.

Le schéma universel d'un test s'appelle **Arrange – Act – Assert** : on **prépare** les données (*arrange*), on **exécute** la fonction (*act*), on **vérifie** le résultat (*assert*). Nous utilisons **pytest**, l'outil standard : il suffit d'écrire des fonctions dont le nom commence par `test_` et d'y mettre des `assert`.

Étape 1 : une fonction de remise, écrite trop vite. Un test attend 60 € pour 80 € avec 25 % de remise :

```
cat > remises.py <<'FIN'
def prix_apres_remise(prix, taux):
    return prix - taux
FIN
cat > test_remises.py <<'FIN'
from remises import prix_apres_remise

def test_remise_de_25_pour_cent():
    assert prix_apres_remise(80.0, 0.25) == 60.0      # 80 * (1 - 0,25) = 60
FIN
python -m pytest -q --color=no --tb=short -p no:cacheprovider test_remises.py 2>&1 | grep -E "^(
  |passed|failed" | sed -E 's/ in [0-9.]+s//'
```

```
E  assert 79.75 == 60.0
E    + where 79.75 = prix_apres_remise(80.0, 0.25)
1 failed
```

Le test a fait son travail : la fonction soustrait le *taux* (0,25 € !) au lieu d'appliquer le pourcentage. Le message affiche la ligne en cause, la valeur obtenue (79,75) et la valeur attendue (60). Notez qu'un second test, `prix_apres_remise(80.0, 0.0) == 80.0`, passerait : un code faux peut réussir un cas particulier, ce qui montre pourquoi **un seul test ne suffit pas**.

Étape 2 : on corrige, on relance.

```
cat > remises.py <<'FIN'
def prix_apres_remise(prix, taux):
    if not 0 <= taux <= 1:
        raise ValueError(f"le taux doit être entre 0 et 1, reçu {taux}")
    return prix * (1 - taux)
FIN
python -m pytest -q --color=no --tb=short -p no:cacheprovider test_remises.py 2>&1 | grep -E
↪ "passed|failed" | sed -E 's/ in [0-9.]+s//'
```

1 passed

Test vert. Remarquez que nous avons aussi ajouté une **validation** : un taux de 25 (au lieu de 0,25) lèverait une erreur claire plutôt que de produire un prix négatif.

Un réflexe d'expert : écrire le test *avant* le correctif. Quand vous découvrez un bogue, écrivez d'abord un test qui l'attrape (il est rouge), puis corrigez le code (il devient vert). Ce bogue ne reviendra jamais sans que quelqu'un le remarque. Cette discipline s'appelle le **développement piloté par les tests (TDD)**.

Étape 3 : le module de la boutique. Il contient quatre classes (un article, un panier, une commande en boutique, une commande sur le site), avec docstrings, annotations de types, validation et une constante pour la TVA (une soixantaine de lignes, données dans le cahier). Voici les passages qui illustrent la section :

```
@dataclass(frozen=True)                # frozen : on ne peut plus modifier un article créé
class Article:
    nom: str
    prix_ht: float

    def __post_init__(self):
        if self.prix_ht < 0:
            raise ValueError(f"prix négatif pour {self.nom!r}")

class Commande:                          # une commande en boutique : retrait gratuit
    def __init__(self, panier):
        self.panier = panier
    def frais_livraison(self):
        return 0.0
    def total_a_payer(self):
        return round(self.panier.total_ttc() + self.frais_livraison(), 2)

class CommandeSite(Commande):            # héritage : on ne redéfinit que ce qui change
    def frais_livraison(self):
        return 0.0 if self.panier.total_ttc() >= 100.0 else 7.0
```

Quelques points de lecture :

- `frozen=True` rend l'article **immuable** : impossible d'écrire `article.prix_ht = -5` après coup. Moins de bogues possibles.

- `__post_init__` est appelé juste après le constructeur généré par la dataclass : c'est l'endroit idéal pour **valider** les données.
- `__len__` est une **méthode spéciale** (on les reconnaît à leurs doubles tirets bas) : elle fait fonctionner `len(panier)`.
- `CommandeSite(Commande)` est un exemple d'**héritage** : la classe fille reprend tout de la classe mère et ne **redéfinit** que ce qui change, ici `frais_livraison`. La méthode `total_a_payer`, écrite une seule fois dans `Commande`, appelle `self.frais_livraison()` et obtient automatiquement le bon comportement selon le type d'objet : c'est le **polymorphisme**.

Quand utiliser l'héritage ? Avec parcimonie. Deux classes dont l'une « *est une sorte de* » l'autre (une commande du site *est une* commande) : oui. Pour simplement réutiliser du code, préférez la **composition** (un objet qui *contient* un autre, comme `Commande` contient un `Panier`). Beaucoup de projets de data science n'ont besoin que de fonctions et de dataclasses.

Étape 4 : les tests du module. Chaque règle métier vérifiée à la main plus haut devient un test (il y en a dix dans le module complet). Le décorateur `@pytest.fixture` prépare le panier de l'exemple (le *arrange*) ; `pytest.approx` compare des nombres décimaux avec une petite tolérance (rappelez-vous, section 1.5 : $0.1 + 0.2 \neq 0.3$ en binaire !) ; `@pytest.mark.parametrize` rejoue le même test avec plusieurs valeurs. Voici deux tests, puis la suite complète :

```
@pytest.fixture
def panier():
    p = Panier()
    p.ajouter(Article("savon", 20.0), 2)
    p.ajouter(Article("plateau", 30.0))
    return p

def test_total_ttc(panier):
    assert panier.total_ttc() == pytest.approx(83.30)

@pytest.mark.parametrize("quantite", [0, -2])
def test_quantite_invalide(quantite):
    with pytest.raises(ValueError):
        Panier().ajouter(Article("savon", 20.0), quantite)
```

```
python -m pytest -q --color=no -p no:cacheprovider 2>&1 | tail -1 | sed -E 's/ in [0-9.]+s//'
```

```
11 passed
```

Les onze tests (dix pour le module, un pour la remise) passent : chaque test vert est une promesse tenue. Ils vérifient les calculs faits à la main : 83,30, 74,97, $83,30 + 7 = 90,30$, et le panier de 90 € HT qui vaut $90 \times 1,19 = 107,10$ € TTC, donc livraison offerte.

Que se passe-t-il si on casse le code ? Modifions le seuil de livraison offerte à 1 000 € dans le module (une faute de frappe plausible : un zéro en trop), et relançons les tests : un seul passe au rouge (« 1 failed, 10 passed »), celui qui protège précisément cette règle ; en remettant la bonne valeur, on retrouve « 11 passed ».

Voilà la valeur d'une suite de tests : une modification « innocente » est détectée **immédiatement**, avec le nom du test et la règle violée.

4.6.6 Tester une fonction d'analyse

Les tests ne servent pas qu'aux classes : une fonction d'analyse de données mérite les mêmes soins, surtout si elle sera réutilisée dans un rapport. Prenons le panier moyen par canal (le même calcul que celui du chapitre 3, `commandes.groupby("canal")["montant"].mean()`). On le teste sur un **petit tableau dont on connaît la réponse à la main** : deux commandes du Site à 10 et 30 € (moyenne $(10 + 30)/2 = 20$) et une commande de la Boutique à 50 € (moyenne 50). Si la fonction renvoie autre chose, un test devient rouge. Elle est alors **nommée, documentée et protégée**. Dans un vrai projet, on range le code dans un dossier `src/` et les tests dans un dossier `tests/` ; la commande `pytest` trouve alors tout seule les fichiers `test_*.py` (chapitre 6.1 pour les ranger sous Git). L'application 4.7 du cahier fait cet exercice complet.

Pour s'entraîner. Cahier, chapitre 4 : application 4.7 (module de la boutique et ses tests) ; exercice 4.13 (un test unitaire qui vise la borne).

À retenir (objets, code propre, tests).

- Une **classe** regroupe des **données** (attributs) et des **opérations** (méthodes) ; `self` désigne l'objet courant. Une **dataclass** génère le constructeur, l'affichage et l'égalité à votre place.
- On **valide** les données à l'entrée (`__post_init__`, `raise ValueError`) : mieux vaut une erreur claire qu'un résultat faux silencieux.
- **Héritage** : la classe fille redéfinit seulement ce qui change ; **composition** : un objet en contient un autre. Dans le doute, préférez la composition.
- **Code propre** : noms parlants, fonctions courtes, constantes nommées (TVA), docstrings, annotations de type (non vérifiées à l'exécution), pas de `[]` comme valeur par défaut.
- **Test unitaire** = Arrange, Act, Assert. Avec **pytest** : fonctions `test_*`, `assert`, `pytest.approx` pour les décimaux, `pytest.raises` pour les erreurs attendues, `@pytest.mark.parametrize` pour plusieurs cas.
- Un bogue trouvé ? Écrivez d'abord le test qui l'attrape, puis corrigez.

4.7 ⊕ Pour aller plus loin : d'autres langages (SAS, MATLAB, Julia)

Section optionnelle. Python et R vous couvriront dans la grande majorité des cas. Mais vous croiserez peut-être, dans une offre d'emploi, un stage ou le code d'un collègue, **SAS**, **MATLAB** ou **Julia**. Cette section ne vous apprend pas ces langages : elle vous donne le **vocabulaire** pour les lire sans panique, en refaisant *la même petite analyse* dans chacun.

Honnêteté sur ce qui a été exécuté. Python et R sont installés sur la machine qui a produit ce livre : leurs sorties ci-dessous sont **réelles**. SAS, MATLAB et Julia ne le sont pas : leurs blocs de code sont marqués « **non exécuté** » et ne sont accompagnés d'aucune sortie. Ils sont écrits à partir de leur documentation, avec les conventions habituelles de chaque langage ; **vérifiez-les chez vous** avant de les utiliser dans un travail important. Le résultat attendu est celui de Python et R (section 3.1 : il est le même partout, car le calcul est le même).

4.7.1 La tâche : un résumé par canal

La gérante demande : « Pour chaque canal de vente, combien de commandes, quel montant moyen, et quel écart-type ? » C'est le calcul du 3.1, appliqué au fichier `donnees/commandes.csv` : lire, regrouper, résumer. Cinq lignes dans chaque langage.

En Python (pandas, section 4.4) :

```
import pandas as pd

commandes = pd.read_csv("donnees/commandes.csv")
resume = (commandes.groupby("canal")["montant"]
          .agg(n="count", moyenne="mean", ecart_type="std")
          .round(1))
print(resume)
```

	n	moyenne	ecart_type
canal			
Boutique	114	74.8	40.6
Réseaux	138	49.0	31.1
Site	148	59.5	38.3

En R (le même calcul avec le paquet `dplyr`, section 4.2) :

```
suppressPackageStartupMessages(library(dplyr))

commandes <- read.csv("donnees/commandes.csv")
commandes |>
  group_by(canal) |>
  summarise(n = n(), moyenne = round(mean(montant), 1), ecart_type = round(sd(montant), 1))
```

```
# A tibble: 3 × 4
  canal      n moyenne ecart_type
  <chr> <int> <dbl> <dbl>
1 Boutique  114  74.8  40.6
2 Réseaux  138  49    31.1
3 Site     148  59.5  38.3
```

Les deux langages donnent exactement les mêmes nombres : 114 commandes en boutique pour un montant moyen d'environ 75 €, comme au 3.1. (Le tri alphabétique des canaux est le même ; seule la présentation du tableau diffère.)

4.7.2 SAS : le langage des grandes organisations

Intuition. SAS (*Statistical Analysis System*) est un logiciel commercial né dans les années 1970. Il est resté très présent dans les **banques, les assurances, l'industrie pharmaceutique et les administrations**, où l'on valorise la stabilité, la traçabilité et le fait que les résultats d'un programme écrit il y a vingt ans soient toujours identiques. C'est souvent le cas dans les métiers de l'actuariat et du risque (le thème du volume V de cette série). Pour vous former gratuitement, SAS propose une version en ligne destinée à l'enseignement (*SAS OnDemand for Academics*).

Un programme SAS est une suite d'**étapes** : les étapes DATA fabriquent ou transforment des tables, les étapes PROC (procédures) appliquent un traitement statistique prêt à l'emploi. Chaque instruction se termine par un point-virgule, et un bloc par `run` ;.

```
/* SAS – non exécuté dans ce livre */
proc import datafile="donnees/commandes.csv"
  out=commandes dbms=csv replace;
```

```

    guessngrows=max;
run;

proc means data=commandes n mean std maxdec=1;
    class canal;          /* un résumé par canal */
    var montant;          /* la variable à résumer */
run;

```

On lit : « importer le CSV dans une table `commandes` » puis « *procédure MEANS* : pour chaque canal (`class`), donner l'effectif (`n`), la moyenne et l'écart-type de montant ». Aucune boucle explicite : SAS parcourt lui-même les lignes.

4.7.3 MATLAB : le calcul numérique des ingénieurs

Intuition. MATLAB (*Matrix Laboratory*) est un environnement commercial centré sur les **matrices**, très utilisé en ingénierie, en traitement du signal et en automatique, souvent enseigné dans les écoles d'ingénieurs. Tout y est une matrice, même un nombre seul (une matrice 1×1). Si vous avez aimé le chapitre 1 (algèbre linéaire), vous serez chez vous. Une alternative libre, **GNU Octave**, exécute une grande partie du même code.

Deux particularités à connaître : les indices **commencent à 1**, et les fichiers de données se lisent dans une table.

```

% MATLAB – non exécuté dans ce livre
T = readtable("donnees/commandes.csv");

G = groupsummary(T, "canal", ["mean" "std"], "montant");
disp(G)

```

`groupsummary` regroupe les lignes par canal et calcule la moyenne et l'écart-type de montant. Le résultat est une nouvelle table avec les colonnes `mean_montant` et `std_montant`.

4.7.4 Julia : la promesse « rapide comme C, simple comme Python »

Intuition. Julia (libre, créé en 2012) vise un compromis : une syntaxe lisible proche de Python et de MATLAB, mais un code **compilé à la volée** presque aussi rapide qu'un programme en C. Il est apprécié pour le calcul scientifique, les simulations lourdes et l'optimisation. Son écosystème de science des données est plus jeune que celui de Python ; une particularité est le temps d'attente à la première exécution (la compilation).

```

# Julia – non exécuté dans ce livre
using CSV, DataFrames, Statistics

commandes = CSV.read("donnees/commandes.csv", DataFrame)

resume = combine(groupby(commandes, :canal),
    nrow => :n,
    :montant => mean => :moyenne,
    :montant => std => :ecart_type)

println(resume)

```

On retrouve la logique de pandas : `groupby` puis `combine` (on lit colonne => fonction => nom_du_résultat). Les colonnes se désignent par des *symboles* (`:canal`).

4.7.5 Le même calcul, vu de près : un dictionnaire de traduction

Voici de quoi passer d'un langage à l'autre. (Les cases SAS, MATLAB et Julia sont **non exécutées**, comme les blocs ci-dessus.)

Idée	Python	R	MATLAB	Julia	SAS
Affecter	<code>x = 5</code>	<code>x <- 5</code>	<code>x = 5;</code>	<code>x = 5</code>	<code>x = 5; (étape DATA)</code>
Premier élément d'une liste	<code>x[0]</code>	<code>x[1]</code>	<code>x(1)</code>	<code>x[1]</code>	—
Moyenne	<code>np.mean(x)</code>	<code>mean(x)</code>	<code>mean(x)</code>	<code>mean(x)</code>	<code>proc means</code>
Écart-type	<code>np.std(x, ddof=1)</code>	<code>sd(x)</code>	<code>std(x)</code>	<code>std(x)</code>	<code>proc means std</code>
Lire un CSV	<code>pd.read_csv(f)</code>	<code>read.csv(f)</code>	<code>readtable(f)</code>	<code>CSV.read(f, DataFrame)</code>	<code>proc import</code>
Résumé par groupe	<code>groupby().agg()</code>	<code>group_by()</code> puis <code>summarise()</code>	<code>groupsummary</code>	<code>proc means;</code> <code>class</code>	
Blocs	indentation	<code>{ }</code>	<code>end</code>	<code>end</code>	<code>run;</code>

Le piège des indices : 0 ou 1 ? Python (comme C, Java) compte à partir de 0 ; R, MATLAB et Julia à partir de 1. De plus, les *tranches* ne se comportent pas pareil, et l'indice négatif signifie des choses opposées. Voyez la différence entre Python et R, tous les deux exécutés :

```
x = [10, 20, 30, 40, 50]
print("x[0] =", x[0])
print("x[0:2] =", x[0:2], " (la borne de droite est exclue)")
print("x[-1] =", x[-1], " (le dernier élément)")
```

```
x[0] = 10
x[0:2] = [10, 20] (la borne de droite est exclue)
x[-1] = 50 (le dernier élément)
```

```
x <- c(10, 20, 30, 40, 50)
cat("x[1] =", x[1], "\n")
cat("x[1:2] =", x[1:2], " (la borne de droite est incluse)\n")
cat("x[-1] =", x[-1], " (tout sauf le premier élément !)\n")
```

```
x[1] = 10
x[1:2] = 10 20 (la borne de droite est incluse)
x[-1] = 20 30 40 50 (tout sauf le premier élément !)
```

Même symbole `x[-1]`, **deux sens opposés** : le dernier élément en Python, « tout sauf le premier » en R. Si vous traduisez du code d'un langage à l'autre, c'est la première chose à vérifier.

Un point commun rassurant : $n - 1$. Les écarts-types par défaut de pandas, R (`sd`), MATLAB (`std`) et Julia (`std`) divisent par $n - 1$ (l'estimateur sans biais démontré au 3.2) ; seul `np.std` de NumPy divise par n si on ne précise pas `ddof=1`. Même formule, mêmes nombres : le fait que vos résultats concordent d'un langage à l'autre est d'ailleurs une excellente manière de **vérifier** un calcul.

4.7.6 Alors, lequel choisir ?

Langage	Atouts	Limites	À privilégier si...
Python	généraliste, immense écosystème, apprentissage automatique, mise en production	graphiques statistiques moins « clés en main » que R	vous voulez un seul langage pour tout (ce livre)
R	statistique de référence, ggplot2, rapports reproductibles	moins à l'aise pour le développement logiciel	votre travail est surtout statistique ou académique
SAS	stabilité, validation réglementaire, très robuste sur de gros fichiers	licence payante, communauté plus fermée	votre employeur (banque, assurance, pharma) l'impose
MATLAB	calcul matriciel, boîtes à outils d'ingénierie	licence payante, moins utilisé en science des données	vous venez de l'ingénierie ou du signal
Julia	très rapide, syntaxe claire, calcul scientifique	écosystème plus jeune, compilation au premier appel	vous faites des simulations lourdes ou de l'optimisation

Le conseil pratique. On n'apprend pas un langage par collection, on l'apprend par **projet**. Maîtrisez Python (et un peu de R pour lire du code de statisticien). Le jour où un poste exige SAS, MATLAB ou Julia, la bonne nouvelle est que *les idées sont les mêmes* : tableau, regroupement, résumé, graphique. Il ne reste qu'à apprendre la syntaxe, avec ce dictionnaire sous la main. Regardez les offres d'emploi de votre domaine et de votre pays pour savoir quels langages y sont demandés avant d'investir du temps.

À retenir (autres langages).

- **SAS** : étapes DATA et PROC, point-virgule partout, très répandu en banque, assurance et pharma ; version d'apprentissage gratuite en ligne.
- **MATLAB** : tout est matrice, indices à partir de 1 ; **Octave** en est une alternative libre.
- **Julia** : rapide et lisible, compilé à la volée ; écosystème plus jeune.
- Python et R ont été exécutés ici ; **SAS, MATLAB et Julia ne l'ont pas été** (blocs « non exécuté »).
- Pièges de traduction : indices à partir de 0 ou de 1, signification de $x[-1]$, bornes des tranches ; l'écart-type par défaut divise par $n - 1$ presque partout.
- Quel que soit le langage, le raisonnement est le même : **lire** → **regrouper** → **résumer** → **vérifier**.

4.8 ⊕ Pour aller plus loin : complexité algorithmique et optimisation du code

Section optionnelle. Un code correct n'est pas toujours un code **utilisable** : le même résultat peut prendre une seconde ou trois jours selon la manière dont on s'y prend. La **complexité algorithmique** donne un langage pour comparer des méthodes *avant* de les écrire, et la me-

sure du temps (le *profilage*) dit où agir *après*. Cette section tient la promesse faite à la section 1.6, où nous avons vu que la boutique ne peut pas « énumérer tous les paniers possibles ».

4.8.1 L'idée : compter les opérations, pas les secondes

Intuition. La gérante cherche un client dans son carnet d'adresses. Si le carnet n'est **pas trié**, elle doit lire les noms un par un : pour 1 000 clients, il lui faut en moyenne 500 lectures, et 1 000 dans le pire cas. Si le carnet est **trié par ordre alphabétique**, elle l'ouvre au milieu, regarde si le nom cherché est avant ou après, et élimine la moitié du carnet à chaque étape : 1 000 clients se règlent en **10 étapes** (car $2^{10} = 1\,024$). Avec un million de clients, la première méthode demande un million de lectures, la seconde **20**.

Ce qui compte n'est pas la vitesse de l'ordinateur ou du langage, mais la **manière dont le nombre d'opérations grandit quand la taille n des données grandit**. C'est la **complexité** de l'algorithme. Vérifions-la en comptant effectivement les comparaisons (par un petit programme de mesure, non reproduit ici) : pour $n = 1\,000$ clients, la lecture linéaire demande **1 000** comparaisons dans le pire cas, la recherche binaire **10** ; pour $n = 1\,000\,000$, **1 000 000** contre **20**.

Ces nombres ne dépendent pas de la machine : c'est ce qui rend la complexité si utile. La première méthode est **linéaire**, la seconde **logarithmique** : multiplier n par 1 000 multiplie le travail de la première par 1 000, mais ajoute seulement une dizaine d'étapes à la seconde.

4.8.2 La notation $O(\cdot)$: une définition rigoureuse

On ne se soucie pas des détails (« 3 opérations par tour de boucle » ou « 5 »). On garde seulement la **forme de la croissance**, d'où une notation qui « oublie » les constantes.

Définition (grand O). On écrit $f(n) = O(g(n))$ s'il existe une constante $c > 0$ et un rang n_0 tels que

$$f(n) \leq c g(n) \quad \text{pour tout } n \geq n_0.$$

En mots : à partir d'un certain rang, f ne dépasse pas un multiple fixe de g .

Exemple fait à la main. Un algorithme effectue $f(n) = 3n^2 + 5n + 2$ opérations. Montrons que $f(n) = O(n^2)$ avec $c = 4$. Il faut $3n^2 + 5n + 2 \leq 4n^2$, c'est-à-dire $n^2 - 5n - 2 \geq 0$. Pour $n = 5$: $25 - 25 - 2 = -2 < 0$ (l'inégalité est fausse). Pour $n = 6$: $36 - 30 - 2 = 4 \geq 0$ (vraie), et le trinôme est croissant ensuite : $n_0 = 6$ convient. Le calcul numérique le confirme : l'inégalité $f(n) \leq 4n^2$ est fausse pour $n = 1, \dots, 5$ et vraie dès $n = 6$.

Le terme dominant (n^2) décide de tout, les termes d'ordre inférieur ($5n, 2$) et le facteur 3 disparaissent. Voici les classes que vous rencontrerez le plus souvent, de la plus rapide à la plus lente :

Classe	Nom	Exemple typique
$O(1)$	constante	lire l'élément d'indice 5 d'une liste ; chercher une clé dans un dictionnaire
$O(\log n)$	logarithmique	recherche binaire dans une liste triée
$O(n)$	linéaire	parcourir une liste ; calculer une moyenne

Classe	Nom	Exemple typique
$O(n \log n)$	quasi-linéaire	trier une liste (tri efficace)
$O(n^2)$	quadratique	comparer toutes les paires d'éléments
$O(2^n)$	exponentielle	énumérer tous les sous-ensembles (tous les paniers possibles)

Pour sentir ce que ces lettres veulent dire en pratique, supposons un ordinateur capable de **un milliard d'opérations par seconde** (c'est du bon matériel) et calculons le temps que chaque classe demande :

n	n	n log ₂ n	n ²	2 ⁿ
10	10 ns	33.2 ns	100 ns	1.02 μs
40	40 ns	213 ns	1.6 μs	18.3 minutes
1,000	1 μs	9.97 μs	1 ms	(astronomique)
1,000,000	1 ms	19.9 ms	16.7 minutes	(astronomique)

Pour $n = 100$: 2^{100} opérations demandent $4.02e+13$ ans
(l'âge de l'Univers est d'environ 13,8 milliards d'années)

Lisez la dernière colonne : pour $n = 40$ éléments seulement, énumérer tous les sous-ensembles prendrait déjà plus de **18 minutes**... et pour $n = 100$, des milliers de fois l'âge de l'Univers (voir les deux lignes sous le tableau). À l'inverse, pour un million de clients, la méthode linéaire ou quasi-linéaire reste à la portée d'un ordinateur portable (de l'ordre de la milliseconde à la vingtaine de millisecondes), alors que la méthode quadratique demande environ **17 minutes**. Même matériel, mêmes données : seule la **complexité** change.

Le grand O décrit la croissance, pas la vitesse. Un algorithme $O(n)$ avec une énorme constante peut être plus lent qu'un $O(n^2)$ pour n petit. Mais, quand les données grossissent, la forme de la courbe finit toujours par gagner : c'est ce que montre la figure ci-dessous.

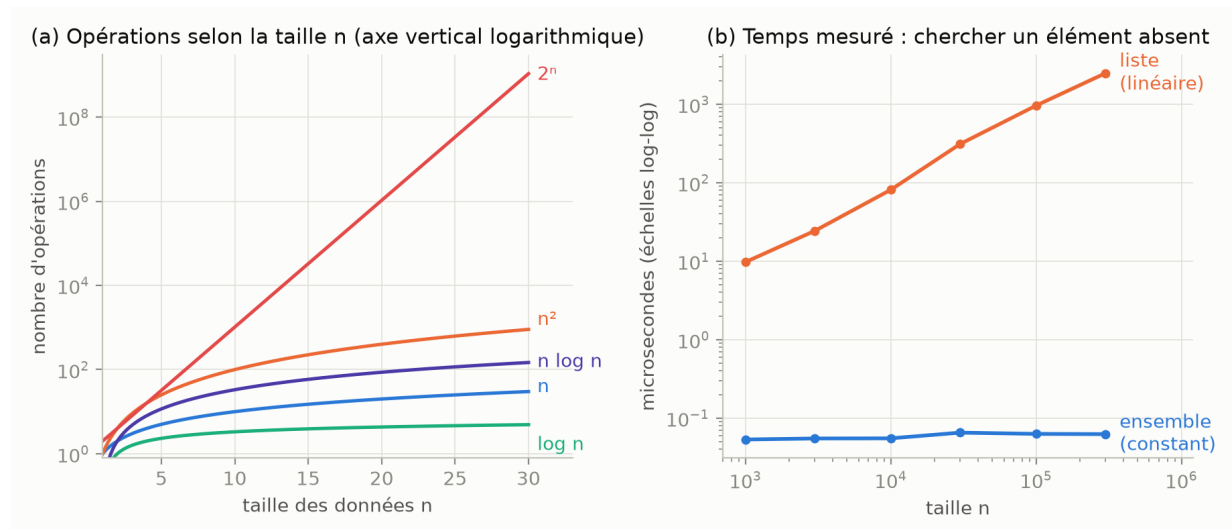


Figure 32: À gauche : nombre d'opérations selon la classe de complexité (échelle logarithmique en ordonnée). À droite : temps mesuré pour chercher un élément absent dans une liste (linéaire) et dans un ensemble (constant).

4.8.3 Mesurer : le temps, avec méthode

La théorie dit ce qui *devrait* se passer, la **mesure** vérifie ce qui se passe *vraiment*. Python fournit le module `timeit`. Deux précautions : répéter la mesure et garder le **minimum** (les autres programmes de la machine ne peuvent que ralentir la mesure, jamais l'accélérer), et comparer des **rapports** plutôt que des durées brutes, car celles-ci dépendent de la machine.

Les temps de cette section varient d'une machine à l'autre, et d'une exécution à l'autre. Nous n'affichons donc que des **rapports arrondis**. Chez vous, les valeurs exactes différeront, mais l'**ordre de grandeur** doit être le même. Si ce n'est pas le cas, c'est intéressant : cherchez pourquoi !

Test 1 : la liste contre l'ensemble. Chercher si un identifiant est présent parmi n clients. Dans une liste (`x in liste`), Python lit les éléments un par un : $O(n)$. Dans un **ensemble** (`set`), qui range ses éléments à la manière d'un dictionnaire, la recherche est en $O(1)$ en moyenne. Nous cherchons un élément **absent** (pire cas pour la liste) et nous multiplions n par 100 (code de mesure non reproduit) :

Le test confirme que la liste a un temps proportionnel à n (multiplier n par 100 multiplie le temps par un nombre de l'ordre de 100), alors que l'ensemble est **insensible** à la taille (rapport proche de 1). Le gain n'est pas de 20 % : à $n = 100\,000$, la figure (b) ci-dessous montre un écart de **plusieurs ordres de grandeur** entre les deux courbes.

Un réflexe à retenir. Vous avez une liste de 100 000 clients à vérifier contre une liste de 50 000 clients « actifs ». Écrire `[c for c in clients if c in actifs]` avec `actifs` en **liste** fait jusqu'à $100\,000 \times 50\,000 = 5 \cdot 10^9$ comparaisons. Une seule ligne, `actifs = set(actifs)`, ramène cela à $\approx 100\,000$ opérations. C'est probablement l'optimisation la plus rentable de toute la data science pratique.

Test 2 : le test du « doublement ». Une technique simple pour deviner la complexité d'un code : doubler n et regarder de combien le temps est multiplié. Environ $\times 2$: linéaire. Environ $\times 4$: quadratique. Environ $\times 8$: cubique. Comparons deux manières de détecter des commandes en double, la première par paires, la seconde en un seul passage :

```
def doublons_naif(ids):
    """Compare chaque paire :  $O(n^2)$ ."""
    n = len(ids)
    return [ids[i] for i in range(n) for j in range(i + 1, n) if ids[i] == ids[j]]

def doublons_ensemble(ids):
    """Un seul passage avec un ensemble de valeurs déjà vues :  $O(n)$ ."""
    vus, doubles = set(), []
    for x in ids:
        if x in vus:
            doubles.append(x)
            vus.add(x)
    return doubles

print(doublons_naif([4, 8, 4, 1, 8]), doublons_ensemble([4, 8, 4, 1, 8]))
```

```
[4, 8] [4, 8]
```

La version par paires est **quadratique** : doubler n multiplie le temps par un nombre proche de 4. La version à un passage est **linéaire** : doubler n multiplie le temps par un nombre proche de 2 (un peu plus, parfois, à cause de la mémoire cache de l'ordinateur). Les mesures sont bruitées : relancez le bloc plusieurs fois et vous verrez ces rapports fluctuer légèrement, mais pas changer

d'ordre de grandeur.

4.8.4 Boucles Python contre calcul vectorisé

Au 4.4, nous avons dit que NumPy et pandas sont « rapides ». Voici de quoi : calculer la somme des carrés de 1 million de montants. Les deux versions sont en $O(n)$, mais la **constante** n'est pas du tout la même : une boucle Python interprète les tours un par un, alors que NumPy exécute une boucle en code compilé.

```
import numpy as np

rng = np.random.default_rng(7)
montants = np.exp(rng.normal(3.9, 0.55, size=1_000_000))    # 1 million de montants simulés
liste = montants.tolist()

total = 0.0
for v in liste:      # une boucle Python
    total += v * v
print(np.isclose(total, np.sum(montants ** 2)))    # la même somme, vectorisée : np.sum(montants ** 2)
```

True

Même complexité théorique, mais un facteur de **dizaines** (voire de centaines selon la machine) en faveur de NumPy : la complexité ne dit donc pas tout, les constantes comptent aussi. La règle pratique du data scientist : **dès que vous écrivez une boucle for sur les lignes d'un tableau, demandez-vous s'il existe une opération vectorisée**. Même chose avec pandas : une opération sur une colonne entière est presque toujours préférable à apply ligne par ligne.

Même verdict avec pandas (`df["montant_ht"].apply(lambda x: x * 1.19)` contre `df["montant_ht"] * 1.19`, sur 200 000 lignes) : apply appelle une fonction Python **pour chaque ligne**, alors que la multiplication de la colonne entière se fait d'un coup, dans du code compilé. Le facteur dépend de la machine (de l'ordre de plusieurs dizaines à plusieurs centaines de fois), mais le message est toujours le même.

4.8.5 La mémoïsation : ne jamais calculer deux fois la même chose

Intuition. Quand on vous demande « combien font 17×23 ? », vous calculez. Si on vous le redemande dix fois, vous n'allez pas refaire le calcul : vous vous souvenez de la réponse. La **mémoïsation** (*memoization*) fait de même : on **mémorise** le résultat d'une fonction pour chaque argument déjà rencontré.

L'exemple classique est la suite de Fibonacci : $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$. La définition récursive est très élégante, mais elle recalcule les mêmes valeurs un nombre énorme de fois : pour calculer $F(5)$, on calcule $F(3)$ deux fois, $F(2)$ trois fois, etc. Comptons les appels de la version naïve (programme de mesure non reproduit ; c'est celui de 4.3.4) : 15 appels pour $F(5)$, 1 973 pour $F(15)$, **242 785** pour $F(25)$. Avec la mémoire, il suffit de 6, 16 et 26 calculs : une seule ligne ajoutée, `@lru_cache(maxsize=None)` au-dessus de la définition de la fonction (voir 4.3.4).

Pour $F(25)$: 242 785 appels contre 26 (un par valeur de 0 à 25). La version naïve est **exponentielle** (le nombre d'appels croît comme $1,6^n$), la version mémoïsée est **linéaire**. On a transformé un calcul impossible en un calcul instantané, en échange d'un peu de **mémoire** : c'est le compromis classique entre le temps et l'espace.

Ce compromis existe partout : un million de montants occupent environ **32 Mo** dans une liste Python, mais **8 Mo** seulement dans un tableau NumPy, qui range ses nombres côte à côte, sans «

emballage » : environ quatre fois moins de mémoire, ce qui est l'une des raisons de sa vitesse.

4.8.6 Profiler : mesurer avant d'optimiser

Règle d'or. « *L'optimisation prématurée est la racine de tous les maux.* » (Donald Knuth). Ne devinez pas où le programme est lent : **mesurez**. Dans un programme de 50 lignes, 95 % du temps se passe typiquement dans 1 ou 2 lignes. Les optimiser donne tout ; optimiser le reste ne change rien.

L'outil s'appelle un **profileur** : cProfile enregistre, pour chaque fonction, le nombre d'appels et le temps passé. Voici un petit rapport qui nettoie des identifiants de commandes, cherche les doublons et calcule une moyenne. Il est écrit sans malice, mais l'une de ses étapes est un piège caché. Laquelle ?

```
def nettoyer(ids):
    return [int(x) for x in ids]

def moyenne(ids):
    return sum(ids) / len(ids)

def rapport(ids):
    propres = nettoyer(ids)
    return len(doublons_naif(propres)), moyenne(propres)
```

On le profile avec cProfile (la sortie, longue, n'est pas reproduite ; nous en résumons l'essentiel) :

```
import cProfile

cProfile.run("rapport(donnees)", sort="cumtime")    # affiche, par fonction, le nombre d'appels et
↳ le temps passé
```

Le rapport du profileur désigne immédiatement le coupable : `doublons_naif` (notre détecteur par paires, quadratique) occupe près de 100 % du temps, loin devant `nettoyer`, `moyenne` et `rapport` (0 %). Inutile d'accélérer `nettoyer` : on remplace plutôt `doublons_naif` par `doublons_ensemble`, qui est linéaire (4.8.3). C'est la démarche en trois temps de tout travail d'optimisation : **mesurer, trouver le goulot d'étranglement, changer d'algorithme ou de structure de données**, puis **mesurer encore** pour confirmer le gain.

Ordre des priorités. (1) D'abord un code **correct** et lisible (4.6, avec ses tests). (2) Ensuite, si c'est trop lent, **mesurer**. (3) Optimiser d'abord l'**algorithme** (complexité), ensuite la **vectorisation**, et seulement en dernier recours les micro-détails. Un test qui reste vert après l'optimisation prouve que vous n'avez rien cassé.

4.8.7 Un exemple : les paires de produits achetés ensemble

Retour à la boutique. La gérante voudrait savoir quels produits sont **souvent achetés ensemble** pour proposer des offres groupées. Son catalogue contient 40 produits. Première idée : examiner **tous les paniers possibles** de 10 produits... Souvenez-vous du 1.6 : il y en a $\binom{40}{10} = 847\,660\,528$. Même à un million de paniers examinés par seconde, cela fait plus de 14 minutes *pour une seule taille de panier*, et le nombre de sous-ensembles totaux est $2^{40} \approx 10^{12}$. Mais la gérante n'a pas besoin de tous les paniers **possibles** : seulement des paniers **réellement achetés**. Il suffit de compter, panier par panier, les paires qu'il contient : la complexité dépend alors de la taille des données, pas de la taille de l'univers des possibles. Ici, paniers est une liste de 20 000 paniers simulés (de

2 à 5 produits chacun), dont 15 % contiennent le couple de produits 7 et 12.

```
compteur = Counter()
for panier in paniers:
    for paire in combinations(panier, 2):          # toutes les paires DU panier
        compteur[paire] += 1
print(compteur.most_common(3))
```

```
[((7, 12), 3017), ((7, 8), 395), ((5, 12), 392)]
```

En à peine plus de 120 000 opérations (au lieu de plusieurs centaines de millions), le couple (7, 12) ressort nettement : c'est l'association que nous avons planifiée dans la simulation, et les autres paires, simplement dues au hasard, ont des effectifs bien plus faibles. La méthode est **linéaire** en nombre de paniers (chaque panier de k produits fournit $\binom{k}{2}$ paires, et $k \leq 5$ ici). C'est exactement l'idée derrière les algorithmes de recommandation (« règles d'association ») : on compte ce qui s'est vraiment passé, on n'explore pas ce qui aurait pu se passer.

Pour s'entraîner. Cahier, chapitre 4 : application 4.8 (règles d'association : support, confiance, lift) ; exercice 4.14 (coût d'un algorithme).

À retenir (complexité et optimisation).

- La **complexité** mesure comment le nombre d'opérations grandit avec n , indépendamment de la machine. $f(n) = O(g(n))$: à partir d'un rang n_0 , $f \leq c g$.
- De la plus lente à la plus rapide : $O(2^n)$ (impraticable dès $n \approx 50$), $O(n^2)$, $O(n \log n)$, $O(n)$, $O(\log n)$, $O(1)$.
- **Structure de données = complexité.** Chercher dans une liste est $O(n)$, dans un set ou un dictionnaire $O(1)$; une liste triée permet la recherche binaire en $O(\log n)$.
- **Test du doublement** : n double, temps $\times 2 \rightarrow$ linéaire ; $\times 4 \rightarrow$ quadratique.
- **Vectorisez** : NumPy et pandas battent les boucles Python de plusieurs ordres de grandeur, à complexité égale.
- **Mémoïsation** (@lru_cache) : échanger de la mémoire contre du temps, en ne calculant jamais deux fois la même chose.
- **Mesurez, ne devinez pas** : profilez (cProfile), trouvez le goulot, changez d'algorithme, remesurez. Les durées varient d'une machine à l'autre ; les ordres de grandeur et les rapports, non.

Bilan du chapitre 4

Vous savez maintenant :

- **programmer en Python** (4.1) : variables et types, collections (liste, tuple, dictionnaire, ensemble), conditions, boucles, fonctions, erreurs et try / except, lecture d'un fichier CSV, et un petit programme complet (le ticket de caisse) testé par assert ;
- **refaire les mêmes analyses en R** (4.2) : vecteurs, data.frame, tests statistiques « de la boîte », dplyr ; et savoir que Python et R donnent les mêmes nombres quand on leur pose la même question ;
- **raisonner comme un informaticien** (4.3) : choisir une structure (liste, dictionnaire, ensemble, pile, file), écrire une récursion avec son cas de base, prouver une dichotomie ou un tri par un invariant, et ne pas trier plus que nécessaire ;
- **manipuler des données** (4.4) : tableaux NumPy et broadcasting ; DataFrames pandas, sélection, groupby, jointures, valeurs manquantes, dates ;

- **dessiner honnêtement** (4.5) : choisir le bon graphique pour la question, l'anatomie d'un graphique matplotlib, seaborn, ggplot2, et les pièges du graphique trompeur (axe tronqué en tête) ;
- (en option, 4.6) **structurer et tester** : classes et dataclasses, code lisible, tests pytest qui visent les bornes ;
- (en option, 4.7) **situer** SAS, MATLAB et Julia par rapport à Python et R ;
- (en option, 4.8) **compter le coût** d'un algorithme (notation $O(\cdot)$), préférer la vectorisation, mémoïser, et mesurer avant d'optimiser.

Trois réflexes à emporter. (1) *Calculer à la main un petit cas avant de coder* : c'est votre oracle. (2) *Lire le message d'erreur par la dernière ligne*. (3) *Vérifier un résultat par une seconde voie* (Python contre R, deux méthodes, une somme de contrôle).

Le chapitre 5 apprend à **aller chercher** les données là où elles vivent réellement : dans des bases de données relationnelles, avec le langage SQL. Vous y retrouverez commandes .csv, des jointures (comme en 4.4.10), des agrégations (comme le groupby) et l'idée de l'**index**, cet arbre trié qui permet une recherche dichotomique.

Pour s'entraîner. Le **cahier** du volume I (chapitre 4) contient huit applications guidées (le ticket de caisse complet, le rapport hebdomadaire, le tableau de bord, le module testé de la boutique, les règles d'association...) et quatorze exercices corrigés.

Chapitre 5 : Bases de données et SQL

« Les données ne vivent presque jamais dans un fichier CSV. Elles vivent dans une **base de données**, et pour leur parler il faut connaître **SQL**. »

Jusqu'ici, nous avons travaillé sur un tableau déjà prêt, chargé en mémoire dans un notebook. Dans une vraie entreprise, ce n'est presque jamais le cas : les commandes, les clients, les stocks sont enregistrés dans une **base de données**, souvent plusieurs millions de lignes réparties dans des dizaines de tables liées entre elles. Avant de calculer la moindre moyenne, il faut donc **aller chercher** l'information, la **croiser**, la **résumer**. Le langage de cette étape s'appelle **SQL** (*Structured Query Language*). Il a plus de cinquante ans, il est partout, et c'est l'une des compétences les plus demandées dans les offres d'emploi de data scientist et de data analyst.

La bonne nouvelle : SQL se lit presque comme de l'anglais (ou, ici, du français traduit), et un petit nombre d'idées suffisent pour répondre à 90 % des questions réelles.

Le chemin de ce chapitre

- **5.1 Modèle relationnel et conception de bases de données** : pourquoi une base plutôt qu'un fichier ? Tables, clés, relations. Nous découvrons la base de la boutique (clients, produits, commandes, lignes de commande).
- **5.2 Requêtes SQL** : sélectionner, filtrer, trier, agréger, **joindre** plusieurs tables, imbriquer des requêtes, et éviter les pièges du NULL et des dates.
- **5.3 Fonctions fenêtres et CTE** : les outils des analystes expérimentés : classements, cumuls, moyennes mobiles, comparaison avec la ligne précédente, requêtes lisibles par étapes, requêtes récursives.
- **5.4 Normalisation et conception de schémas** : pourquoi on découpe les données en plusieurs tables, comment le faire proprement (formes normales), puis les index et les transactions.
- **⊕ 5.5 Pour aller plus loin : bases NoSQL** (MongoDB, Redis) : quand et pourquoi sortir du modèle relationnel.

Le fil conducteur : la base de données de la boutique. Au chapitre 3, nous avions un seul tableau de 400 commandes. Ici, la gérante passe à la vitesse supérieure : ses **400 commandes** (le fichier `donnees/commandes.csv`, inchangé) sont rangées dans une vraie base, à côté de la liste de ses **80 clients**, de ses **16 produits** et du **détail de chaque commande**. Tout est simulé avec une graine fixe : vous retrouverez exactement les mêmes résultats que dans le livre.

Rien à installer. Nous utilisons **SQLite**, une base de données complète qui tient dans un simple fichier et qui est déjà fournie avec Python (module `sqlite3`). Pas de serveur à configurer, pas de mot de passe. Le SQL que vous apprendrez ici fonctionne, à de petites différences près (nous les signalons), sur PostgreSQL, MySQL, SQL Server, Oracle ou BigQuery. Le fichier `donnees/boutique.db`, fourni avec le livre, contient la base toute faite : vous pouvez aussi l'ouvrir avec n'importe quel outil graphique (DB Browser for SQLite, DBeaver...) ou avec la commande `sqlite3 donnees/boutique.db`.

Comment lire les blocs sql. Chaque requête est suivie de **sa sortie réelle** (raccourcie à quelques lignes, avec `LIMIT`). Pour exécuter une requête depuis Python et récupérer le résultat dans un tableau pandas (section 4.4), on écrit `pd.read_sql_query("SELECT ...", con)`, où `con` est la connexion à la base : c'est ce que fait l'outil de fabrication de ce livre en coulisses.

Le cahier. Les applications guidées et les exercices corrigés de ce chapitre se trouvent dans le **cahier d'exercices et d'applications** du volume, chapitre 5 ; le livre y renvoie à la fin des sections.

5.1 Modèle relationnel et conception de bases de données

Intuition. Une base de données relationnelle, c'est **un classeur de tableaux bien rangés** (les *tables*) **qui se parlent entre eux** grâce à des numéros d'identification (les *clés*). Au lieu de tout recopier dans un seul gros tableau, on range chaque chose à **un seul endroit** : les clients dans un tableau, les produits dans un autre, les commandes dans un troisième, et on les relie par des numéros.

5.1.1 Pourquoi pas simplement un fichier CSV ?

La gérante a commencé avec un fichier Excel. Un jour, elle s'est retrouvée avec `commandes_v3_FINAL.xlsx`, `commandes_v3_FINAL_corrige.xlsx` et `commandes_v3_VRAIMENT_FINAL.xlsx`. Dans l'un, la cliente « Léa Martin » habite Ville F, dans l'autre Ville G : laquelle est la bonne ? Personne ne sait. Les problèmes d'un fichier à plat sont toujours les mêmes :

Problème	Exemple chez la boutique	Ce que fait une base de données
Redondance	l'adresse d'un client est copiée sur chacune de ses 28 commandes	elle est stockée une seule fois
Incohérence	deux lignes du même client donnent deux villes différentes	impossible : une seule ligne client, référencée par un numéro
Données invalides	une note de satisfaction de 7, un canal « Marché » qui n'existe pas	des contraintes refusent la saisie
Volume	10 millions de lignes : le fichier ne s'ouvre plus	la base lit seulement ce dont elle a besoin (grâce aux <i>index</i> , 5.4)
Accès simultané	deux employés modifient le fichier en même temps : l'un écrase l'autre	des transactions gèrent les accès concurrents (5.4)

Problème	Exemple chez la boutique	Ce que fait une base de données
Questions complexes	« les clients de Ville F qui ont acheté des bijoux deux mois de suite »	un langage fait pour ça : SQL

Un **SGBD** (*système de gestion de bases de données*) est le logiciel qui garantit tout cela. Les plus courants : **PostgreSQL** et **MySQL** (libres, très répandus sur le web), **SQL Server** et **Oracle** (grandes entreprises), **SQLite** (une base dans un simple fichier, présente dans votre téléphone, votre navigateur et Python), **BigQuery**, **Snowflake**, **Redshift** (entrepôts de données dans le *cloud*). Tous parlent SQL, avec de petits accents régionaux (*dialectes*).

5.1.2 Le modèle relationnel

Le modèle relationnel date de 1970 (Edgar F. Codd, chez IBM). Il tient en quelques mots.

- Une **table** (ou *relation*) représente un type de chose : les clients, les produits...
- Une **colonne** (ou *attribut*) est une propriété de cette chose : prenom, ville...
- Une **ligne** (ou *enregistrement*, ou *n-uplet*) est une chose précise : la cliente n° 1.
- Chaque colonne a un **type** (entier, texte, date...).

Voici trois clients de la boutique sous forme de table :

id_client	prenom	nom	ville
1	Yann	Lambert	Ville C
2	Sam	Fontaine	Ville A
3	Adam	Michel	Ville F

Définition rigoureuse. Soient $D_1, \dots, D_p$ des ensembles appelés **domaines** (par exemple $D_1 = \mathbb{N}$ pour les numéros, $D_2 =$ les chaînes de caractères). Une **relation** R de **schéma** $R(A_1 : D_1, \dots, A_p : D_p)$ est un **sous-ensemble fini** du produit cartésien

$$R \subseteq D_1 \times D_2 \times \dots \times D_p.$$

Deux conséquences importantes, souvent oubliées : (1) R étant un **ensemble**, il n'y a **ni doublons ni ordre** entre les lignes : demander « la première ligne » n'a pas de sens sans préciser un critère de tri ; (2) chaque case contient **une seule valeur** de son domaine (nous y reviendrons avec la première forme normale, 5.4).

Les clés. C'est là que la magie opère.

- Une **clé candidate** est un ensemble minimal de colonnes qui identifie **sans ambiguïté** une ligne. Dans `clients`, `id_client` en est une. Le couple (`prenom`, `nom`) n'en est pas une : nous verrons au 5.2.4 que notre propre base contient trois « Léa Fontaine » dans trois villes différentes, et même deux « Anna Faure » qui habitent toutes deux Ville B. Un nom n'est **jamais** un bon identifiant.
- La **clé primaire** (*primary key*, PK) est la clé candidate choisie comme identifiant officiel. Elle ne peut être ni vide (NULL) ni répétée. Par convention, on utilise un **numéro** sans signification (une *clé de substitution*, ou *surrogate key*) : il ne change jamais, même si la personne déménage ou change de nom.

- Une **clé étrangère** (*foreign key*, FK) est une colonne qui **référence la clé primaire d'une autre table**. Dans commandes, la colonne `id_client` désigne le client qui a passé la commande. C'est ainsi que les tables « se parlent ».
- L'**intégrité référentielle** est la règle qui en découle : une clé étrangère ne peut contenir que des valeurs qui **existent** dans la table référencée. Impossible d'enregistrer une commande du client n° 9999 s'il n'existe pas.

L'analogie du carnet d'adresses. Dans votre téléphone, vous ne recopiez pas toutes les informations d'un contact dans chaque SMS ; vous gardez une fiche « Camille » et vous écrivez à « Camille ». La fiche est la ligne de clients, son numéro est la clé primaire, et chaque SMS porte une clé étrangère vers elle.

Les **relations entre tables** se décrivent en trois types, selon le nombre de lignes de chaque côté :

- **Un à plusieurs (1-N)** : un client passe **plusieurs** commandes, mais chaque commande est passée par **un seul** client. La clé étrangère se place du côté « plusieurs » (commandes.`id_client`).
- **Plusieurs à plusieurs (N-N)** : une commande contient **plusieurs** produits, et un produit figure dans **plusieurs** commandes. Une clé étrangère ne suffit plus : on crée une **table d'association** (ou *de jonction*) qui contient une ligne par couple (commande, produit). C'est la table `lignes_commande`, qui porte aussi les attributs du lien : `quantite` et `prix_unitaire`.
- **Un à un (1-1)** : rare ; en pratique, on fusionne souvent les deux tables.

Un cas particulier amusant : la relation **réflexive**. La boutique a un programme de parrainage : un client peut avoir été **parrainé par un autre client**. La table `clients` se référence donc elle-même (`id_parrain` pointe vers `id_client`). Nous nous en servons pour les requêtes récursives (5.3).

Voici le **schéma** complet de notre base. Chaque boîte est une table ; l'étiquette **PK** désigne une clé primaire, **FK** une clé étrangère. Un trait relie chaque clé étrangère (côté « N », plusieurs) à la clé primaire qu'elle référence (côté « 1 »).

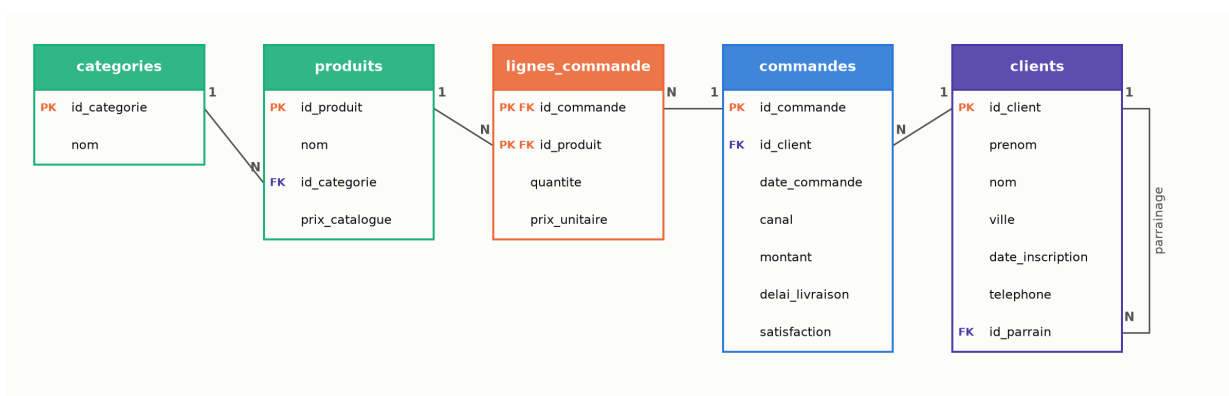


Figure 33: Schéma de la base de la boutique : cinq tables. Un trait relie chaque clé étrangère (N) à la clé primaire qu'elle référence (1). La table `lignes_commande` est la table d'association entre commandes et produits.

L'algèbre relationnelle : les maths derrière SQL. Les requêtes SQL sont la traduction de cinq opérations sur les ensembles, que Codd a décrites dès 1970. Les connaître donne une compréhension durable, qui dépasse les différences de syntaxe entre les SGBD :

Opération	Notation	Sens	Équivalent SQL
Sélection	$\sigma_{\text{condition}}(R)$	garder certaines lignes	WHERE
Projection	$\pi_{A_1, \dots, A_k}(R)$	garder certaines colonnes	SELECT col1, col2
Produit cartésien	$R \times S$	toutes les paires (une ligne de R , une ligne de S)	CROSS JOIN
Jointure	$R \bowtie_{\text{cond}} S = \sigma_{\text{cond}}(R \times S)$	les paires qui vérifient une condition	JOIN ... ON
Union / différence	$R \cup S, R \setminus S$	ensembles de lignes	UNION, EXCEPT

La ligne la plus importante est celle de la **jointure** : c'est un produit cartésien **filtré**. Nous le verrons « en vrai » au 5.2.5 : 80 clients $\times$ 400 commandes = 32 000 paires, dont seules 400 sont les « bonnes ».

5.1.3 Créer des tables : un petit exemple, puis la base fournie

Une table se crée avec l'instruction CREATE TABLE, qui énumère les colonnes, leur type et leurs règles ; on la remplit avec INSERT. Voici, sur une base vide et temporaire, la création de deux tables du schéma ci-dessus, avec deux produits.

```
import sqlite3

essai = sqlite3.connect(":memory:")          # une base vide, en mémoire
essai.executescript("""
CREATE TABLE categories (id_categorie INTEGER PRIMARY KEY, nom TEXT NOT NULL UNIQUE);
CREATE TABLE produits (
    id_produit INTEGER PRIMARY KEY, nom TEXT NOT NULL,
    id_categorie INTEGER NOT NULL REFERENCES categories(id_categorie),
    prix_catalogue REAL NOT NULL CHECK (prix_catalogue > 0));
INSERT INTO categories VALUES (1, 'Poterie');
INSERT INTO produits VALUES (1, 'Plat décoratif', 1, 45.0), (2, 'Bol en céramique', 1, 18.0);
""")
print(essai.execute("SELECT nom, prix_catalogue FROM produits").fetchall())

[('Plat décoratif', 45.0), ('Bol en céramique', 18.0)]
```

Chaque ligne de CREATE TABLE est la traduction directe du schéma : PRIMARY KEY pour la clé primaire, REFERENCES pour la clé étrangère, NOT NULL et CHECK pour les règles (nous les détaillons au 5.1.4). Le fichier donnees/boutique.db, **fourni avec le livre**, contient la base complète construite de la même façon, avec ses cinq tables. Elle a été générée par le script build/base_sql.py à partir du fichier donnees/commandes.csv du chapitre 3, avec une graine fixée : vous retrouverez exactement les mêmes résultats que dans le livre. Reconstruire toute la base, étape par étape, est l'objet de l'application 5.1 du cahier.

Table	Lignes	Contenu
categories	4	les familles de produits (poterie, textile, bijoux, cosmétiques)
produits	16	le catalogue, avec le prix catalogue
clients	80	les inscrits (dont 14 qui n'ont jamais commandé), avec leur éventuel parrain

Table	Lignes	Contenu
commandes	400	les 400 commandes du chapitre 3, avec une date et un client ajoutés
lignes_commande	693	le détail de chaque commande, produit par produit

Les lignes de commande sont construites pour que la **somme** (quantité × prix) de chaque commande retombe **exactement** sur son montant. Le prix payé peut différer de quelques pour cent du prix du catalogue (promotions, variations de prix dans l'année) ; nous verrons au 5.4 pourquoi il est essentiel de **recopier** le prix dans la ligne de commande au lieu de le relire dans le catalogue.

Un premier regard sur le contenu : combien de lignes par table, et un aperçu des clients. (SELECT * signifie « toutes les colonnes » et LIMIT 5 « seulement 5 lignes » ; le mot-clé UNION ALL empile les résultats de plusieurs requêtes, nous y reviendrons.)

```
SELECT 'clients' AS "table", COUNT(*) AS lignes FROM clients
UNION ALL SELECT 'commandes', COUNT(*) FROM commandes
UNION ALL SELECT 'lignes_commande', COUNT(*) FROM lignes_commande;
```

table	lignes
clients	80
commandes	400
lignes_commande	693

```
SELECT * FROM clients LIMIT 5;
```

id_client	prenom	nom	ville	date_inscription	telephone	id_parrain
1	Yann	Lambert	Ville C	2024-12-14	7725588	None
2	Sam	Fontaine	Ville A	2024-12-16	NaN	None
3	Adam	Michel	Ville F	2024-12-16	NaN	None
4	Anna	Blanc	Ville A	2024-12-18	NaN	None
5	Anna	Michel	Ville A	2024-12-30	7402250	None

Remarquez la colonne id_parrain : vide pour la plupart des clients (c'est un NULL), mais quand elle est remplie, elle contient le numéro d'un autre client. Le telephone vide de certains clients est lui aussi un NULL : il nous réservera quelques surprises au 5.2.7.

Lisons maintenant ensemble les lignes des quatre premières commandes, car cette table est le cœur de la base :

```
SELECT * FROM lignes_commande WHERE id_commande <= 4;
```

id_commande	id_produit	quantite	prix_unitaire
1	1	1	44.80
2	10	1	34.50
3	2	1	17.84
3	3	1	64.42
3	13	1	5.94
4	11	1	15.57
4	14	1	14.53

La commande n° 1 a un montant de 44,80 € dans la table commandes : une seule ligne ici, le produit n° 1, en un exemplaire à 44,80 €. La commande n° 3 a un montant de 88,20 € : trois lignes, trois produits différents, chacun en un exemplaire (17,84 + 64,42 + 5,94 = 88,20). La table commandes ne dit **pas** ce qu'il y a dans le panier ; c'est lignes_commande qui le dit. Ce découpage est le propre d'une bonne base de données.

Pourquoi une clé primaire à deux colonnes ? Dans `lignes_commande`, la clé primaire est le **couple** (`id_commande`, `id_produit`) : une même commande ne peut pas contenir deux fois le même produit sur deux lignes séparées ; si le client en prend trois, on écrit `quantite = 3`. C'est une **clé composite**.

5.1.4 Types, contraintes : la base se défend

Relisez les `CREATE TABLE`. Outre les noms et les types, on y trouve des **contraintes** : des règles que la base fait respecter **toute seule**.

Contrainte	Exemple dans notre schéma	Effet
PRIMARY KEY	<code>id_client INTEGER PRIMARY KEY</code>	identifiant unique et non vide
NOT NULL	<code>prenom TEXT NOT NULL</code>	la valeur est obligatoire
UNIQUE	<code>nom TEXT NOT NULL UNIQUE</code> (catégories)	pas de doublon
CHECK	<code>satisfaction BETWEEN 1 AND 5</code>	la valeur doit vérifier une condition
REFERENCES	<code>id_client ... REFERENCES</code> <code>clients(id_client)</code>	clé étrangère (intégrité référentielle)

Voyons-les à l'œuvre en essayant d'enregistrer des données **absurdes** :

```
essais = {
    "client inexistant": "INSERT INTO commandes VALUES (9001, 9999, '2025-06-01', 'Site', 50, 2, 4)",
    "canal inconnu":     "INSERT INTO commandes VALUES (9002, 1, '2025-06-01', 'Marché', 50, 2, 4)",
    "note de 7 sur 5":   "INSERT INTO commandes VALUES (9003, 1, '2025-06-01', 'Site', 50, 2, 7)",
    "prénom manquant":  "INSERT INTO clients VALUES (500, NULL, 'Test', 'Ville H', '2025-06-01',
        ↪ NULL, NULL)",
}
for nom, requete in essais.items():
    try:
        con.execute(requete)
    except sqlite3.IntegrityError as erreur:
        print(f"{nom:18s} -> refusé : {erreur}")
```

```
client inexistant -> refusé : FOREIGN KEY constraint failed
canal inconnu     -> refusé : CHECK constraint failed: canal IN ('Réseaux', 'Site', 'Boutique')
note de 7 sur 5   -> refusé : CHECK constraint failed: satisfaction BETWEEN 1 AND 5
prénom manquant   -> refusé : NOT NULL constraint failed: clients.prenom
```

Chaque donnée absurde est **refusée avec un message clair**, et les tables restent intactes (la base garde ses 400 commandes). C'est la grande différence avec une feuille Excel, où rien n'empêche de taper « sept » dans la colonne des notes. Une base bien conçue rend les erreurs de saisie **impossibles** au lieu de les laisser se glisser puis fausser silencieusement une analyse.

Particularités de SQLite (à savoir). (1) SQLite n'applique les clés étrangères que si on le demande par `PRAGMA foreign_keys = ON` (le script de construction l'active ; avec le fichier fourni, il faut l'exécuter à chaque connexion, car ce réglage n'est pas mémorisé dans le fichier). Les autres SGBD les appliquent toujours. (2) SQLite est « souple » avec les types : il accepterait du texte dans une colonne d'entiers. PostgreSQL, lui, est strict. (3) SQLite n'a **pas de type date** : on stocke les dates en texte au format **ISO 8601** ('2025-06-01', année-mois-jour). Ce format a l'avantage que **l'ordre alphabétique est l'ordre chronologique**, ce qui permet de comparer et de trier correctement les dates comme des textes. N'utilisez jamais

'01/06/2025' : le tri alphabétique donnerait n'importe quoi.

Et l'argent ? Nous stockons les montants en REAL (nombres à virgule flottante) par simplicité. En production, c'est une **mauvaise idée** : au chapitre 1 (analyse numérique, 1.5), nous avons vu que $0,1 + 0,2 \neq 0,3$ en binaire. Les bons choix sont un type décimal exact (NUMERIC / DECIMAL), ou bien des **entiers** exprimés dans la plus petite unité : en **centimes**, 44,80 € s'enregistre 4480. Les comptables vous remercieront.

5.1.5 SQL sur un fichier CSV : l'import en cinq lignes

Une dernière remarque pratique. Vous n'aurez pas toujours une base toute faite : souvent, vous recevrez un CSV. La bibliothèque pandas sait le charger dans SQLite d'un trait, et vous pouvez alors utiliser SQL dessus : très pratique pour des agrégations compliquées ou pour s'entraîner.

```
import pandas as pd

con_csv = sqlite3.connect(":memory:") # une base temporaire, vide
pd.read_csv("donnees/commandes.csv").to_sql("commandes_csv", con_csv, index=False)
requete = """SELECT canal, COUNT(*) AS commandes, ROUND(AVG(montant), 2) AS panier_moyen
FROM commandes_csv GROUP BY canal ORDER BY panier_moyen DESC"""
print(pd.read_sql_query(requete, con_csv))
```

	canal	commandes	panier_moyen
0	Boutique	114	74.81
1	Site	148	59.50
2	Réseaux	138	49.01

On retrouve les effectifs par canal du chapitre 3 (114 commandes en boutique, 148 sur le site, 138 sur Réseaux) et, en prime, les paniers moyens : la boutique est la plus généreuse. Remarquez la différence avec la vraie base : ici la table a été **devinée** par pandas (aucune clé, aucune contrainte : l'application 5.2 du cahier l'examine), alors que notre base de la boutique a été **conçue**. Un import rapide convient pour explorer ; une base conçue est indispensable pour travailler à plusieurs et dans la durée.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.1 (reconstruire la base) et 5.2 (importer un CSV et lui donner un vrai schéma), exercice 5.1.

À retenir

- Une base relationnelle range chaque information à **un seul endroit** dans des **tables**, reliées par des **clés** (primaires, étrangères).
- Les **contraintes** (NOT NULL, CHECK, REFERENCES...) font respecter les règles métier **automatiquement**.
- Une relation **plusieurs à plusieurs** passe par une **table d'association** (lignes_commande).
- Les requêtes SQL traduisent l'**algèbre relationnelle** : sélection, projection, produit cartésien, jointure (= produit filtré), union.
- Dates : format ISO 8601 en texte. Argent : décimaux exacts ou entiers (millimes), jamais de flottants en production.

5.2 Requêtes SQL : sélection, jointures, agrégation

Intuition. SQL est un langage **déclaratif** : on ne dit pas *comment* trouver le résultat (« parcourir la table, comparer, recopier... »), on décrit *ce qu'on veut* (« les commandes de plus de 100 € passées par le canal Réseaux »), et la base choisit seule la meilleure méthode. C'est comme commander au restaurant : on dit « le plat du jour, sans oignons », pas la recette.

5.2.1 Anatomie d'une requête : SELECT ... FROM ...

La requête de base a deux morceaux obligatoires : **quoi** (SELECT, les colonnes) et **où** (FROM, la table).

```
SELECT id_commande, canal, montant
FROM commandes
LIMIT 4;
```

id_commande	canal	montant
1	Boutique	44.8
2	Site	34.5
3	Réseaux	88.2
4	Réseaux	30.1

On peut **calculer** de nouvelles colonnes et les **renommer** avec AS (un *alias*). Les montants de la boutique sont TTC, avec une TVA de 19 % ; calculons le hors taxe et la TVA :

```
SELECT id_commande,
       montant AS ttc,
       ROUND(montant / 1.19, 2) AS ht,
       ROUND(montant - montant / 1.19, 2) AS tva
FROM commandes
LIMIT 4;
```

id_commande	ttc	ht	tva
1	44.8	37.65	7.15
2	34.5	28.99	5.51
3	88.2	74.12	14.08
4	30.1	25.29	4.81

Voici la requête complète « tout-en-un », avec toutes les clauses possibles, **dans l'ordre où on les écrit** :

SELECT	colonnes ou calculs	-- quoi afficher
FROM	table	-- d'où ça vient
JOIN	autre_table ON ...	-- relier d'autres tables
WHERE	condition sur les lignes	-- quelles lignes garder
GROUP BY	colonnes	-- regrouper
HAVING	condition sur les groupes	-- quels groupes garder
ORDER BY	colonnes	-- trier
LIMIT	n	-- n lignes au maximum

L'ordre d'écriture n'est pas l'ordre d'exécution. Logiquement, la base procède ainsi : (1) FROM / JOIN (assembler les lignes), (2) WHERE (filtrer les lignes), (3) GROUP BY (regrouper), (4) HAVING (filtrer les groupes), (5) SELECT (calculer les colonnes demandées), (6) ORDER BY (trier), (7) LIMIT (couper). Conséquence classique : un alias défini dans le SELECT n'existe **pas encore** quand le WHERE s'exécute (certains SGBD tolèrent l'alias dans ORDER BY ou GROUP BY, pas dans WHERE). Si une requête refuse votre alias dans un WHERE, c'est pour cette raison.

Lien avec l'algèbre relationnelle (5.1.2). Une requête `SELECT a, b FROM T WHERE c` se lit : $\pi_{a,b}(\sigma_c(T))$: d'abord la **sélection** (les lignes), puis la **projection** (les colonnes). Le vocabulaire est trompeur : le `SELECT` de SQL correspond à la *projection* de l'algèbre, pas à sa *sélection* ! Le `WHERE` est la vraie sélection.

5.2.2 Filtrer avec WHERE

`WHERE` garde les lignes pour lesquelles la condition est **vraie**. Les comparaisons sont `=` (un seul signe égal, contrairement à Python), `<>` (différent), `<`, `<=`, `>`, `>=`. On combine avec `AND`, `OR`, `NOT`.

```
SELECT COUNT(*) AS nb
FROM commandes
WHERE canal = 'Réseaux' AND montant > 100;
```

```
nb
11
```

(Les textes se mettent entre **apostrophes simples** : `'Réseaux'`. Les guillemets doubles servent aux noms de colonnes.) Il y a donc peu de grosses commandes par le canal Réseaux. Pour tester une liste de valeurs, `IN` ; pour un intervalle (**bornes incluses**), `BETWEEN` ; pour une recherche de motif dans un texte, `LIKE` (`%` remplace n'importe quelle suite de caractères, `_` un seul caractère) :

```
SELECT COUNT(*) AS commandes_de_decembre
FROM commandes
WHERE date_commande BETWEEN '2025-12-01' AND '2025-12-31';
```

```
commandes_de_decembre
57
```

```
SELECT id_produit, nom, prix_catalogue
FROM produits
WHERE nom LIKE '%parfum%';
```

```
id_produit      nom      prix_catalogue
14      Eau parfumée      14.0
16      Bougie parfumée    20.0
```

Attention aux priorités. `AND` passe **avant** `OR`, comme la multiplication passe avant l'addition. L'expression `A OR B AND C` se lit donc `A OR (B AND C)`, ce qui n'est pas du tout `(A OR B) AND C`. Mesurons l'écart sur un exemple. On veut compter « les commandes de plus de 100 € passées par le canal Réseaux ou par le site » :

```
SELECT 'sans parenthèses' AS version, COUNT(*) AS nb
FROM commandes
WHERE canal = 'Réseaux' OR canal = 'Site' AND montant > 100
UNION ALL
SELECT 'avec parenthèses', COUNT(*)
FROM commandes
WHERE (canal = 'Réseaux' OR canal = 'Site') AND montant > 100;
```

```
version  nb
sans parenthèses 152
avec parenthèses 25
```

Sans parenthèses, la requête compte **toutes** les commandes du canal Réseaux (quel que soit le montant) *plus* les commandes du site de plus de 100 € : 152 lignes. Avec parenthèses, elle compte les grosses commandes (plus de 100 €) venues des réseaux sociaux *ou* du site : 25 lignes. Un écart de 1 à 6 dans le résultat, à cause de deux caractères.

Règle d'or. Dès qu'on mélange AND et OR, **mettez des parenthèses**, même quand elles sont techniquement inutiles. Votre lecteur (et vous dans six mois) vous en sera reconnaissant.

5.2.3 Trier, limiter, dédoublonner

ORDER BY trie (ASC croissant par défaut, DESC décroissant). Rappelez-vous (5.1.2) que sans ORDER BY, **l'ordre des lignes n'est pas garanti**. LIMIT n ne garde que les n premières : indispensable pour des « top 5 ».

```
SELECT id_commande, date_commande, canal, montant
FROM commandes
ORDER BY montant DESC
LIMIT 5;
```

id_commande	date_commande	canal	montant
157	2025-06-23	Site	255.7
61	2025-03-28	Site	243.8
208	2025-07-31	Site	217.1
243	2025-08-26	Boutique	212.4
362	2025-12-16	Boutique	208.8

On peut trier sur plusieurs colonnes (ORDER BY canal, montant DESC : d'abord par canal, puis, à canal égal, du plus gros au plus petit montant). DISTINCT supprime les doublons :

```
SELECT DISTINCT ville
FROM clients
ORDER BY ville;
```

```
ville
Ville A
Ville B
Ville C
Ville D
Ville E
Ville F
Ville G
Ville H
```

Trois précisions utiles sur ces clauses. (1) On peut trier sur un **alias** défini dans le SELECT (ORDER BY chiffre_affaires) ou sur la **position** de la colonne (ORDER BY 2), mais la première forme est bien plus lisible. (2) Les NULL (5.2.7) sont placés **en premier** par SQLite dans un tri croissant, et en dernier dans un tri décroissant ; d'autres SGBD font l'inverse, et la plupart acceptent NULLS FIRST / NULLS LAST pour le préciser. (3) LIMIT n OFFSET m saute les m premières lignes avant d'en garder n : c'est la façon classique de **paginer** un résultat (page 3 de 10 lignes : LIMIT 10 OFFSET 20). Rappelez-vous enfin que LIMIT sans ORDER BY donne un résultat **arbitraire** : pour un « top 5 », le tri est obligatoire.

5.2.4 Agréger : COUNT, SUM, AVG, GROUP BY

Jusqu'ici, chaque ligne du résultat correspondait à une ligne de la table. Le cœur de l'analyse de données, c'est l'inverse : **résumer** beaucoup de lignes en peu de chiffres. Les **fonctions d'agrégation** sont COUNT (nombre de lignes), SUM (somme), AVG (moyenne), MIN, MAX.

```
SELECT COUNT(*)           AS nb_commandes,
       COUNT(DISTINCT id_client) AS nb_clients_distincts,
       ROUND(SUM(montant), 2)   AS chiffre_affaires,
       ROUND(AVG(montant), 2)   AS panier_moyen,
       MIN(montant)            AS plus_petite,
```

MAX(montant)		AS plus_grande			
FROM commandes;					
nb_commandes	nb_clients_distincts	chiffre_affaires	panier_moyen	plus_petite	plus_grande
400	66	24098.3	60.25	8.6	255.7

Les 400 commandes totalisent environ 24 098 €, soit un panier moyen de **60,25 €** : c'est exactement la moyenne du chapitre 3. SQL et pandas calculent la même chose ; seule la syntaxe change. Autre information intéressante : sur les 80 clients inscrits, **66 seulement** ont passé au moins une commande.

Une table de correspondance pandas ↔ SQL (que vous retrouverez en 4.4) : `df[df.canal == "Site"] ↔ WHERE canal = 'Site' ; df.groupby("canal")["montant"].mean() ↔ SELECT canal, AVG(montant) ... GROUP BY canal ; df.sort_values("montant") ↔ ORDER BY montant.`

GROUP BY découpe la table en **groupes** (un par valeur de la colonne) et applique les agrégations **dans chaque groupe**. Voici, par canal, le nombre de commandes, le panier moyen, le chiffre d'affaires, la satisfaction moyenne et le délai de livraison moyen :

```
SELECT canal,
  COUNT(*)           AS commandes,
  ROUND(AVG(montant), 2) AS panier_moyen,
  ROUND(SUM(montant))   AS chiffre_affaires,
  ROUND(AVG(satisfaction), 2) AS satisfaction,
  ROUND(AVG(delai_livraison), 2) AS delai_moyen
FROM commandes
GROUP BY canal
ORDER BY chiffre_affaires DESC;
```

canal	commandes	panier_moyen	chiffre_affaires	satisfaction	delai_moyen
Site	148	59.50	8807.0	3.79	4.70
Boutique	114	74.81	8528.0	4.49	0.00
Réseaux	138	49.01	6764.0	3.72	4.49

En une requête de six lignes, nous avons la photographie de l'activité. Le **site** fait le plus de chiffre d'affaires parce qu'il a le plus de commandes ; la **boutique**, avec moins de commandes, a le panier moyen et la satisfaction les plus élevés (le délai de livraison y est nul : les clients repartent avec leur achat). Le canal **Réseaux** a les paniers les plus modestes.

La règle du GROUP BY. Dans un SELECT avec GROUP BY, chaque colonne affichée doit être **soit dans le GROUP BY, soit à l'intérieur d'une fonction d'agrégation**. Demander `SELECT canal, montant ... GROUP BY canal` n'a pas de sens : quelle valeur de montant choisir parmi les 148 lignes du groupe « Site » ? (SQLite en choisit une, arbitrairement, sans protester ; PostgreSQL refuse avec une erreur : c'est plus prudent.)

WHERE contre HAVING. WHERE filtre les **lignes avant** le regroupement ; HAVING filtre les **groupes après** l'agrégation. On ne peut pas écrire `WHERE COUNT(*) > 10`, car le décompte n'existe pas encore à ce moment-là.

```
SELECT ville, COUNT(*) AS clients
FROM clients
GROUP BY ville
HAVING COUNT(*) >= 11
ORDER BY clients DESC, ville;
```

ville	clients
Ville A	13
Ville B	11
Ville G	11
Ville H	11

Un usage très pratique de `GROUP BY ... HAVING` : **détecter les doublons**. Reprenons notre remarque du 5.1.2 : peut-on identifier un client par son prénom et son nom ? Cherchons les couples qui apparaissent plus d'une fois (`GROUP_CONCAT` recolle les villes en un seul texte) :

```
SELECT prenom, nom, COUNT(*) AS homonymes, GROUP_CONCAT(ville, ' / ') AS villes
FROM clients
GROUP BY prenom, nom
HAVING COUNT(*) > 1
ORDER BY homonymes DESC, nom;
```

prenom	nom	homonymes	villes
Léa	Fontaine	3	Ville A / Ville H / Ville E
Théo	Simon	3	Ville A / Ville C / Ville F
Anna	Faure	2	Ville B / Ville B
Zoé	Garcia	2	Ville G / Ville F
Elsa	Girard	2	Ville D / Ville H
Yann	Lambert	2	Ville C / Ville A
Lou	Michel	2	Ville H / Ville G
Zoé	Moreau	2	Ville E / Ville B

Huit couples de prénom et nom apparaissent plusieurs fois, dont trois « Léa Fontaine » ! Et deux « Anna Faure » habitent la même ville : sans le numéro `id_client`, il serait **impossible** de les distinguer. La clé primaire n'est pas un luxe.

COUNT(*) ou COUNT(colonne) ? `COUNT(*)` compte les **lignes**. `COUNT(colonne)` compte les lignes où la colonne **n'est pas vide** (NULL) : nous y reviendrons au 5.2.7, c'est un grand classique des erreurs de comptage.

5.2.5 Les jointures : relier les tables

Le problème : la table `commandes` dit *quel numéro de client* a passé chaque commande, mais pas son nom ni sa ville. Pour les afficher ensemble, il faut **joindre** les deux tables.

Une jointure est un produit cartésien filtré. Reprenons l'algèbre relationnelle du 5.1.2 : $R \bowtie_{\text{cond}} S = \sigma_{\text{cond}}(R \times S)$. Voyons-le concrètement. Le produit cartésien « clients × commandes » forme **toutes les paires possibles** (une cliente, une commande), même absurdes :

```
SELECT COUNT(*) AS paires_possibles
FROM clients CROSS JOIN commandes;
```

```
paires_possibles
32000
```

Il y a bien $80 \times 400 = 32\,000$ paires. Mais seules **400** sont « les bonnes » : celles où le numéro de client de la commande est celui de la cliente. En filtrant, on obtient la jointure :

```
SELECT COUNT(*) AS paires_valides
FROM clients CROSS JOIN commandes
WHERE clients.id_client = commandes.id_client;
```

```
paires_valides
400
```

C'est exactement ce que fait `JOIN ... ON` (qu'on écrit ainsi pour la lisibilité, et parce que le moteur l'exécute beaucoup plus finement que de fabriquer les 32 000 paires). L'équivalent :

```
SELECT cl.prenom, cl.nom, cl.ville, c.date_commande, c.montant
FROM commandes AS c
JOIN clients AS cl ON cl.id_client = c.id_client
ORDER BY c.montant DESC
LIMIT 5;
```

prenom	nom	ville	date_commande	montant
Jules	Garcia	Ville G	2025-06-23	255.7
Lou	Simon	Ville F	2025-03-28	243.8
Nina	Faure	Ville H	2025-07-31	217.1
Sam	Fontaine	Ville A	2025-08-26	212.4
Sam	Fontaine	Ville A	2025-12-16	208.8

Deux détails de lisibilité : on donne des **alias courts aux tables** (`c`, `cl`), et on **préfixe** chaque colonne par la table d'où elle vient (`cl.nom`), ce qui évite les ambiguïtés quand deux tables ont une colonne du même nom (comme `id_client`).

Lire une jointure. `JOIN clients AS cl ON cl.id_client = c.id_client` se lit : « pour chaque commande `c`, va chercher la ligne de `clients` dont le numéro est `c.id_client` et accole ses colonnes ». Toujours : **clé étrangère = clé primaire**.

Joindre plus de deux tables se fait en enchaînant les `JOIN`. Quel est le chiffre d'affaires par **catégorie de produit** ? Il faut aller de `lignes_commande` à `produits`, puis à `categories` :

```
SELECT ca.nom                                AS categorie,
       SUM(l.quantite)                       AS unites_vendues,
       ROUND(SUM(l.quantite * l.prix_unitaire)) AS chiffre_affaires
FROM lignes_commande AS l
JOIN produits AS p ON p.id_produit = l.id_produit
JOIN categories AS ca ON ca.id_categorie = p.id_categorie
GROUP BY ca.nom
ORDER BY chiffre_affaires DESC;
```

categorie	unites_vendues	chiffre_affaires
Bijoux	200	7006.0
Textile	184	6994.0
Poterie	180	6950.0
Cosmétiques	196	3148.0

Les trois premières catégories sont presque à égalité (les **bijoux** devançant de justesse le **textile** et la **poterie**, autour de 7 000 € chacun) ; les **cosmétiques** se vendent en grand nombre (196 unités, presque autant que les bijoux avec 200) mais à petits prix, donc leur total est bien plus faible. Ce genre d'écart entre « ce qui se vend le plus » et « ce qui rapporte le plus » est exactement le type d'information qu'on cherche.

Souvenez-vous aussi de la promesse faite en 5.1.3 : les lignes de commande devaient avoir été construites pour que chaque commande « retombe » sur son montant. Vérifions-le, avec une requête qui compte les commandes **incohérentes** (dont le montant diffère de la somme de leurs lignes) :

```
SELECT COUNT(*) AS commandes_incoherentes
FROM (
  SELECT c.id_commande
  FROM commandes AS c
  JOIN lignes_commande AS l ON l.id_commande = c.id_commande
  GROUP BY c.id_commande
  HAVING ABS(c.montant - SUM(l.quantite * l.prix_unitaire)) > 0.005
```

```
);
```

```
commandes_incoherentes
0
```

Zéro : tout est cohérent. Ce type de **contrôle de cohérence** (on appelle cela un test de qualité de données) est un réflexe à avoir à chaque fois qu'on reçoit une base. Vous avez aussi vu une **sous-requête** (la requête entre parenthèses dans le FROM) ; nous y revenons au 5.2.6.

INNER JOIN contre LEFT JOIN. Le JOIN simple (en réalité INNER JOIN) ne garde que les lignes qui **ont une correspondance des deux côtés**. Un client qui n'a jamais commandé **disparaît** du résultat ! Si l'on veut garder *tous* les clients, même sans commande, on utilise LEFT JOIN : toutes les lignes de la table de **gauche** sont conservées, et les colonnes de la table de droite sont remplies de NULL quand il n'y a pas de correspondance.

```
SELECT cl.id_client, cl.prenom, cl.nom, cl.ville
FROM clients AS cl
LEFT JOIN commandes AS c ON c.id_client = cl.id_client
WHERE c.id_commande IS NULL
ORDER BY cl.id_client
LIMIT 6;
```

id_client	prenom	nom	ville
10	Lou	Michel	Ville H
23	Hugo	Girard	Ville G
31	Yann	Simon	Ville E
38	Théo	Faure	Ville D
42	Hugo	Garcia	Ville B
56	Lina	Fontaine	Ville B

Les six premières lignes ci-dessus sont les premiers des quatorze clients qui se sont inscrits mais n'ont jamais acheté : une liste précieuse pour une campagne de relance (« votre premier achat à -10 % »). Le motif **LEFT JOIN ... WHERE droite IS NULL** (« les lignes de gauche sans correspondance à droite ») est l'un des plus utiles de tout SQL. Avec un INNER JOIN, ces quatorze clients n'auraient jamais été trouvés, puisqu'ils n'ont aucune commande à joindre.

Les quatre jointures, en une phrase. INNER : l'intersection (couples valides seulement). LEFT : tous les couples valides, plus les lignes de gauche sans partenaire (complétées par NULL). RIGHT : l'inverse (rarement utilisé : on échange simplement les deux tables). FULL : les deux. SQLite gère INNER, LEFT, CROSS et, depuis la version 3.39, RIGHT et FULL.

L'auto-jointure. Rien n'interdit de joindre une table **avec elle-même** (en lui donnant deux alias différents). C'est ce qu'il faut pour afficher, pour chaque client parrainé, le nom de son parrain :

```
SELECT f.prenom || ' ' || f.nom AS filleul,
       p.prenom || ' ' || p.nom AS parrain
FROM clients AS f
JOIN clients AS p ON p.id_client = f.id_parrain
ORDER BY f.id_client
LIMIT 5;
```

filleul	parrain
Luc Garcia	Paul Martin
Zoé Laurent	Hugo Lefebvre
Théo Garcia	Paul Martin
Jules Garcia	Lou Michel
Alex Lefebvre	Anna Michel

(L'opérateur || recolle deux textes.) Imaginez la même table lue deux fois : une fois dans le rôle

des filleuls (f), une fois dans celui des parrains (p).

Les opérations ensemblistes UNION (réunion sans doublon), UNION ALL (réunion en gardant les doublons), INTERSECT (intersection) et EXCEPT (différence) combinent **deux requêtes de même forme**. Combien de clients ont acheté à la fois en boutique et sur le site ?

```
SELECT COUNT(*) AS clients_boutique_et_site
FROM (
    SELECT id_client FROM commandes WHERE canal = 'Boutique'
    INTERSECT
    SELECT id_client FROM commandes WHERE canal = 'Site'
);
```

```
clients_boutique_et_site
21
```

5.2.6 Les sous-requêtes : une requête dans une requête

Une **sous-requête** est une requête placée entre parenthèses à l'intérieur d'une autre. Trois usages courants.

(a) **Une valeur unique**, utilisable comme un nombre : « les commandes dont le montant dépasse la moyenne ».

```
SELECT COUNT(*) AS commandes_au-dessus_de_la_moyenne,
    ROUND(AVG(montant), 2) AS leur_montant_moyen
FROM commandes
WHERE montant > (SELECT AVG(montant) FROM commandes);
```

```
commandes_au-dessus_de_la_moyenne  leur_montant_moyen
160                                95.12
```

La sous-requête (SELECT AVG(montant) FROM commandes) vaut 60,25 ; la requête externe garde donc les commandes au-dessus de ce seuil. Il y en a **160 sur 400** : 40 %, bien moins que la moitié, signe d'une distribution asymétrique à droite (la moyenne dépasse la médiane, 3.1.3).

(b) **Une liste de valeurs**, avec IN : « les clients qui ont passé au moins une commande de plus de 200 € ».

```
SELECT cl.id_client, cl.prenom, cl.nom
FROM clients AS cl
WHERE cl.id_client IN (SELECT id_client FROM commandes WHERE montant > 200);
```

```
id_client prenom      nom
2         Sam Fontaine
14        Jules Garcia
39        Lou Simon
71        Nina Faure
```

(c) **Une condition d'existence**, avec EXISTS : « les clients ayant déjà acheté en boutique ». La sous-requête est ici **corrélée** : elle fait référence à la ligne courante de la requête externe (clients.id_client).

```
SELECT COUNT(*) AS clients_deja_venus_en_boutique
FROM clients
WHERE EXISTS (SELECT 1 FROM commandes AS c
    WHERE c.id_client = clients.id_client AND c.canal = 'Boutique');
```

```
clients_deja_venus_en_boutique
26
```

Quand préférer une jointure à une sous-requête ? Les deux marchent souvent. Une règle simple : si vous avez besoin des colonnes de l'autre table dans le résultat, **jointure**. Si l'autre table ne sert que de **filtre** (existence, appartenance), **EXISTS** / **IN**. Les CTE (5.3) rendront les requêtes imbriquées beaucoup plus lisibles.

5.2.7 Les pièges : NULL et dates

Deux sources d'erreurs reviennent sans cesse dans les requêtes des débutants (et des autres) : les valeurs **absentes** (NULL), qui suivent une logique à trois valeurs que l'intuition ne devine pas, et les **dates**, qui sont des textes déguisés. Cette sous-section les traite l'une après l'autre.

Le NULL : ni zéro, ni texte vide, mais « inconnu »

NULL signifie « **valeur absente ou inconnue** ». Ce n'est pas zéro, ce n'est pas le texte vide. Et il suit une logique à **trois valeurs** : vrai, faux et *inconnu*. Toute comparaison avec NULL donne *inconnu* : NULL = NULL n'est pas vrai (deux inconnues sont-elles égales ? On ne sait pas !), NULL <> 5 n'est pas vrai non plus. Et WHERE ne garde que les lignes **vraies**.

Dans notre base, la colonne telephone contient des NULL (le client n'a pas donné son numéro). Dénombrons :

```
SELECT COUNT(*)           AS clients,
       COUNT(telephone)   AS avec_telephone,
       SUM(telephone IS NULL) AS sans_telephone
FROM clients;
```

clients	avec_telephone	sans_telephone
80	65	15

COUNT(*) compte les 80 lignes ; COUNT(telephone) ignore les NULL et n'en compte que 65. (Dans SQLite, telephone IS NULL vaut 1 ou 0, d'où la somme.) La **bonne** façon de tester l'absence de valeur est IS NULL (ou IS NOT NULL). Voici ce qui arrive si l'on écrit naïvement = NULL :

```
SELECT COUNT(*) AS avec_egal_null
FROM clients
WHERE telephone = NULL;
```

avec_egal_null
0

Résultat : **zéro**, alors qu'il y a des clients sans téléphone, et aucun message d'erreur. Un deuxième piège, plus sournois : le **filtre « différent de »** qui oublie les inconnus.

```
SELECT (SELECT COUNT(*) FROM clients WHERE telephone <> '7725588') AS differents,
       (SELECT COUNT(*) FROM clients WHERE telephone <> '7725588' OR telephone IS NULL) AS
       differents_ou_inconnus;
```

differents	differents_ou_inconnus
64	79

Parmi les 80 clients, un seul a le numéro 7725588 ; on s'attendrait donc à 79 « autres » clients. Le filtre <> n'en trouve que **64** : les quinze clients sans téléphone ont disparu, car pour eux la condition n'est pas « vraie » mais « inconnue ». Il faut ajouter explicitement OR telephone IS NULL.

Les fonctions d'agrégation, elles, **ignorent** les NULL : AVG(id_parrain) ne moyennise que les clients parrainés (ce qui d'ailleurs n'a aucun sens, un numéro n'est pas une quantité). Pour

remplacer un NULL par une valeur par défaut, on utilise **COALESCE**, qui renvoie son premier argument non vide :

```
SELECT id_client, prenom, COALESCE(telphone, '(non renseigné)') AS telephone
FROM clients
LIMIT 4;
```

id_client	prenom	telephone
1	Yann	7725588
2	Sam	(non renseigné)
3	Adam	(non renseigné)
4	Anna	(non renseigné)

Enfin, le piège **le plus dangereux** du chapitre : **NOT IN** avec une sous-requête qui contient un NULL. Cherchons les clients qui **n'ont parrainé personne**, c'est-à-dire dont le numéro n'apparaît jamais dans la colonne `id_parrain` :

```
SELECT (SELECT COUNT(*) FROM clients
WHERE id_client NOT IN (SELECT id_parrain FROM clients)) AS naif,
(SELECT COUNT(*) FROM clients
WHERE id_client NOT IN (SELECT id_parrain FROM clients WHERE id_parrain IS NOT NULL)) AS
↳ corrige,
(SELECT COUNT(*) FROM clients AS c
WHERE NOT EXISTS (SELECT 1 FROM clients AS f WHERE f.id_parrain = c.id_client)) AS
↳ avec_not_exists;
```

naif	corrige	avec_not_exists
0	55	55

La version naïve donne **zéro** (aucun client n'est « sans filleul » ? C'est absurde) ; les deux autres donnent 55. Pourquoi ? La liste (`SELECT id_parrain FROM clients`) contient des NULL (tous les clients non parrainés). Or `x NOT IN (a, b, NULL)` se lit `x <> a AND x <> b AND x <> NULL` ; le dernier terme est *inconnu*, donc le tout n'est jamais vrai, quelle que soit la valeur de `x`. Le filtre élimine toutes les lignes sans le moindre avertissement.

Règle pratique. Pour exprimer « n'est pas dans l'autre table », utilisez **NOT EXISTS** (insensible aux NULL) ou **LEFT JOIN ... IS NULL**, ou à défaut ajoutez `WHERE col IS NOT NULL` dans la sous-requête. Évitez **NOT IN** (sous-requête) sur une colonne qui peut contenir des NULL.

Les dates

Comme dit en 5.1.4, SQLite stocke les dates en texte ISO 8601 ; on les manipule avec les fonctions `strftime`, `date` et `julianday`. Le premier réflexe de l'analyste : **regrouper par mois**.

```
SELECT strftime('%Y-%m', date_commande) AS mois,
COUNT(*) AS commandes,
ROUND(SUM(montant)) AS chiffre_affaires
FROM commandes
GROUP BY mois
ORDER BY mois;
```

mois	commandes	chiffre_affaires
2025-01	16	996.0
2025-02	20	1168.0
2025-03	25	1687.0
2025-04	36	1843.0
2025-05	36	2314.0
2025-06	33	2491.0
2025-07	42	2571.0
2025-08	44	2435.0

2025-09	31	1713.0
2025-10	20	1272.0
2025-11	40	2284.0
2025-12	57	3325.0

`strftime('%Y-%m', ...)` extrait « année-mois » (%Y année, %m mois, %d jour, %w jour de la semaine, 0 = dimanche). On voit la saisonnalité : un creux en janvier-février, une belle période estivale, un petit creux en octobre, et le pic des fêtes en **décembre** (57 commandes, 3 325 €).

Autres opérations utiles : `date('2025-12-31', '-90 days')` (soustraire une durée), `julianday(d2) - julianday(d1)` (nombre de jours entre deux dates).

```
SELECT date('2025-12-31', '-90 days')      AS il_y_a_90_jours,
       date('2025-03-15', 'start of month') AS debut_du_mois,
       CAST(julianday('2025-12-31') - julianday('2025-01-01') AS INTEGER) AS jours_ecoules;
```

il_y_a_90_jours	debut_du_mois	jours_ecoules
2025-10-02	2025-03-01	364

N'utilisez jamais `date('now')` dans une analyse censée être reproductible. 'now' change tous les jours : votre requête donnera un résultat différent demain, et nul ne pourra la rejouer. Dans ce livre, la « date du jour » est **fixée** au **31 décembre 2025** (date d'arrêt de la base). En entreprise, on passe la date d'arrêt en paramètre.

5.2.8 CASE WHEN : classer et croiser

CASE WHEN est le « si... alors... sinon » de SQL. Il sert à créer des **classes** à partir d'une variable numérique, et, combiné avec SUM, à fabriquer des **tableaux croisés**. Voici la première utilisation : répartir les commandes selon leur délai de livraison, pour voir si les clients en retard sont moins satisfaits.

```
SELECT CASE WHEN delai_livraison = 0 THEN '0 j (retrait)'
           WHEN delai_livraison <= 3 THEN '1 à 3 j'
           WHEN delai_livraison <= 6 THEN '4 à 6 j'
           ELSE '7 j et plus' END      AS delai,
       COUNT(*)                       AS commandes,
       ROUND(AVG(satisfaction), 2)    AS satisfaction_moyenne
FROM commandes
GROUP BY delai
ORDER BY MIN(delai_livraison);
```

delai	commandes	satisfaction_moyenne
0 j (retrait)	114	4.49
1 à 3 j	88	4.03
4 à 6 j	156	3.76
7 j et plus	42	3.17

La satisfaction **baisse régulièrement** avec le délai : de 4,49 pour un retrait en boutique à 3,17 au-delà d'une semaine. C'est exactement la relation que nous avons mise en évidence par la corrélation au chapitre 3, retrouvée ici avec de simples agrégations SQL. (Attention, comme toujours, à ne pas conclure trop vite à la causalité : le retrait en boutique diffère des commandes en ligne à bien d'autres égards.)

La seconde utilisation, `SUM(CASE WHEN canal = 'Site' THEN montant ELSE 0 END)`, place chaque canal dans sa propre colonne : c'est un **tableau croisé** (*pivot*). SQL n'en a pas de commande universelle, et cette astuce suffit presque toujours.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.3 (les dix meilleurs clients), 5.4 (chiffre d'affaires par mois et par canal) et 5.5 (clients dormants), exercices 5.2 à 5.7.

À retenir

- Ordre d'écriture : SELECT, FROM, JOIN, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT. Ordre d'exécution : FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.
- WHERE filtre les **lignes** (avant regroupement) ; HAVING filtre les **groupes** (après).
- Une **jointure** est un produit cartésien filtré par clé étrangère = clé primaire. LEFT JOIN conserve les lignes sans correspondance : c'est l'outil pour trouver « ceux qui n'ont jamais... ».
- NULL = « inconnu » : on le teste avec IS NULL, jamais avec =. COUNT(col) ignore les NULL. Méfiez-vous de NOT IN quand la liste peut contenir un NULL.
- Dates en ISO 8601, strftime pour regrouper, jamais de date('now') dans une analyse reproductible.
- Mettez des **parenthèses** dès que AND et OR se mélangent.

5.3 Fonctions fenêtres et CTE

Intuition. Avec GROUP BY (5.2.4), on **écrase** les lignes : cent commandes deviennent une seule ligne « total ». Mais souvent on veut garder chaque commande **et**, à côté, la comparer au reste : « cette commande est-elle au-dessus du panier moyen de son canal ? », « quel est son rang ? », « combien de jours depuis la commande précédente du même client ? ». Une **fonction fenêtre** calcule un résultat sur un *groupe de lignes voisines*, **sans réduire le nombre de lignes**. C'est l'outil qui distingue l'analyste débutant de l'analyste expérimenté.

5.3.1 Le principe : fonction(...) OVER (...)

Reprenons les commandes. On aimerait voir, pour chaque commande, **le panier moyen de son canal** et l'écart entre les deux. Avec GROUP BY, on obtiendrait 3 lignes (une par canal) et on perdrait les commandes. Avec une fenêtre :

```
SELECT id_commande, canal, montant,
       ROUND(AVG(montant) OVER (PARTITION BY canal), 2) AS moyenne_du_canal,
       ROUND(montant - AVG(montant) OVER (PARTITION BY canal), 2) AS ecart
FROM commandes
ORDER BY id_commande
LIMIT 6;
```

id_commande	canal	montant	moyenne_du_canal	ecart
1	Boutique	44.8	74.81	-30.01
2	Site	34.5	59.50	-25.00
3	Réseaux	88.2	49.01	39.19
4	Réseaux	30.1	49.01	-18.91
5	Boutique	110.1	74.81	35.29
6	Site	39.8	59.50	-19.70

Chaque commande est toujours là, et trois colonnes se sont ajoutées. Par exemple, la commande n° 1 (boutique, 44,80 €) est 30,01 € **en dessous** du panier moyen de la boutique (74,81 €), alors que la commande n° 3 (Réseaux, 88,20 €) est 39,19 € **au-dessus** de celui du canal Réseaux (49,01

€). Une commande de 88 € est « grosse » sur le canal Réseaux mais « moyenne » en boutique : l'écart à son propre canal est plus parlant que le montant brut.

La syntaxe est toujours : **fonction(...)** **OVER (PARTITION BY ... ORDER BY ... cadre)**.

Morceau	Rôle	Analogie
fonction(...)	le calcul (AVG, SUM, RANK, LAG...)	ce qu'on mesure
PARTITION BY	découpe en groupes indépendants (facultatif ; sans lui, une seule partition = toute la table)	« à l'intérieur de chaque canal »
ORDER BY	ordonne les lignes dans chaque partition	« du plus ancien au plus récent »
cadre (ROWS BETWEEN ...)	quelles lignes voisines sont prises en compte pour la ligne courante	« les 3 dernières lignes »

Définition précise. Pour **chaque ligne** r , la base détermine sa **partition** $P(r)$ (les lignes qui ont la même valeur de PARTITION BY), les trie selon ORDER BY, puis retient un **cadre** $F(r) \subseteq P(r)$ (par exemple « de la première ligne de la partition jusqu'à r »). Le résultat de la ligne r est $f(F(r))$: la fonction appliquée à ce sous-ensemble. GROUP BY est le cas dégénéré où toutes les lignes d'une partition reçoivent **le même** résultat, qu'on réduit alors à une ligne. Le nombre de lignes du résultat d'une fenêtre, lui, est **le même** qu'en entrée.

5.3.2 Les classements : ROW_NUMBER, RANK, DENSE_RANK

Trois fonctions numérotent les lignes d'une partition, selon l'ordre demandé. Elles diffèrent **seulement** par leur façon de traiter les **ex æquo**. Un exemple minimal, avec un classement de cinq clients selon leur note de satisfaction (deux notes à 5, deux à 3), construit directement avec VALUES :

```
WITH notes(client, note) AS (
  VALUES ('Léa', 5), ('Noé', 5), ('Mia', 4), ('Hugo', 3), ('Zoé', 3)
)
SELECT client, note,
  ROW_NUMBER() OVER (ORDER BY note DESC) AS row_number,
  RANK() OVER (ORDER BY note DESC) AS rank,
  DENSE_RANK() OVER (ORDER BY note DESC) AS dense_rank
FROM notes;
```

client	note	row_number	rank	dense_rank
Léa	5	1	1	1
Noé	5	2	1	1
Mia	4	3	3	2
Hugo	3	4	4	3
Zoé	3	5	4	3

Lisez-le colonne par colonne :

- ROW_NUMBER : 1, 2, 3, 4, 5. Aucun ex æquo n'est reconnu : le départage entre Léa et Noé est **arbitraire** (si vous voulez un résultat reproductible, ajoutez un second critère d'ordre).
- RANK : 1, 1, 3, 4, 4. Les ex æquo partagent le même rang, et **le rang suivant saute** (comme aux Jeux olympiques : deux médailles d'or, pas d'argent, puis le bronze).
- DENSE_RANK : 1, 1, 2, 3, 3. Les rangs sont **consécutifs**, sans trou.

(Au passage : `WITH notes(...) AS (...)` est une **CTE**, nous l'étudions au 5.3.5.) Voici un cas réel : le classement des **trois plus grosses commandes de chaque canal**. Le plus naturel serait d'écrire `WHERE ROW_NUMBER() OVER (...) <= 3`, mais c'est **interdit** : le `WHERE` s'exécute *avant* le calcul des fenêtres (voir l'ordre d'exécution au 5.2.1). On calcule donc le rang dans une requête intérieure, et on filtre dans la requête extérieure :

```
SELECT canal, rang, id_commande, date_commande, montant
FROM (
    SELECT canal, id_commande, date_commande, montant,
           ROW_NUMBER() OVER (PARTITION BY canal ORDER BY montant DESC) AS rang
    FROM commandes
)
WHERE rang <= 3
ORDER BY canal, rang;
```

canal	rang	id_commande	date_commande	montant
Boutique	1	243	2025-08-26	212.4
Boutique	2	362	2025-12-16	208.8
Boutique	3	115	2025-05-15	189.2
Réseaux	1	140	2025-06-10	166.1
Réseaux	2	59	2025-03-27	159.9
Réseaux	3	125	2025-05-23	127.3
Site	1	157	2025-06-23	255.7
Site	2	61	2025-03-28	243.8
Site	3	208	2025-07-31	217.1

Ce motif (« **top N par groupe** ») est l'un des plus fréquents en entretien d'embauche comme en entreprise : « les 3 meilleurs vendeurs par région », « le dernier achat de chaque client » (`ROW_NUMBER() ... ORDER BY date DESC, puis rang = 1`).

5.3.3 Cumuls et moyennes mobiles : le cadre de la fenêtre

Ajoutons `ORDER BY` dans la fenêtre d'une somme : on obtient une **somme cumulée**. Voici l'évolution du chiffre d'affaires mensuel : le total mois par mois, le **cumul depuis janvier**, et une **moyenne mobile sur trois mois** (le mois courant et les deux précédents), qui lisse les fluctuations pour mieux voir la tendance.

```
WITH mensuel AS (
    SELECT strftime('%Y-%m', date_commande) AS mois,
           ROUND(SUM(montant)) AS ca
    FROM commandes
    GROUP BY mois
)
SELECT mois, ca,
       SUM(ca) OVER (ORDER BY mois) AS cumul,
       ROUND(AVG(ca) OVER (ORDER BY mois ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)) AS
       ↳ moyenne_mobile_3_mois
FROM mensuel
ORDER BY mois;
```

mois	ca	cumul	moyenne_mobile_3_mois
2025-01	996.0	996.0	996.0
2025-02	1168.0	2164.0	1082.0
2025-03	1687.0	3851.0	1284.0
2025-04	1843.0	5694.0	1566.0
2025-05	2314.0	8008.0	1948.0
2025-06	2491.0	10499.0	2216.0
2025-07	2571.0	13070.0	2459.0
2025-08	2435.0	15505.0	2499.0
2025-09	1713.0	17218.0	2240.0
2025-10	1272.0	18490.0	1807.0

2025-11	2284.0	20774.0	1756.0
2025-12	3325.0	24099.0	2294.0

La clause **ROWS BETWEEN 2 PRECEDING AND CURRENT ROW** définit le cadre : « les deux lignes précédentes plus la ligne courante ». En janvier, il n'y a pas de ligne précédente : la « moyenne sur trois mois » n'utilise qu'une valeur (996), puis deux en février. Dans un rapport sérieux, on masquerait ces deux premières valeurs.

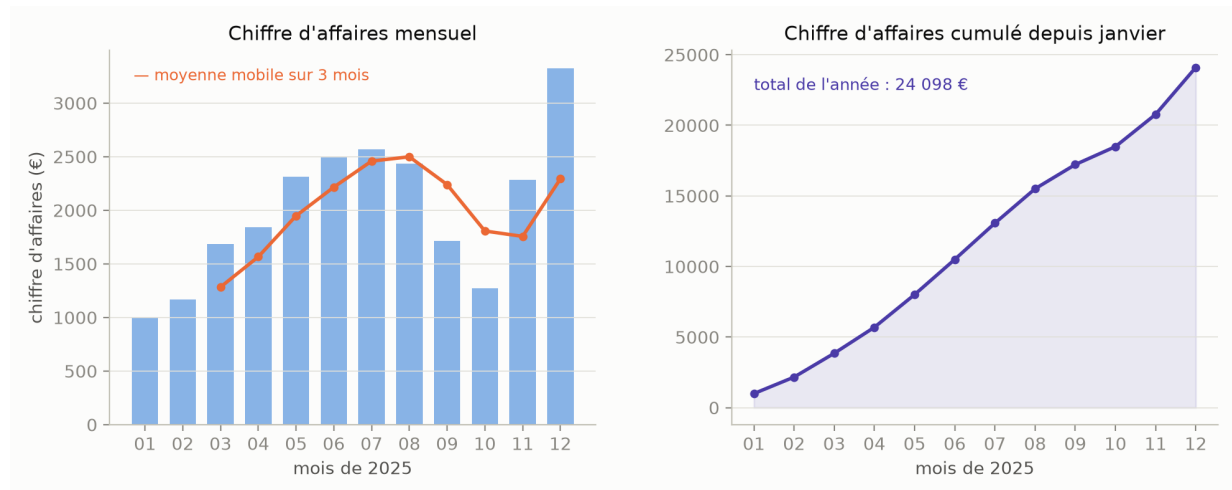


Figure 34: À gauche : chiffre d'affaires mensuel de la boutique en 2025 (barres) et sa moyenne mobile sur trois mois (courbe orange). À droite : chiffre d'affaires cumulé depuis janvier. Les données sont celles de la requête précédente.

Le graphique fait apparaître ce que les chiffres cachent : la moyenne mobile (courbe orange) gomme le creux d'octobre et la remontée de décembre, mais montre bien la **tendance** : une montée jusqu'à l'été, un repli à l'automne, puis un rebond de fin d'année. Le cumul atteint 24 099 € en décembre (à l'arrondi près : chaque mois a été arrondi au euro avant d'être additionné ; le vrai total est 24 098,30 €, comme le montre le graphique de droite).

Piège des ex æquo avec ORDER BY. Quand on écrit `SUM(...) OVER (ORDER BY ...)` sans préciser de cadre, le cadre par défaut est `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, et `RANGE` regroupe toutes les lignes **ex æquo** sur la colonne de tri : elles reçoivent le **même** cumul. Illustration sur les six premières commandes, dont deux (n° 2 et n° 3) sont du même jour :

```
SELECT id_commande, date_commande, montant,
       SUM(montant) OVER (ORDER BY date_commande) AS cumul_range,
       SUM(montant) OVER (ORDER BY date_commande, id_commande ROWS UNBOUNDED PRECEDING) AS
       ↳ cumul_ligne_a_ligne
FROM commandes
WHERE id_commande <= 6
ORDER BY date_commande, id_commande;
```

id_commande	date_commande	montant	cumul_range	cumul_ligne_a_ligne
1	2025-01-03	44.8	44.8	44.8
2	2025-01-04	34.5	167.5	79.3
3	2025-01-04	88.2	167.5	167.5
4	2025-01-05	30.1	197.6	197.6
5	2025-01-08	110.1	307.7	307.7
6	2025-01-09	39.8	347.5	347.5

Les commandes 2 et 3 (même jour) affichent le **même** `cumul_range` : la somme des deux d'un

coup, ce qui n'est pas un « cumul ligne à ligne ». Pour un cumul ligne à ligne, précisez toujours **ROWS** et un ordre complet (qui départage les ex æquo, ici par `id_commande`). C'est une des rares occasions où une fenêtre sans cadre explicite donne un résultat inattendu.

5.3.4 Comparer avec la ligne précédente : LAG et LEAD

`LAG(colonne)` renvoie la valeur de la ligne **précédente** (dans l'ordre de la fenêtre) ; `LEAD(colonne)`, de la ligne **suivante**. C'est l'outil des évolutions : « par rapport au mois dernier », « depuis la dernière commande ». Commençons par la **croissance mensuelle du chiffre d'affaires**, en pourcentage :

```
WITH mensuel AS (
    SELECT strftime('%Y-%m', date_commande) AS mois,
           ROUND(SUM(montant))              AS ca
    FROM commandes
    GROUP BY mois
)
SELECT mois, ca,
       LAG(ca) OVER (ORDER BY mois)              AS mois_precedent,
       ROUND(100.0 * (ca - LAG(ca) OVER (ORDER BY mois)) / LAG(ca) OVER (ORDER BY mois), 1) AS
       ↪ evolution_pct
FROM mensuel
ORDER BY mois;
```

mois	ca	mois_precedent	evolution_pct
2025-01	996.0	NaN	NaN
2025-02	1168.0	996.0	17.3
2025-03	1687.0	1168.0	44.4
2025-04	1843.0	1687.0	9.2
2025-05	2314.0	1843.0	25.6
2025-06	2491.0	2314.0	7.6
2025-07	2571.0	2491.0	3.2
2025-08	2435.0	2571.0	-5.3
2025-09	1713.0	2435.0	-29.7
2025-10	1272.0	1713.0	-25.7
2025-11	2284.0	1272.0	79.6
2025-12	3325.0	2284.0	45.6

La première ligne n'a pas de précédent : `LAG` renvoie `NULL` (que `pandas` affiche `NaN`). Les plus fortes hausses sont en novembre (+79,6 %), en décembre (+45,6 %) et en mars (+44,4 %), la plus forte baisse en septembre (-29,7 %).

Une deuxième utilisation, plus riche : **le délai entre deux commandes successives du même client**. `LAG(date_commande)` calculé *dans la partition du client* donne la date de sa commande précédente (les dates sont en texte ISO, `julianday` les convertit en nombres de jours). Combien de jours séparent en moyenne deux achats successifs d'un même client ? Nous calculons d'abord les intervalles, puis nous les résumons :

```
WITH achats AS (
    SELECT id_client, date_commande,
           LAG(date_commande) OVER (PARTITION BY id_client ORDER BY date_commande, id_commande) AS
           ↪ precedente
    FROM commandes
)
SELECT COUNT(*)              AS intervalles,
       ROUND(AVG(julianday(date_commande) - julianday(precedente)), 1) AS delai_moyen_jours,
       MIN(julianday(date_commande) - julianday(precedente))          AS minimum,
       MAX(julianday(date_commande) - julianday(precedente))          AS maximum
FROM achats
WHERE precedente IS NOT NULL;
```

intervalles	delai_moyen_jours	minimum	maximum
334	40.2	0.0	320.0

334 intervalles : 400 commandes moins 66 « premières commandes » (une par client actif, qui n'ont pas de précédente). En moyenne, **40 jours** séparent deux achats d'un même client, avec des extrêmes allant de 0 (deux commandes le même jour) à 320 jours. C'est exactement l'ingrédient d'un indicateur classique du commerce : le **cycle de réachat**. Il guide, par exemple, la date idéale d'un courriel de relance : passé 40 jours sans achat, un client est déjà « en retard » par rapport à la normale.

5.3.5 Les CTE : donner un nom à une étape

Vous les avez déjà croisées dans ce chapitre. Une **CTE** (*Common Table Expression*) est une requête **nommée**, déclarée en tête avec `WITH nom AS ( ... )`, que l'on utilise ensuite comme si c'était une table. Elle ne change **rien** au résultat par rapport à une sous-requête imbriquée (5.2.6) : elle change la **lisibilité**. Comparez, pour la même question, l'imbrication :

```
SELECT ... FROM (SELECT ... FROM (SELECT ... FROM commandes GROUP BY ...) GROUP BY ...) WHERE ...
```

et le découpage en étapes nommées :

```
WITH etape1 AS (SELECT ... FROM commandes GROUP BY ...),
     etape2 AS (SELECT ... FROM etape1 ...)
SELECT ... FROM etape2 WHERE ...
```

La deuxième forme se lit **de haut en bas**, comme un script : on peut vérifier chaque étape isolément, ce qui est précieux pour le débogage. On peut enchaîner plusieurs CTE (séparées par des virgules) ; chacune peut utiliser celles qui précèdent.

Deux remarques pratiques. (1) Une CTE n'est visible que **dans la requête** qui la déclare : elle n'est pas enregistrée dans la base (pour cela, il existe les vues, 5.4.4). (2) Rien n'oblige à découper : une requête simple n'a pas besoin de CTE. La règle est celle de la lisibilité : dès que vous imbriquez deux niveaux de sous-requêtes, ou que vous avez besoin d'utiliser le même résultat intermédiaire à deux endroits, nommez l'étape.

5.3.6 Les CTE récursives : des requêtes qui se rappellent elles-mêmes

Une CTE peut **se référencer elle-même** : c'est une CTE **récursive** (`WITH RECURSIVE`). Elle comporte toujours deux parties reliées par `UNION ALL` : un **cas de base** (le point de départ) et un **pas récursif** (comment passer de la ligne précédente à la suivante), avec une condition d'arrêt. Comme en Python, il faut un cas de base *et* une condition d'arrêt, faute de quoi la requête tourne sans fin.

L'exemple le plus simple : générer les carrés des entiers de 1 à 5.

```
WITH RECURSIVE n(i) AS (
    SELECT 1                                -- cas de base : on part de 1
    UNION ALL
    SELECT i + 1 FROM n WHERE i < 5        -- pas : on ajoute 1, tant que i < 5
)
SELECT i, i * i AS carre FROM n;
```

i	carre
1	1
2	4
3	9
4	16
5	25

La base procède ainsi : elle écrit la ligne 1 ; pour chaque ligne nouvellement produite, elle applique le pas (1 donne 2, 2 donne 3...) jusqu'à ce que `WHERE i < 5` bloque la production.

Premier usage, fabriquer un calendrier : on génère *tous* les jours de l'année, puis on les joint aux commandes par un `LEFT JOIN` pour ne pas oublier les jours sans vente (sans lui, les moyennes quotidiennes seraient surestimées) ; c'est l'application 5.7 du cahier.

Second usage : parcourir une hiérarchie. La table `clients` contient le parrainage : un client peut avoir été parrainé par un autre, lui-même parrainé... C'est un **arbre**. À quelle profondeur ? Combien de filleuls (directs ou non) a chaque ambassadeur ? Une jointure ne peut parcourir qu'un nombre fixe de niveaux ; une CTE récursive les parcourt tous. Cas de base : les clients **sans parrain** (les « racines »). Pas : on ajoute leurs filleuls, puis les filleuls de leurs filleuls...

```
WITH RECURSIVE reseau(id_client, racine, profondeur) AS (
    SELECT id_client, id_client, 0 FROM clients WHERE id_parrain IS NULL
    UNION ALL
    SELECT c.id_client, r.racine, r.profondeur + 1
    FROM clients AS c JOIN reseau AS r ON c.id_parrain = r.id_client
)
SELECT r.racine, cl.prenom || ' ' || cl.nom AS ambassadeur,
       COUNT(*) - 1 AS filleuls, MAX(r.profondeur) AS generations
FROM reseau AS r JOIN clients AS cl ON cl.id_client = r.racine
GROUP BY r.racine ORDER BY filleuls DESC LIMIT 4;
```

racine	ambassadeur	filleuls	generations
10	Lou Michel	5	4
7	Paul Martin	4	3
1	Yann Lambert	4	3
5	Anna Michel	3	3

Le premier réseau est celui du client n° 10 : **5 filleuls répartis sur 4 générations**. (`COUNT(*) - 1` : on ne compte pas la racine elle-même.) L'application 5.8 du cahier affiche la chaîne complète de parrainage de chaque membre (un « chemin » recollé le long de l'arbre) et calcule le chiffre d'affaires généré par un réseau, base d'une prime au parrainage.

Récursion et sécurité. Si le parrainage contenait un **cycle** (A parraine B qui parraine A), la récursion tournerait indéfiniment. Ici c'est impossible, puisqu'un parrain a toujours un numéro plus petit, donc une inscription plus ancienne, que son filleul (nous l'avons garanti à la construction de la base, 5.1.3), mais, sur des données réelles, ajoutez toujours une **condition d'arrêt** (profondeur maximale, par exemple `WHERE profondeur < 10`). C'est l'équivalent du `while` qui ne se termine jamais.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.6 (segmentation RFM avec `NTILE`), 5.7 (un calendrier complet) et 5.8 (réseau de parrainage), exercices 5.8 à 5.10.

À retenir

- Une fonction **fenêtre** (`... OVER (PARTITION BY ... ORDER BY ...)`) calcule sur un groupe de lignes **sans réduire** leur nombre ; `GROUP BY` les réduit.
- **Classements** : `ROW_NUMBER` (jamais d'ex æquo), `RANK` (ex æquo et trous), `DENSE_RANK` (ex æquo sans trous). Le **top N par groupe** passe par une requête intérieure ou une CTE.
- **Cumuls, moyennes mobiles** : `SUM/AVG(...) OVER (ORDER BY ... ROWS BETWEEN ...)`. Précisez `ROWS` pour éviter le piège des ex æquo.
- **LAG / LEAD** donnent la valeur de la ligne précédente / suivante : évolutions et délais

entre événements.

- Une CTE (WITH nom AS (...)) nomme une étape et rend la requête lisible de haut en bas. Une CTE **récursive** (WITH RECURSIVE) génère des suites (calendriers) et parcourt des hiérarchies (parrainage, organigrammes).

5.4 Normalisation et conception de schémas

Intuition. Pourquoi avoir découpé les données de la boutique en cinq tables, au lieu d'un seul grand tableau, comme dans Excel ? Parce qu'un tableau unique **répète** les mêmes informations (la ville d'une cliente apparaît sur chacune de ses commandes), et que **tout ce qui est répété finit par se contredire**. La **normalisation** est la méthode qui consiste à ranger chaque fait **une seule fois**, à sa place. Cette section explique *pourquoi* le schéma du 5.1 est bon, et vous donne la méthode pour en concevoir un vous-même. Nous terminerons par deux sujets de **performance et de fiabilité** : les index et les transactions.

5.4.1 Le problème : la grande feuille unique

Imaginons que la gérante ait gardé son habitude du tableur : **une seule feuille** avec une ligne par article vendu, contenant tout ce qu'on sait sur la commande, la cliente et le produit. Fabriquons cette feuille pour les **huit premières commandes**, en recollant par jointures les cinq tables du 5.1 (nous créons pour cela dans la base des tables de travail préfixées `ex_`, supprimées à la fin de la section).

Voici un extrait de la feuille : les lignes des clients n° 3 et n° 5 (quelques colonnes seulement pour tenir en largeur).

```
SELECT id_commande, id_produit, id_client, prenom, ville, nom_produit, nom_categorie, quantite
FROM ex_feuille
WHERE id_client IN (3, 5)
ORDER BY id_commande, id_produit;
```

id_commande	id_produit	id_client	prenom	ville	nom_produit	nom_categorie	quantite
3	2	3	Adam	Ville F	Bol en céramique	Poterie	1
3	3	3	Adam	Ville F	Vase peint à la main	Poterie	1
3	13	3	Adam	Ville F	Savon à l'huile d'olive	Cosmétiques	1
5	8	5	Anna	Ville A	Pochette brodée	Textile	2
5	9	5	Anna	Ville A	Bague en argent	Bijoux	1
5	15	5	Anna	Ville A	Huile de soin	Cosmétiques	1
8	4	5	Anna	Ville A	Grand plat de service	Poterie	2
8	5	5	Anna	Ville A	Plaid en coton	Textile	1

Regardez la cliente n° 5, Anna Michel : elle apparaît sur **cinq lignes**, et sa ville « Ville A » est écrite cinq fois. Même chose pour le produit n° 9 (Bague en argent) et sa catégorie « Bijoux ». Cette redondance cause trois catégories de problèmes, appelées **anomalies**.

1. Anomalie de mise à jour. Anna déménage à Ville G. L'employé de la gérante ne corrige qu'une ligne sur cinq (il a oublié les autres) :

```
UPDATE ex_feuille SET ville = 'Ville G'
WHERE id_client = 5 AND id_commande = 8 AND id_produit = 5;
```

```
SELECT id_client, prenom, nom, ville, COUNT(*) AS lignes
FROM ex_feuille
```

```
WHERE id_client = 5
GROUP BY id_client, prenom, nom, ville;
```

id_client	prenom	nom	ville	lignes
5	Anna	Michel	Ville A	4
5	Anna	Michel	Ville G	1

Deux villes pour la même personne : on ne sait plus laquelle est vraie. C'est exactement l'histoire de « Léa Martin » du 5.1.1. Dans notre vraie base, cette erreur est **impossible** : la ville d'une cliente n'est écrite qu'à un seul endroit.

Une incohérence ne se résorbe pas toute seule. Même un `SELECT DISTINCT id_client, ville` ne sait pas « choisir » la bonne ville : il renvoie les deux. Les doublons contradictoires sont de la **vraie** information fausse, que seul un humain peut trancher. Remettons la ville d'origine pour la suite, par un `UPDATE` inverse.

2. Anomalie d'insertion. La gérante veut ajouter au catalogue un nouveau produit, pas encore vendu. Impossible : la feuille n'a de place que pour des *lignes de commande*, et la clé primaire exige un numéro de commande. La base répond : `NOT NULL constraint failed: ex_feuille.id_commande`.

(Et en supprimant la contrainte, on aurait une ligne avec des trous partout.)

3. Anomalie de suppression. Quels produits n'apparaissent que sur **une seule** ligne de la feuille ? Pour ceux-là, supprimer cette ligne (par exemple parce que la commande est annulée) effacerait **toute trace du produit** : son nom, sa catégorie, son prix catalogue.

```
SELECT id_produit, nom_produit, prix_catalogue, MIN(id_commande) AS seule_commande
FROM ex_feuille
GROUP BY id_produit
HAVING COUNT(*) = 1
ORDER BY id_produit;
```

id_produit	nom_produit	prix_catalogue	seule_commande
1	Plat décoratif	45.0	1
3	Vase peint à la main	65.0	3
4	Grand plat de service	38.0	8
10	Pendentif	35.0	2
11	Bracelet de perles	15.0	4
13	Savon à l'huile d'olive	6.0	3
14	Eau parfumée	14.0	4
15	Huile de soin	24.0	5

Annuler la commande n° 5 ferait disparaître l'huile de soin du catalogue : on a voulu supprimer *une vente* et on a perdu *un produit*.

Résumé : dans une table, **tout fait doit être associé à un seul sujet**. Un client, un produit, une commande et une vente sont quatre sujets différents ; les mélanger dans une même table provoque ces trois anomalies.

5.4.2 Dépendances fonctionnelles : la théorie derrière la normalisation

Définition. Dans une table R d'attributs A , on dit que X **détermine fonctionnellement** Y , noté $X \rightarrow Y$ (avec $X, Y \subseteq A$), si deux lignes qui ont la **même valeur de X** ont **forcément la**

même valeur de Y :

$$\forall t_1, t_2 \in R, \quad t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y].$$

Exemples dans notre feuille : $\text{id_client} \rightarrow \text{ville}$ (un client n'a qu'une ville) ; $\text{id_produit} \rightarrow \text{prix_catalogue}$; mais **pas** $\text{ville} \rightarrow \text{id_client}$ (plusieurs clientes habitent Ville A).

Une clé primaire est un cas particulier : K est une **clé** de R si $K \rightarrow A$ (elle détermine *toutes* les colonnes) et si aucun sous-ensemble strict de K n'en fait autant (minimalité).

Les dépendances fonctionnelles obéissent à trois règles, les **axiomes d'Armstrong** (1974), qui permettent d'en déduire d'autres :

1. **Réflexivité** : si $Y \subseteq X$, alors $X \rightarrow Y$ (trivial).
2. **Augmentation** : si $X \rightarrow Y$, alors $XZ \rightarrow YZ$ pour tout Z .
3. **Transitivité** : si $X \rightarrow Y$ et $Y \rightarrow Z$, alors $X \rightarrow Z$.

La **fermeture** X^+ d'un ensemble d'attributs est l'ensemble de tout ce qu'il détermine, directement ou par transitivité. On la calcule en partant de X et en ajoutant les attributs déterminés tant que c'est possible. Appliquons-le à notre feuille. Les dépendances que nous croyons vraies (règles de gestion de la boutique) sont les cinq suivantes :

Dépendance	Signification
$\text{id_commande} \rightarrow \text{date_commande}, \text{id_client}$	une commande a une date et un client
$\text{id_client} \rightarrow \text{prenom}, \text{nom}, \text{ville}$	un client a un nom et une ville
$\text{id_produit} \rightarrow \text{nom_produit}, \text{id_categorie}, \text{prix_catalogue}$	un produit a un nom, une catégorie, un prix
$\text{id_categorie} \rightarrow \text{nom_categorie}$	une catégorie a un nom
$(\text{id_commande}, \text{id_produit}) \rightarrow \text{quantite}, \text{prix_unitaire}$	une ligne est identifiée par le couple

Pour la fermeture, l'algorithme est celui du point fixe : on part de X , et tant qu'une dépendance dont le membre gauche est déjà inclus apporte de nouveaux attributs, on les ajoute. (Un court programme l'exécute en coulisses sur la feuille.)

Avant de s'en servir, vérifions que ces dépendances sont **respectées par les données** de la feuille : pour chaque dépendance $X \rightarrow Y$, aucun groupe de lignes de même X ne doit contenir deux valeurs différentes de Y .

Le test confirme que les cinq dépendances sont respectées ; les deux « fausses » dépendances ($\text{ville} \rightarrow \text{id_client}$, $\text{id_commande} \rightarrow \text{prix_unitaire}$) sont **contredites** par les données, comme prévu. Attention à la nuance logique : un jeu de données peut seulement **réfuter** une dépendance (une paire de lignes suffit), jamais la **démontrer** ; seule la connaissance du métier (« un client n'a qu'une ville de livraison par défaut ») l'affirme.

Calculons maintenant des fermetures et cherchons la clé de la feuille. La fermeture de $\{\text{id_client}\}$ est $\{\text{id_client}, \text{prenom}, \text{nom}, \text{ville}\}$; celle de $\{\text{id_commande}\}$ ajoute la date et le client ($\text{date_commande}, \text{id_client}$) et, par transitivité, ses prenom, nom et ville. La seule clé candidate est le couple $(\text{id_commande}, \text{id_produit})$:

La seule clé est le couple $(\text{id_commande}, \text{id_produit})$: c'est la clé primaire que nous avons déclarée. Le calcul explique aussi l'origine des anomalies : beaucoup d'attributs dépendent seulement d'**une partie** de la clé ($\text{prenom}, \text{ville}$... dépendent de id_commande seul ; nom_produit ,

prix_catalogue... de id_produit seul), ou d'un attribut qui n'est pas la clé (ville dépend de id_client, qui dépend de id_commande). On a trouvé la source du mal. Il reste à la soigner.

5.4.3 Les trois premières formes normales

Les **formes normales** sont des niveaux de « propreté » d'un schéma, chaque niveau supprimant un type de redondance. Retenez la formule qui résume les trois premières, dans l'esprit du serment d'un témoin au tribunal : *chaque attribut dépend de la clé, de toute la clé, et rien que de la clé* (« so help me Codd »).

Forme	Exigence	Anomalie évitée
1FN	chaque case contient une seule valeur (atomique) ; pas de listes ni de colonnes répétées	cases du type « Plat ; Plaid »
2FN	1FN et aucun attribut ne dépend d'une partie de la clé (utile quand la clé est composite)	informations sur le produit répétées sur chaque vente
3FN	2FN et aucun attribut non-clé ne dépend d'un autre attribut non-clé (pas de dépendance transitive)	ville du client recopiée parce que id_client détermine ville

1FN. Si la gérante avait noté le panier dans une seule case (articles = "Plaid en coton ; Pochette brodée"), elle n'aurait pu ni compter les ventes par produit, ni les joindre au catalogue : on ne sait pas joindre un morceau de texte. La solution est **une ligne par article** : c'est ce que fait notre feuille, qui est donc déjà en 1FN (et nous aurions eu le même problème avec des colonnes produit1, produit2, produit3).

2FN. La clé de la feuille est composite (id_commande, id_produit). Or les infos du produit ne dépendent que de id_produit, et celles de la commande que de id_commande : ce sont des **dépendances partielles**. On découpe : chaque groupe d'attributs va dans une table dont la clé est l'attribut qui le détermine.

```
CREATE TABLE ex_lignes AS
SELECT id_commande, id_produit, quantite, prix_unitaire FROM ex_feuille;

CREATE TABLE ex_commandes AS
SELECT DISTINCT id_commande, date_commande, id_client, prenom, nom, ville FROM ex_feuille;

CREATE TABLE ex_produits AS
SELECT DISTINCT id_produit, nom_produit, id_categorie, nom_categorie, prix_catalogue FROM ex_feuille;
```

(DISTINCT supprime les doublons : chaque commande et chaque produit n'est plus écrit qu'une fois.) La redondance a déjà fortement baissé, mais elle n'a pas disparu : dans ex_commandes, la ville d'Anna est encore écrite pour **chacune** de ses commandes (n° 5 et n° 8). Cause : id_commande → id_client → ville : c'est une **dépendance transitive**.

3FN. On extrait ce qui dépend d'un attribut non-clé dans sa propre table : les clients (clé id_client) d'un côté, les catégories (clé id_categorie) de l'autre.

```
CREATE TABLE ex_clients AS
SELECT DISTINCT id_client, prenom, nom, ville FROM ex_commandes;

CREATE TABLE ex_categories AS
SELECT DISTINCT id_categorie, nom_categorie FROM ex_produits;
```

```
CREATE TABLE ex_commandes3 AS
SELECT DISTINCT id_commande, date_commande, id_client FROM ex_commandes;

CREATE TABLE ex_produits3 AS
SELECT DISTINCT id_produit, nom_produit, id_categorie, prix_catalogue FROM ex_produits;
```

Comptons les lignes de chaque table pour voir la différence (un comptage fait en coulisses) : la grande feuille de 16 lignes devient cinq tables : 16 lignes de commande, 8 commandes, 7 clients, 12 produits et 4 catégories.

La grande feuille est devenue cinq petites tables : exactement la structure du 5.1 ! La normalisation est ce qui a conduit à notre schéma, et un dessin préalable des entités (5.1.2) aurait donné le même résultat plus vite. Les anomalies ont disparu : changer la ville d'Anna se fait par **un seul** UPDATE sur **une seule** ligne ; ajouter un produit au catalogue est un simple INSERT dans produits ; supprimer une vente laisse produit et client intacts.

Le test de sécurité : « décomposer sans rien perdre ». Découper une table en plusieurs ne doit pas **perdre d'information**. Vérifions que, si l'on recolle les morceaux par des jointures, on retrouve **exactement** la feuille d'origine, ni plus ni moins. On recolle les cinq petites tables par des jointures (5.2.5) et on compare dans les deux sens avec EXCEPT : les lignes reconstituées absentes de la feuille, puis l'inverse (le contrôle est exécuté en coulisses).

Le contrôle renvoie zéro et zéro : la décomposition est **sans perte**. (Et grâce à la remarque du 5.4.1 sur la ville d'Anna, nous avons pris soin de remettre la feuille en état avant de la découper : normaliser des données **déjà contradictoires** aurait recopié fidèlement la contradiction dans la table ex_clients, avec deux lignes pour la même cliente. Un schéma normalisé rend les *nouvelles* incohérences impossibles, par exemple en déclarant id_client clé primaire de clients, mais ne répare pas les anciennes.)

Pourquoi la décomposition sans perte fonctionne (théorème de Heath). Soit $R(X, Y, Z)$ une table avec la dépendance $X \rightarrow Y$. Alors $R = \pi_{X,Y}(R) \bowtie \pi_{X,Z}(R)$. *Preuve.* L'inclusion $R \subseteq \pi_{X,Y}(R) \bowtie \pi_{X,Z}(R)$ est évidente (toute ligne se retrouve en recollant ses propres morceaux). Réciproquement, soit (x, y, z) dans la jointure : (x, y) vient d'une ligne $(x, y, z') \in R$ et (x, z) d'une ligne $(x, y', z) \in R$. Ces deux lignes ont la **même valeur** x , donc, comme $X \rightarrow Y$, la **même valeur** $y = y'$. La ligne $(x, y', z) = (x, y, z)$ est donc dans R . $\square$ Sans la dépendance, la jointure pourrait fabriquer de **fausses lignes** : c'est le piège de la décomposition faite « au feeling ».

Pour finir proprement, nos tables de travail sont supprimées en coulisses (pour que la base redevenue celle du 5.1).

5.4.4 Aller plus loin, ou s'arrêter plus tôt ?

La forme normale de Boyce-Codd (BCNF) renforce la 3FN : pour *toute* dépendance $X \rightarrow Y$ non triviale, X doit être une clé. Dans la pratique, la 3FN suffit presque toujours, et un schéma en 3FN est souvent déjà en BCNF. Il existe des formes supérieures (4FN, 5FN) pour des cas plus rares.

Mais attention : « plus normalisé » n'est pas toujours « mieux ». Un schéma normalisé est idéal pour **enregistrer** des données (les systèmes *transactionnels*, dits **OLTP** : caisse, site de vente, réservations) : peu de redondance, mises à jour sûres. Pour **analyser** (les systèmes **OLAP** : entrepôts de données, tableaux de bord), l'analyste doit en revanche joindre cinq tables pour la moindre question, ce qui est lent et fastidieux. On **dénormalise** donc volontairement dans les entrepôts, par exemple avec un **schéma en étoile** : au centre, une grosse table de **faits** (les ventes : une ligne

par article vendu, avec des montants et des clés), autour d'elle de petites tables de **dimensions** (client, produit, date) qui décrivent le contexte. Notre lignes_commande entourée de commandes, produits, clients ressemble déjà à une étoile.

Règle pratique : **normalisez pour écrire, dénormalisez pour lire**. En attendant de construire un entrepôt, une **vue** donne à l'analyste une « grande table » toute prête, sans dupliquer les données : une vue est une **requête enregistrée sous un nom**, qu'on interroge comme une table.

```
CREATE VIEW v_ventes AS
SELECT l.id_commande, c.date_commande, c.canal, c.satisfaction,
       cl.id_client, cl.ville,
       ca.nom AS categorie, p.nom AS produit,
       l.quantite, l.prix_unitaire, l.quantite * l.prix_unitaire AS montant_ligne
FROM lignes_commande AS l
JOIN commandes AS c ON c.id_commande = l.id_commande
JOIN clients AS cl ON cl.id_client = c.id_client
JOIN produits AS p ON p.id_produit = l.id_produit
JOIN categories AS ca ON ca.id_categorie = p.id_categorie;
```

```
SELECT categorie, canal, ROUND(SUM(montant_ligne)) AS chiffre_affaires
FROM v_ventes
GROUP BY categorie, canal
ORDER BY categorie, canal;
```

categorie	canal	chiffre_affaires
Bijoux	Boutique	2256.0
Bijoux	Réseaux	1914.0
Bijoux	Site	2836.0
Cosmétiques	Boutique	878.0
Cosmétiques	Réseaux	1099.0
Cosmétiques	Site	1170.0
Poterie	Boutique	2397.0
Poterie	Réseaux	2143.0
Poterie	Site	2410.0
Textile	Boutique	2997.0
Textile	Réseaux	1607.0
Textile	Site	2390.0

Une requête qui exigeait quatre jointures tient maintenant en trois lignes, et la vue peut évoluer (changer de définition) sans que les analystes changent leurs requêtes.

5.4.5 Les index : retrouver une ligne sans tout lire

Quand on écrit `WHERE id_client = 2`, comment la base trouve-t-elle les commandes du client ? Sans aide, elle doit **lire toutes les lignes** de la table, une par une (un *balayage complet*, ou *full scan*). Avec 400 lignes, c'est instantané ; avec 100 millions, c'est insupportable. Un **index** est une structure annexe, triée, comparable à l'**index alphabétique à la fin d'un livre** : au lieu de feuilleter tout l'ouvrage pour trouver « Khi-deux », on consulte l'index qui renvoie à la bonne page. Techniquement c'est le plus souvent un **arbre B** (*B-tree*) : on trouve une valeur parmi N en environ $\log_2 N$ comparaisons au lieu de N .

Pour $N = 500\,000$, c'est $\log_2 N \approx 19$ comparaisons contre 500 000 : un facteur **plus de 25 000**. On peut demander à la base **comment elle compte exécuter** une requête, avec `EXPLAIN QUERY PLAN` (c'est le premier outil de l'analyste qui s'occupe de performance) :

```
-- On demande à la base comment elle compte exécuter la requête, puis on crée l'index :
EXPLAIN QUERY PLAN SELECT * FROM commandes WHERE id_client = 2;
CREATE INDEX idx_commandes_client ON commandes(id_client);
```

```

Avant l'index :
    SCAN commandes
Après CREATE INDEX :
    SEARCH commandes USING INDEX idx_commandes_client (id_client=?)
Recherche par la clé primaire (index automatique) :
    SEARCH commandes USING INTEGER PRIMARY KEY (rowid=?)
Filtre sur une colonne non indexée :
    SCAN commandes

```

SCAN signifie « lire toute la table » ; SEARCH ... USING INDEX signifie « chercher via l'arbre ». La clé primaire est **toujours** indexée automatiquement ; c'est aussi pourquoi les clés étrangères mal indexées sont une cause classique de lenteur. Le filtre sur montant reste un SCAN : aucun index ne l'aide.

Pour mesurer le gain « pour de vrai », nous construisons en coulisses une table de **500 000 lignes** (par une CTE récursive, 5.3.6) et chronométrons la même recherche avant et après un index (l'application 5.9 du cahier refait l'expérience pas à pas). Les durées exactes dépendent de votre machine ; nous n'affichons donc que le **facteur de gain**, arrondi à la puissance de dix inférieure, pour que le résultat reste le même d'une exécution à l'autre.

```

lignes dans la table : 500000
gain au moins égal à : 100 fois

```

Chez nous, le gain dépasse un facteur cent (et la précision de l'affichage est volontairement grossière, pour rester reproductible) : voilà pourquoi les index sont **la première optimisation**. Mais ils ne sont pas gratuits : un index occupe de la place, et il doit être **mis à jour à chaque insertion ou modification**, ce qui ralentit l'écriture. Règle d'usage : indexer les colonnes **souvent utilisées dans un WHERE ou un JOIN** (surtout les clés étrangères) et ne pas indexer à tout va. Notez enfin que les index peuvent changer le **temps** d'une requête, **jamais son résultat**.

5.4.6 Les transactions : tout ou rien

Enregistrer une commande, c'est plusieurs écritures : une ligne dans commandes, puis une ligne par article dans lignes_commande (et, dans un vrai système, la mise à jour du stock). Que se passe-t-il si le courant saute **entre** deux de ces écritures ? On aurait une commande **sans** articles : une base incohérente. Une **transaction** regroupe plusieurs opérations en un bloc **indivisible** : soit toutes réussissent (COMMIT), soit aucune n'a d'effet (ROLLBACK). On résume les garanties d'une transaction par l'acronyme **ACID** :

Lettre	Garantie	Signification
Atomicité	tout ou rien	si une étape échoue, tout est annulé
Cohérence	les règles tiennent toujours	les contraintes (clés, CHECK) sont respectées avant et après
Isolation	pas d'interférence	deux transactions simultanées ne voient pas les états intermédiaires l'une de l'autre
Durabilité	c'est définitif	une fois validée, la modification survit à une panne

Voyons l'atomicité en action. On tente d'enregistrer une commande dont la **deuxième** étape échoue (un produit n° 999 qui n'existe pas) :

```
def nb_commandes():
    return con.execute("SELECT COUNT(*) FROM commandes").fetchone()[0]

print("avant :", nb_commandes(), "commandes")
try:
    with con:
        # ouvre une transaction ; ROLLBACK automatique si
        ↪ exception
        con.execute("INSERT INTO commandes VALUES (9001, 1, '2025-12-31', 'Site', 80, 3, 5)")
        print("pendant la transaction :", nb_commandes(), "commandes (la nouvelle est visible pour
        ↪ nous)")
        con.execute("INSERT INTO lignes_commande VALUES (9001, 999, 1, 80)") # produit inexistant :
        ↪ échec
except sqlite3.IntegrityError as erreur:
    print("échec :", erreur)
print("après :", nb_commandes(), "commandes")
```

```
avant : 400 commandes
pendant la transaction : 401 commandes (la nouvelle est visible pour nous)
échec : FOREIGN KEY constraint failed
après : 400 commandes
```

La commande n° 9001, pourtant bien insérée à l'étape 1, a **disparu** : l'échec de l'étape 2 a annulé toute la transaction. Sans transaction, on aurait gardé une commande fantôme sans article. Dans le bloc `with con:`, Python déclenche un `COMMIT` si tout se passe bien, un `ROLLBACK` à la première exception.

Pourquoi l'isolation compte. Deux caissiers enregistrent en même temps la vente du dernier exemplaire d'un vase. Sans isolation, chacun lit « stock = 1 », chacun vend, et le stock tombe à -1. Avec une transaction isolée, la seconde attend (ou échoue) et lit « stock = 0 ». Les SGBD offrent plusieurs **niveaux d'isolation**, du plus laxiste au plus strict, avec un compromis entre sécurité et vitesse ; le détail dépasse ce chapitre, mais retenez que les bases relationnelles gèrent pour vous un problème que les fichiers CSV ignorent complètement.

Pour s'entraîner. Cahier, chapitre 5 : application 5.9 (mesurer le gain d'un index), exercice 5.11 (dépendances fonctionnelles et décomposition en 3FN).

À retenir

- Une table qui mélange plusieurs sujets provoque des **anomalies** de mise à jour, d'insertion et de suppression.
- Une **dépendance fonctionnelle** $X \rightarrow Y$ dit que X détermine Y . La fermeture X^+ permet de trouver les **clés**. Les données peuvent **réfuter** une dépendance, pas la prouver.
- **1FN** : cases atomiques. **2FN** : rien ne dépend d'une partie de la clé. **3FN** : rien ne dépend d'un non-clé. Formule : « la clé, toute la clé, et rien que la clé ».
- Une décomposition doit être **sans perte** (théorème de Heath). Normaliser ne répare pas des données déjà contradictoires.
- **Normalisez pour écrire, dénormalisez pour lire** (OLTP contre OLAP, schéma en étoile, vues).
- Un **index** accélère les recherches (de N à $\log N$ comparaisons) au prix de l'espace et de l'écriture ; `EXPLAIN QUERY PLAN` montre si la base lit tout (`SCAN`) ou utilise l'index (`SEARCH`).
- Une **transaction** est un bloc « tout ou rien » (ACID).

5.5 ☉ Pour aller plus loin : les bases NoSQL (MongoDB, Redis)

Section optionnelle. Le reste du livre ne dépend pas de cette section. Elle vous donne la carte du territoire « au-delà du relationnel », utile si vous lisez des offres d'emploi (MongoDB, Redis, Cassandra, Neo4j y sont souvent cités) ou si un projet vous met un jour devant l'une de ces bases.

Honnêteté sur l'exécution. MongoDB et Redis sont des **serveurs** qui ne sont pas installés dans l'environnement qui a servi à fabriquer ce livre. Les commandes qui leur sont destinées sont donc marquées **non exécutées** : elles n'ont pas de sortie, et nous ne prétendons pas en avoir vu une. À la place, nous reproduisons leur **logique** avec du Python et du JSON, exécutés, de sorte que vous puissiez voir à *quoi ressemblent* les données et les requêtes.

Intuition. « NoSQL » signifie « *Not only SQL* » (« pas seulement SQL »). Ce n'est pas *un* modèle, mais une famille de bases nées dans les années 2000 chez des géants du web (Google, Amazon, Facebook), qui avaient des besoins pour lesquels les bases relationnelles étaient mal adaptées : des **milliards** d'enregistrements répartis sur des **centaines de machines**, des données de **formes variées** qui changent sans cesse, des temps de réponse de **quelques millisecondes**.

5.5.1 Pourquoi sortir du relationnel ?

Le modèle relationnel est excellent, mais il a des prix : le **schéma est rigide** (ajouter un champ à une table de 10 milliards de lignes est lourd), les **jointures coûtent cher** quand les données sont réparties sur plusieurs machines, et les **garanties ACID** (5.4.6) ralentissent. Les bases NoSQL font des compromis différents. Il en existe quatre familles principales :

Famille	Idée	Exemples	Cas d'usage typique
Clé-valeur	un énorme dictionnaire : on retrouve une valeur par sa clé, très vite	Redis , DynamoDB	cache, paniers, sessions, compteurs, classements
Documents	des documents JSON imbriqués, de forme libre	MongoDB , CouchDB	catalogues, profils, contenus, données semi-structurées
Colonnes larges	tables géantes réparties, écritures très rapides	Cassandra, HBase	journaux, mesures de capteurs, historiques
Graphes	nœuds et relations en première classe	Neo4j	réseaux sociaux, recommandations, détection de fraude

Le théorème CAP (Brewer, 2000 ; démontré par Gilbert et Lynch, 2002). Dans un système de données **réparti** sur plusieurs machines, on ne peut pas garantir en même temps les trois propriétés suivantes : **Cohérence** (tous les lecteurs voient la dernière écriture), **Availability**, la **disponibilité** (chaque requête reçoit une réponse), et la **Partition tolérance** (le système continue de fonctionner quand le réseau entre machines est coupé). Comme les coupures réseau **arrivent** (on ne peut pas les exclure), il faut choisir en cas de coupure : **cohérence** (on refuse de répondre pour ne pas donner une donnée périmée) ou **disponibilité** (on répond, quitte à donner une donnée un peu ancienne : on parle de *cohérence à terme*). Les bases

relationnelles classiques penchent vers la cohérence ; beaucoup de bases NoSQL, vers la disponibilité. Ce n'est pas « meilleur » ou « moins bon » : c'est un **choix de conception** selon l'application (un virement bancaire exige la cohérence ; le nombre de « j'aime » d'une publication peut être approximatif quelques secondes).

5.5.2 Les bases de documents : l'exemple de MongoDB

Dans une base de documents, une commande n'est pas répartie sur plusieurs tables : c'est **un seul document JSON** qui contient tout (le client, les lignes) **imbriqué**. Voici, construit à partir de notre base relationnelle (par un court programme exécuté en coulisses ; l'application 5.10 du cahier le détaille), le document de la commande n° 3, celle du 5.1.3 aux trois lignes :

```
400 documents ; le troisième :
{"_id": 3, "date": "2025-01-04", "canal": "Réseaux", "montant": 88.2,
 "client": {"id": 3, "nom": "Adam Michel", "ville": "Ville F"},
 "lignes": [{"produit": "Bol en céramique", "categorie": "Poterie", "quantite": 1, "prix": 17.84},
            {"produit": "Vase peint à la main", "categorie": "Poterie", "quantite": 1, "prix": 64.42},
            {"produit": "Savon à l'huile d'olive", "categorie": "Cosmétiques", "quantite": 1, "prix":
↵ 5.94}]}
```

Ce document **contient tout ce qu'il faut** pour afficher la commande : pas de jointure à faire, une seule lecture suffit. C'est l'argument central des bases de documents : ce qu'on lit ensemble est rangé ensemble. On les interroge avec des filtres sur les champs, y compris dans les listes imbriquées. Voici « les commandes d'au moins 100 € qui contiennent un bijou », d'abord à la manière de MongoDB (les filtres se décrivent avec des documents) :

```
// Non exécuté : nécessite un serveur MongoDB.
db.commandes.countDocuments({
  montant: { $gte: 100 },
  "lignes.categorie": "Bijoux"
})
```

Un court programme Python (exécuté en coulisses) applique la même logique, un filtre sur les champs imbriqués, à nos documents ; comparons son résultat avec la réponse **relationnelle** (jointure SQL du 5.2) pour vérifier qu'on obtient bien la même chose :

```
version documents (Python) : 27
version relationnelle (SQL) : 27
```

Même résultat, deux philosophies : dans l'une, on **reconstruit** les liens à la lecture (jointure) ; dans l'autre, on les a **pré-assemblés** à l'écriture (imbrication).

Notez qu'il n'est même pas nécessaire de quitter SQLite pour jouer avec des documents : il sait stocker du JSON dans une colonne de texte et l'interroger avec les fonctions `json_extract` et `json_each`. C'est aussi le cas de PostgreSQL (type `jsonb`), ce qui permet un mélange des deux mondes. Plaçons les 400 documents dans une table `ex_docs` (une colonne de texte JSON) et interrogeons-la.

```
SELECT json_extract(doc, '$.canal')           AS canal,
       COUNT(*)                             AS commandes,
       ROUND(AVG(json_extract(doc, '$.montant')), 2) AS panier_moyen
FROM ex_docs
GROUP BY canal
ORDER BY canal;
```

canal	commandes	panier_moyen
Boutique	114	74.81
Réseaux	138	49.01

Site	148	59.50
------	-----	-------

On retrouve les paniers moyens par canal du 5.2.4. (`'$.canal'` est un *chemin* dans le document ; `$.client.ville` atteindrait un champ imbriqué.) La même question du bijou, avec `json_each` qui « déplie » la liste des lignes :

```
SELECT COUNT(*) AS commandes_avec_bijou_100
FROM ex_docs
WHERE json_extract(doc, '$.montant') >= 100
      AND EXISTS (SELECT 1 FROM json_each(ex_docs.doc, '$.lignes') AS l
                  WHERE json_extract(l.value, '$.categorie') = 'Bijoux');
```

```
commandes_avec_bijou_100
                        27
```

Pour un agrégat complet, MongoDB utilise un **pipeline** d'étapes qui s'enchaînent (le même esprit que les CTE du 5.3) :

```
// Non exécuté : chiffre d'affaires et nombre de commandes par canal, pour les commandes de 100 € et
↳ plus.
db.commandes.aggregate([
  { $match: { montant: { $gte: 100 } } },
  { $group: { _id: "$canal", ca: { $sum: "$montant" }, commandes: { $sum: 1 } } },
  { $sort: { ca: -1 } }
])
```

Le revers de la médaille : la redondance. Chaque document contient la ville du client. Combien de documents faudrait-il modifier si Sam Fontaine (client n° 2) déménageait ? Un comptage exécuté en coulisses répond :

Vingt-trois documents (autant que de commandes de ce client) ! C'est exactement l'anomalie de mise à jour du 5.4.1 : en choisissant l'**imbrication**, on assume la **redondance**, et c'est à l'application de la gérer. Dans le monde des documents, on décide au cas par cas ce qu'on imbrique (ce qui est lu ensemble, qui change rarement) et ce qu'on référence (ce qui change souvent). Aucun modèle n'est gratuit.

5.5.3 Les bases clé-valeur : l'exemple de Redis

Redis est le plus simple des modèles : un dictionnaire géant **en mémoire** (donc extrêmement rapide : des centaines de milliers d'opérations par seconde). On y range des valeurs sous une clé : `SET cle valeur`, `GET cle`. Il propose aussi des types pratiques : compteurs, listes, ensembles, **ensembles triés** (classements). Voici quelques commandes typiques pour une boutique en ligne :

```
# Non exécuté : nécessite un serveur Redis.
redis-cli SET panier:2 '{"articles": 3, "total": 88.2}' EX 3600 # panier du client 2, expire dans 1
↳ heure
redis-cli GET panier:2
redis-cli INCR visites:page_accueil # compteur atomique
redis-cli ZADD classement_clients 1874.3 "Sam Fontaine" 1701.7 "Yann Lambert" # ensemble trié par
↳ score
redis-cli ZREVRANGE classement_clients 0 2 WITHSCORES # les 3 meilleurs
```

Le cas d'usage numéro un est le **cache** : stocker le résultat d'une requête lente (par exemple le tableau de bord du chiffre d'affaires) pour ne pas la recalculer à chaque visite. Imaginons sept demandes successives (Site, Site, Boutique, Site, Boutique, Réseaux, Site) : la première fois qu'un canal est demandé, la requête SQL est exécutée (*cache miss*) et son résultat est rangé dans un dictionnaire ; les fois suivantes, la réponse vient directement du « cache » (*cache hit*). Résultat :

sept demandes, **trois** requêtes SQL seulement (l'application 5.11 du cahier programme ce petit cache). Il reste le problème classique : **quand périmé le cache ?** (si une nouvelle commande arrive, la valeur en cache devient fausse). Redis résout cela avec une **durée de vie** (EX 3600, comme ci-dessus) : la clé s'efface toute seule. Un mot célèbre résume la difficulté : « *il n'y a que deux choses difficiles en informatique : invalider un cache et nommer les choses.* »

5.5.4 Alors, que choisir ?

Critère	Relationnel (SQL)	NoSQL
Structure des données	stable, bien définie	variable, évolutive
Relations entre données	nombreuses (jointures)	peu, ou imbriquées
Cohérence	forte (ACID)	souvent « à terme »
Requêtes	très riches (SQL)	plus limitées, spécialisées
Volume	de petit à très grand	pensé pour le très grand, réparti
Analyse de données	excellent	souvent à exporter d'abord

Pour un data scientist, la réponse pratique est la suivante. **Commencez par le relationnel** (PostgreSQL en particulier : fiable, gratuit, et capable de stocker du JSON) : il répond à l'immense majorité des besoins, et l'analyse y est la plus confortable. N'allez vers le NoSQL que lorsqu'un besoin précis l'impose (cache ultra-rapide, volume réparti, données de forme libre, graphes). Dans les grandes entreprises, on trouve d'ailleurs souvent **plusieurs bases à la fois** (on parle de *persistance polyglotte*) : un SGBD relationnel pour les commandes, Redis pour le cache, un moteur de documents pour le catalogue, un entrepôt de données pour l'analyse. En tant qu'analyste, vous serez souvent celui qui les **réunit**, d'où l'intérêt de connaître chacune de ces familles.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.10 (documents JSON) et 5.11 (un cache en Python).

À retenir

- **NoSQL** = « pas seulement SQL » : quatre familles (clé-valeur, documents, colonnes, graphes), nées pour le **très gros volume**, la **souplesse** du schéma et la **répartition**.
- Le **théorème CAP** impose un choix, en cas de coupure réseau, entre cohérence et disponibilité.
- Une base de **documents** imbrique ce qui est lu ensemble : lecture sans jointure, mais **redondance** à gérer (anomalie de mise à jour du 5.4.1).
- Une base **clé-valeur** (Redis) est un dictionnaire ultra-rapide, idéal comme **cache** ; tout l'art est de savoir quand le périmé.
- En pratique : **commencez par le relationnel**, choisissez le NoSQL quand un besoin précis l'exige.
- Les commandes MongoDB et Redis de cette section n'ont **pas** été exécutées ; leur logique a été reproduite et vérifiée en Python/SQLite.

Bilan du chapitre 5

Vous savez maintenant :

- **expliquer pourquoi** une base relationnelle vaut mieux qu'un fichier (redondance, incohérence, contraintes, volume, accès simultanés), et lire un schéma : tables, **clés primaires et étrangères**, relations 1–N et N–N ;
- **écrire des requêtes SQL** complètes : SELECT, WHERE, ORDER BY, GROUP BY/HAVING, **jointures** (INNER, LEFT, auto-jointure), sous-requêtes, opérations ensemblistes ;
- **éviter les pièges classiques** : priorité de AND/OR, NULL (trois valeurs de vérité, NOT IN), dates et date('now'), WHERE contre HAVING ;
- **calculer sans écraser les lignes** grâce aux **fonctions fenêtres** (classements, cumuls, moyennes mobiles, LAG/LEAD), structurer une requête en CTE, et parcourir des hiérarchies par **réursion** ;
- **concevoir un schéma** : dépendances fonctionnelles, 1FN/2FN/3FN, décomposition sans perte, et savoir quand dénormaliser (OLTP contre OLAP, vues, schéma en étoile) ;
- comprendre ce que font les **index** (EXPLAIN QUERY PLAN) et les **transactions** (ACID) ;
- (en option) situer les bases **NoSQL** (documents, clé-valeur) et le théorème **CAP**.

Pour s'entraîner. Le cahier du volume consacre son chapitre 5 à ce chapitre : onze applications guidées (reconstruire la base, importer un CSV, les meilleurs clients, tableaux croisés, clients dormants, segmentation RFM, calendrier, réseaux de parrainage, mesure d'un index, documents JSON et cache) et onze exercices corrigés.

Le chapitre 6 clôt la partie « boîte à outils » avec les habitudes de travail du professionnel : **Git** pour garder l'historique de vos analyses (y compris vos requêtes SQL, qui sont du code comme les autres), les **notebooks Jupyter** pour mélanger code, résultats et explications, et la **ligne de commande** pour tout automatiser. Ensuite, le projet de clôture du volume, proposé dans le cahier, réunira tout ce que vous avez appris, des mathématiques au SQL, dans une seule étude de bout en bout.

Chapitre 6 : Outils de travail

« Un bon artisan ne se reconnaît pas seulement à ses gestes, mais à l'ordre de son atelier. »

Vous savez maintenant calculer (chapitres 1 à 3), programmer (chapitre 4) et interroger une base de données (chapitre 5). Reste une question que personne n'ose poser en cours, mais qui décide de la vie quotidienne d'un data scientist : **comment ne pas se perdre dans son propre travail ?**

Voici la scène, que tout le monde a vécue au moins une fois. La gérante ouvre le dossier de son analyse des ventes :

```
analyse.py
analyse_v2.py
analyse_v2_corrige.py
analyse_FINAL.py
analyse_FINAL_vrai.py
analyse_FINAL_vrai_ne_pas_toucher.py
rapport_mars.docx
rapport_mars (copie).docx
```

Quelle version a produit le graphique envoyé à la banque la semaine dernière ? Impossible de le savoir. Que s'est-il passé quand elle a « juste changé un petit truc » mardi soir, et que tous les chiffres se sont mis à bouger ? Mystère. Et si un collègue veut l'aider, comment fusionnent-ils leurs deux versions sans écraser le travail de l'autre ?

Ce chapitre vous donne **quatre outils** qui répondent à ces questions, et qui sont utilisés dans toutes les équipes de données du monde.

Le chemin de ce chapitre

- **6.1 Git et gestion de versions** : une machine à remonter le temps pour vos fichiers. On y apprend à photographier son travail (*commit*), à comparer, à défaire, à travailler à plusieurs (*branches, fusions, conflits*) et à partager (*dépôts distants*).
- **6.2 Notebooks Jupyter** : le carnet de laboratoire interactif où code, résultats et explications cohabitent. On y apprend aussi à **s'en méfier** : un notebook mal utilisé produit des résultats que personne ne peut reproduire.
- **6.3 Ligne de commande et environnements** : parler directement à l'ordinateur avec du texte, enchaîner de petits outils, et isoler les bibliothèques de chaque projet dans un **environnement virtuel**.
- **⊕ Pour aller plus loin** : scripts shell et bases de Docker (6.4) ; recherche reproductible, de Pandoc à Quarto en passant par LaTeX et Make (6.5).

- **Bilan du chapitre.** Les applications guidées et les exercices corrigés de ce chapitre se trouvent dans le **cahier** (chapitre 6) ; chaque section y renvoie.

Le fil conducteur : la reproductibilité. Un résultat n'a de valeur que si **quelqu'un d'autre (ou vous-même dans six mois) peut le refaire**. Chaque outil de ce chapitre attaque une cause différente de non-reproductibilité : Git (« quelle version du code ? »), les notebooks bien tenus (« dans quel ordre a-t-on exécuté les cellules ? »), les environnements (« avec quelles versions des bibliothèques ? »), les graines aléatoires et les Makefile (« avec quelles données et quelles étapes ? »). Le terme technique est la *reproductibilité computationnelle*.

Comment lire ce chapitre. Contrairement aux précédents, les exemples se tapent dans un **terminal** (aussi appelé *console* ou *shell*), pas dans Python. Sous Linux et macOS, ouvrez l'application « Terminal ». Sous Windows, installez **Git for Windows** (il fournit *Git Bash*, un terminal compatible avec tous nos exemples) ou, mieux, **WSL** (le sous-système Linux de Windows). Les lignes à taper sont celles des blocs de code ; les blocs gris qui suivent montrent ce que l'ordinateur répond. Le livre n'en garde que l'essentiel : les séances complètes, pas à pas, sont dans le cahier.

L'atelier de la gérante

Pour que les exemples soient concrets, nous préparons un petit **atelier** : un dossier de travail dans lequel nous copions le fichier de données du livre.

À propos de \$DONNEES. Le fichier `commandes.csv` est fourni avec le livre dans le dossier `donnees/`. Dans les exemples qui suivent, la variable `$DONNEES` désigne le **chemin de ce dossier** sur la machine qui exécute le code. Chez vous, remplacez `"$DONNEES"` par l'endroit où vous avez rangé le fichier (par exemple `~/livre/donnees`). Le caractère `~` est une abréviation pour votre dossier personnel ; nous l'expliquons au 6.3.

```
mkdir -p ~/atelier/boutique
cd ~/atelier/boutique
cp "$DONNEES/commandes.csv" .
ls
```

```
commandes.csv
```

Chaque ligne tapée est une **commande** : un nom (`mkdir`, `cd`, `cp`, `ls`) suivi d'**arguments**. Ici : créer un dossier (`mkdir`, pour *make directory* ; l'option `-p` crée aussi les dossiers intermédiaires), s'y placer (`cd`, *change directory*), y copier le fichier (`cp`, *copy* ; le point `.` désigne « le dossier où je suis »), puis lister le contenu (`ls`, *list*).

Honnêteté sur l'exécution. Tous les exemples de ce chapitre qui peuvent s'exécuter sans réseau ni logiciel spécial ont été **réellement exécutés** dans un atelier jetable, et la sortie affichée est la vraie. Les rares exceptions (Docker, Quarto, services en ligne comme GitHub) sont clairement signalées « **non exécuté** » : nous n'y affichons que la commande, jamais une sortie inventée.

6.1 Git et gestion de versions

Intuition. Imaginez que vous puissiez, à tout moment, prendre une **photographie complète** de votre dossier de travail, lui donner un titre (« ajout du calcul par canal de vente »), et la ranger dans un album. Plus tard, vous pouvez feuilleter l'album, comparer deux photos, revenir à celle d'il y a un mois, ou même ouvrir **deux albums parallèles** pour tester une idée folle sans toucher à la version qui marche. **Git** est exactement cet album, avec trois cadeaux en plus : il est gratuit, il fonctionne hors ligne, et il permet à plusieurs personnes de travailler sur le même dossier sans se marcher dessus.

Cette section présente les idées et les commandes essentielles, avec de courts exemples. Les séances complètes, pas à pas, sont proposées dans le cahier (applications 6.1 à 6.3).

6.1.1 Pourquoi pas simplement « analyse_FINAL_vrai.py » ?

Copier ses fichiers avec un suffixe (`_v2`, `_FINAL`) semble naturel, mais les défauts apparaissent vite :

Problème	Avec des copies de fichiers	Avec Git
« Qu'est-ce qui a changé entre hier et aujourd'hui ? »	Ouvrir deux fichiers et comparer à l'œil	<code>git diff</code> liste chaque ligne modifiée
« Pourquoi ai-je changé ça ? »	Personne ne s'en souvient	Chaque version porte un message daté et signé
« Je veux revenir à la version de mars »	Si elle existe encore et si son nom est clair	Une commande
« Je veux tester une idée sans risque »	Dupliquer tout le dossier	Une branche , en une seconde
« Un collègue et moi avons modifié le même fichier »	L'un des deux écrase l'autre, ou on recopie à la main	Git fusionne et signale seulement les vrais conflits
« Quel code a produit ce graphique ? »	Aucune idée	On retrouve la version exacte

Git a été créé en 2005 par Linus Torvalds pour développer le noyau Linux ; il est aujourd'hui le standard de fait, bien au-delà de la programmation : vous l'utiliserez pour du code, des notebooks, des rapports en texte, des fichiers de configuration.

Git et GitHub, ce n'est pas pareil. *Git* est le logiciel qui tourne **sur votre machine** et garde l'historique. *GitHub*, *GitLab* ou *Bitbucket* sont des **sites web** qui hébergent des copies de dépôts Git pour les partager. On peut utiliser Git des années sans jamais toucher à GitHub.

6.1.2 Premiers réglages

Chaque version enregistrée est **signée** : il faut dire à Git qui vous êtes, une fois pour toutes.

```
git config --global user.name "La gérante"
git config --global user.email "gerante@boutique.example"
git config --global init.defaultBranch main
git config --global --list
```

```
user.name=La gérante
user.email=gerante@boutique.example
init.defaultbranch=main
```

Les réglages sont rangés dans le fichier `.gitconfig` de votre dossier personnel. La branche par défaut s'appellera `main` (l'ancien nom était `master`).

Une astuce de reproductibilité (à ne pas reproduire chez vous). Chaque version Git est identifiée par une empreinte qui dépend, entre autres, **de la date**. Pour que le livre affiche toujours les mêmes empreintes, nous avons fixé artificiellement la date de toutes les versions de cette section. Chez vous, Git utilisera l'horloge et vos empreintes seront différentes des nôtres : aucun raisonnement ne dépend de leur valeur.

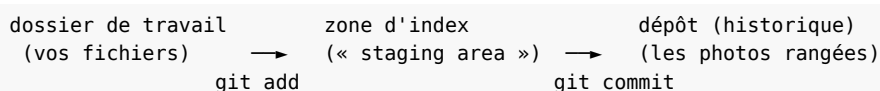
6.1.3 Le premier dépôt : `init`, `add`, `commit`

Un **dépôt** (*repository*, ou *repo*) est un dossier que Git surveille. On l'initialise avec `git init`. Écrivons le tout premier script d'analyse de la gérante, qui calcule le montant moyen des commandes, puis demandons à Git ce qu'il en pense.

```
cd ~/atelier/boutique
git init -q
cat > analyse.py <<'FIN'
import pandas as pd
df = pd.read_csv("commandes.csv")
print("Montant moyen :", round(df["montant"].mean(), 2), "€")
FIN
git status --short
```

```
?? analyse.py
?? commandes.csv
```

Le double point d'interrogation `??` signifie **non suivi** (*untracked*) : Git a remarqué les deux fichiers mais ne les surveille pas encore. Pour comprendre la suite, il faut connaître les **trois zones** de Git :



- Le **dossier de travail** contient vos fichiers, tels que vous les éditez.
- La **zone d'index** (*staging area*) est la table où l'on **prépare la prochaine photo** : on y dépose seulement ce qu'on veut y voir.
- Le **dépôt** contient les photos prises, à jamais (les *commits*).

Pourquoi cette zone intermédiaire ? Parce qu'on a souvent modifié trois fichiers pour trois raisons différentes : on peut ainsi faire **trois commits distincts et propres** plutôt qu'un fourre-tout.

```
git add analyse.py commandes.csv
git commit -q -m "Premier script et données (400 commandes)"
git log --oneline
```

```
04c558e Premier script et données (400 commandes)
```

`git log --oneline` montre une ligne par version : l'empreinte courte, puis le message.

Écrire de bons messages. Un message de commit répond à la question « **que fait** ce changement, et pourquoi ? », dans un langage que vous comprendrez dans six mois. « maj » ou « modifs » sont inutiles ; « Ajout du montant moyen par canal de vente » est excellent. Convention courante : une première ligne courte (une cinquantaine de caractères).

Pour s'entraîner. Cahier, chapitre 6 : application 6.1, exercice 6.3.

6.1.4 Ce qu'il y a vraiment dans un commit

Rigueur : le modèle d'objets de Git. Un *commit* n'est pas une liste de différences : c'est une **photographie complète** du dossier, accompagnée de métadonnées et d'un **pointeur vers le commit précédent** (son *parent*). Chaque objet est stocké sous un nom qui est l'**empreinte de son contenu** (une fonction de hachage SHA-1, qui transforme n'importe quel contenu en un nombre de 40 chiffres hexadécimaux). Il y a trois sortes d'objets :

- un **blob** : le contenu d'un fichier, sans son nom ;
- un **arbre** (*tree*) : la liste des fichiers d'un dossier, avec pour chacun le nom et l'empreinte de son blob ;
- un **commit** : l'empreinte d'un arbre, l'empreinte du ou des parents, l'auteur, la date, le message.

Conséquence cruciale : si on change **un seul caractère** dans un fichier, l'empreinte du blob change, donc celle de l'arbre, donc celle du commit, et de tous ses descendants. Un historique Git ne peut donc pas être modifié en douce : c'est ce qui le rend digne de confiance. Autre conséquence : deux fichiers de contenu identique partagent le même blob, et Git ne les stocke qu'une fois.

C'est vérifiable à la main. L'empreinte d'un blob est simplement le SHA-1 de la chaîne blob, suivie de la taille du contenu, d'un caractère nul, puis du contenu. Calculons-la avec Git, puis sans Git, avec l'outil `shasum` :

```
echo "bonjour" > bonjour.txt
git hash-object bonjour.txt
printf 'blob 8\0bonjour\n' | shasum
```

```
1cd909e05d33f0f6bc4ealcaf19b5749b434ceb3
1cd909e05d33f0f6bc4ealcaf19b5749b434ceb3 -
```

Les deux empreintes sont **identiques** : Git ne fait rien de magique, il hache le contenu (8 octets : les 7 lettres de « bonjour » plus le saut de ligne). Voyons le dernier commit « de l'intérieur » :

```
git cat-file -p HEAD
```

```
tree 14f9f852d095c04f65343c0f4fb89e500c3460ad
author La gérante <gerante@boutique.example> 1772442000 +0100
committer La gérante <gerante@boutique.example> 1772442000 +0100

Premier script et données (400 commandes)
```

On lit l'empreinte de l'arbre (*tree*), l'auteur, la date (en secondes) et le message. Ce premier commit n'a pas de ligne parent ; les suivants en auront une. Le mot HEAD désigne « le commit où je suis en ce moment » ; HEAD~1 est son parent.

6.1.5 Modifier, comparer, enregistrer : le cycle de travail

Le quotidien avec Git est une boucle de quatre gestes : **modifier** des fichiers, **regarder** ce qui a changé (`git status`, `git diff`), **choisir** ce qui part dans la prochaine photo (`git add`), **enregistrer** (`git commit`). La gérante veut maintenant le montant moyen **par canal de vente**.

```
echo 'print(df.groupby("canal")["montant"].mean().round(2))' >> analyse.py
git diff
git commit -q -am "Ajout du montant moyen par canal"
git log --oneline
```

```
diff --git a/analyse.py b/analyse.py
index 4f821d9..51a90f6 100644
--- a/analyse.py
+++ b/analyse.py
@@ -1,3 +1,4 @@
import pandas as pd
df = pd.read_csv("commandes.csv")
print("Montant moyen :", round(df["montant"].mean(), 2), "€")
+print(df.groupby("canal")["montant"].mean().round(2))
5dca424 Ajout du montant moyen par canal
04c558e Premier script et données (400 commandes)
```

`git diff` montre les différences entre le dossier de travail et la dernière photo : les lignes précédées de + sont ajoutées, celles précédées de - seraient supprimées, et le bloc @@ ... @@ situe la modification dans le fichier. L'option -a de `git commit` ajoute d'office les fichiers **déjà suivis** qui ont changé.

Trois variantes utiles de `git diff`. `git diff` compare le dossier de travail à l'index (« ce que je n'ai pas encore préparé ») ; `git diff --staged` compare l'index au dernier commit (« ce qui partira au prochain commit ») ; `git diff HEAD~1 HEAD` compare deux commits. Pour relire l'historique avec le détail des fichiers touchés : `git log --stat`.

6.1.6 Défaire une erreur

C'est la plus grande qualité de Git : **presque tout se rattrape**, à condition de choisir la bonne commande.

Situation	Commande	Effet
« J'ai cassé un fichier, je veux la version du dernier commit. »	<code>git restore fichier</code>	le fichier revient à l'état du dernier commit
« J'ai enregistré une erreur dans un commit. »	<code>git revert <commit></code>	un nouveau commit fait l'inverse ; l'historique reste intact
« Je veux revoir l'état du dossier à une date passée. »	<code>git switch --detach <commit></code>	on regarde le passé sans rien détruire ; <code>git switch main</code> pour revenir

Voici les deux premières en action : la gérante vide son script par erreur et le retrouve, puis commite une ligne de debug qu'elle annule proprement.

```
> analyse.py # oups : le fichier est vidé
git restore analyse.py # on retrouve le dernier commit
wc -l analyse.py
echo 'print("TODO")' >> analyse.py
git commit -q -am "Ligne de debug oubliée"
git revert --no-edit HEAD
```

```
4 analyse.py
[main 02d76fa] Revert "Ligne de debug oubliée"
Date: Mon Mar 2 10:00:00 2026 +0100
1 file changed, 1 deletion(-)
```

Le fichier a retrouvé ses 4 lignes ; l'historique contient maintenant l'erreur *et* son annulation, et le script est redevenu propre.

git restore est irréversible pour les modifications non enregistrées. Ce que Git n'a jamais photographié, il ne peut pas le retrouver. D'où la règle d'or : **faites des commits petits et fréquents**. Et méfiez-vous de `git reset --hard`, `git clean -fd` et `git push --force`, qui détruisent du travail sans confirmation ; en cas de doute, créez d'abord une branche de secours (`git branch sauvegarde`).

6.1.7 Ce qu'il ne faut pas suivre : .gitignore

Certains fichiers n'ont rien à faire dans l'historique : les fichiers temporaires de Python (`__pycache__`/), l'environnement virtuel (`.venv`/, section 6.3), les mots de passe, les données volumineuses ou confidentielles. On les déclare dans un fichier texte nommé `.gitignore`.

```
cat > .gitignore <<'FIN'
.venv/
__pycache__/
*.env
FIN
git status --short
git check-ignore -v secrets.env
```

```
?? .gitignore
.gitignore:3:*.env secrets.env
```

Git ne voit plus que `.gitignore` lui-même (qu'on **veut** suivre, pour que toute l'équipe ignore les mêmes choses). `git check-ignore -v` explique pourquoi un fichier est ignoré : il cite la règle et la ligne responsable.

Un secret commité est un secret perdu. Si vous enregistrez par erreur un mot de passe ou une clé d'accès, le retirer dans un commit suivant ne suffit pas : il reste lisible dans l'historique. Considérez-le comme compromis et **changez-le**. Prévenez l'erreur en mettant `.gitignore` en place **avant** de créer le fichier secret.

6.1.8 Les branches : tester une idée sans risque

Une **branche** est une ligne de travail parallèle. Techniquement, c'est trivial : un simple **pointeur** (une étiquette) vers un commit, qui avance à chaque nouveau commit. Créer une branche ne copie rien : cela prend une milliseconde, quelle que soit la taille du dépôt.

Une expérience sur une branche, puis la fusion

La gérante se demande à partir de quel montant offrir la livraison. Elle ouvre une branche dédiée à cette expérience, plutôt que de modifier le script principal ; quand elle est satisfaite, elle la **fusionne** (*merge*) dans main. Comme main n'a pas bougé entretemps, Git n'a rien à réconcilier : il se contente d'**avancer le pointeur** (*fast-forward*).

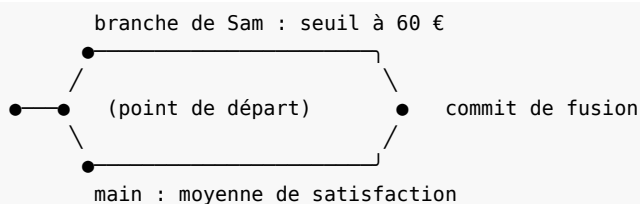
```
git switch -c seuil-livraison
printf 'SEUIL = 80\nprint("part >= SEUIL :", (df["montant"] >= SEUIL).mean())\n' >> analyse.py
git commit -q -am "Seuil de livraison gratuite à 80 €"
git switch main
git merge seuil-livraison
```

```
Switched to a new branch 'seuil-livraison'
Switched to branch 'main'
Updating 5798a46..a21fa8b
Fast-forward
 analyse.py | 2 ++
 1 file changed, 2 insertions(+)
```

Tant qu'on est sur main, avant la fusion, les lignes du seuil **n'existent pas** dans le fichier : elles ne vivent que sur la branche, et rien n'est perdu. Après la fusion, main les contient.

Quand les deux ont bougé : fusion automatique, puis conflit

Le cas intéressant arrive quand **deux personnes** modifient le dépôt en parallèle. Un collègue, Sam, teste sur sa branche un seuil de 60 € ; la gérante, sur main, ajoute une moyenne de satisfaction. Les deux historiques divergent, en « Y », puis se rejoignent dans un **commit de fusion** à deux parents :



```
git merge --no-edit seuil-sam
```

```
Auto-merging analyse.py
Merge made by the 'ort' strategy.
 analyse.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Git a réussi **automatiquement** : les deux changements portaient sur des zones différentes du fichier. Un **conflit** survient quand les deux branches modifient **la même ligne** de façons différentes : Git ne sait pas laquelle choisir et vous demande de trancher. Ce n'est pas une catastrophe, c'est le fonctionnement normal du travail à plusieurs. Provoquons-en un : la gérante veut un seuil de 70 €, Sam de 65 €.

```
git merge --no-edit seuil-gerante
grep -n -A4 '<<<<<<' analyse.py
```

```
Auto-merging analyse.py
CONFLICT (content): Merge conflict in analyse.py
Automatic merge failed; fix conflicts and then commit the result.
5:<<<<<< HEAD
6-SEUIL = 65
7-=====
8-SEUIL = 70
9->>>>>> seuil-gerante
```

Git s'arrête (CONFLICT (content)) et écrit dans le fichier, autour de la ligne litigieuse, trois **marqueurs** : <<<<<< HEAD (début de la version de la branche où l'on se trouve), ===== (séparation) et >>>>>> seuil-gerante (fin de la version qu'on fusionne). Résoudre le conflit, c'est **éditer le fichier à la main** pour ne garder que ce qu'on veut (ici, après discussion, 70 €), supprimer les marqueurs, puis déclarer le conflit réglé avec `git add` et conclure par un `commit`. Avant de valider, **relancez toujours le code** : une fusion peut réussir sans conflit et pourtant produire un programme qui ne marche plus.

```
# (après avoir édité analyse.py : on garde SEUIL = 70 et on supprime les marqueurs)
git add analyse.py
git commit -q --no-edit
git log --oneline --graph | head -n 5
```

```
* 9161812 Merge branch 'seuil-gerante'
|\
| * 885efd7 Seuil à 70 €
* | 3dcee2e Seuil à 65 €
|/
```

Réduire les conflits. Ils sont d'autant plus rares que les branches vivent peu de temps, qu'on rapatrie souvent main dans sa branche, qu'on évite de reformater un fichier entier pendant que d'autres y travaillent, et qu'on se répartit les fichiers. Un conflit est surtout un signal de **communication** à rattraper.

Pour s'entraîner. Cahier, chapitre 6 : application 6.2, exercice 6.5.

6.1.9 Étiqueter une version : les tags

Quand une version a une signification particulière (« le rapport remis à la banque »), on lui pose une **étiquette** (*tag*) qui ne bouge plus, contrairement à une branche.

```
git tag -a v1.0-rapport-banque -m "Version remise à la banque"
git tag
```

```
v1.0-rapport-banque
```

C'est la réponse à la question qui ouvrirait ce chapitre : « quel code a produit le graphique envoyé à la banque ? » Il suffit de noter le *tag* dans le rapport.

6.1.10 Dépôts distants : partager et sauvegarder

Jusqu'ici tout se passe sur une seule machine. Pour collaborer, ou simplement se **sauvegarder**, on utilise un **dépôt distant** (*remote*) : un autre dépôt Git, hébergé sur un serveur ou un site comme GitHub. Quatre commandes suffisent à comprendre l'essentiel :

Commande	Effet
<code>git clone <adresse></code>	copie complète d'un dépôt distant (historique compris) sur votre machine
<code>git push</code>	envoie vos nouveaux commits vers le dépôt distant
<code>git fetch</code>	récupère les nouveaux commits du distant sans toucher à vos fichiers
<code>git pull</code>	fetch puis fusion dans votre branche courante

Commande	Effet
----------	-------

Un dépôt distant n'a rien de mystérieux : c'est un dépôt Git comme les autres, souvent « nu » (*bare*, sans dossier de travail). On peut donc le **simuler sans Internet** avec un second dossier de la machine : c'est l'objet de l'application 6.3 du cahier. Avec un vrai service, les commandes ressemblent à ceci (*non exécuté ici : il faut un compte et une connexion*) :

```
git remote add origin git@github.com:nom/depot.git    # déclarer l'adresse du dépôt distant
git push -u origin main                               # envoyer son travail
git pull                                              # récupérer celui des autres
```

Le flux de travail standard en équipe est : (1) créer une **branche** pour chaque tâche ; (2) la pousser sur le site ; (3) ouvrir une **pull request** (*demande de fusion*) : une page où les collègues relisent les changements, commentent, puis acceptent la fusion dans main ; (4) supprimer la branche. Le *fork* est une copie personnelle d'un dépôt qui ne vous appartient pas, pour y proposer des modifications.

Pour s'entraîner. Cahier, chapitre 6 : application 6.3.

6.1.11 Bonnes pratiques pour la data science

1. **Un commit = une idée.** Corriger un bug et ajouter une analyse, ce sont deux commits.
2. **Ne versionnez pas les gros fichiers de données ni les secrets.** Pour des jeux de données volumineux, on les place ailleurs et on versionne un petit fichier décrivant leur origine, ou l'on utilise un outil dédié (*Git LFS*, *DVC*).
3. **Versionnez le code qui produit les résultats, pas les résultats.** Un graphique se régénère ; un script ne se reconstitue pas.
4. **Commitez souvent, poussez régulièrement.** Un disque dur peut mourir ; un dépôt distant est une sauvegarde.
5. **Ajoutez un fichier README :** à quoi sert le projet, comment l'exécuter (6.3 et 6.5).
6. **Posez des tags** sur les versions livrées.

Le mini-aide-mémoire des commandes de la section :

Je veux...	Commande
démarrer un dépôt	git init
voir l'état du dossier	git status
préparer / enregistrer	git add fichier puis git commit -m "message"
voir ce qui a changé	git diff, git diff --staged
lire l'historique	git log --oneline --graph
jeter mes modifications d'un fichier	git restore fichier
annuler un commit proprement	git revert <commit>
créer / changer de branche	git switch -c nom, git switch nom
fusionner	git merge nom
poser une étiquette	git tag -a nom -m "message"
copier / envoyer / récupérer	git clone, git push, git pull

À retenir

- Git garde un **album de photographies complètes** de votre dossier ; chaque photo (*commit*) porte un message, un auteur, une date et un pointeur vers sa parente.

- Trois zones : dossier de travail → `git add` → index → `git commit` → dépôt.
- Presque tout se rattrape (`restore`, `revert`, `switch --detach`) **si l'on a enregistré**. Faites des commits petits et fréquents.
- Une **branche** est un pointeur : en créer ne coûte rien ; on y teste une idée avant de **fusionner**.
- Un **conflit** n'apparaît que lorsque deux changements touchent la même ligne ; on édite, on `add`, on `commit`, et on **relance le code**.
- `.gitignore` pour les fichiers temporaires et secrets ; les secrets commités sont perdus.
- Un **dépôt distant** (GitHub...) est un dépôt Git ordinaire qui sert à partager et à sauvegarder.

6.2 Notebooks Jupyter

Intuition. Un **notebook** (« carnet ») est un document où l'on mélange, dans l'ordre où l'on réfléchit : du **texte** (« voici la question »), du **code** (« voici le calcul ») et le **résultat** de ce code (tableau, graphique), juste en dessous. C'est le cahier de laboratoire du data scientist : on explore, on essaie, on commente, et le tout se lit comme un récit. Le projet s'appelle **Jupyter** (de *Julia*, *Python*, *R*, les trois langages d'origine) ; ses fichiers portent l'extension `.ipynb`.

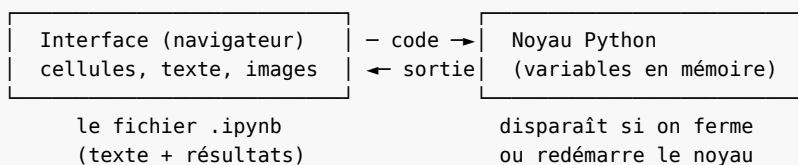
Dans un notebook, vous tapez un calcul, appuyez sur Maj + Entrée, et le résultat s'affiche à l'instant. Pas de cycle « écrire le fichier, l'enregistrer, lancer le programme, regarder l'écran » : c'est idéal pour **explorer** des données, **enseigner**, ou **raconter** une analyse. Mais cet outil a un défaut sournois, et toute la seconde moitié de cette section lui est consacrée : **un notebook mal tenu produit des résultats qu'on ne peut pas reproduire**.

6.2.1 Les ingrédients : cellules et noyau

Un notebook est une suite de **cellules** de deux sortes principales :

Type de cellule	Contenu	Résultat de l'exécution
Code	du Python (ou un autre langage)	la sortie du code : texte, tableau, graphique, erreur
Markdown	du texte mis en forme, titres, listes, formules $\$...\$$	du texte formaté (rien n'est « calculé »)

Derrière l'interface se cache un second ingrédient, essentiel pour tout comprendre : le **noyau** (*kernel*). C'est un **programme Python qui tourne en arrière-plan**, avec sa **mémoire**. Quand vous exécutez une cellule, son code est envoyé au noyau, qui l'exécute et renvoie le résultat. Les variables créées par une cellule **restent en mémoire dans le noyau** et sont donc visibles par toutes les cellules exécutées ensuite.



Retenez bien cette image : le **fichier** .ipynb et la **mémoire** du noyau sont deux choses distinctes. Le fichier garde le texte, le code et les derniers résultats affichés ; la mémoire, elle, contient l'état *actuel* des variables, qui peut très bien ne plus correspondre à ce que le code écrit dans le fichier produirait.

Pour installer et lancer l'interface (*non exécuté ici : l'interface s'ouvre dans un navigateur et ne peut pas être reproduite dans un livre*) :

```
pip install jupyterlab      # installe JupyterLab (l'interface moderne)
jupyter lab                # lance le serveur et ouvre le navigateur
```

On peut aussi ouvrir des notebooks directement dans **VS Code** (extension Jupyter), ou en ligne, sans rien installer, dans **Google Colab** (service gratuit de Google ; non testé ici). Le format des fichiers est le même partout.

6.2.2 Un notebook est un fichier texte (JSON)

Il n'y a rien de magique dans un fichier .ipynb : c'est du **JSON**, un format texte qui décrit des données imbriquées (dictionnaires et listes), lisible par n'importe quel langage. Pour s'en convaincre, fabriquons un notebook de deux cellules avec la bibliothèque nbformat, et regardons la cellule de code **comme du simple texte** :

```
import json
import nbformat
from nbformat import v4 as nbformat

nb = nbformat.new_notebook(cells=[
    nbformat.new_markdown_cell("# Ventes de la boutique", id="titre"),
    nbformat.new_code_cell("import pandas as pd\nndf = pd.read_csv('donnees/commandes.csv')\nndf.shape",
        id="chargement"),
])
nb.metadata["kernelspec"] = {"name": "python3", "display_name": "Python 3", "language": "python"}
print(json.dumps(nb.cells[1], indent=1))
```

```
{
  "id": "chargement",
  "cell_type": "code",
  "metadata": {},
  "execution_count": null,
  "source": "import pandas as pd\nndf = pd.read_csv('donnees/commandes.csv')\nndf.shape",
  "outputs": []
}
```

On y reconnaît le `cell_type` (code), la source (le texte tapé), et deux champs propres aux cellules de code : `execution_count` (le numéro d'exécution, vide tant que la cellule n'a pas tourné) et `outputs` (la liste des résultats affichés, vide pour l'instant). Exécutons maintenant le carnet avec `nbclient`, la bibliothèque qui pilote un noyau depuis un programme (c'est ce que fait le bouton « Exécuter tout ») :

```
from nbclient import NotebookClient

NotebookClient(nb, kernel_name="python3", resources={"metadata": {"path": "."}}).execute()
cellule = nb.cells[1]
print(cellule.execution_count, cellule.outputs[0].data["text/plain"])
```

```
1 (400, 4)
```

La cellule porte désormais son **numéro d'exécution** (1) et sa **sortie** (le tableau compte 400 lignes et 4 colonnes), enregistrées dans le fichier lui-même. Un graphique serait stocké sous forme

d'image encodée en texte : un notebook avec beaucoup de graphiques devient donc volumineux.

Ce que cela implique. Comme les résultats sont *écrits dans le fichier*, vous pouvez ouvrir un notebook et voir des résultats **sans rien exécuter**. C'est pratique pour partager... et dangereux, car **rien ne garantit que ces résultats correspondent au code affiché** (voir 6.2.3).

Pour s'entraîner. Cahier, chapitre 6 : application 6.4.

6.2.3 Le piège : l'état caché

Dans un script Python ordinaire, l'ordre d'exécution est celui du fichier, de haut en bas, toujours. Dans un notebook, **vous choisissez l'ordre** : vous pouvez relancer la cellule 5 trois fois, sauter la cellule 3, retourner modifier la cellule 1 sans relancer les suivantes, ou même supprimer une cellule dont la variable continue d'exister en mémoire. Le noyau, lui, se souvient de tout.

Pour voir ce phénomène sans notebook, **simulons** un noyau avec Python pur : un dictionnaire `memoire` joue la mémoire du noyau, et `executer` joue le rôle de `Maj + Entrée`. Trois cellules calculent le prix d'une commande de trois articles à 50 €, TVA à 19 % comprise. Puis la gérante corrige le prix à 80 €, relance la cellule A et la cellule C... **en oubliant de relancer B**.

```
memoire = {}
def executer(code): exec(code, memoire)          # joue « Maj + Entrée »

A, B, C = "prix = 50", "total = prix * 3 * 1.19", "print('Total TTC :', round(total, 2), '€')"
for cellule in (A, B, C): executer(cellule)      # exécution normale : 50 × 3 × 1,19 = 178,5
A = "prix = 80"
executer(A); executer(C)                        # on corrige A, on relance C, mais pas B
```

```
Total TTC : 178.5 €
Total TTC : 178.5 €
```

Le total affiché est **toujours 178,5 €** alors que le prix est maintenant de 80 € : la variable `total` en mémoire date de l'ancienne exécution de B. Sur l'écran, la cellule A affiche 80, la cellule C un total faux, et rien ne signale l'incohérence. Pire : un collègue qui exécute le notebook de haut en bas obtient $80 \times 3 \times 1,19 = 285,6$ €, un résultat **différent** de celui de la gérante. Autre variante du même piège, la **cellule supprimée** : une variable définie dans une cellule effacée « pour faire propre » continue de vivre dans le noyau, jusqu'à ce que quelqu'un d'autre ouvre le notebook et obtienne une erreur `NameError` (voir l'application 6.5 du cahier).

Ces scénarios ont un point commun : **le résultat dépendait d'un état invisible**, la mémoire du noyau, qui n'est écrite nulle part dans le fichier. C'est la première cause de notebooks non reproductibles. L'antidote est simple :

La règle du « Restart & Run All ». Avant de partager un notebook, de le commiter ou d'en tirer un chiffre pour un rapport, faites **Noyau → Redémarrer et tout exécuter** (*Restart Kernel and Run All Cells*). Cela efface la mémoire et rejoue toutes les cellules **dans l'ordre du fichier**. Si tout passe et que les résultats ne changent pas, votre notebook est sain. S'il casse, vous venez de découvrir un état caché, et mieux vaut le découvrir maintenant que devant votre client.

Pourquoi cela revient à un problème d'ordre. Un programme est reproductible si son résultat est une **fonction** de ses entrées (code + données + graine aléatoire). Dans un notebook, le résultat dépend en plus de la **suite des cellules exécutées** : une *séquence*, pas seulement

un ensemble. Il existe $n!$ façons d'ordonner n cellules, et le fichier n'en garde qu'une trace partielle (les numéros [1], [2]...). Imposer l'ordre du fichier (« Run All ») ramène la dépendance à **une seule** séquence canonique : celle qu'on lit.

6.2.4 Repérer un notebook douteux

Les numéros d'exécution enregistrés dans le fichier permettent de repérer les notebooks douteux **sans les exécuter**. Un notebook exécuté d'un seul trait, sur un noyau neuf, affiche [1], [2], [3]... sans trou ni saut. Si les numéros sont désordonnés (3, 1, 2), si une cellule n'a jamais tourné, ou si une erreur est enregistrée, l'alarme sonne. Un tel détecteur tient en une quinzaine de lignes de Python (c'est un bon exemple de petite application des fonctions et des listes du chapitre 4 à un fichier de notebook) ; il ne *prouve* pas qu'un notebook est reproductible (la seule preuve est de le réexécuter), mais il repère les cas flagrants en une milliseconde, par exemple dans un script de vérification avant chaque commit. Vous l'écrirez dans l'application 6.5 du cahier.

6.2.5 Bonnes pratiques

1. **Restart & Run All avant tout partage.** Règle numéro un.
2. **Les imports et les réglages en première cellule** : bibliothèques, graine aléatoire (`rng = np.random.default_rng(42)`), chemins des fichiers. On voit d'emblée ce dont le notebook dépend.
3. **Un notebook, une question.** Un notebook de 200 cellules est ingérable ; découpez : 01-nettoyage.ipynb, 02-exploration.ipynb, 03-modele.ipynb.
4. **Sortez le code réutilisable dans des fichiers .py** (fonctions de nettoyage, de calcul) et importez-les : `from outils import nettoyer`. Ce code se teste (4.6), se versionne proprement avec Git, et sert à d'autres notebooks.
5. **Écrivez du texte entre les cellules** : titres, hypothèses, interprétation. Le notebook est un récit, pas un brouillon.
6. **Chemins relatifs** (`donnees/commandes.csv`), jamais un chemin absolu propre à votre ordinateur : le notebook doit marcher sur une autre machine.
7. **N'utilisez pas le notebook pour la production.** Une fois l'analyse stabilisée, un script .py lancé depuis la ligne de commande (6.3) est plus fiable qu'un carnet qu'on clique à la main.

6.2.6 Notebooks et Git : le problème des sorties

Un notebook est un fichier texte : Git peut donc le suivre. Mais, comme les **sorties** et les **numéros d'exécution** sont stockés dans le fichier, la moindre ré-exécution modifie des dizaines de lignes (graphiques encodés, numéros qui changent) : les comparaisons `git diff` deviennent illisibles et les conflits de fusion cauchemardesques. La pratique courante est de **ne garder dans Git que les sources**, en effaçant les sorties avant de commiter. Effacer les sorties revient à vider la liste `outputs` et à remettre `execution_count` à vide dans chaque cellule de code : l'application 6.5 le fait en quelques lignes avec `nbformat`. Dans la pratique, on n'écrit pas ce nettoyage à la main : l'outil `nbstripout` s'installe en « crochet » Git et efface les sorties automatiquement à chaque commit. Le revers de la médaille : le fichier commité n'affiche plus de résultats ; on publie alors, à côté, une version **exportée** (voir ci-dessous).

6.2.7 Exporter : du notebook au rapport, au script

La bibliothèque nbconvert transforme un notebook en d'autres formats : **HTML** (à envoyer par e-mail), **PDF**, **Markdown**, ou **script Python** (le code seul). En ligne de commande, l'option `--execute` rejoue d'abord tout le notebook dans un noyau neuf, c'est-à-dire exactement la règle « Restart & Run All », automatisée :

```
jupyter nbconvert --to html --execute analyse.ipynb      # page web, après avoir tout rejoué
jupyter nbconvert --to script analyse.ipynb              # le code seul, en fichier .py
jupyter nbconvert --to markdown analyse.ipynb            # document Markdown
```

(Ces commandes supposent un fichier `analyse.ipynb` ; elles sont exécutées pour de vrai dans l'application 6.4 du cahier.) Pour un PDF, `--to pdf` demande en plus une installation de LaTeX (6.5).

6.2.8 Pour finir : choisir son outil

Je veux...	J'utilise...
explorer des données, essayer des idées	un notebook
un traitement répétable, automatisé (tous les soirs)	un script .py
une fonction que je vais réutiliser	un module .py, importé dans le notebook
un rapport reproductible de bout en bout	un document qui mélange texte et code, exécuté d'un trait (6.5)

À retenir

- Un notebook = des **cellules** (code / Markdown) + un **noyau** (la mémoire). Le fichier .ipynb est du JSON qui stocke le code **et** les derniers résultats.
- Piège principal : l'**état caché**. La mémoire du noyau peut contenir des variables qui ne correspondent plus au code écrit (cellule modifiée mais pas relancée, cellule supprimée...).
- Avant de partager : **Restart & Run All**. Des numéros d'exécution [1], [2], [3]... sans trou sont bon signe.
- Rangez le code réutilisable dans des .py, ne versionnez pas les sorties, exportez avec nbconvert.
- Un notebook sert à **explorer et raconter**, un script à **produire**.

Pour s'entraîner. Cahier, chapitre 6 : applications 6.4 et 6.5, exercice 6.6.

6.3 Ligne de commande et environnements

Intuition. Une interface graphique, c'est un **menu de restaurant** : on montre du doigt ce qu'on veut parmi ce qui est proposé. La **ligne de commande** (*command line*, *terminal*), c'est parler directement au cuisinier : on peut demander des choses que le menu n'a jamais prévues, et surtout **écrire sa commande sur un papier** pour la rejouer à l'identique demain, ou la donner à un collègue. Pour un data scientist, ce second point est capital : une suite de commandes est une **recette reproductible**, alors qu'une série de clics ne laisse aucune trace.

Le terminal fait peur au début : un écran noir, un curseur clignotant, aucune indication. Il suffit pourtant d'une douzaine de commandes pour devenir efficace, et elles servent pour toute

une carrière : sur votre ordinateur, sur un serveur distant, dans un conteneur, dans un outil d'automatisation. Dans ce livre, nous utilisons **Bash**, le terminal de Linux, de macOS et de Git Bash/WSL sous Windows.

6.3.1 S'orienter dans les dossiers

Les fichiers sont rangés dans une **arborescence** : un dossier contient des fichiers et d'autres dossiers. Le terminal est toujours « placé » dans un dossier, le **dossier courant**. Cinq commandes pour s'y repérer :

Commande	Signification	Mnémonique
<code>pwd</code>	afficher le dossier courant	<i>print working directory</i>
<code>ls</code>	lister le contenu du dossier	<i>list</i>
<code>cd dossier</code>	se déplacer dans un dossier	<i>change directory</i>
<code>mkdir dossier</code>	créer un dossier	<i>make directory</i>
<code>touch fichier</code>	créer un fichier vide (ou mettre à jour sa date)	

Pour **copier, déplacer, renommer, supprimer**, on a de même `cp`, `mv` (qui sert aux deux usages : déplacer et renommer) et `rm`. Quelques **chemins spéciaux** sont à connaître : `.` est le dossier courant, `..` le dossier **parent**, `~` votre dossier personnel, `/` la racine de toute l'arborescence. Un chemin est **absolu** s'il part de la racine et **relatif** s'il part du dossier courant (`donnees/commmandes.csv`).

rm n'a pas de corbeille. Ce qui est supprimé dans le terminal est **définitivement perdu**. Ne tapez jamais `rm -r` sans relire la ligne entière, et n'écrivez jamais de commande de suppression avec un chemin que vous n'avez pas vérifié avec `ls` juste avant. Avec Git (6.1), au moins, vous pouvez toujours récupérer une version enregistrée.

Quatre astuces qui changent la vie. (1) La touche **Tab** complète les noms de fichiers : tapez `cd` et puis **Tab**. (2) La flèche **↑** rappelle les commandes précédentes ; **Ctrl + R** cherche dans l'historique. (3) **Ctrl + C** interrompt une commande qui tourne. (4) commande `--help` (ou `man commande`) affiche la documentation de n'importe quelle commande.

Voici l'arborescence d'un projet d'analyse bien rangé, créée d'un coup. La syntaxe `{a,b,c}` est une **expansion d'accolades** : une seule ligne `mkdir` crée les quatre dossiers.

```
cd ~/atelier
mkdir -p etude-ventes/{donnees,notebooks,src,rapports}
cp boutique/commmandes.csv etude-ventes/donnees/
cd etude-ventes
find . -not -name '.' | sort
```

```
./donnees
./donnees/commmandes.csv
./notebooks
./rapports
./src
```

Chaque dossier a un rôle : les données brutes ne se modifient jamais, les notebooks explorent, `src` contient le code réutilisable, `rapports` reçoit les résultats.

6.3.2 Regarder le contenu d'un fichier

Un fichier CSV est un fichier **texte** : on peut le lire dans le terminal sans l'ouvrir dans un tableur. C'est très pratique pour vérifier rapidement un fichier de plusieurs millions de lignes que Excel ne saurait pas ouvrir.

```
cd donnees
head -n 3 commandes.csv
tail -n 2 commandes.csv
wc -l commandes.csv

canal,montant,livraison,satisfaction
Boutique,44.8,0,4
Site,34.5,2,4
Réseaux,37.5,3,4
Site,31.4,4,4
401 commandes.csv
```

`head -n 3` affiche les 3 premières lignes, `tail -n 2` les 2 dernières, `wc -l` compte les lignes : le CSV a une ligne d'en-tête (les noms de colonnes) puis une commande par ligne, soit 401 lignes pour 400 commandes. Pour parcourir un long fichier page par page, il existe `less` (on avance avec la barre d'espace, on quitte avec `q`) ; `cat` fichier affiche tout le fichier d'un coup.

6.3.3 Interroger des données avec des outils de texte

Il existe quelques outils très anciens, très rapides, et parfaitement adaptés aux fichiers de données en colonnes. Chaque outil fait **une chose** :

Outil	Rôle
<code>grep motif</code>	garder les lignes qui contiennent le motif
<code>cut -d, -f2</code>	extraire la colonne n°2 (séparateur ,)
<code>sort</code>	trier (-n : numérique, -r : décroissant, -t, : séparateur, -k2 : colonne)
<code>uniq -c</code>	compter les lignes identiques consécutives (d'où le sort avant)
<code>awk</code>	mini-langage pour calculer sur les colonnes

Combien de commandes viennent du canal « Réseaux » ? Et combien par canal ? Pour la seconde question, il faut extraire la colonne des canaux (sans l'en-tête), la trier pour regrouper les valeurs identiques, puis compter :

```
grep -c Réseaux commandes.csv
tail -n +2 commandes.csv | cut -d, -f1 | sort | uniq -c | sort -rn

138
  148 Site
  138 Réseaux
  114 Boutique
```

La seconde ligne est un **pipeline** (« tuyau ») : `tail -n +2` supprime l'en-tête (« commence à la ligne 2 »), `cut -d, -f1` garde la première colonne, `sort` regroupe les canaux, `uniq -c` compte chaque groupe, et `sort -rn` classe du plus fréquent au plus rare. Le symbole `|` (« pipe ») envoie la sortie d'une commande à l'entrée de la suivante. Le site est le premier canal en nombre de commandes, devant les réseaux et la boutique. Pour calculer, par exemple un montant moyen, on utilise `awk` : il lit le fichier ligne par ligne, `$2` désigne la 2^e colonne, `NR` le numéro de ligne.

```
awk -F, 'NR > 1 { somme += $2; n++ } END { printf "montant moyen : %.2f € sur %d commandes\n",
↪ somme/n, n }' commandes.csv
```

```
montant moyen : 60.25 € sur 400 commandes
```

L'option `-F`, fixe le séparateur. Pour chaque ligne sauf l'en-tête (`NR > 1`), on ajoute le montant à une somme et on compte ; à la fin (`END`) on affiche la moyenne : on retrouve bien les 60,25 € du chapitre 3.

Quand utiliser quoi ? Ces commandes brillent pour un **coup d'œil rapide**, pour traiter des fichiers trop gros pour la mémoire, ou pour assembler une chaîne de traitement dans un script. Dès que l'analyse devient subtile (jointures, valeurs manquantes, statistiques), on passe à `pandas` (4.4). Les deux approches sont complémentaires.

6.3.4 Pipes et redirections : assembler de petits outils

Toutes ces commandes obéissent à la même philosophie, dite « **philosophie Unix** » : *chaque programme fait une seule chose, la fait bien, et lit/écrit du texte*. Ainsi tous les programmes se branchent les uns sur les autres, comme des briques de LEGO. Il y a trois flux : l'**entrée standard** (clavier ou tuyau), la **sortie standard** (écran) et la **sortie d'erreur** (écran aussi, mais distincte). Les opérateurs de redirection vers ou depuis des fichiers :

Opérateur	Effet	Exemple
<code>></code>	écrire la sortie dans un fichier (écrase l'existant)	<code>ls > liste.txt</code>
<code>>></code>	ajouter la sortie à la fin du fichier	<code>echo "fin" >> liste.txt</code>
<code><</code>	lire l'entrée depuis un fichier	<code>sort < liste.txt</code>
<code>2></code>	rediriger les messages d'erreur	<code>ls absent 2> erreurs.txt</code>

Sauvegardons un résumé des ventes dans un fichier, puis provoquons une erreur pour voir le **code de sortie** :

```
cd ..
tail -n +2 donnees/commandes.csv | cut -d, -f1 | sort | uniq -c > rapports/commandes-par-canal.txt
cat rapports/commandes-par-canal.txt
ls donnees/inexistant.csv 2> rapports/erreurs.txt
echo "code de sortie : $?"
```

```
114 Boutique
138 Réseaux
148 Site
code de sortie : 2
```

La variable spéciale `$?` contient le **code de sortie** de la dernière commande : **0** signifie « **tout s'est bien passé** », toute autre valeur signale une erreur (ici, `ls` a échoué : le message est allé dans `erreurs.txt` au lieu de s'afficher). Les scripts s'appuient sur ces codes pour enchaîner des étapes : `commande1 && commande2` lance la seconde **seulement si** la première a réussi ; `commande1 || commande2` la lance seulement si la première a échoué.

Pour s'entraîner. Cahier, chapitre 6 : application 6.6, exercices 6.1, 6.2 et 6.4.

6.3.5 Lancer des programmes Python depuis le terminal

Le plus utile est de pouvoir **lancer ses propres scripts Python**, avec des paramètres. Les arguments tapés après le nom du script sont disponibles en Python dans la liste `sys.argv` (le premier élément, `sys.argv[0]`, est le nom du script lui-même). Voici un petit outil qui résume un fichier CSV donné en argument :

```
cat > src/resume.py <<'FIN'
import sys
import pandas as pd

df = pd.read_csv(sys.argv[1])
print(f"{len(df)} lignes, {df.shape[1]} colonnes")
print(df.describe().loc[["mean", "max"]].round(2))
FIN
python src/resume.py donnees/commandes.csv
```

```
400 lignes, 4 colonnes
      montant  livraison  satisfaction
mean    60.25         3.29           3.96
max    255.70        13.00           5.00
```

Un bon outil respecte la convention des codes de sortie : **0** quand tout va bien, un code non nul (avec un message clair) pour un fichier absent ou un mauvais usage. En Python, on le fait avec `sys.exit(code)`. L'application 6.7 du cahier ajoute cette gestion d'erreurs à notre outil. Deux variantes utiles de python : `python -c "print(2+3)"` exécute une ligne, `python -m module` exécute un module de la bibliothèque (par exemple `python -m venv`, juste après).

6.3.6 Le problème des bibliothèques : les environnements virtuels

Voici un scénario qui arrive à tout le monde. En janvier, la gérante installe pandas pour son analyse. En juin, elle commence un autre projet qui nécessite une **ancienne** version de la même bibliothèque. Si tout est installé au même endroit, mettre à jour casse l'ancien projet, et rétrograder casse le nouveau. Pire : quand elle envoie son code à un collègue, comment sait-il quelles versions utiliser ? D'où l'**environnement virtuel** :

Définition. Un environnement virtuel est un **dossier isolé** contenant une copie de l'interpréteur Python et ses propres bibliothèques. Chaque projet a le sien : les bibliothèques de l'un n'affectent jamais celles de l'autre. L'environnement se **crée**, s'**active**, puis on y installe ce dont le projet a besoin avec **pip**, le gestionnaire de paquets de Python.

Voici le cycle complet. Pour ne pas télécharger des centaines de mégaoctets dans cet exemple, nous installons une toute petite bibliothèque, `tabulate` (qui formate joliment des tableaux en texte), à la place de pandas.

```
python -m venv .venv
source .venv/bin/activate
pip install --quiet tabulate 2>&1 | grep -v -i -E "notice|warning"
pip freeze > requirements.txt
cat requirements.txt
deactivate
```

```
tabulate==0.10.0
```

`python -m venv .venv` crée l'environnement dans un dossier caché nommé `.venv`. `source .venv/bin/activate` l'**active** : dès cet instant, python et pip désignent ceux du dossier `.venv` (sous Windows, la commande d'activation est `.venv\Scripts\activate`). `pip install` installe la bibliothèque **dans .venv uniquement**, et `pip freeze` liste tout ce qui est installé avec les

numéros de version exacts. Ce dernier point est la clé de la reproductibilité : on enregistre cette liste dans un fichier, traditionnellement nommé `requirements.txt`, que l'on versionne avec Git (6.1). Quelqu'un qui reçoit le projet reconstruit **le même environnement** en trois commandes : créer un environnement neuf, l'activer, puis `pip install -r requirements.txt`. Le dossier `.venv` lui-même ne se partage pas (il est propre à la machine et volumineux) : il figure dans le `.gitignore` de 6.1, et seul `requirements.txt` voyage avec le projet.

requirements.txt fige les bibliothèques, pas Python lui-même. Mentionnez aussi la version de Python dans le README (« testé avec Python 3.13 »). Pour des besoins plus avancés, il existe des outils qui gèrent aussi la version de Python : `conda` (très répandu en data science, notamment sous Windows), `uv` (récent et très rapide), ou `Poetry`. Ils répondent au même besoin : isoler, figer, reproduire.

Pour le livre complet, l'installation recommandée dans l'avant-propos serait donc (*non exécutée ici : elle télécharge environ 200 Mo de bibliothèques*) :

```
python -m venv .venv
source .venv/bin/activate          # sous Windows : .venv\Scripts\activate
pip install numpy pandas scipy matplotlib seaborn
pip freeze > requirements.txt
```

Pour s'entraîner. Cahier, chapitre 6 : application 6.9, exercice 6.7.

6.3.7 Variables d'environnement et PATH

Le terminal garde en mémoire des **variables** (texte nommé), que l'on affiche avec `$` : c'est ainsi que nous avons utilisé `$?` et `$HOME`. On en crée avec `NOM=valeur` (sans espaces autour du `=`) ; `export` les rend visibles aux programmes lancés ensuite.

```
export TAUX_TVA=0.19
python -c "import os; print('TVA lue depuis Python : ', float(os.environ['TAUX_TVA']))"
```

```
TVA lue depuis Python : 0.19
```

Ce mécanisme sert à **transmettre des réglages** à un programme sans les écrire dans son code : un chemin de fichier, un taux, ou un **mot de passe de base de données** (qu'on ne met jamais dans le code ni dans Git !). Une variable est particulièrement importante : **PATH**, la liste des dossiers où le terminal cherche les programmes. Quand vous tapez `python`, le terminal parcourt les dossiers de **PATH** dans l'ordre et lance le premier `python` trouvé. **Activer un environnement virtuel, c'est simplement placer son dossier `bin` en tête du `PATH`** : voilà pourquoi `python` désigne alors celui de `.venv`.

6.3.8 Et sous Windows ?

Trois options : **WSL** (un vrai Linux intégré à Windows, recommandé), **Git Bash** (fourni avec Git for Windows), ou **PowerShell** qui a ses propres noms de commandes :

Bash	PowerShell	Rôle
<code>pwd</code>	<code>pwd</code> ou <code>Get-Location</code>	dossier courant
<code>ls</code>	<code>ls</code> ou <code>dir</code>	lister
<code>cp, mv, rm</code>	<code>cp, mv, rm</code>	copier, déplacer, supprimer
<code>cat</code>	<code>cat</code> ou <code>type</code>	afficher un fichier

Bash	PowerShell	Rôle
source .venv/bin/activate	.venv\Scripts\Activate.ps1	activer l'environnement

À retenir

- Une suite de commandes est une **recette reproductible** ; un clic n'en est pas une.
- Se repérer : pwd, ls, cd, mkdir, cp, mv, rm (sans corbeille !). . = ici, .. = parent, ~ = maison.
- Lire et interroger du texte : head, tail, wc, grep, cut, sort, uniq, awk. On les assemble avec |, on redirige avec >, >>, 2>.
- Un programme renvoie un **code de sortie** (0 = succès) ; && et || s'appuient dessus.
- Un **environnement virtuel par projet** : python -m venv .venv, source .venv/bin/activate, pip install, pip freeze > requirements.txt. On partage requirements.txt, pas .venv.

6.4 ⊕ Pour aller plus loin : scripts shell, environnements avancés et Docker

Section optionnelle. Elle prolonge la section 6.3 pour celles et ceux qui veulent **automatiser** leurs analyses (scripts shell), **isoler** plus finement leurs projets, et découvrir **Docker**, l'outil qui emballe un projet avec tout son système. Vous pouvez passer directement à 6.5 sans rien perdre du fil.

6.4.1 Écrire un script shell

Quand on retape trois fois la même suite de commandes, on la range dans un **script shell** : un fichier texte contenant des commandes, exécuté d'un coup. Notre premier script affiche, pour un fichier de commandes, le nombre de commandes par canal. Il introduit les ingrédients de base : **arguments** (\$1 est le premier argument, \$# leur nombre), **variables**, **test** (if), **boucle** (for) et **substitution de commande** (\$(...)) insère le résultat d'une commande dans une autre).

```
mkdir -p scripts
cat > scripts/rapport.sh <<'FIN'
#!/usr/bin/env bash
set -euo pipefail
[ "$#" -eq 1 ] || { echo "usage : $0 fichier.csv" >&2; exit 2; }
total=$(( $(wc -l < "$1") - 1 ))
echo "== $total commandes dans $1 =="
for canal in Boutique Réseaux Site; do
    echo "  $canal : $(grep -c "^$canal," "$1")"
done
FIN
bash scripts/rapport.sh donnees/commandes.csv
```

```
== 400 commandes dans donnees/commandes.csv ==
Boutique : 114
Réseaux : 138
Site : 148
```

Passons ce fichier en revue :

- La première ligne, `#!/usr/bin/env bash`, s'appelle le **shebang** : elle indique avec quel programme exécuter le fichier. Les lignes qui commencent par `#` sont des **commentaires**.
- `set -euo pipefail` est une **ceinture de sécurité** que l'on met en tête de tout script sérieux : `-e` arrête le script à la première commande qui échoue, `-u` refuse d'utiliser une variable jamais définie (souvent une faute de frappe), `-o pipefail` fait échouer un pipeline si l'une de ses commandes échoue (et pas seulement la dernière).
- La ligne `[ "$#" -eq 1 ] || { ... }` vérifie qu'on a reçu exactement un argument ; sinon elle affiche un mode d'emploi **sur la sortie d'erreur** (`>&2`) et quitte avec le code 2.
- `total=$(( $(wc -l < "$1") - 1 ))` calcule le nombre de lignes moins l'en-tête (la double parenthèse `$(( ... ))` fait de l'arithmétique entière).
- La boucle `for` passe en revue chaque canal ; `grep -c "^$canal,"` compte les lignes **commençant** par le nom du canal suivi d'une virgule.

Un script peut aussi être rendu directement exécutable (`chmod +x scripts/rapport.sh`, puis `./scripts/rapport.sh`), appliquer un traitement à **chaque fichier** d'un groupe avec une boucle (le joker `*` désigne tous les fichiers d'un motif, comme `donnees/*.csv`), et s'enchaîner avec d'autres scripts dans un petit **pipeline**.

6.4.2 Enchaîner des étapes : échouer bruyamment

Un vrai projet d'analyse est une **chaîne d'étapes** : vérifier les données, calculer, produire le rapport. Le point crucial est ce qui se passe quand une étape échoue. Avec `set -e`, le script **s'arrête net** à la première étape défailante et renvoie un code d'erreur : on sait immédiatement que quelque chose ne va pas. Sans cette ceinture, il continue comme si de rien n'était, annonce « terminé » et renvoie le code 0 (succès !) alors que **rien n'a été calculé** : le pire des scénarios, car personne ne se doute que le rapport est vide ou partiel. D'où la règle : **un script qui échoue doit échouer bruyamment**. L'application 6.8 du cahier met les deux comportements côte à côte.

Les limites du shell. Le shell est parfait pour **enchaîner des programmes**, manipuler des fichiers et des dossiers. Mais dès qu'il faut des calculs, des structures de données ou de la logique non triviale, passez à Python : ses 300 lignes seront plus lisibles et plus testables que 300 lignes de shell. Règle pratique : si le script dépasse une cinquantaine de lignes ou nécessite des tableaux, c'est un script Python.

6.4.3 Faire tourner un script tout seul : cron

Sous Linux et macOS, le programme **cron** exécute des commandes à heure fixe. On le configure en ajoutant des lignes à la **table cron** (`crontab -e`). Une ligne comporte cinq champs de temps (minute, heure, jour du mois, mois, jour de la semaine) suivis de la commande. Par exemple, pour lancer notre script tous les jours à 7 h du matin :

```
# minute heure jour mois jour-semaine  commande
0 7 * * * cd ~/etude-ventes && bash scripts/rapport.sh donnees/commandes.csv
```

(Non exécuté : la table `cron` appartient à l'utilisateur de la machine, et le résultat ne se verrait qu'à 7 h.) Le `*` signifie « toutes les valeurs ». Sous Windows, l'équivalent est le *Planificateur de tâches*. Dans le cloud, on utilise des services d'orchestration plus riches (comme Airflow), que vous rencontrerez dans les volumes suivants de la série.

6.4.4 Isoler plus finement : plusieurs versions d'une même bibliothèque

Au 6.3.6, nous avons dit que des projets distincts peuvent avoir besoin de **versions différentes** d'une même bibliothèque : deux environnements virtuels coexistent sans se gêner, chacun avec sa version, exigée avec `==` dans la commande d'installation (l'application 6.9 du cahier le démontre). Dans un fichier `requirements.txt`, les **numéros de version** peuvent s'écrire avec plus ou moins de rigueur :

Écriture	Signification
<code>pandas==3.0.0</code>	exactement cette version (reproductibilité maximale)
<code>pandas>=3.0,<4</code>	n'importe quelle version 3.x, à partir de la 3.0
<code>pandas</code>	n'importe quelle version (déconseillé : le résultat change avec le temps)

La bonne pratique à retenir : **`==` pour reproduire un résultat** (rapport remis à un client, article), **fourchette** pour une bibliothèque en développement qui doit rester compatible avec d'autres. Deux outils alternatifs que vous croiserez (*non exécutés ici : non installés*) :

```
conda create --name etude-ventes python=3.13 pandas # conda installe aussi Python lui-même
uv venv && uv pip install -r requirements.txt # uv : un installateur très rapide, compatible
↪ avec pip
```

6.4.5 Docker : emballer le projet avec son système

Un environnement virtuel isole les **bibliothèques Python**. Mais votre analyse peut dépendre d'autre chose : une version précise de Python, une bibliothèque système, un outil comme R, une configuration. « Chez moi, ça marche ! » reste possible. **Docker** résout ce problème en emballant le projet avec **tout son système d'exploitation de base**, dans une sorte de mini-ordinateur virtuel appelé **conteneur**.

Une analogie. Un environnement virtuel, c'est une **étagère personnelle** dans une cuisine partagée : vos ingrédients sont à vous, mais la cuisine (le système) est celle de tout le monde. Docker, c'est un **repas livré dans une boîte hermétique** : la boîte contient les aliments, les couverts et même la table. Où qu'on ouvre la boîte, le repas est identique.

Trois mots à retenir :

- Une **image** est le **modèle** figé : le système de base, Python, vos bibliothèques, votre code. On la construit une fois à partir d'une **recette** (le Dockerfile).
- Un **conteneur** est une **instance en marche** d'une image. On peut en lancer dix à partir de la même image.
- Un **registre** (comme *Docker Hub*) est un entrepôt d'images, un peu comme GitHub pour le code : on y trouve des images prêtes à l'emploi (python, postgres, jupyter...).

Voici le Dockerfile d'un projet d'analyse comme le nôtre :

```
# une image Python toute prête
FROM python:3.13-slim
# le dossier de travail dans le conteneur
WORKDIR /app
# la liste des bibliothèques, installées pendant la construction
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
# puis le reste du projet
COPY . .
# la commande lancée au démarrage du conteneur
CMD ["python", "src/resume.py", "donnees/commandes.csv"]
```

FROM choisit la base, WORKDIR fixe le dossier, COPY copie des fichiers de votre machine vers l'image, RUN exécute une commande **pendant la construction**, CMD définit ce que fera le conteneur **au démarrage**. L'ordre des lignes n'est pas anodin : Docker garde en cache chaque étape, et la copie de requirements.txt **avant** celle du code permet de ne pas réinstaller les bibliothèques à chaque modification d'un script. On construit puis on lance :

```
docker build -t etude-ventes .           # construit l'image à partir du Dockerfile
docker run --rm etude-ventes             # lance un conteneur, le supprime à la fin
docker run --rm -v "$PWD/rapports:/app/rapports" etude-ventes # partage le dossier rapports/
```

La troisième commande montre un point essentiel : un conteneur est **éphémère**, ce qu'il écrit disparaît avec lui. Pour récupérer des résultats, on **monte** un dossier de votre machine dans le conteneur (option -v).

Non exécuté, honnêtement. Docker n'est pas disponible dans l'environnement où ce livre a été rédigé. Vérifions-le plutôt que de le supposer :

```
command -v docker || echo "docker : absent de cet environnement"
```

```
docker : absent de cet environnement
```

Le Dockerfile et les commandes ci-dessus sont donc donnés **sans sortie**, et n'ont pas été testés ici. Ils suivent le modèle standard de la documentation de Docker ; si vous les essayez, une erreur de syntaxe ou de version est possible, et le message d'erreur vous guidera.

Docker en data science, à quoi ça sert ? (1) **Reproduire** une analyse des années plus tard, à l'identique. (2) **Déployer** un modèle sur un serveur (volume III). (3) **Travailler en équipe** sans que chacun installe la même pile logicielle. (4) Lancer facilement des logiciels complexes (une base de données, Jupyter) sans les installer : `docker run postgres`. Le coût : une courbe d'apprentissage et des images volumineuses (quelques centaines de Mo à plusieurs Go).

À retenir

- Un **script shell** range des commandes dans un fichier : shebang, arguments (\$1, \$#), if, for, \$(...). Commencez-le par `set -euo pipefail` pour qu'il échoue **bruyamment**.
- Pour tout ce qui est calcul ou logique complexe, passez à Python ; gardez le shell pour **enchaîner**.
- **cron** lance des commandes à heure fixe (5 champs de temps + commande).
- Plusieurs environnements virtuels peuvent contenir **des versions différentes** de la même bibliothèque ; `==` fige une version, `>=`, `<` la borne.
- **Docker** emballe un projet avec son système (image → conteneur). Ici : concepts et recette donnés, mais **non exécutés**.

Pour s'entraîner. Cahier, chapitre 6 : applications 6.8 et 6.9, exercices 6.7 et 6.8.

6.5 ⊕ Pour aller plus loin : recherche reproductible

Section optionnelle. Elle rassemble les outils du chapitre pour répondre à une exigence centrale de la science des données : **n'importe qui doit pouvoir refaire votre analyse, et obtenir les mêmes résultats.** Vous y verrez comment produire un rapport **qui se régénère tout seul** à partir des données, des graines aléatoires, un fichier Makefile, R Markdown, Quarto et LaTeX.

6.5.1 Reproductible, répliquable : de quoi parle-t-on ?

Deux mots proches, deux idées différentes :

Terme	Question posée	Ce qu'on garde fixe	Ce qu'on change
Reproductibilité	« Si je relance exactement la même analyse, ai-je les mêmes chiffres ? »	données, code, environnement	rien (c'est une exigence technique)
Répliquabilité	« Si quelqu'un refait l'étude avec de nouvelles données , arrive-t-il à la même conclusion ? »	la méthode	les données (c'est une exigence scientifique)

On ne peut pas espérer la seconde sans la première : si je ne sais même pas refaire mes propres calculs, personne ne pourra juger si ma conclusion est solide. Une partie des « crises de réplication » observées dans plusieurs disciplines vient de simples **défauts de reproductibilité** : données modifiées à la main dans un tableur, numéros de version oubliés, code introuvable, étapes faites « à la souris ». Les outils des sections précédentes en sont le remède. Voici la **liste de contrôle** que nous allons dérouler :

Élément	Question à se poser	Outil (section)
Données	Ai-je gardé les données brutes intactes ? Sont-elles identifiables ?	dossier donnees/ en lecture seule, empreinte sha256 (6.5.8)
Code	Est-il versionné ? Retrouve-t-on la version qui a produit le résultat ?	Git (6.1)
Aléa	Mes simulations donnent-elles les mêmes nombres à chaque fois ?	graines aléatoires (6.5.2)
Environnement	Quelles versions de Python et des bibliothèques ?	requirements.txt (6.3), relevé de versions (6.5.3)
Exécution	Les étapes s'enchaînent-elles sans intervention manuelle ?	scripts, Makefile (6.4, 6.5.5)
Rapport	Les chiffres du texte viennent-ils <i>directement</i> du code ?	documents dynamiques (6.5.4)

6.5.2 L'aléa maîtrisé : les graines

Beaucoup d'analyses font appel au hasard : échantillonnage, simulation, validation croisée, bootstrap (3.7), initialisation de modèles. Or un ordinateur ne tire pas vraiment au hasard : il calcule une suite de nombres qui **a l'air** aléatoire à partir d'un point de départ, la **graine** (*seed*). Même graine, même suite. Sans graine explicite, Python en choisit une différente à chaque exécution (à partir de l'horloge, par exemple), et les résultats changent. Voici la moyenne d'un échantillon de 10 commandes tiré au hasard, avec et sans graine :

```
import numpy as np
import pandas as pd

montants = pd.read_csv("donnees/commandes.csv")["montant"].to_numpy()
def moyenne(rng, taille=10):
    return montants[rng.choice(len(montants), size=taille, replace=False)].mean()

print("graine 42, deux fois :", round(moyenne(np.random.default_rng(42)), 2),
      ↪ round(moyenne(np.random.default_rng(42)), 2))
print("sans graine, deux fois identiques ?", moyenne(np.random.default_rng()) ==
      ↪ moyenne(np.random.default_rng()))
```

```
graine 42, deux fois : 46.55 46.55
sans graine, deux fois identiques ? False
```

Avec la même graine 42, on obtient **exactement** le même nombre à chaque fois, sur n'importe quel ordinateur (pour la même version de NumPy) ; sans graine, les deux tirages diffèrent. C'est ce qui rend les nombres du livre reproductibles.

Reste à savoir **quelle graine choisir**. La réponse est contre-intuitive : *n'importe laquelle*, mais **sans la choisir en regardant le résultat**. Changer de graine jusqu'à obtenir le résultat qui nous arrange est une forme de triche appelée *p-hacking* (voir 3.5). La graine sert à **figer** une exécution, pas à l'améliorer. Pour vérifier qu'un résultat n'est pas un accident de graine, on l'observe pour **plusieurs graines** :

Pour les graines 1 à 8, les moyennes d'échantillons de 10 commandes vont de 43,3 € à 73,3 €, alors que la vraie moyenne de la population est de 60,25 € : elles varient fortement d'une graine à l'autre (voir l'erreur-type, 3.2). La graine 42 n'a donc rien de spécial, et une conclusion qui ne tiendrait que pour elle serait suspecte. Deux règles pratiques : (1) créer **un seul** générateur `rng = np.random.default_rng(graine)` en début de programme, et le passer aux fonctions qui en ont besoin, plutôt que de multiplier les graines cachées ; (2) se souvenir que chaque bibliothèque a sa propre source d'aléa (random de Python, NumPy, et plus tard PyTorch ou scikit-learn avec leur paramètre `random_state`) : il faut fixer **chacune** de celles qu'on utilise.

Une graine ne garantit pas l'identité entre versions. Les mêmes graine et code peuvent donner des tirages différents si la **version** de NumPy change (les algorithmes évoluent). D'où la nécessité de noter aussi les versions, juste après.

6.5.3 Relever l'environnement

Le fichier `requirements.txt` (6.3) fige les bibliothèques à *installer*. On peut aussi, en fin de rapport, **imprimer ce qui a réellement servi**. Cela prend deux lignes et sauve bien des enquêtes (en R, la fonction `sessionInfo()` joue ce rôle) :

```
import sys, importlib.metadata as meta
print("Python", sys.version.split()[0], "| numpy", meta.version("numpy"), "| pandas",
      ↪ meta.version("pandas"))
```

Python 3.13.3 | numpy 2.5.3 | pandas 3.0.6

La sortie est le **relevé des versions** de cette exécution : on l'ajoute au bas de chaque rapport.

6.5.4 Un rapport qui se fabrique tout seul

Le défaut le plus courant d'un rapport : on calcule un chiffre dans un notebook, on le **recopie à la main** dans un document, et trois semaines plus tard on met à jour les données sans mettre à jour le texte. Le remède s'appelle le **document dynamique** (*literate programming*, « programmation lettrée ») : le texte et les chiffres sont produits **par le même programme**, donc ils ne peuvent pas diverger. Nous construisons cette chaîne avec ce que nous avons déjà :

```
donnees/commandes.csv → src/faire_rapport.py → rapport.md → pandoc → HTML, PDF
      (données)           (code : calcule et      (texte et chiffres      (mise en forme)
                        écrit le texte)          cohérents)
```

Première étape : un programme Python qui **écrit** un document Markdown en y insérant les chiffres calculés. Le Markdown est le format qui sert à écrire ce livre.

```
cat > src/faire_rapport.py <<'FIN'
import sys
import pandas as pd

df = pd.read_csv(sys.argv[1])
m = df["montant"]
print('---\ntitle: "Les ventes de la boutique"\nlang: fr\n---\n')
print(f"Le fichier contient **{len(df)} commandes** ; le montant moyen est de "
      f"**{m.mean():.2f} €**", la médiane de **{m.median():.2f} €**.")
for canal, g in df.groupby("canal"):
    print(f"- {canal} : {len(g)} commandes, panier moyen {g['montant'].mean():.2f} €")
FIN
python src/faire_rapport.py donnees/commandes.csv > rapports/rapport-ventes.md
cat rapports/rapport-ventes.md
```

```
---
title: "Les ventes de la boutique"
lang: fr
---

Le fichier contient **400 commandes** ; le montant moyen est de **60.25 €**, la médiane de **51.00 €**.
- Boutique : 114 commandes, panier moyen 74.81 €
- Réseaux : 138 commandes, panier moyen 49.01 €
- Site : 148 commandes, panier moyen 59.50 €
```

Le programme n'écrit **aucun chiffre en dur** : le nombre de commandes, les moyennes et le détail par canal sont calculés. Si les données changent demain, il suffit de relancer. Le résultat est un fichier Markdown propre ; **Pandoc**, le « couteau suisse » de la conversion de documents, le transforme en page web, en PDF ou en document Word :

```
pandoc --standalone rapports/rapport-ventes.md -o rapports/rapport-ventes.html
ls rapports
```

```
commandes-par-canal.txt
erreurs.txt
rapport-ventes.html
rapport-ventes.md
```

L'application 6.10 du cahier ajoute à ce rapport un intervalle de confiance, une formule, un tableau et un PDF. Retenez le principe : **pas de date du jour ni de valeur aléatoire non figée dans un rapport reproductible**, sinon deux exécutions ne produisent plus le même fichier.

6.5.5 make : ne refaire que ce qui a changé

Tant que la chaîne tient en deux commandes, on les retape. Mais un vrai projet compte des dizaines d'étapes : nettoyage, tableaux, graphiques, rapport. Relancer **tout** à chaque modification est lent ; relancer **à la main** les bonnes étapes est source d'oublis. L'outil **make** (né en 1976, toujours en pleine forme) résout les deux problèmes. On lui décrit dans un fichier nommé **Makefile** des **règles** : une cible, ses dépendances, et la commande qui fabrique la cible à partir d'elles. **make** compare les **dates de modification** : si une dépendance est plus récente que la cible, il **refait** la cible ; sinon il ne fait rien. Voici le **Makefile** de notre projet (attention : l'indentation des commandes **doit être une vraie tabulation**, pas des espaces) :

```
rapports/rapport-ventes.html: rapports/rapport-ventes.md
    pandoc --standalone rapports/rapport-ventes.md -o $@

rapports/rapport-ventes.md: donnees/commandes.csv src/faire_rapport.py
    python src/faire_rapport.py donnees/commandes.csv > $@
```

Lisons-le de haut en bas. Le rapport HTML dépend du fichier Markdown ; le fichier Markdown dépend des **données** et du **programme** qui le fabrique. Dans les commandes, `$@` désigne la cible. Supprimons les fichiers produits, puis lançons **make** deux fois, et enfin « touchons » le fichier de données (ce qui met à jour sa date) :

```
make
make
touch donnees/commandes.csv
make
```

```
python src/faire_rapport.py donnees/commandes.csv > rapports/rapport-ventes.md
pandoc --standalone rapports/rapport-ventes.md -o rapports/rapport-ventes.html
make: 'rapports/rapport-ventes.html' is up to date.
python src/faire_rapport.py donnees/commandes.csv > rapports/rapport-ventes.md
pandoc --standalone rapports/rapport-ventes.md -o rapports/rapport-ventes.html
```

make affiche chaque commande qu'il lance : le premier appel refait les **deux** étapes, le deuxième répond qu'il n'y a **rien à faire**, et, quand on « touche » le fichier de données, les **deux** étapes sont de nouveau déclenchées.

Le grand intérêt. Il suffit de modifier le texte du rapport, et seule la conversion se refait. Dans un grand projet, c'est un gain de temps considérable, et, surtout, **la documentation de la chaîne de traitement** est le **Makefile** lui-même : un lecteur voit d'un coup d'œil de quoi dépend quoi. Le README du projet peut alors se résumer à : « Pour tout reproduire : **make** ».

6.5.6 Les outils « tout-en-un » : R Markdown et Quarto

Notre chaîne maison (Python qui écrit du Markdown, puis Pandoc) fonctionne, mais le monde de la data science a standardisé la même idée dans des outils dédiés : un **seul fichier** mélange texte Markdown et **blocs de code** ; à la compilation, le code est exécuté et **ses résultats sont insérés** dans le document final (HTML, PDF, Word, diaporama).

R Markdown (pour R, mais aussi Python) fonctionne ainsi : le fichier `.Rmd` contient un en-tête, du texte, des blocs de code `{r}` et du code R **en ligne** dans le texte (une expression entre accents graves, précédée de `r`). La moyenne écrite dans une phrase est alors **calculée** à la compilation, jamais recopiée. L'application 6.11 du cahier compile un vrai document R Markdown.

Quarto est le successeur de R Markdown : même idée, mais indépendant du langage (Python, R, Julia), avec davantage de formats (sites, livres, présentations). Un fichier `.qmd` ressemble à ceci :

```
---
title: "Les ventes de la boutique"
format: html
---
```

Le montant moyen est de `{python} round(moyenne, 2)` €.

```
{python}
import pandas as pd
df = pd.read_csv("donnees/commandes.csv")
moyenne = df["montant"].mean()
{python}
```

```
quarto render rapport.qmd          # produit rapport.html
quarto render rapport.qmd --to pdf  # produit un PDF (via LaTeX)
```

Non exécuté. Quarto n'est pas installé dans l'environnement de rédaction : l'exemple .qmd et les deux commandes ci-dessus sont donnés à titre d'illustration, sans avoir été testés ici. (R Markdown, lui, est exécuté dans le cahier.) La syntaxe exacte, notamment celle du code en ligne {python}, peut dépendre de la version de Quarto : consultez sa documentation si vous l'installez.

6.5.7 LaTeX : composer proprement les formules et les PDF

LaTeX (on prononce « latèk ») est le langage de composition de documents scientifiques : c'est lui qui met en forme les formules des chapitres 1 à 3 de ce livre. Vous l'avez déjà écrit sans le savoir : $\bar{x}$, $\sum$, $\frac{a}{b}$ sont des commandes LaTeX. Un document complet se compose d'un **préambule** (les réglages) et d'un **corps** :

```
\documentclass{article}
\usepackage{amsmath}
\begin{document}
Intervalle de confiance à 95\,% de la moyenne :
\[ \bar{x} \pm 1{,}96 \frac{s}{\sqrt{n}} \]
\end{document}
```

Un compilateur (xelatex, installé avec ce livre) transforme ce fichier en PDF ; l'application 6.11 du cahier le fait pour de vrai. En pratique, vous n'écrirez presque jamais du LaTeX *complet* : vous le laisserez Pandoc ou Quarto générer à partir du Markdown, et vous n'écrirez à la main que les formules. Pour écrire à plusieurs un document LaTeX sans rien installer, le service en ligne **Overleaf** est très utilisé (non testé ici).

6.5.8 Garder une empreinte des données

Dernier maillon de la chaîne : s'assurer que les **données** n'ont pas changé en cachette. Une **empreinte cryptographique** (*hash*) est un court texte, calculé à partir du contenu d'un fichier : le moindre changement du fichier (même un seul chiffre) change complètement l'empreinte. C'est le principe que nous avons vu au 6.1.4 pour les objets de Git. On calcule celle de commandes.csv, on la **note dans le README**, et n'importe qui peut vérifier. Ici, une faute de frappe glisse dans le fichier (le premier montant, 44,8, devient 448,0) :

```
sha256sum donnees/commandes.csv > donnees/EMPREINTES.sha256
sha256sum -c donnees/EMPREINTES.sha256
sed -i '2s/44.8/448.0/' donnees/commandes.csv
sha256sum -c donnees/EMPREINTES.sha256 || echo "ALERTE : les données ont été modifiées"
```

```
donnees/commandes.csv: OK
donnees/commandes.csv: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
ALERTE : les données ont été modifiées
```

La première vérification répond OK ; après la modification d'un seul nombre, elle échoue : l'alerte sonne avant même qu'on lance l'analyse.

Le « kit de reproductibilité » d'un projet, en une page. Un dossier de projet bien tenu contient : un README (but, comment reproduire, version de Python), un fichier `requirements.txt`, les données brutes **non modifiées** avec leur empreinte, le code versionné avec Git, un Makefile (ou un script) qui enchaîne les étapes, et le rapport généré. Pour archiver une version définitive de manière pérenne, on dépose le tout sur un service qui attribue un identifiant stable (DOI), comme **Zenodo** (non testé ici).

À retenir

- **Reproductible** = mêmes données + même code + même environnement $\Rightarrow$ mêmes résultats. **Répliquable** = mêmes conclusions sur de nouvelles données.
- **Graine aléatoire** : fixer *une* graine par exécution, la choisir *avant* de voir les résultats, et vérifier sur plusieurs graines que la conclusion tient. Noter aussi les versions des bibliothèques.
- **Document dynamique** : le texte et les chiffres sortent du **même** programme (Python + Pandoc, R Markdown, Quarto) ; on ne recopie jamais un chiffre à la main.
- **make** ne refait que ce qui est périmé ; le Makefile documente la chaîne de traitement.
- **LaTeX** compose les formules et les PDF ; on le fait généralement générer par Pandoc ou Quarto.
- Une **empreinte** (sha256sum) détecte toute modification des données.

Pour s'entraîner. Cahier, chapitre 6 : applications 6.10 et 6.11, exercices 6.9 et 6.10.

Bilan du chapitre 6

Vous savez maintenant :

- **versionner** votre travail avec Git : photographier (add, commit), comparer (diff), défaire (restore, revert), travailler en parallèle (branches, fusions, conflits) et partager (dépôts distants) ;
- **utiliser un notebook** pour explorer et raconter, **sans tomber dans le piège de l'état caché** (Restart & Run All, détecteur d'ordre, ne pas versionner les sorties) ;
- **vous servir du terminal** : vous repérer, lire, filtrer et résumer des fichiers avec grep, cut, sort, uniq et awk, assembler des commandes avec les tuyaux et les redirections, comprendre les codes de sortie ;
- **isoler chaque projet** dans un environnement virtuel et **figer** ses dépendances dans `requirements.txt` ;
- (en option) **automatiser** avec des scripts shell et make, comprendre Docker, fabriquer un **rapport dynamique** (Python + Pandoc, R Markdown, Quarto, LaTeX), maîtriser les graines aléatoires et surveiller l'intégrité des données avec une empreinte.

Le fil rouge de tout le chapitre tient en une phrase : **un résultat n'existe que s'il peut être refait.**

Vous disposez désormais de l'ensemble des fondations du volume : les mathématiques (chapitre 1), les probabilités (2), la statistique (3), la programmation (4), les bases de données (5) et les outils de travail (6). Il est temps de tout assembler dans le **projet du volume** (au cahier) : une étude complète des ventes de la boutique, des données brutes jusqu'au rapport.

Pour s'entraîner. Le cahier (chapitre 6) rassemble les onze applications guidées et les dix exercices corrigés de ce chapitre.

Points clés

« On ne sait vraiment une chose que lorsqu'on peut l'expliquer à quelqu'un d'autre sans regarder ses notes. »

Ce dernier chapitre court **fixe l'essentiel** de chaque chapitre en quelques lignes, pour pouvoir y revenir. La vérification, elle, se fait dans le cahier : un projet de bout en bout et trente questions d'auto-évaluation avec leurs réponses.

Les points clés, chapitre par chapitre

Chapitre	Ce que vous devez emporter
1. Mathématiques	Un vecteur est une liste de nombres <i>et</i> une flèche ; une matrice transforme l'espace. Les valeurs propres disent de combien une direction est étirée, la SVD en donne la meilleure approximation. La dérivée mesure un taux de variation, le gradient pointe vers la plus forte montée : on descend dans l'opposé pour optimiser. Les ordinateurs calculent avec des approximations ($0,1 + 0,2 \neq 0,3$) : on compare avec une tolérance.
2. Probabilités	Une probabilité mesure l'incertitude ; la formule de Bayes met à jour une croyance quand on observe un fait (et une alerte rare est souvent une fausse alerte). Les lois (Bernoulli, binomiale, Poisson, normale...) sont des modèles ; l'espérance et la variance les résument. La loi des grands nombres dit que la moyenne converge ; le théorème central limite dit comment elle fluctue (en $\sigma/\sqrt{n}$).
3. Statistique	Dessiner avant de calculer. Un estimateur se juge à son biais et à sa variance. Un intervalle de confiance quantifie l'incertitude ; une p-valeur n'est <i>pas</i> la probabilité que l'hypothèse soit vraie ; « significatif » n'est pas « important ». Tester plusieurs fois impose de corriger (Bonferroni, Holm, Benjamini-Hochberg). Le bootstrap et les tests de permutation fonctionnent sans hypothèse de loi.
4. Programmation	Python pour tout faire, R pour comparer. Une bonne structure de données (dictionnaire, ensemble) vaut mieux qu'un calcul plus rapide. Vectorisez avec NumPy et pandas au lieu de boucler. Un graphique répond à une question et ne ment pas (axes honnêtes). Un test automatique est un filet de sécurité.
5. SQL	Une base relationnelle sépare l'information en tables reliées par des clés. WHERE filtre les lignes, HAVING filtre les groupes. LEFT JOIN garde les lignes sans correspondance, NULL se teste avec IS NULL. Les fonctions fenêtres calculent sans écraser les lignes ; les CTE donnent un nom à chaque étape. Normaliser évite la redondance et les incohérences.
6. Outils	Git garde l'historique et permet d'essayer sans risque (branches). Un notebook mélange code et texte, mais cache un état : <i>Restart & Run All</i> avant de partager. La ligne de commande assemble de petits outils ; un environnement virtuel et un <code>requirements.txt</code> rendent l'analyse reproductible.

Chapitre	Ce que vous devez emporter
Projet (cahier)	Le cycle complet : question → contrôle des données → description → comparaison ou liaison rigoureuse → conclusion prudente → reproductibilité .

Trois idées qui traversent tout le volume

1. **Les données varient** : un chiffre calculé sur un échantillon est une estimation, jamais la vérité (chapitres 2 et 3).
2. **Un calcul juste n'est pas une conclusion juste** : il faut vérifier les hypothèses, dessiner, contrôler les données, distinguer association et causalité (chapitres 3 à 5 et projet).
3. **Ce qui n'est pas reproductible n'existe pas** : graines fixées, code versionné, environnement décrit (chapitres 4 à 6).

Pour s'entraîner. Cahier, chapitre 7 : le projet du volume (une étude complète de la boutique, de la question au rapport) et les trente questions d'auto-évaluation avec leur corrigé et leur grille.

Et maintenant ?

Le volume I vous a donné le **socle**. Il ne contient volontairement aucun modèle « à la mode » : ceux-ci demandent justement les bases que vous venez d'acquérir. Dans le **volume II : Modélisation statistique**, vous apprendrez à **modéliser** : la régression linéaire (la droite des moindres carrés du projet du cahier, généralisée à plusieurs variables), les modèles linéaires généralisés, les séries temporelles (la saisonnalité des ventes, enfin traitée proprement), l'analyse de survie et la statistique bayésienne.

Un conseil pour la suite. Ne passez pas au volume II en vous reprochant de ne pas tout retenir du volume I : personne ne retient tout. Retenez **où chercher**. Gardez ce livre à portée de main, et revenez-y chaque fois qu'une notion (une p-valeur, une jointure, un gradient) revient dans un contexte nouveau. C'est ainsi, en revenant, que les fondations deviennent solides.

À retenir, tout simplement. La data science est un artisanat : des mathématiques simples, du code propre, beaucoup de bon sens, et la méthode. Vous avez désormais les gestes. Il ne reste qu'à les répéter.